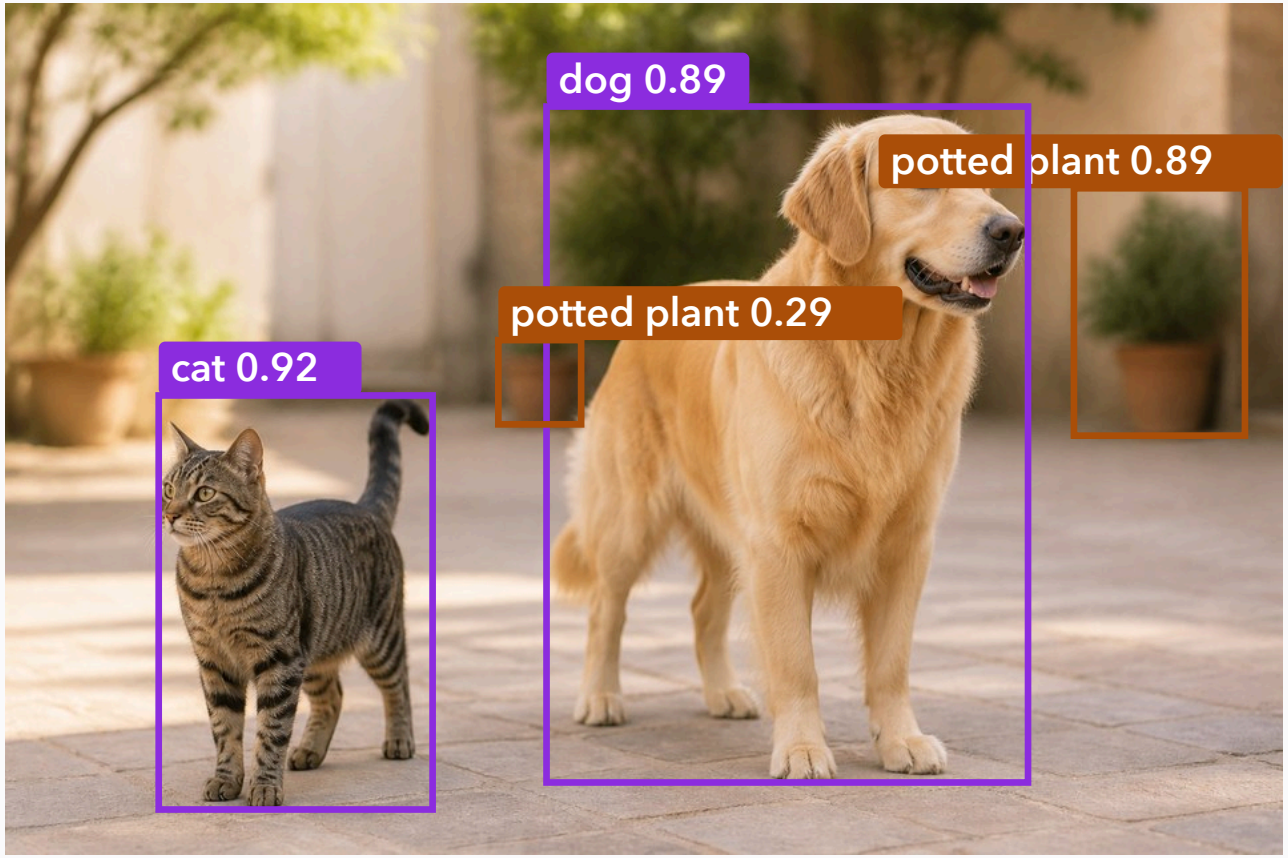
Object detection: from one label to a set of objects

A detector looks at a photo and returns a list: one entry per object, each a class and a box. How can a network with a fixed-size output return a list whose length changes from photo to photo?

ES 667 Deep Learning · Lecture 22
Nipun Batra · IIT Gandhinagar



> The real output of a small detector, YOLOv8n, on today's photo: four objects, each with a class, a score and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Part 1 · Why detect objects?

1 · Why detect? | 2 · A list | 3 · Boxes, IoU | 4 · One box, many | 5 · Dense grid | 6 · NMS | 7 · Three grids | 8 · Measuring

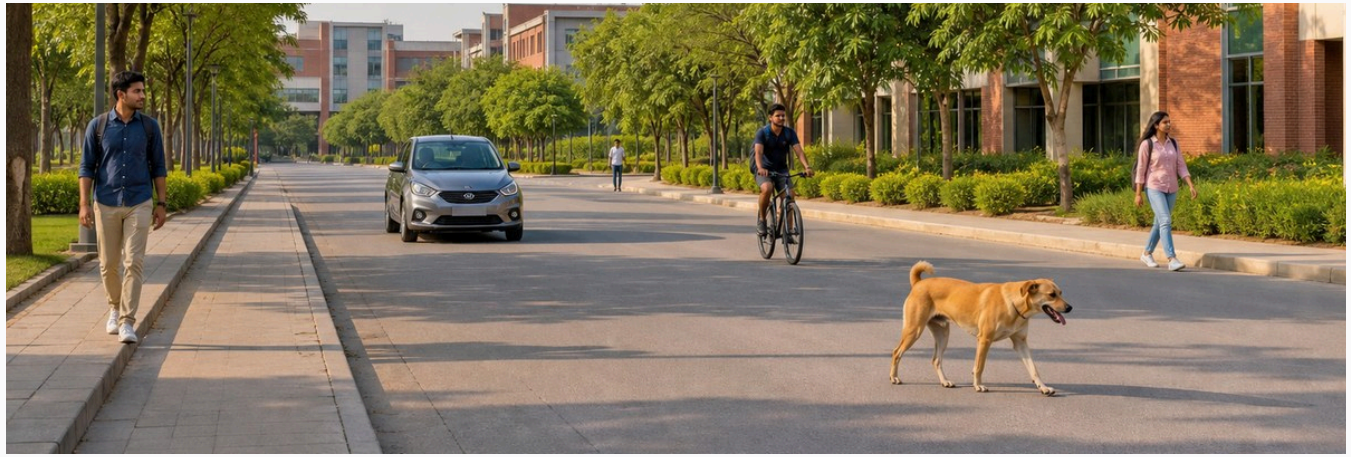
A camera becomes useful when it can say what is where. Start with six jobs that need exactly that.

1. Six photos, six jobs: say what you would want to know, then see the answer.
2. Notice that every answer has the same shape, and what it is used for.
3. Three things that make the job hard.

> Why detection matters: six jobs, one kind of answer, three difficulties.

▶ Notebook, part 1

On the road: who is where?



A car’s camera sees this street. What must the car know before it brakes or steers?

> A car’s camera sees this street. Before it brakes or steers, what does it need to know?

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

On the road: who is where?



A car's camera sees this street. What must the car know before it brakes or steers?

find people, cyclists, vehicles, animals

use brake or steer; follow each one from frame to frame

In US crash data, cars that brake automatically for pedestrians had 27% fewer pedestrian crashes, but no fewer on dark, unlit roads (IIHS, 2022).

> YOLOv8n's answer: 4 people, a car, a bicycle and a dog, each with a box and a score. This list is what the car acts on.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

In a crowd: how many people?



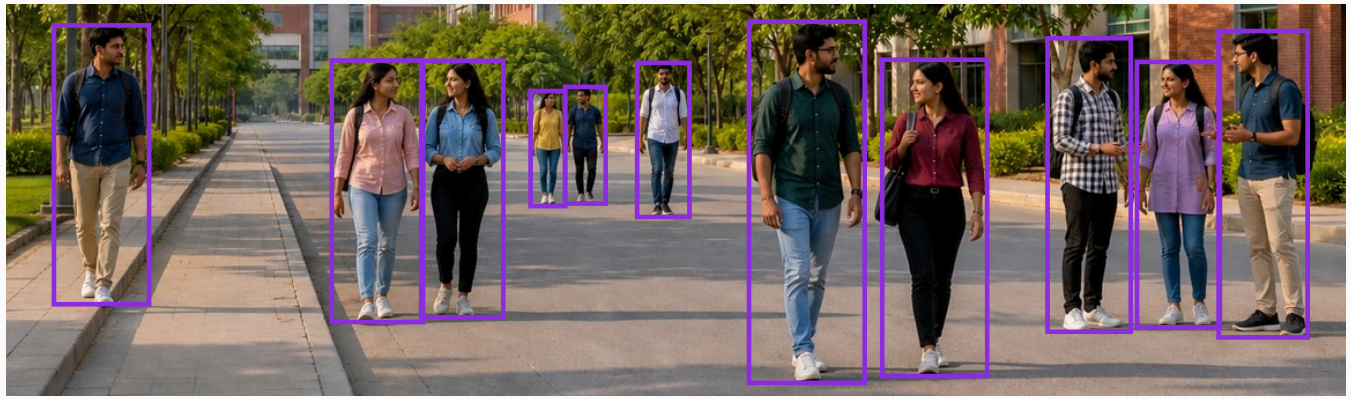
How many people are on this path? Count them before the next click.

> How many people are on this path? Count them.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

In a crowd: how many people?



How many people are on this path? Count them before the next click.

find people

use count them; warn when a place gets too crowded

Stations, stadiums and large gatherings count people camera by camera to spot dangerous crowding early.

> YOLOv8n boxes 11 people. The count is the length of the list, and the list also says where each person is.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

At a camera trap: which animals walked past?



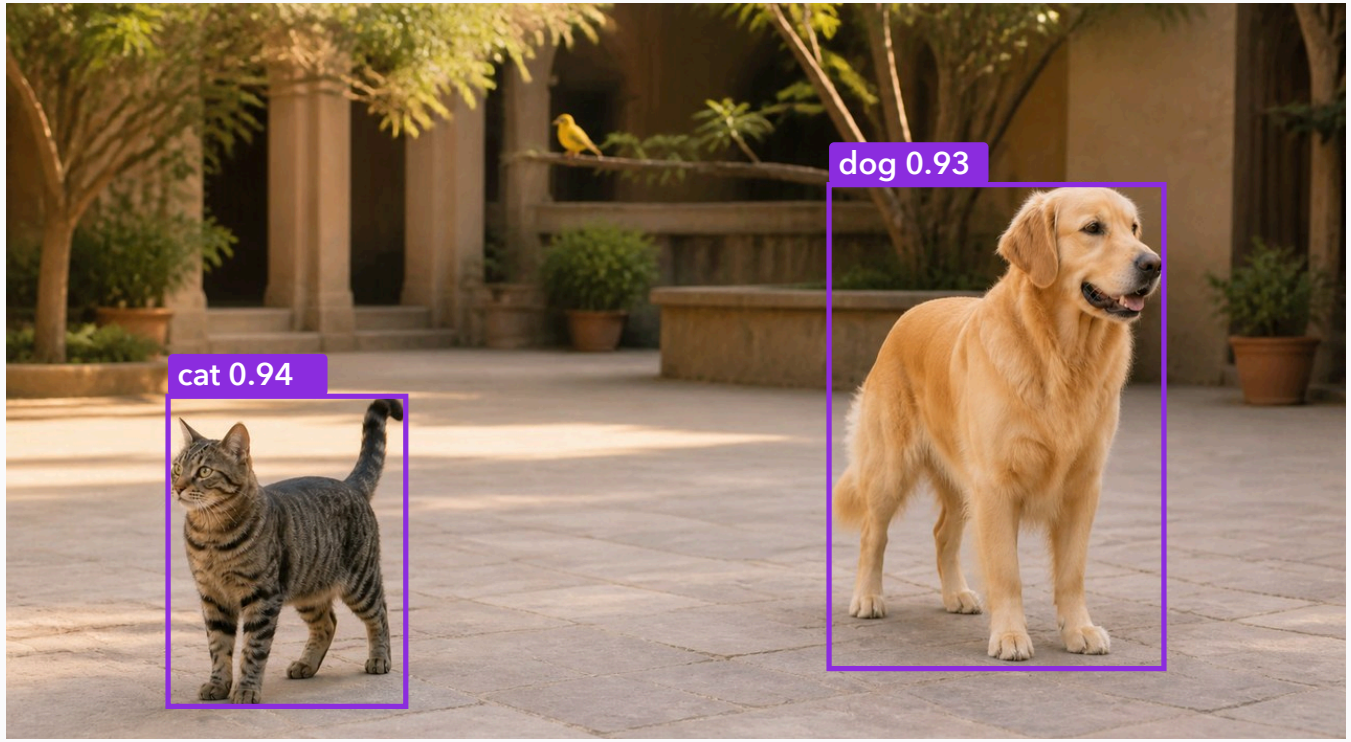
A camera on a trail takes a photo whenever something moves. Which animals are in this one?

A camera trap takes a photo whenever something moves. Which animals are in this one?

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

At a camera trap: which animals walked past?



A camera on a trail takes a photo whenever something moves. Which animals are in this one?

find	animals by species, people, vehicles
use	count each species; skip the empty photos

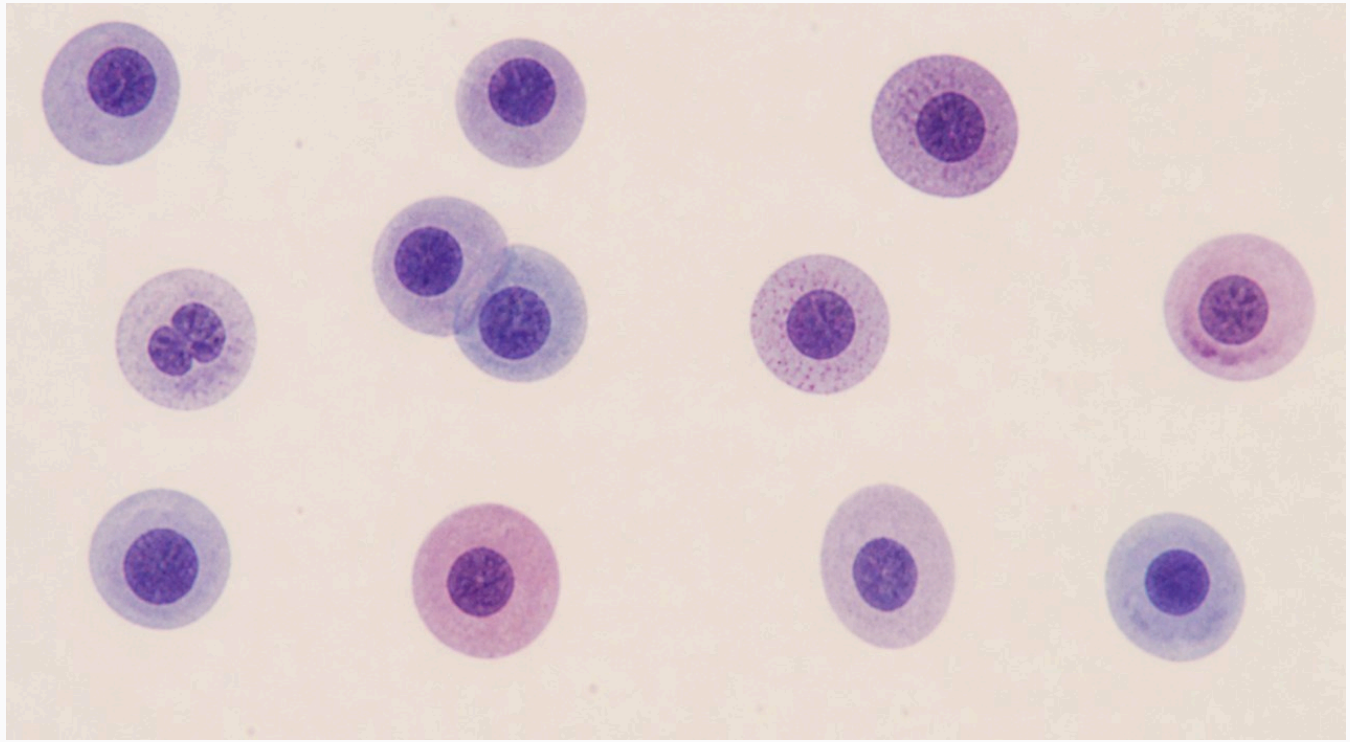
In an Arctic study of 548,627 camera-trap photos, a workflow built on the free MegaDetector left only 4–9% for people to check (Celis et al., 2024).

> YOLOv8n: a cat (0.94) and a dog (0.93). Look again at the photo: did it find every animal? We come back to this in a few slides.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Under a microscope: count the cells



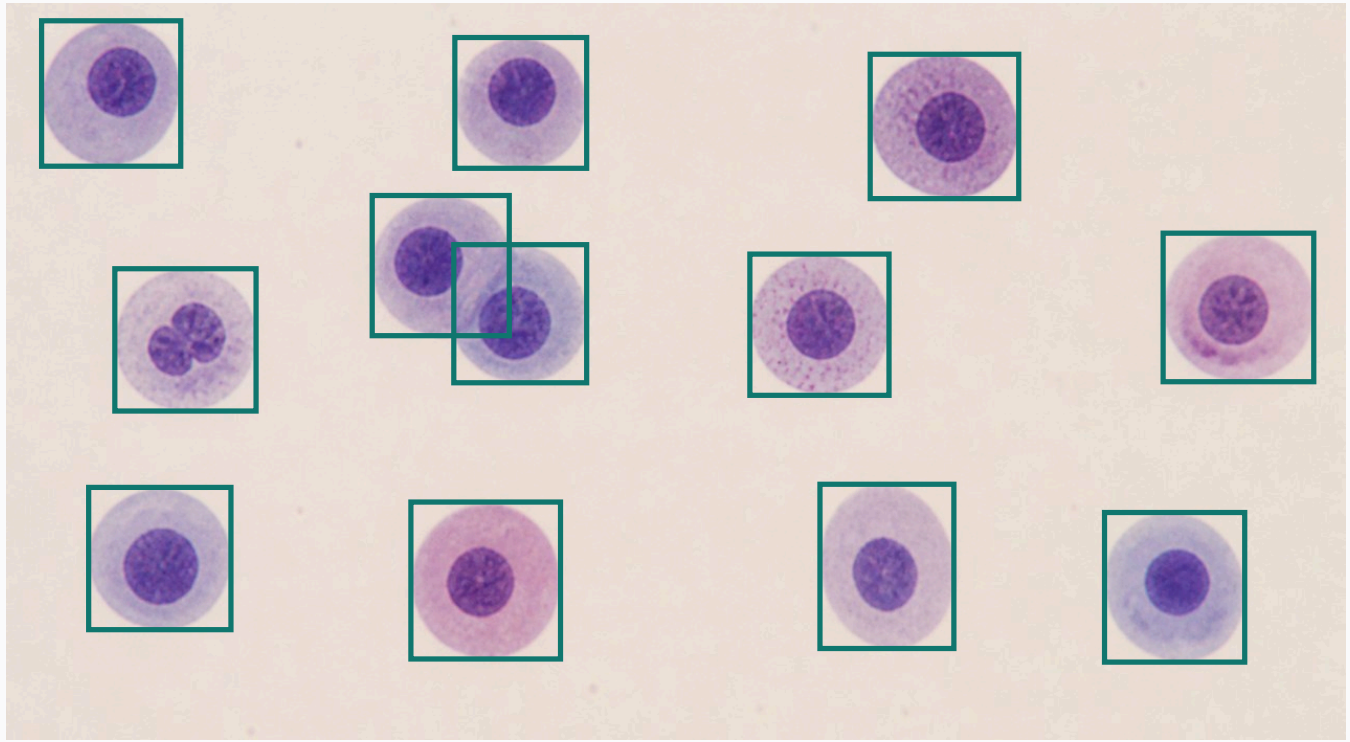
Stained cells under a microscope. A lab wants to know how many there are, and of which kinds.

> Stained cells under a microscope. What would a lab want to know?

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Under a microscope: count the cells



Stained cells under a microscope. A lab wants to know how many there are, and of which kinds.

find	cells, by type
use	count them; crop each one and classify its type

Finding cells starts with labelled examples: the 2018 Data Science Bowl's 841 images carry 37,333 nuclei, each marked by hand (Nature Methods, 2019).

12 cells, each boxed. These teal boxes are ours: COCO has no class "cell", so no detector in this lecture knows one. A cell detector is trained on lists exactly like this.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

In a warehouse: where is the parcel?



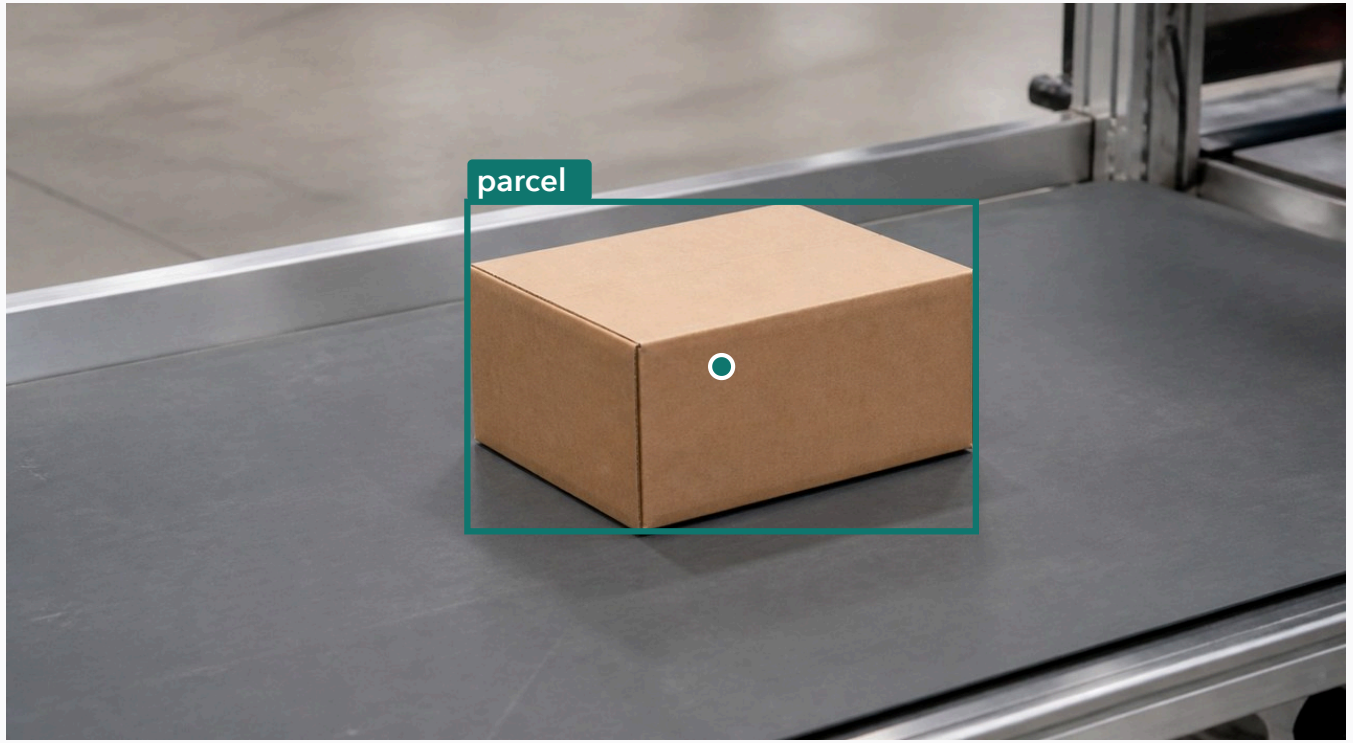
A robot arm waits at the end of this belt. What must it know to pick the parcel up?

A robot arm waits at the end of the belt. What must it know to pick the parcel up?

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

In a warehouse: where is the parcel?



A robot arm waits at the end of this belt. What must it know to pick the parcel up?

find	parcels, sacks, envelopes
use	aim the gripper at the box; count parcels per hour

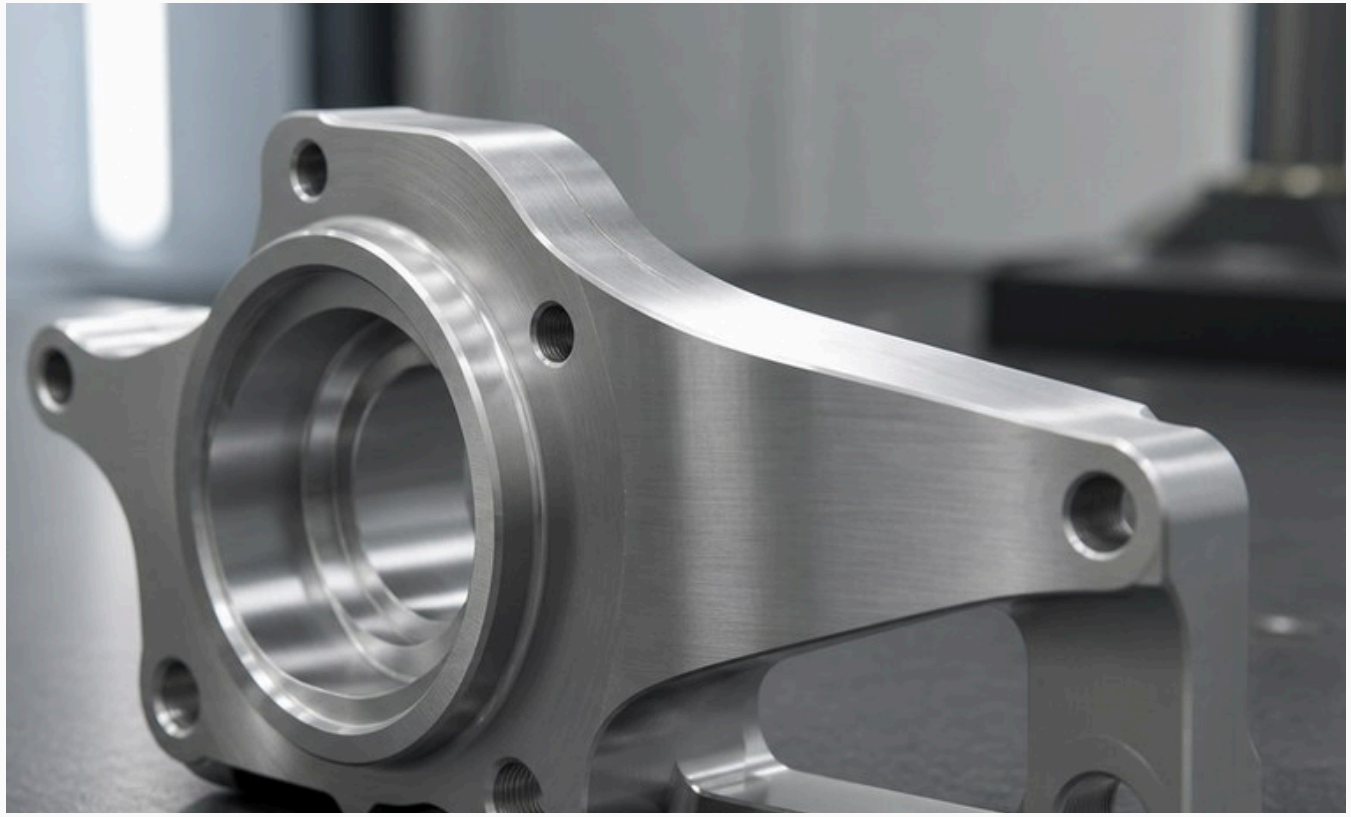
A harder relative: the SKU-110K photos of shop shelves hold about 147 boxed products each, packed edge to edge (Goldman et al., 2019).

> One entry: a parcel, 486 × 315 pixels. The arm aims at the box's centre, the dot; a counter just counts entries.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

On a production line: is anything wrong?



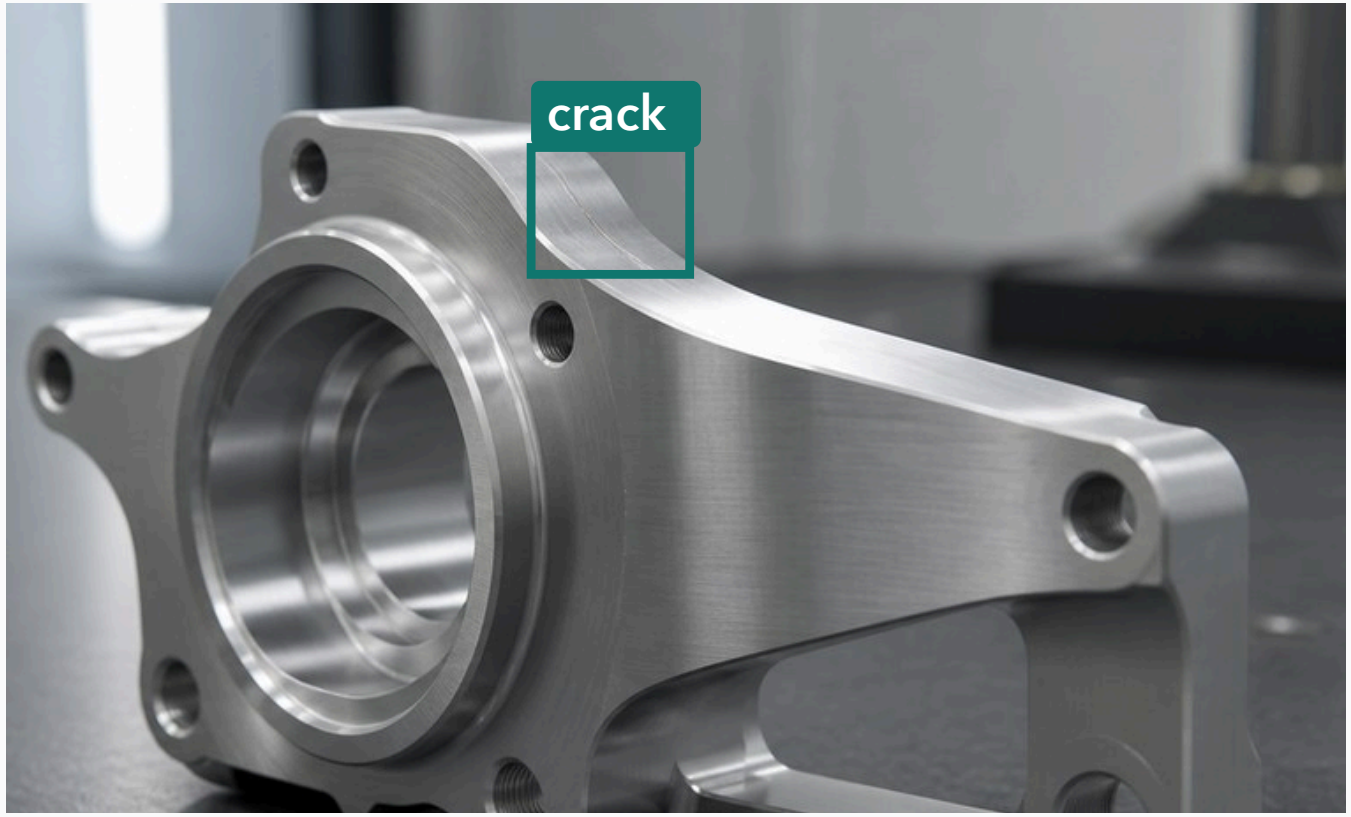
This part has just been machined. Look closely: is anything wrong with it?

> A freshly machined part. Look closely: is anything wrong?

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

On a production line: is anything wrong?



This part has just been machined. Look closely: is anything wrong with it?

find	cracks, scratches, dents, missing holes
use	reject the part; show the inspector where to look

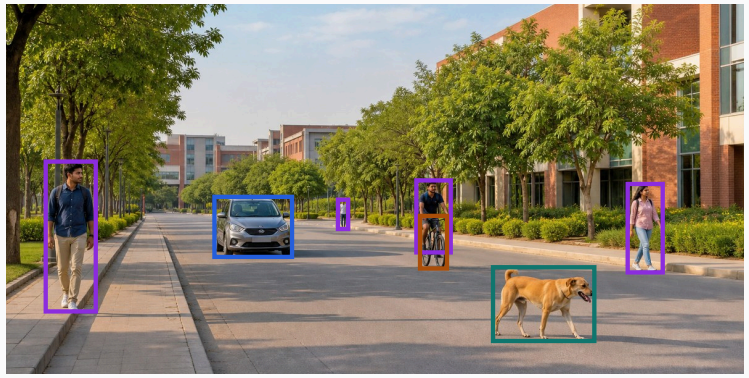
Inspection data sets look like this list: NEU-DET labels six kinds of steel-surface defect, 300 photos of each, every defect boxed.

> A hairline crack on the arm, inside a box 90 × 72 pixels. The box tells the inspector exactly where to look.

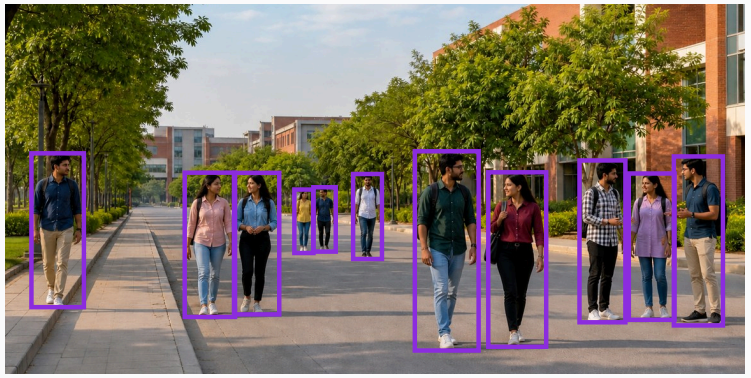
Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

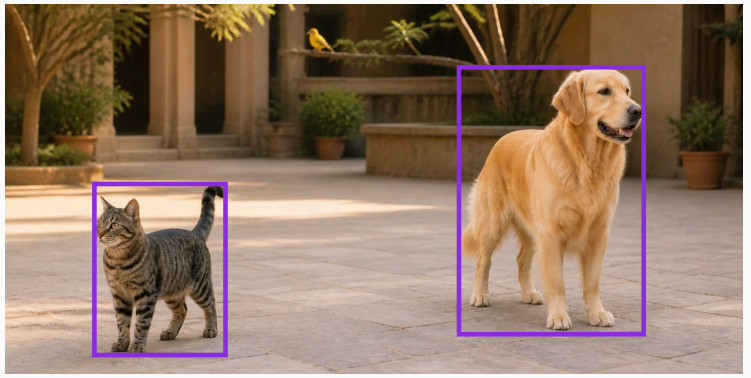
Six jobs, one kind of answer



road · **7** objects



crowd · **11** people



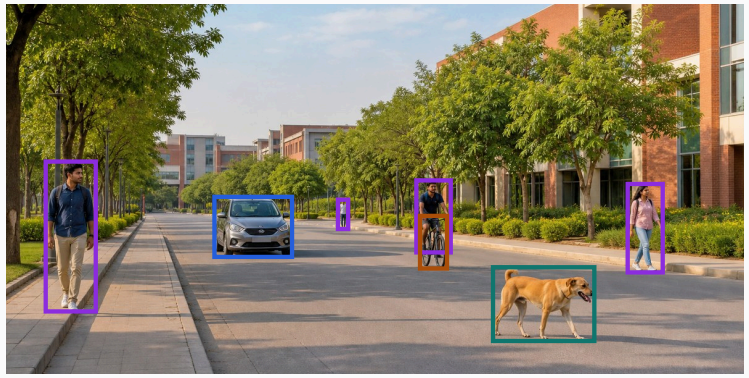
camera trap · **2** animals

Three jobs from everyday photos: 7 objects, 11 people, 2 animals. Purple: a detector’s answer.

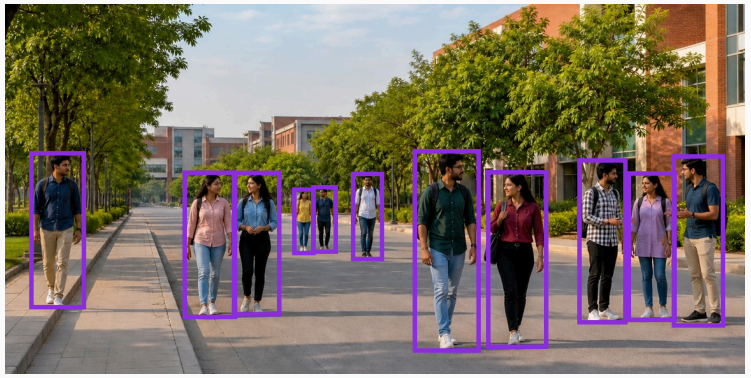
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells, parcel and crack: our annotation.

▶ Notebook, part 1

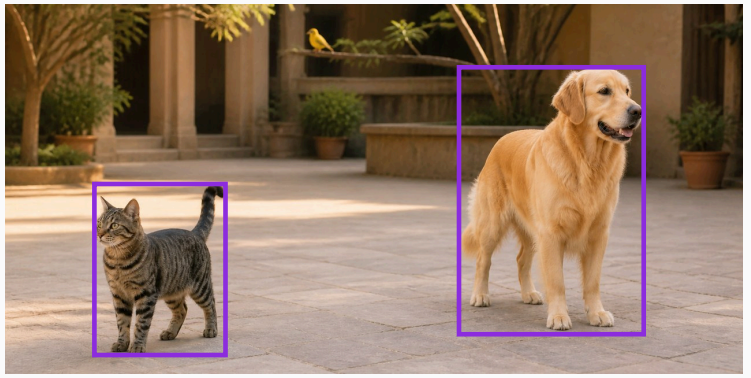
Six jobs, one kind of answer



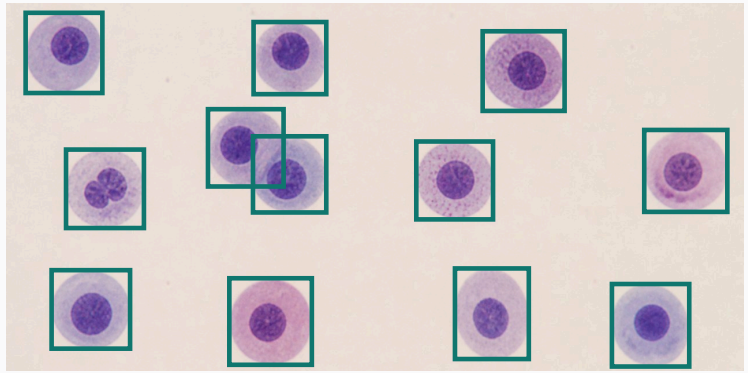
road · **7** objects



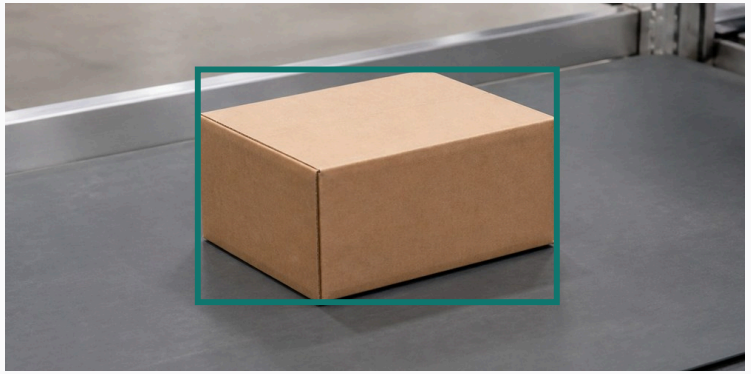
crowd · **11** people



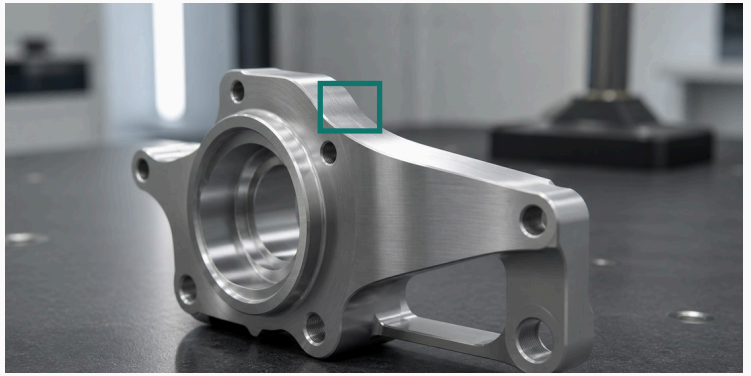
camera trap · **2** animals



microscope · **12** cells



warehouse · **1** parcel



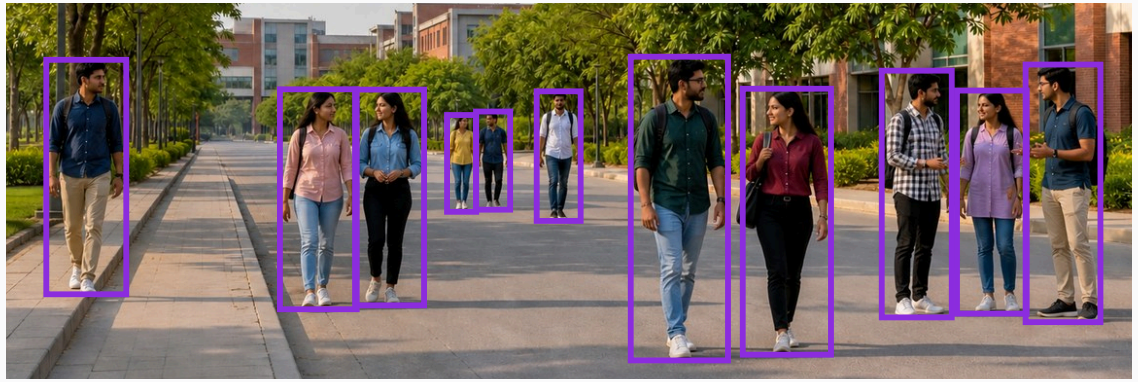
factory · **1** crack

Three more: 12 cells, 1 parcel, 1 crack (teal: our annotation). Every answer has the same shape: a list, one entry per object, each a class and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells, parcel and crack: our annotation.

▶ Notebook, part 1

What the list is used for



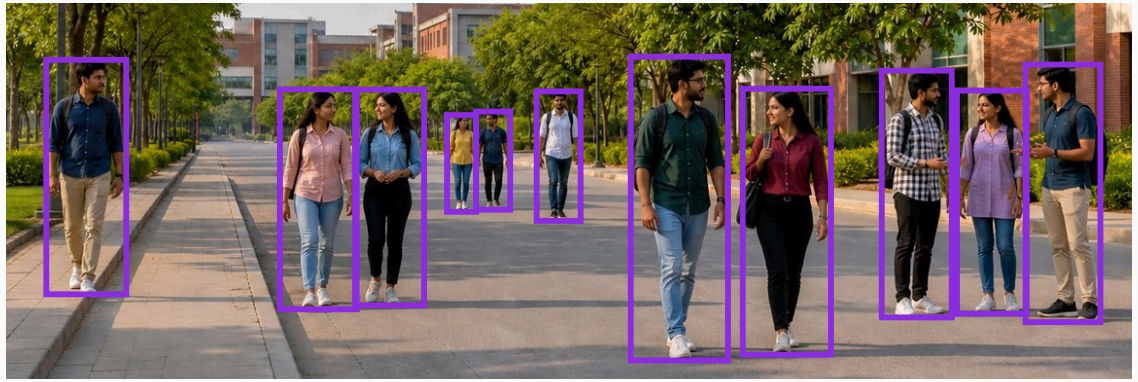
count · 11 people

> Count: the number of entries of a class. Eleven people; twelve cells.

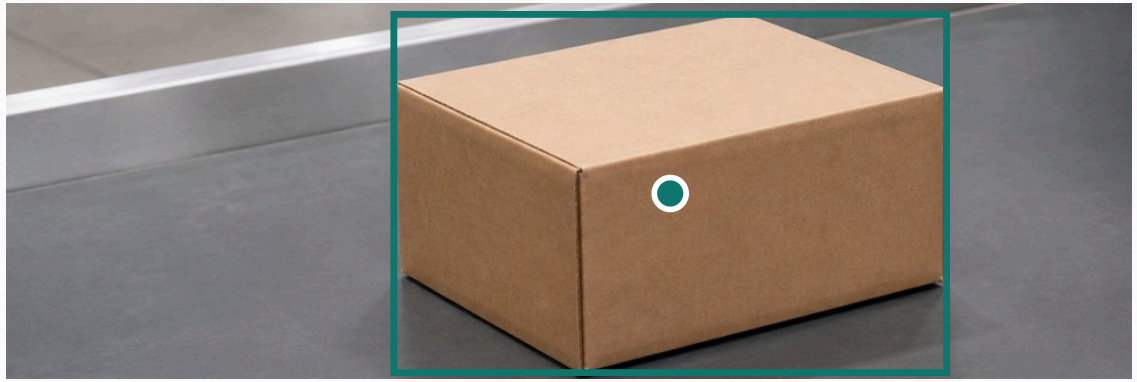
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

What the list is used for



count · 11 people



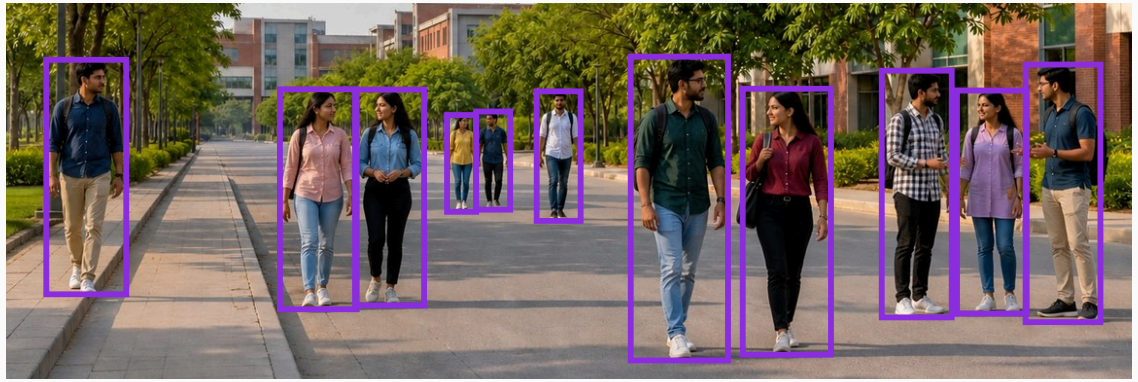
locate · grip at the box's centre

> Locate: the box says where to act. The arm grips at the parcel's centre; a car brakes for the person in its lane.

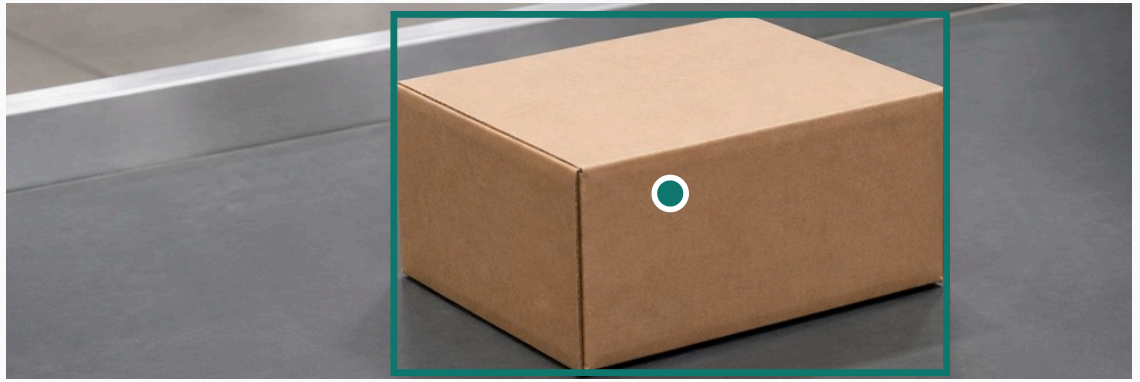
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

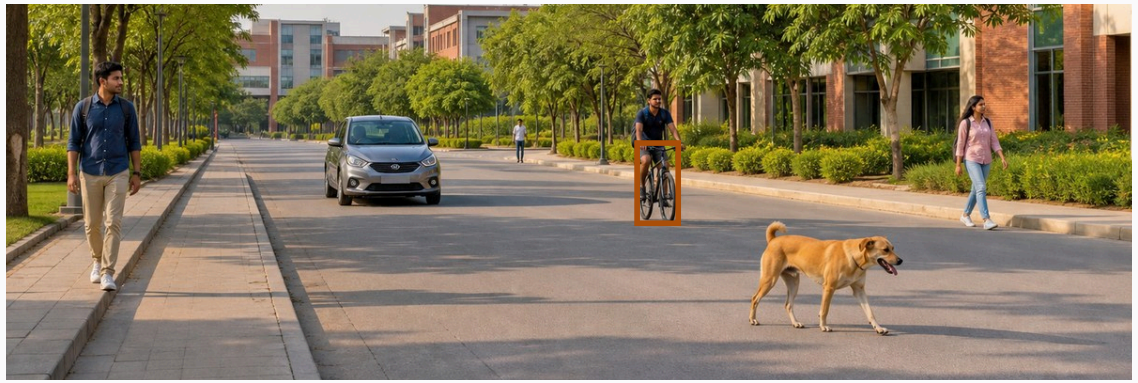
What the list is used for



count · 11 people



locate · grip at the box's centre



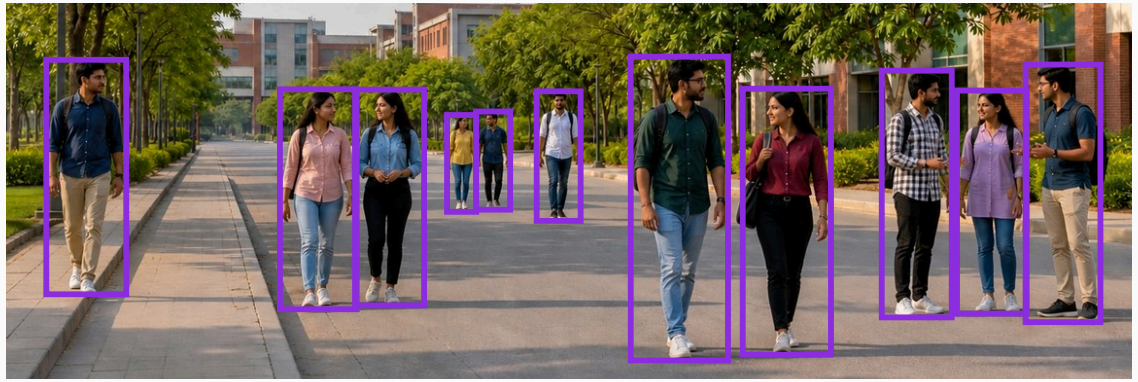
track · link each box to the next frame's

> Track: in a video, run the detector on every frame and link each box to the box that overlaps it most in the next frame.

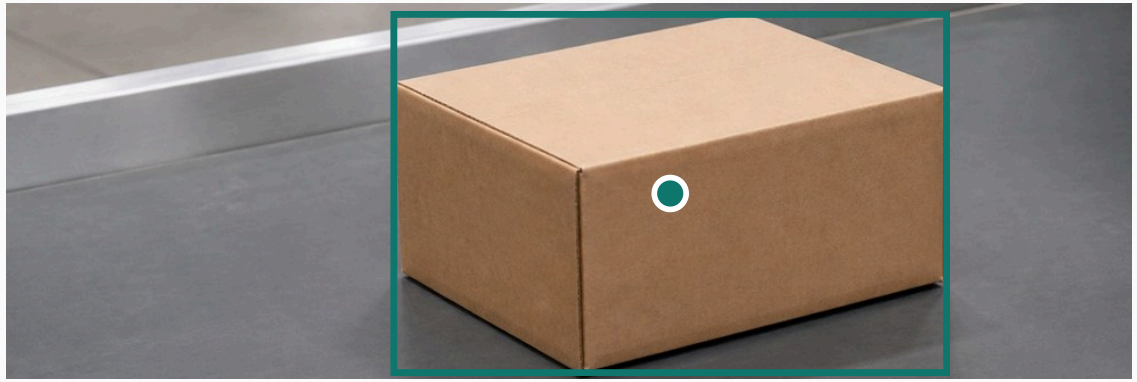
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

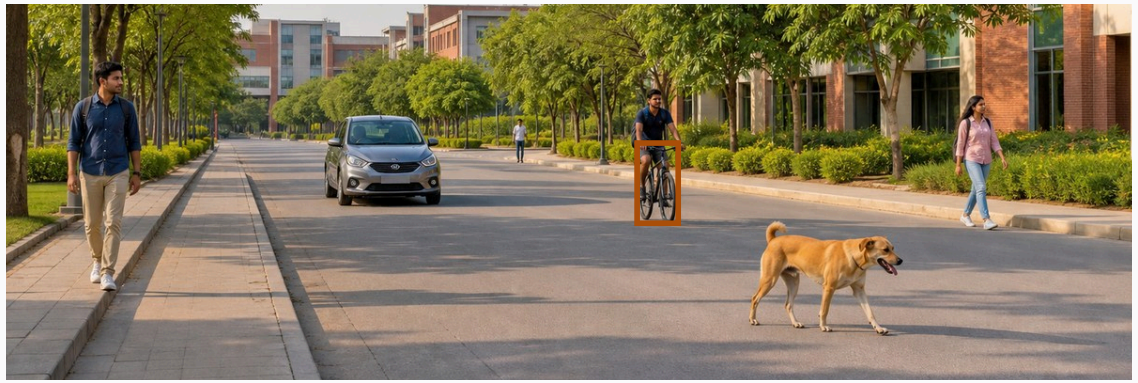
What the list is used for



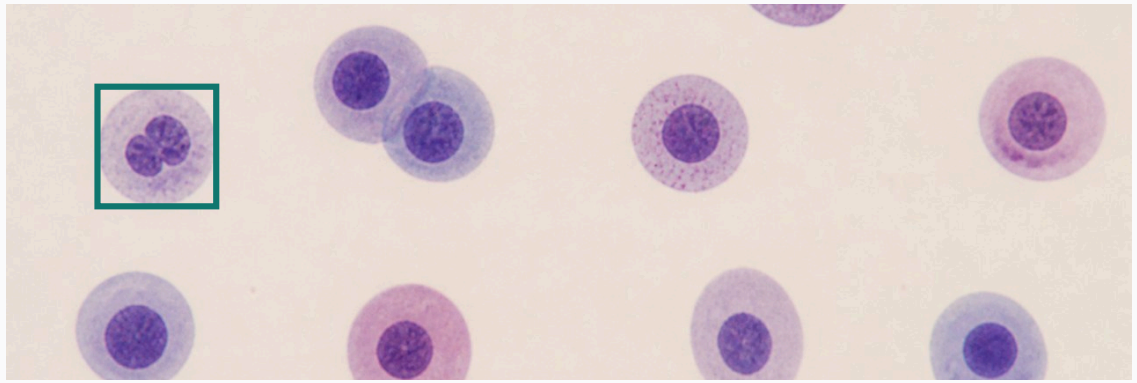
count · 11 people



locate · grip at the box's centre



track · link each box to the next frame's



crop · then name the cell's type

> Crop: cut out each box and pass it to a second model: read a number plate, identify a face, name a cell's type.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

Hard, 1: a bus and a distant person in one photo



The bus is 502 pixels wide.

> One photo, a bus 502 pixels wide.

Boxes: our annotation (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Hard, 1: a bus and a distant person in one photo



The bus is 502 pixels wide.

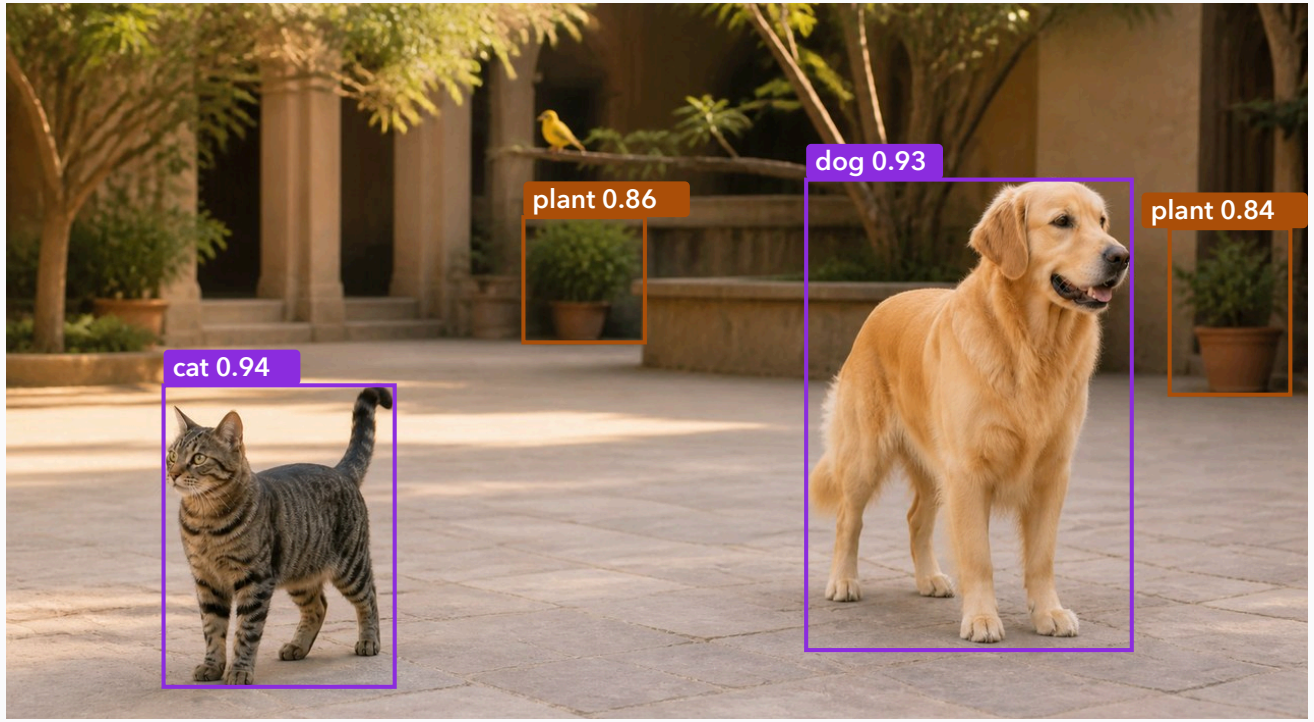
The far pedestrian: 13 pixels, 39 times narrower. One detector must box both.

> And a pedestrian 13 pixels wide, far down the road: 39 times narrower. A classifier sees one main thing; a detector must handle both sizes at once.

Boxes: our annotation (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



YOLOv8n's four detections: cat, dog, two plants.

> Back to the camera trap. YOLOv8n reports a cat, a dog and two plants. Where is the bird?

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



The bird is 50 × 43 pixels. One cell did draw a box on it, IoU **0.913** with ours, but scored it **0.098**.

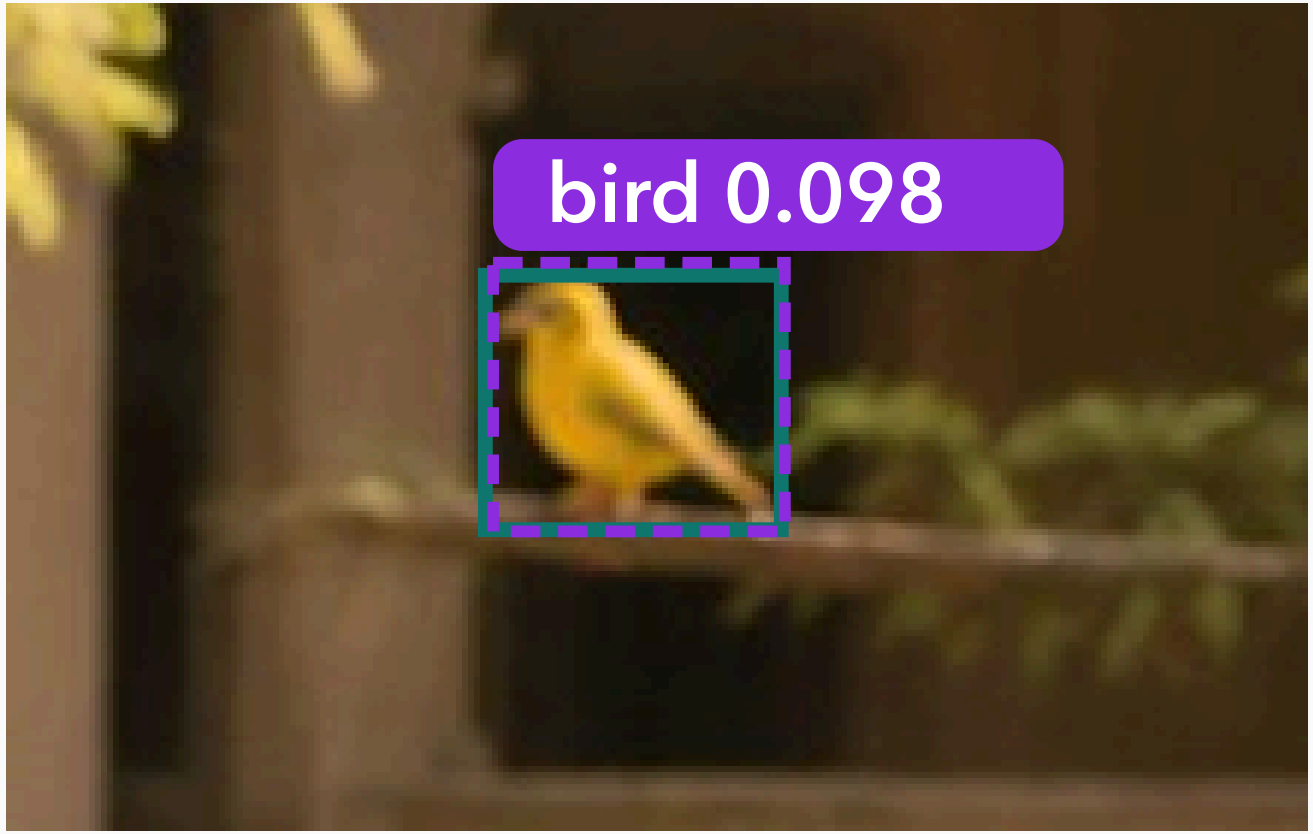
Below the usual cut-off of 0.25, so the box is thrown away.

> Zoom in. The bird is 50 × 43 pixels. One cell boxed it almost perfectly, IoU 0.91, but gave “bird” a score of only 0.098, so it was dropped.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



The bird is 50 × 43 pixels. One cell did draw a box on it, IoU **0.913** with ours, but scored it **0.098**.

Below the usual cut-off of 0.25, so the box is thrown away.

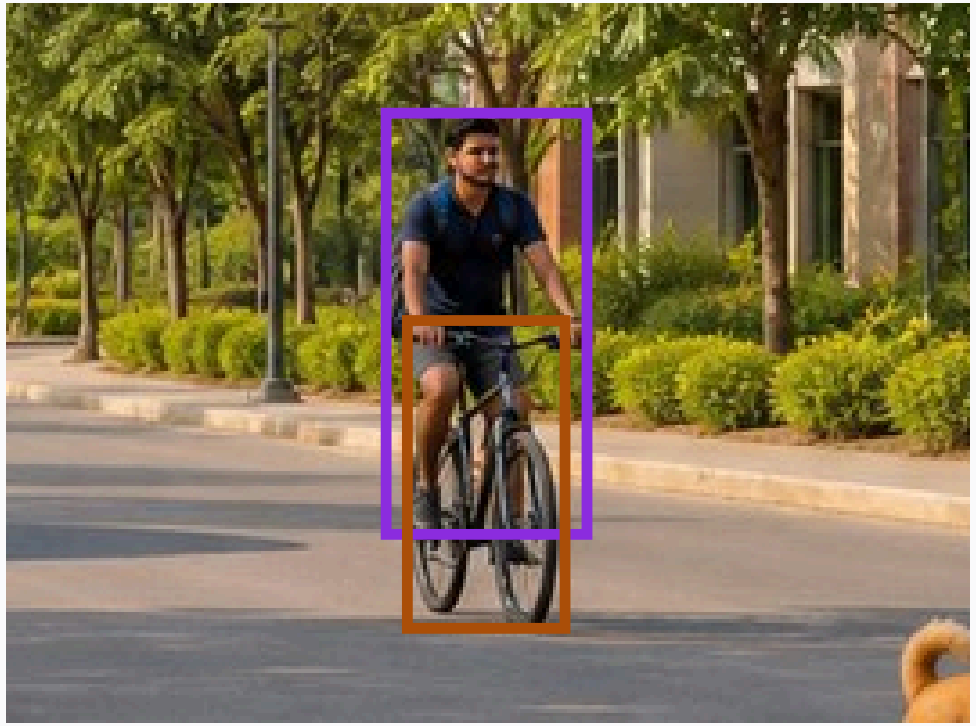
Its next guess for that cell: banana, 0.059. A small yellow thing.

> That cell's second guess was "banana" at 0.059. Few pixels, little detail: small objects get low, unsure scores.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 3: objects overlap



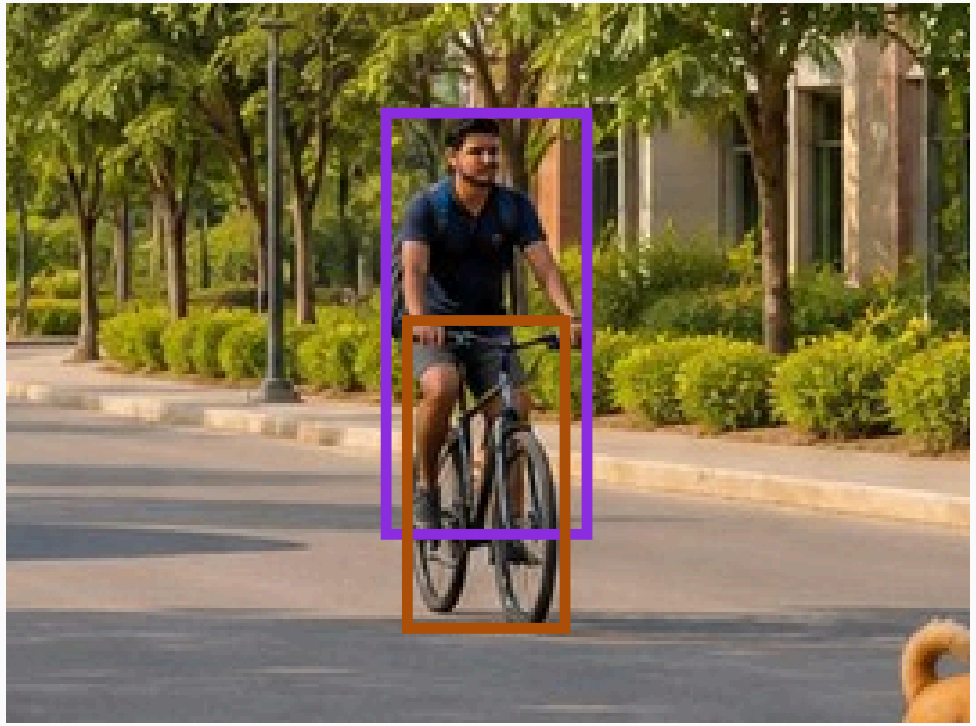
a cyclist: a person and a bicycle

> A cyclist is two objects in one place: a person box and a bicycle box, one largely on top of the other. Both are correct.

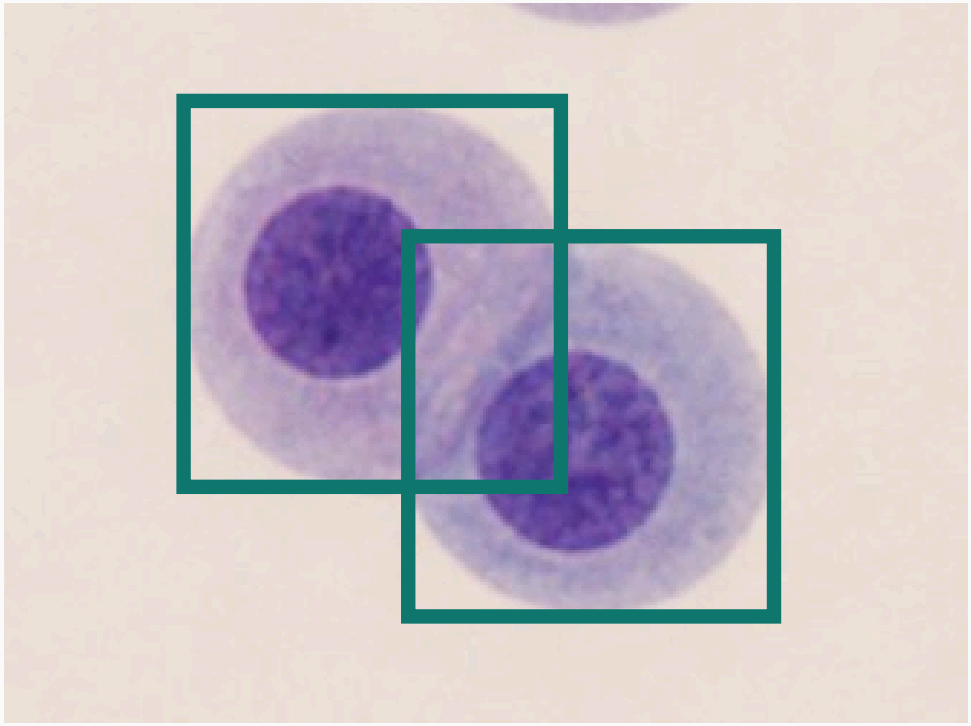
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells: our annotation.

▶ Notebook, part 1

Hard, 3: objects overlap



a cyclist: a person and a bicycle



two touching cells

> Two touching cells: their boxes overlap too. A detector must keep both entries, and yet must not report one object twice.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells: our annotation.

Part 1 in one slide

1. Detection answers **what is where**: a list of (class, box, score), one entry per object.
2. The same answer serves roads, crowds, camera traps, microscopes, warehouses and factories.
3. The list is used to **count, locate, track** and **crop** for a second look.
4. Hard: sizes vary about 40×, small objects get unsure scores, objects overlap.

Next: why an ordinary classifier cannot give this list, and what a network would have to output instead.

> Part 1: what detection is for. Part 2: why a network finds a list hard to produce.

▶ Notebook, part 2

Part 2 · From one label to a list

A classifier answers with one word. We need a list. Why is a list hard for a network?

1. What a classifier says about one photo, and what we want instead.
2. Two awkward facts about the list: its length changes, and its order means nothing.
3. The route through the rest of the lecture.

> From one label to a list of (class, box) entries.

▶ Notebook, part 2

What is in this photo?



> Look at the photo. In one sentence, what is in it?

ResNet-50 with torchvision's IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What is in this photo?



ImageNet class	probability
golden retriever	0.192
flat-coated retriever	0.015
tiger cat	0.013
Labrador retriever	0.012
tabby	0.010

A classifier answers with one label for the whole photo. ResNet-50, trained on ImageNet: golden retriever, 0.19. Every row is a guess about the same single label.

ResNet-50 with torchvision's IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What is in this photo?



ImageNet class	probability
golden retriever	0.192
flat-coated retriever	0.015
tiger cat	0.013
Labrador retriever	0.012
tabby	0.010

One label for the whole photo. Its 118 dog classes together hold 0.39; the five cat classes, 0.03.

> The cat appears only as a weak runner-up, tiger cat at 0.013. The answer cannot say “a dog and a cat”, how many, or where.

ResNet-50 with torchvision’s IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What we want: a list, one entry per object



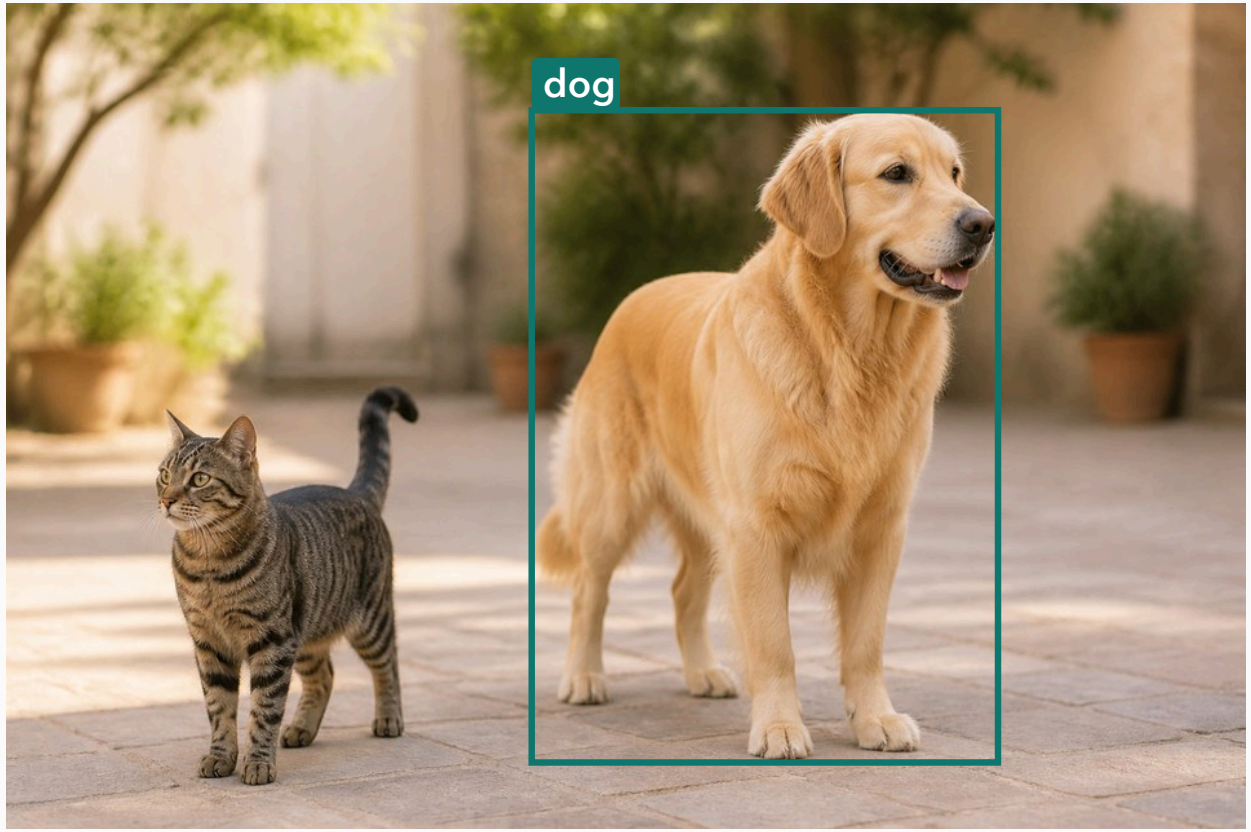
class	box: left, top, right, bottom (pixels)

> Detection answers with a list. Each entry is one object: a class, and a box around it.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

What we want: a list, one entry per object



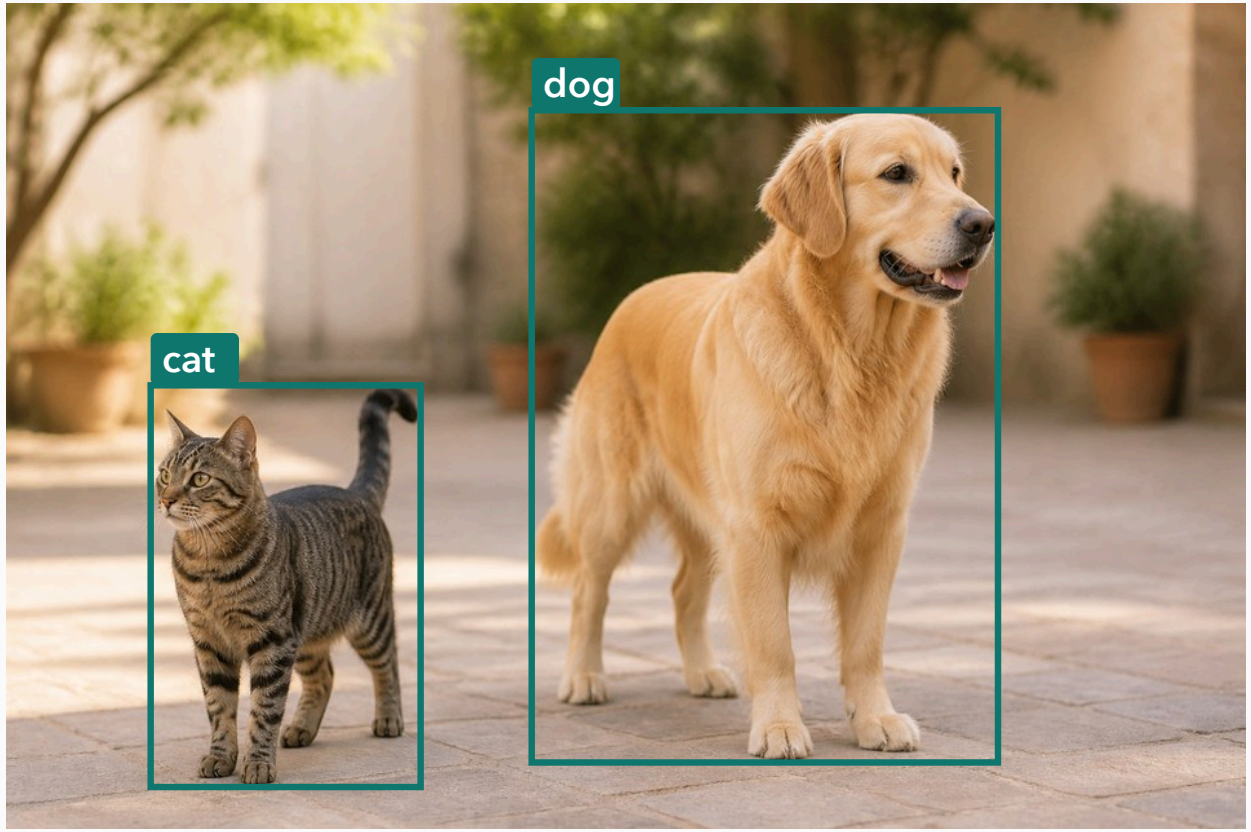
class	box: left, top, right, bottom (pixels)
dog	(407, 83, 768, 588)

First entry: dog. Its box has left edge 407, top 83, right 768 and bottom 588, in pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

What we want: a list, one entry per object



class	box: left, top, right, bottom (pixels)
dog	(407, 83, 768, 588)
cat	(112, 296, 321, 607)

The boxes are our annotation of this photo, in pixels from its top-left corner.

> Second entry: cat. Two objects, two entries. A photo with five objects gets five.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

Four tasks, four kinds of answer



Classification: one label

> Classification: one label for the whole photo.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box

> Localisation: one object, its label and one box.

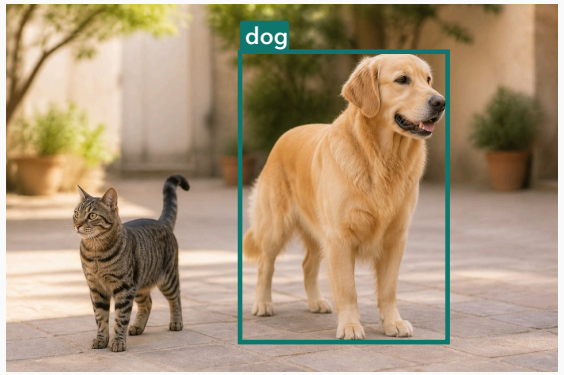
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

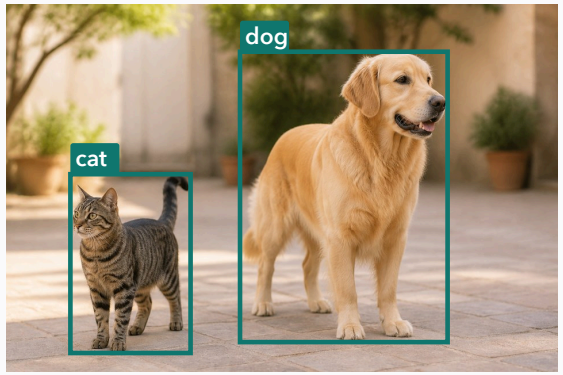
Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box



Detection: a class and a box per object

> Detection: every object, each with a class and a box. Today's task.

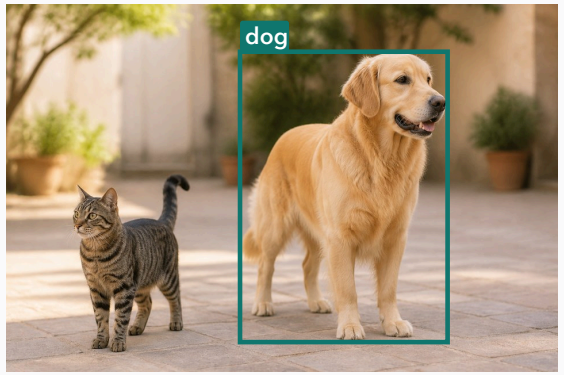
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

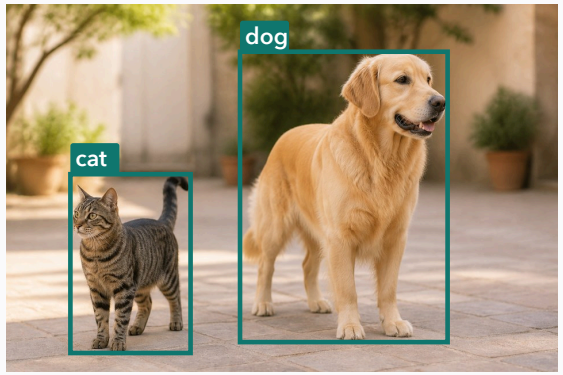
Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box



Detection: a class and a box per object



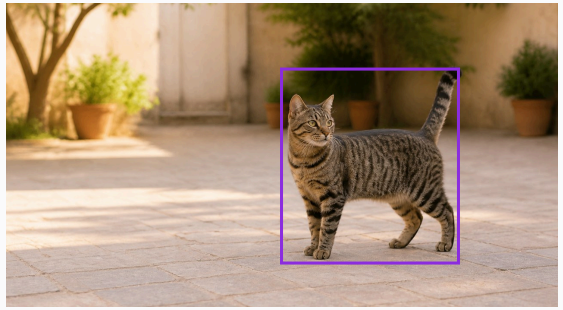
Segmentation: which pixels (Lecture 23)

> Segmentation: which pixels belong to each object. The masks here are from a real YOLOv8n-seg run; this is next lecture.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

The number of objects changes from photo to photo



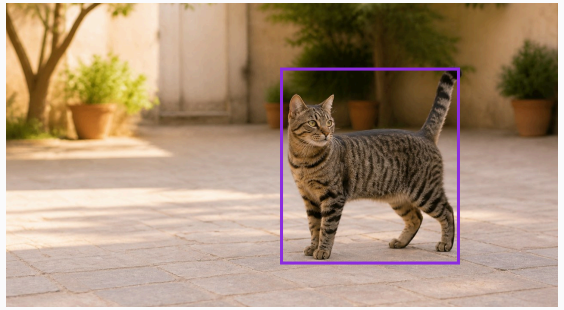
1 · one cat

> One cat: a list of one.

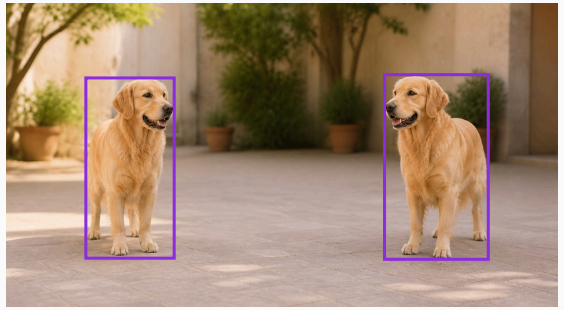
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

The number of objects changes from photo to photo



1 · one cat



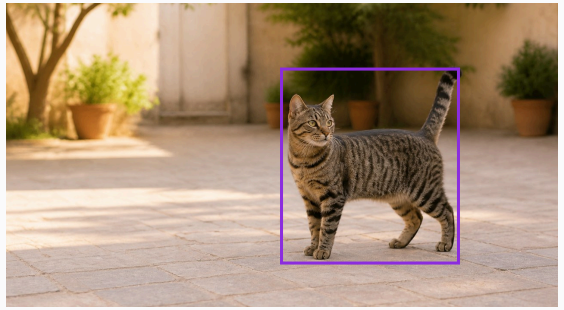
2 · two dogs

> Two dogs: a list of two.

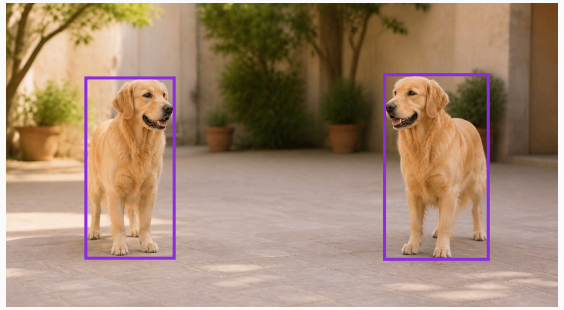
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

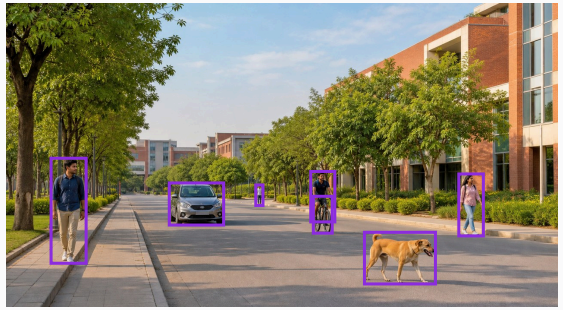
The number of objects changes from photo to photo



1 · one cat



2 · two dogs



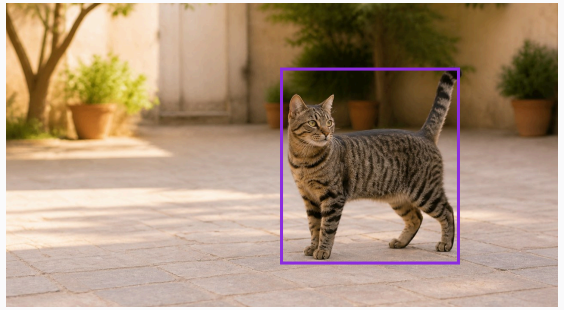
7 · a street

> A street: seven.

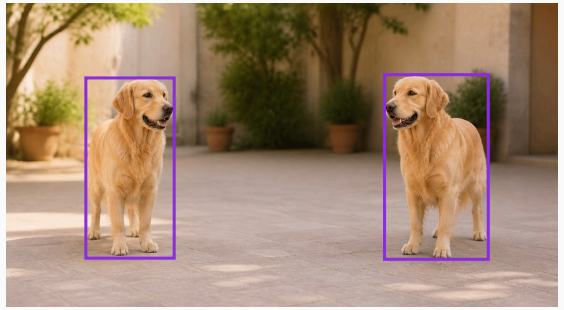
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

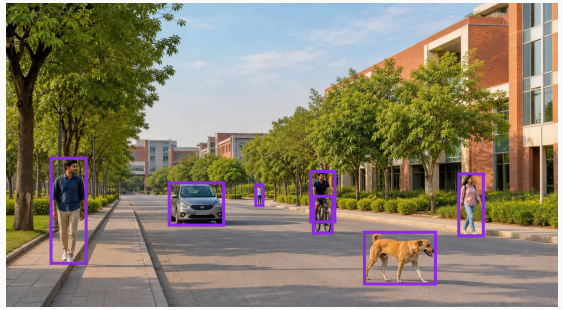
The number of objects changes from photo to photo



1 · one cat



2 · two dogs



7 · a street



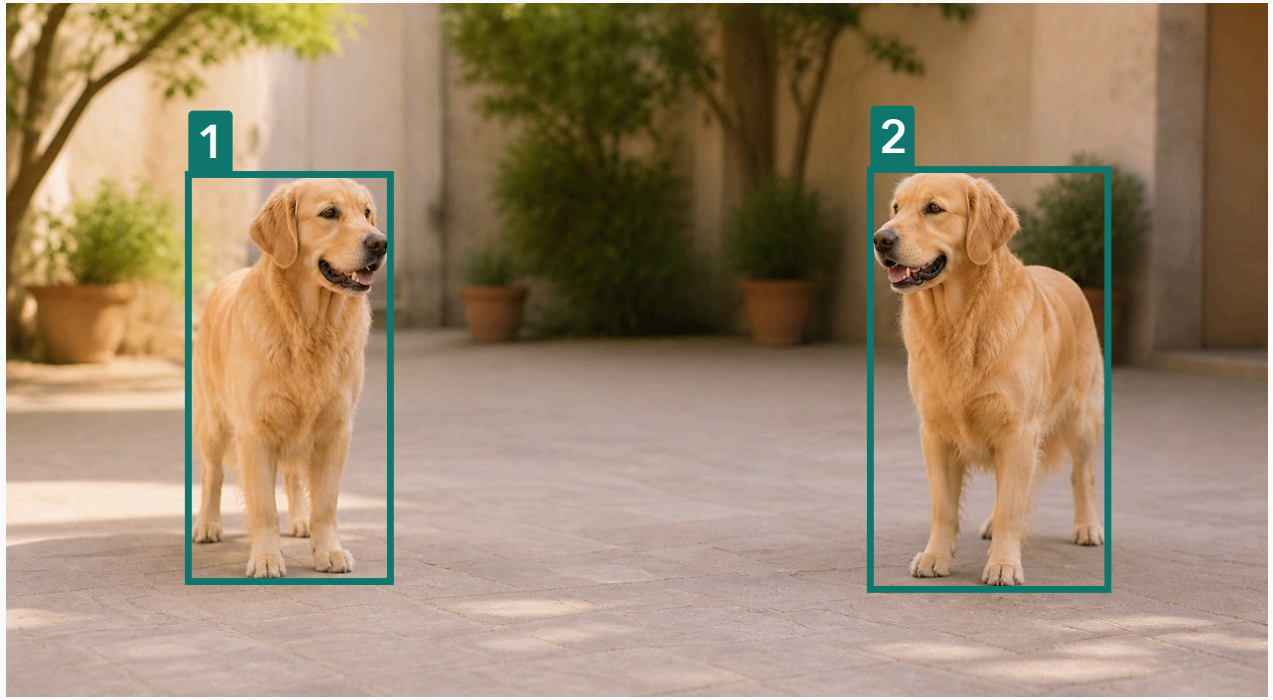
11 · a path of people

A busy path: eleven people. The length of the answer depends on the photo.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

A list with no order: a set



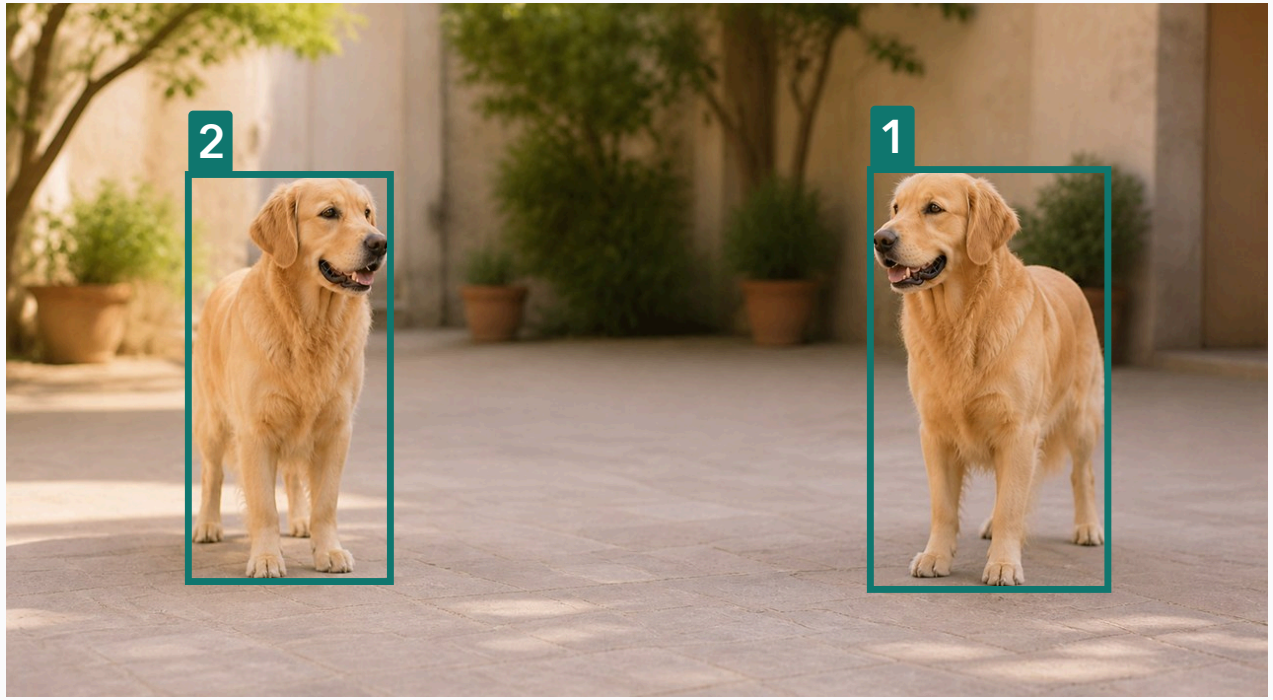
$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

> Number the two dogs: left one first.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

A list with no order: a set



$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

Number the dogs either way: it is the same answer. Only each class stays paired with its own box.

> Or right one first. Both lists describe the same scene: the answer is a set of N (class, box) pairs, N varies, and order means nothing.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



A classifier's last layer always has the same number of outputs: 1,000 for ImageNet, for every photo.

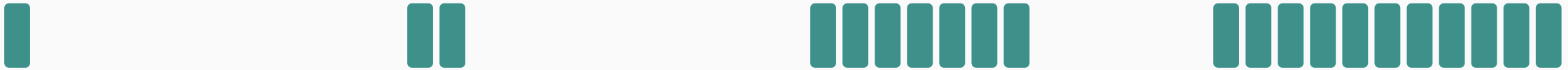
▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

We need a list whose length changes with the photo, and whose order means nothing.

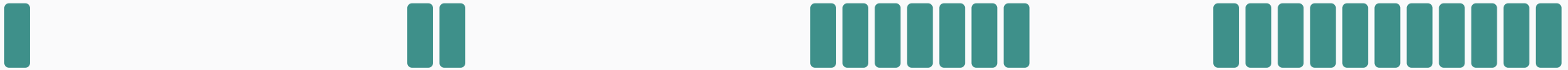
▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

Today's question: **how can a fixed-size output describe a set of any size, in no order?**

> That is the question of the lecture.

▶ Notebook, part 2

Today's route: build a detector step by step

3. **Boxes and IoU**: four numbers for a box; a score for how well two boxes agree
4. **One box, then many**: a network that outputs one box works; several boxes break the loss

> First the tools (Part 3), then a network that outputs boxes, to see exactly where a fixed output breaks (Part 4).

▶ Notebook, part 2

Today's route: build a detector step by step

3. **Boxes and IoU**: four numbers for a box; a score for how well two boxes agree
4. **One box, then many**: a network that outputs one box works; several boxes break the loss
5. **A candidate at every place**: every cell of a grid writes a box and class scores; train the cells near each object
6. **NMS**: keep one box per object
7. **Three grids**: small and large objects
8. **Measuring**: precision, recall and AP

Appendix A: the other answer, DETR, a fixed set of slots matched to objects, with no NMS.

> Then the answer used by YOLO and most fast detectors: a candidate at every place, cleaned up by NMS, at three sizes; and how to measure the result.

▶ Notebook, part 3

Part 3 · Describe a box, compare two boxes

Every later step needs two small tools.

1. Write a box as four numbers, in two common formats, and see how a labelled photo is stored.
2. Measure how well two boxes agree: intersection over union, IoU, computed by hand.
3. Try IoU live, then write it in six lines of PyTorch.

> Two tools: four numbers for a box, and IoU to compare two boxes.

▶ Notebook, part 3

Positions in a photo: pixels from the top-left corner



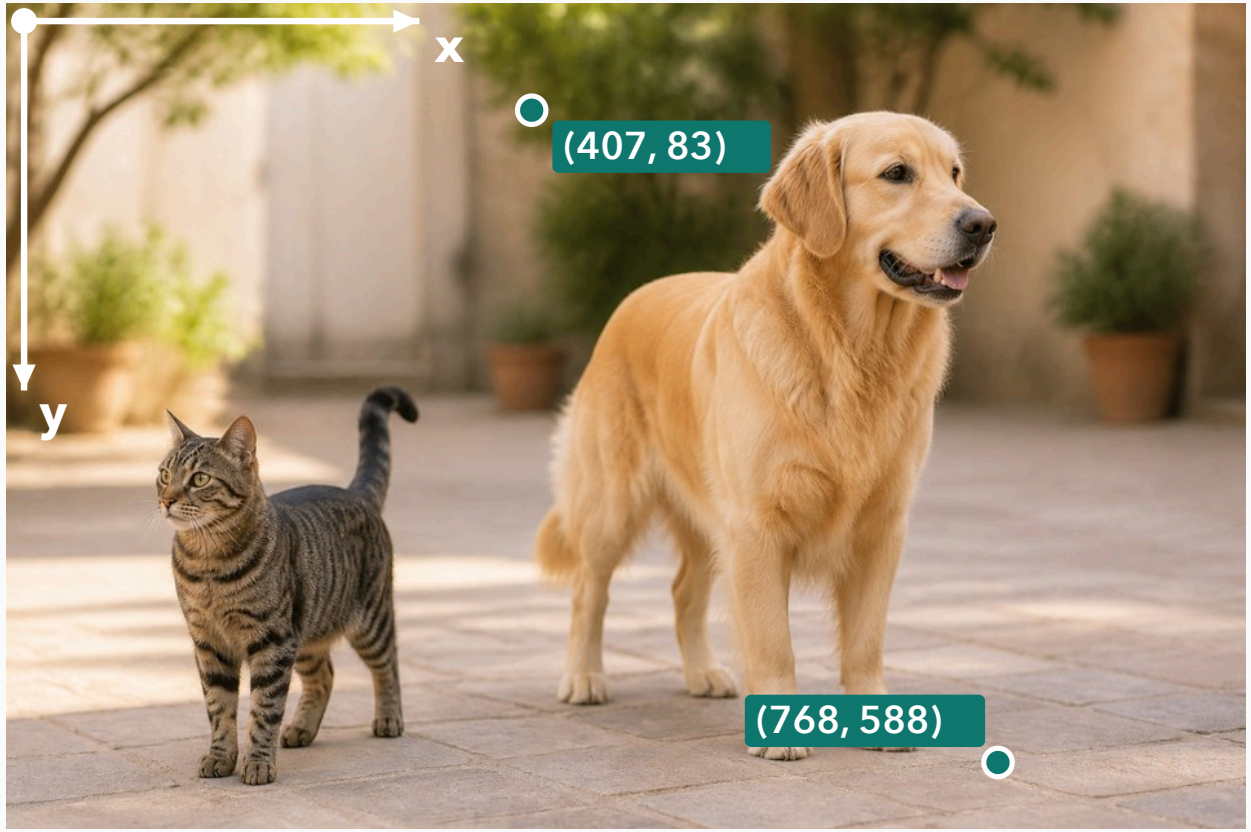
The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

> Every position is a pair of pixel counts from the top-left corner: x to the right, y downwards. This photo is 960 × 640.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Positions in a photo: pixels from the top-left corner



The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

Two corners of the dog: top-left (407, 83), bottom-right (768, 588).

> The dog's top-left corner is at (407, 83); its bottom-right corner at (768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

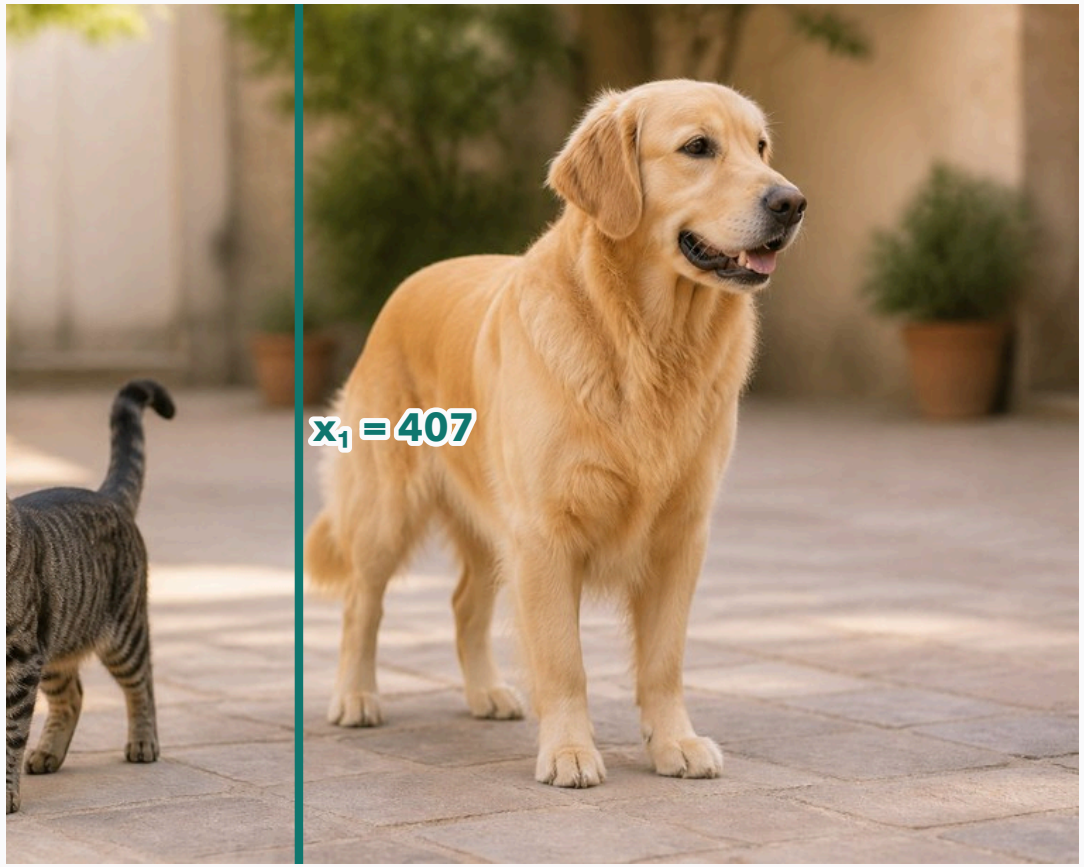
$$(x_1,\ y_1,\ x_2,\ y_2)$$

A box is the region between two vertical and two horizontal lines. Draw them one at a time.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

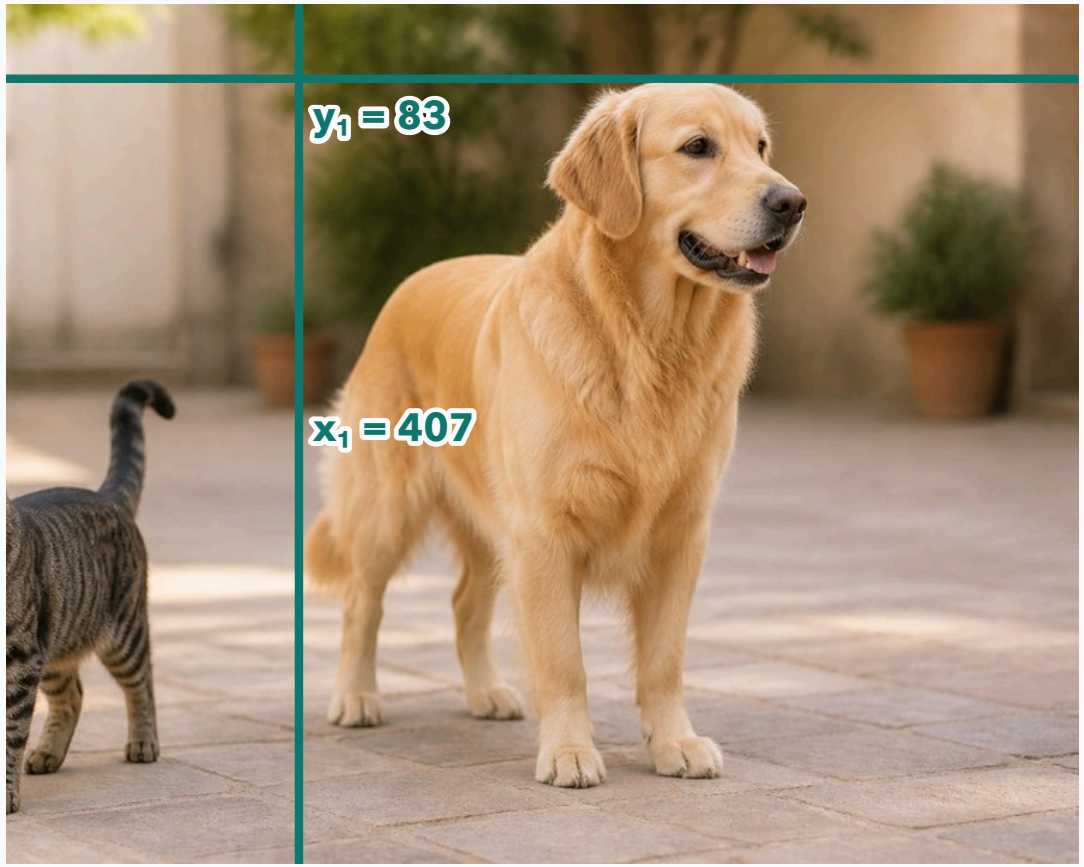
$$(x_1,\ y_1,\ x_2,\ y_2)$$

> Left edge: every point of the dog has x at least 407.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

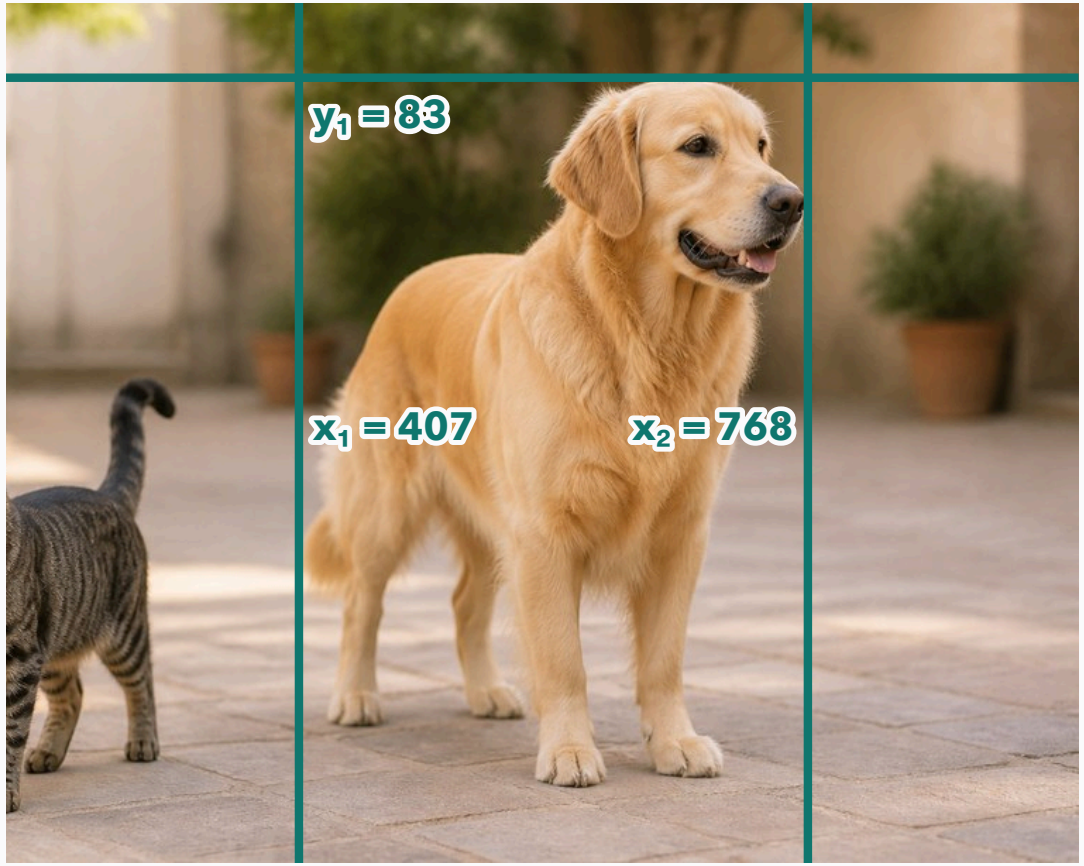
$$(x_1,\ y_1,\ x_2,\ y_2)$$

> Top edge: y at least 83, the tips of the ears.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

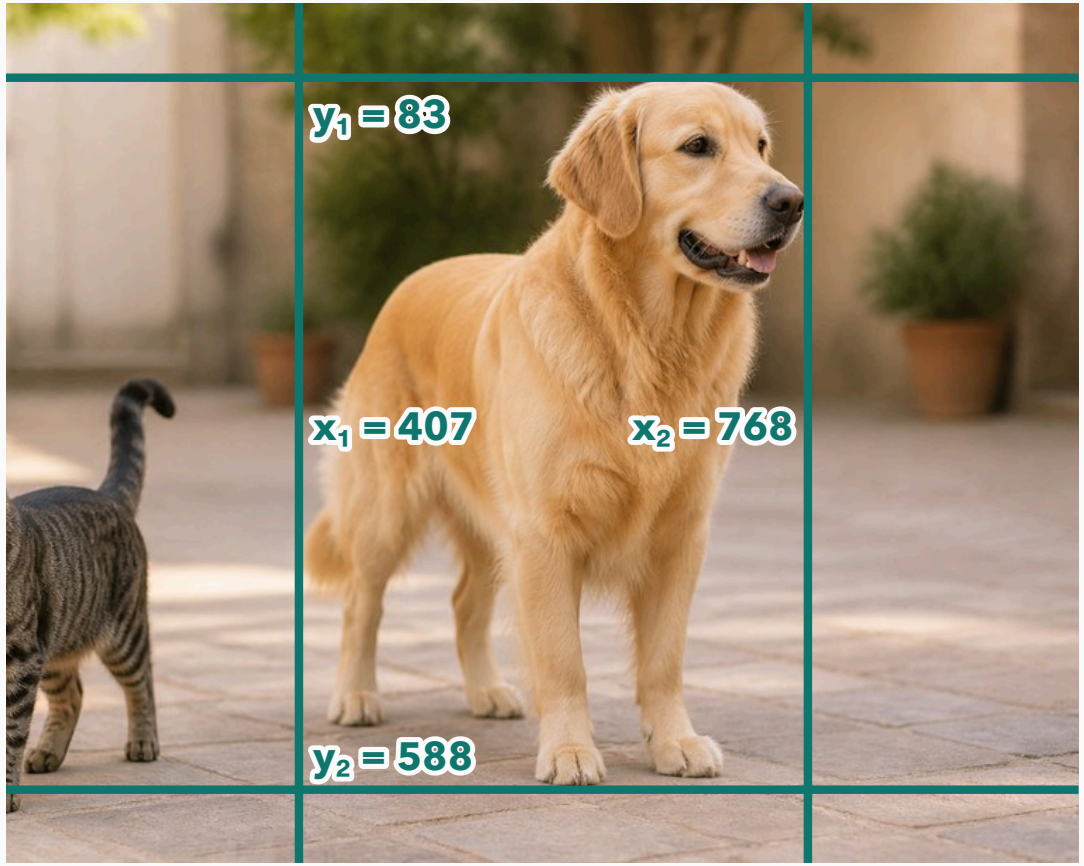
$$(x_1,\ y_1,\ x_2,\ y_2)$$

Right edge: x at most 768, the nose.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

$$(x_1,\ y_1,\ x_2,\ y_2)$$

$$= (407,\ 83,\ 768,\ 588)$$

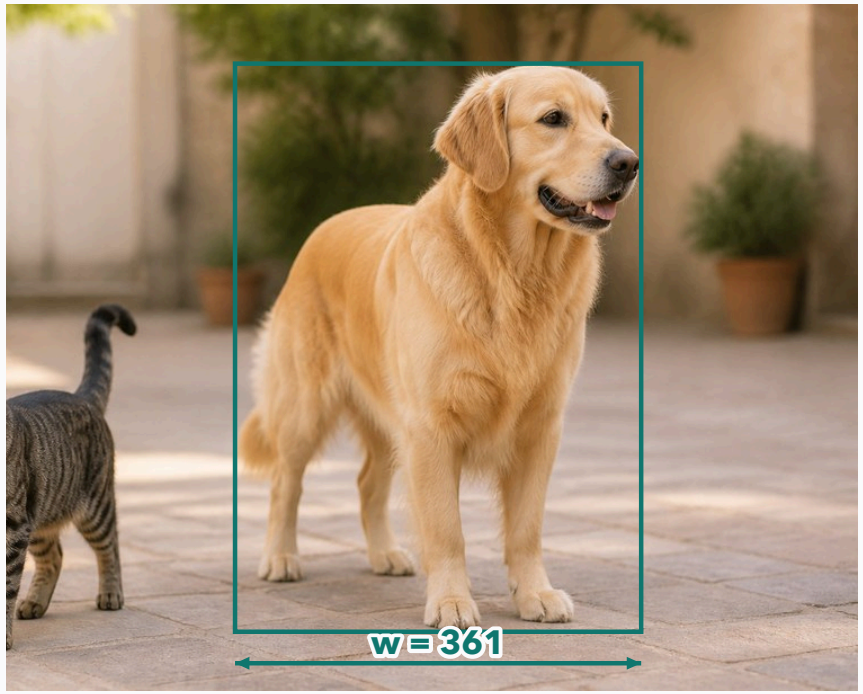
(x_1, y_1) is the top-left corner, (x_2, y_2) the bottom-right. torchvision uses this format.

> Bottom edge: y at most 588, the paws. Four numbers: (407, 83, 768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The box's size: width and height



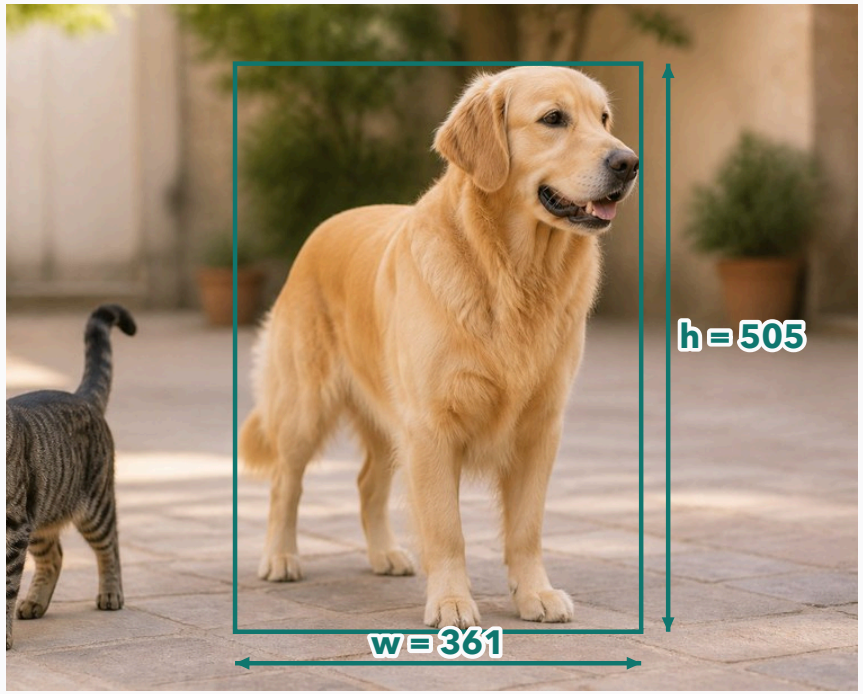
$$w = x_2 - x_1 = 768 - 407 = 361$$

Width: the right edge minus the left edge, 768 − 407 = 361 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The box's size: width and height



$$w = x_2 - x_1 = 768 - 407 = 361$$

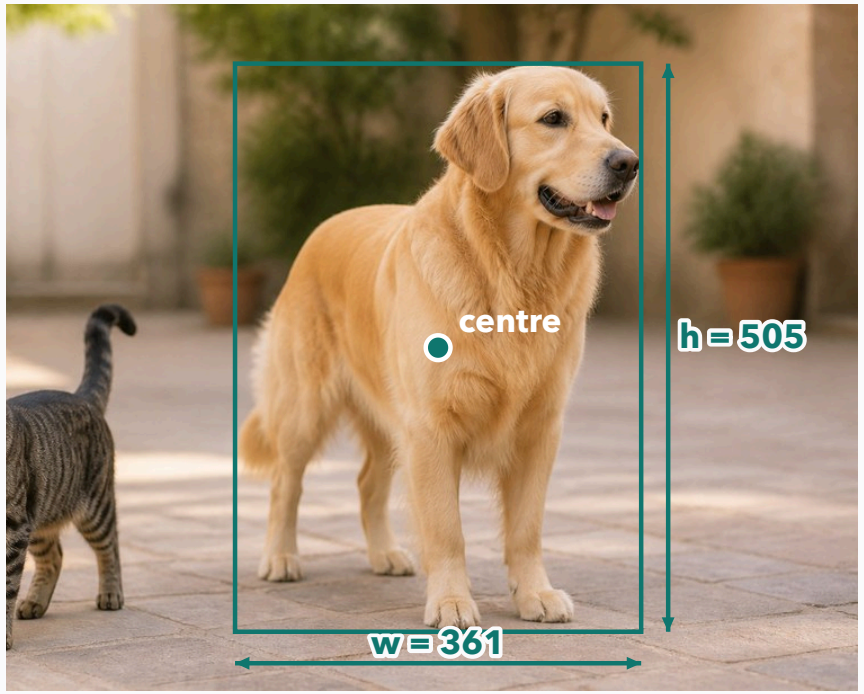
$$h = y_2 - y_1 = 588 - 83 = 505$$

> Height: the bottom minus the top, 588 − 83 = 505. Bottom minus top, because y grows downwards.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same box as a centre and a size



$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

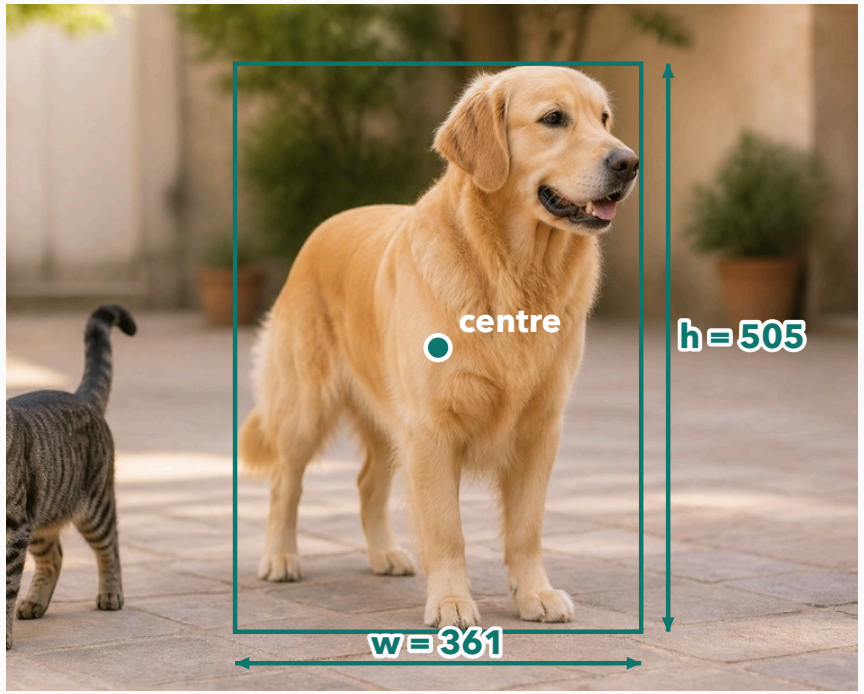
$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

> The centre is halfway across and halfway down: start at the left edge and add half the width; start at the top and add half the height.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same box as a centre and a size



$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

$$(c_x, c_y, w, h) = (587.5,\ 335.5,\ 361,\ 505)$$

> Same box, second format: centre (587.5, 335.5), size 361 × 505. Either format can be computed from the other.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

> Divide horizontal numbers by the photo's width, vertical ones by its height.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

$$= (0.612, 0.524, 0.376, 0.789)$$

> The dog: (0.612, 0.524, 0.376, 0.789). Every number is between 0 and 1.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

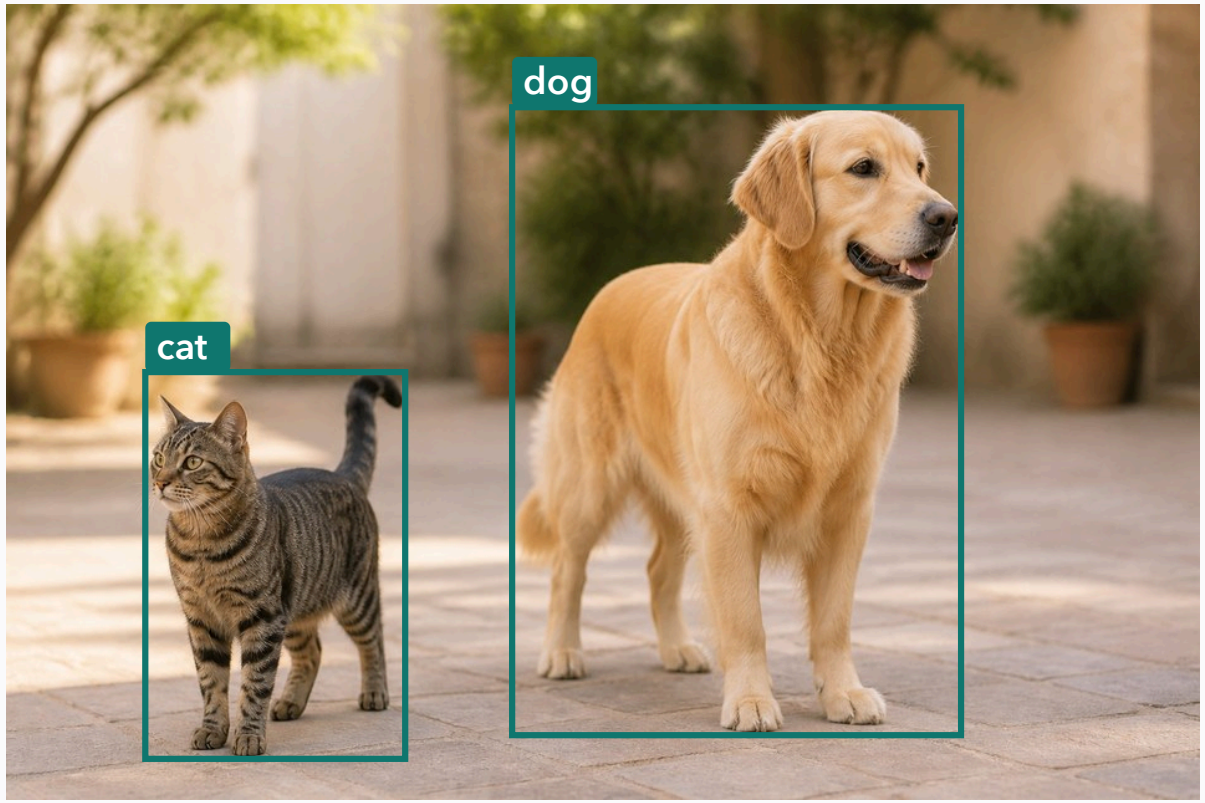
$$= (0.612, 0.524, 0.376, 0.789)$$

Halve the photo to 480 × 320: every pixel number halves, the normalised numbers do not change. The dog's centre is 61% of the way across and 52% of the way down, at any resolution.

> These numbers survive resizing, which is why label files and many networks use them.

▶ Notebook, part 3

A labelled photo on disk: one line per object



`cat-dog.txt`, next to `cat-dog.jpg`:

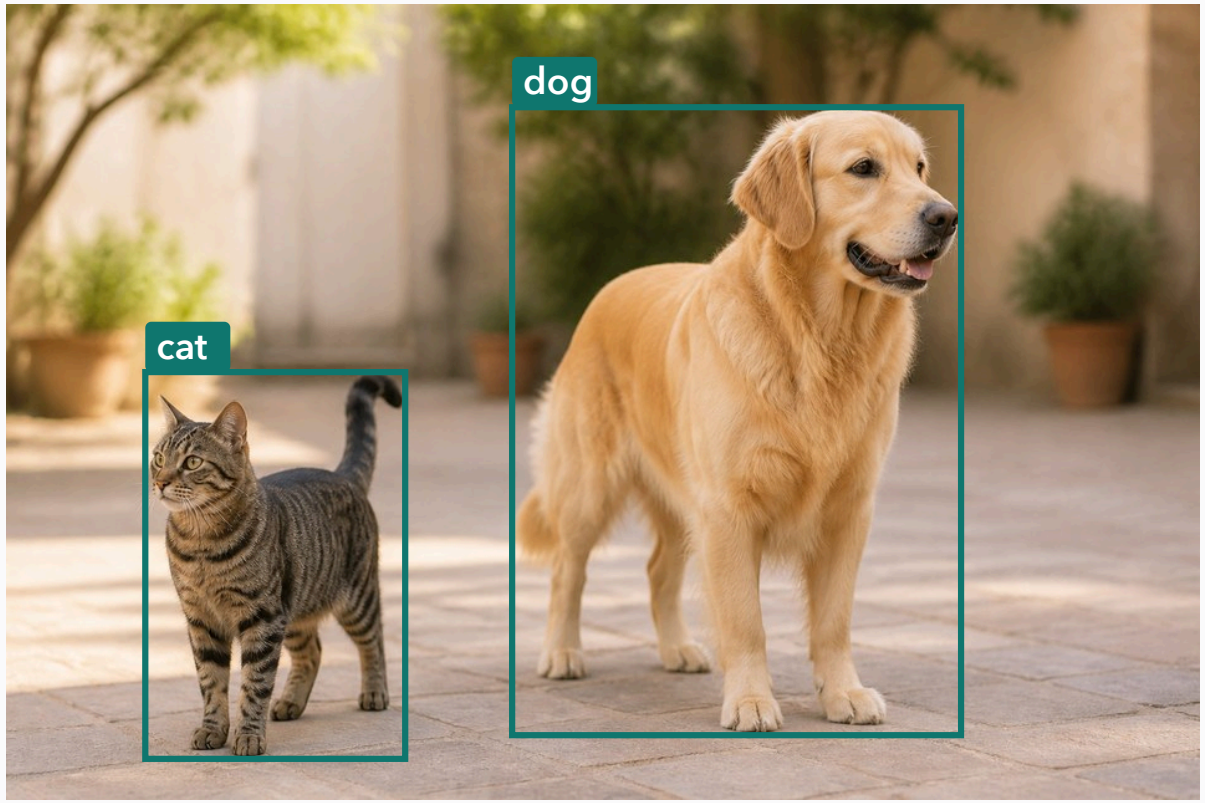
```
16 0.612 0.524 0.376 0.789
15 0.226 0.705 0.218 0.486
```

A training photo is stored with a small text file: one line per object. This is the YOLO format, for our two annotated animals.

Our annotation of the photo (data/annotations.json), written in the YOLO label format.

▶ Notebook, part 3

A labelled photo on disk: one line per object



`cat-dog.txt`, next to `cat-dog.jpg`:

```
16 0.612 0.524 0.376 0.789
15 0.226 0.705 0.218 0.486
```

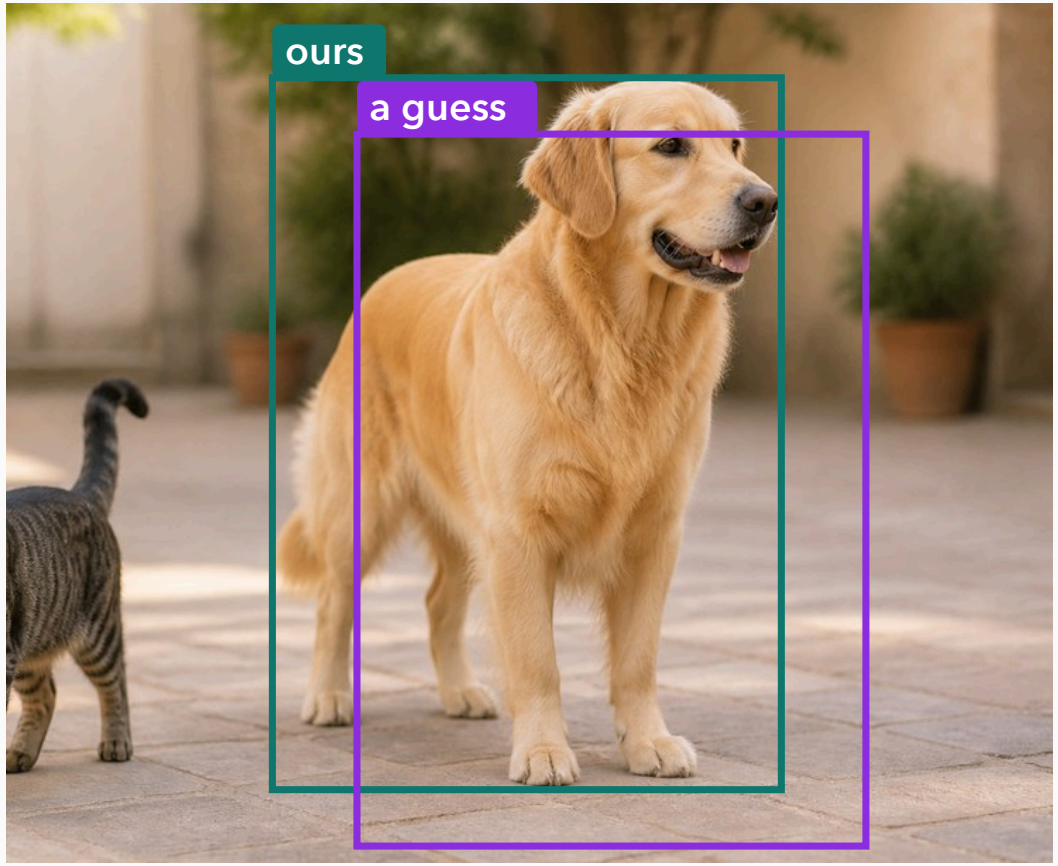
	class	c_x	c_y	w	h
line 1	16 = dog	0.612	0.524	0.376	0.789
line 2	15 = cat	0.226	0.705	0.218	0.486

Read a line: a class number (COCO counts from 0: 15 is cat, 16 is dog), then the box's normalised centre and size. Training a detector means learning to write these lines.

Our annotation of the photo (data/annotations.json), written in the YOLO label format.

▶ Notebook, part 3

Two boxes for one dog: how well do they agree?



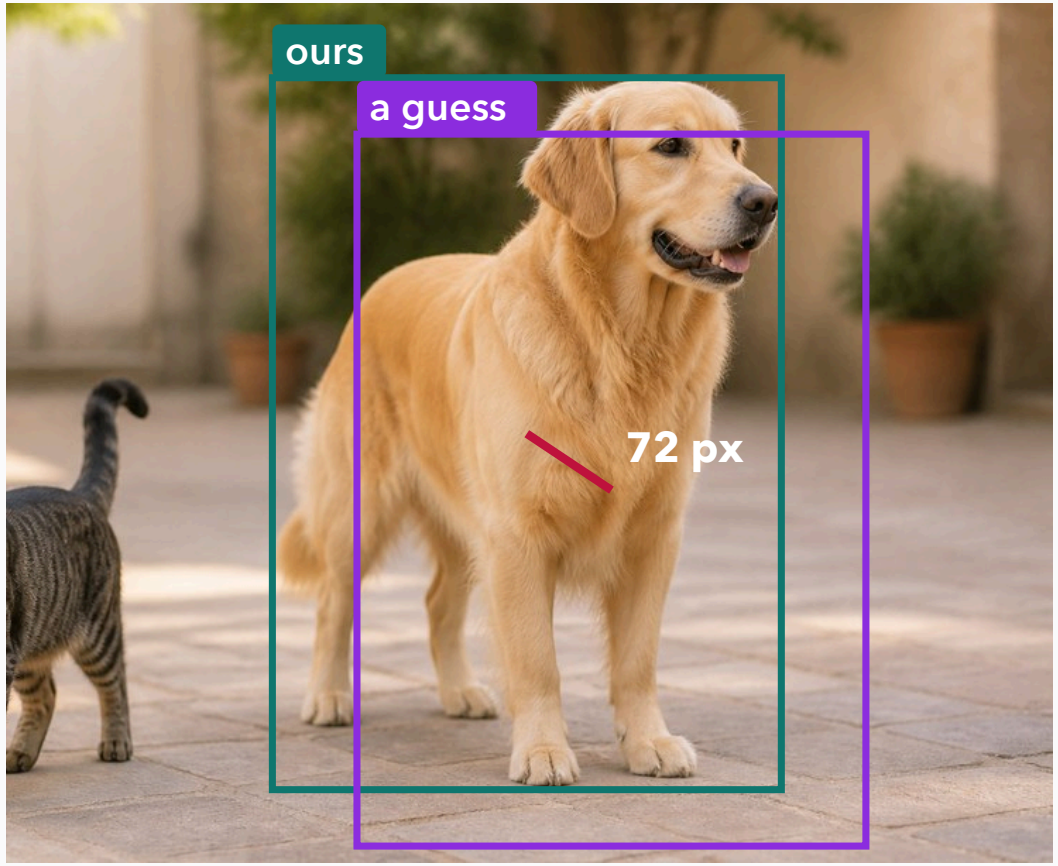
Teal: our box. Purple: a guess, moved 60 px right and 40 px down.

> Teal is our box for the dog. Purple is a guess, moved 60 px right and 40 px down. How good is the guess?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Two boxes for one dog: how well do they agree?



Teal: our box. Purple: a guess, moved 60 px right and 40 px down.

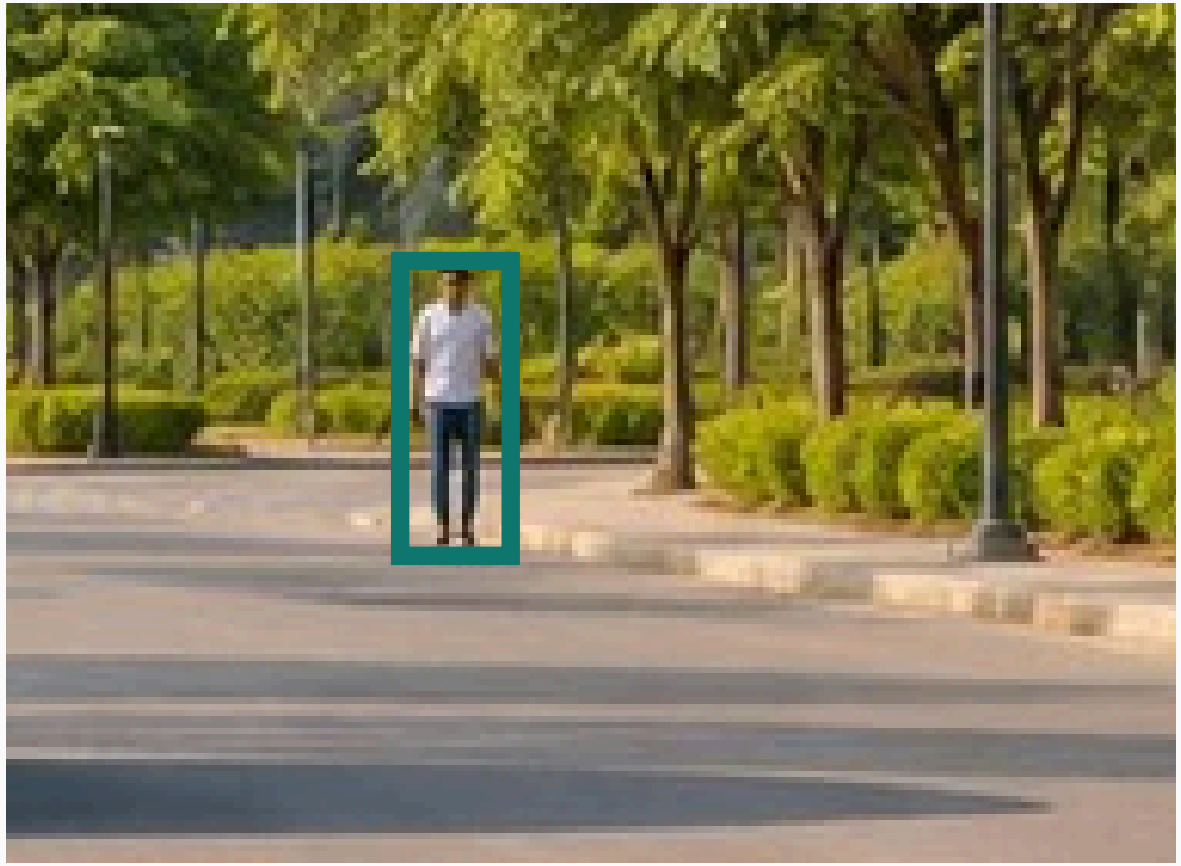
Idea 1: the distance between the centres, $\sqrt{(60^2 + 40^2)} = 72$ px.

> One idea: measure how far apart the centres are. Here 72 px. Is that a lot?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same slip on a small object



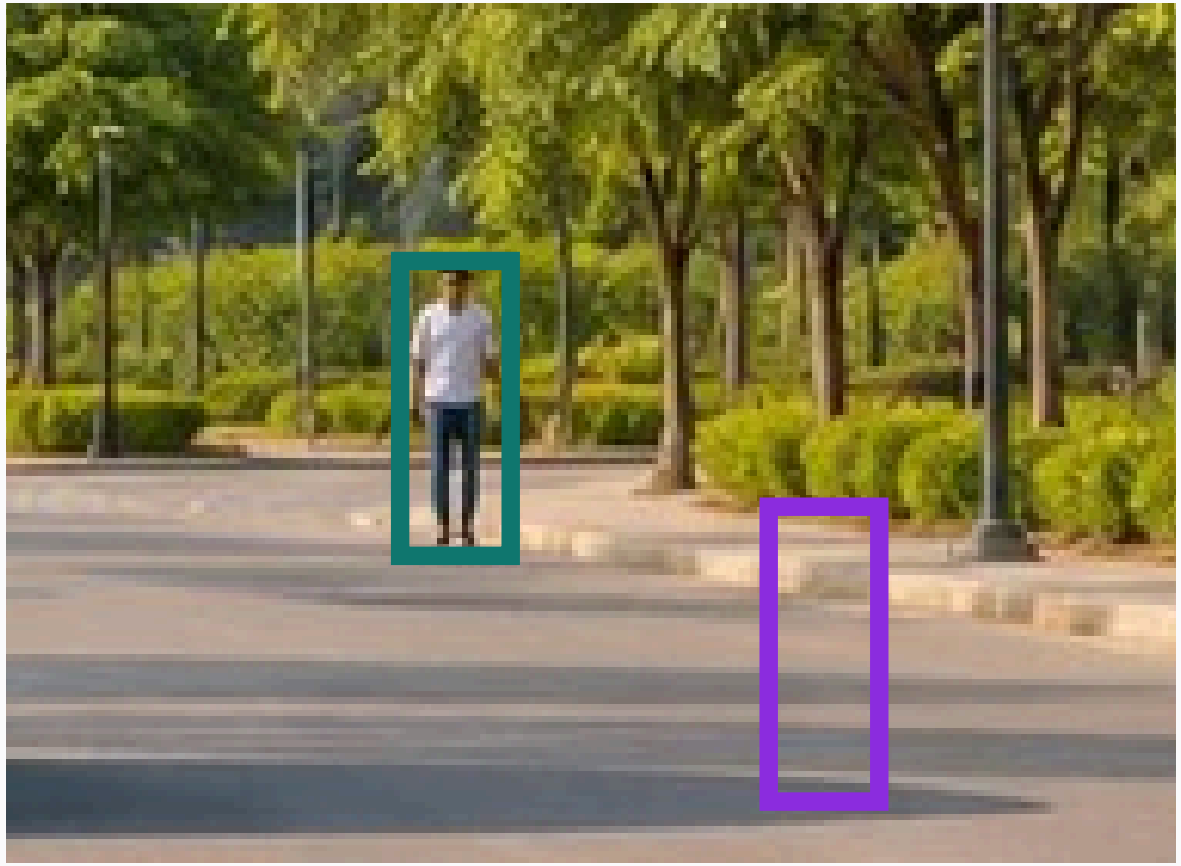
A person far down the street, 18 px wide.

> A person far down the street: 18 pixels wide.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same slip on a small object



A person far down the street, 18 px wide.

Move the box by the same 60 px right and 40 px down: it no longer touches the person.

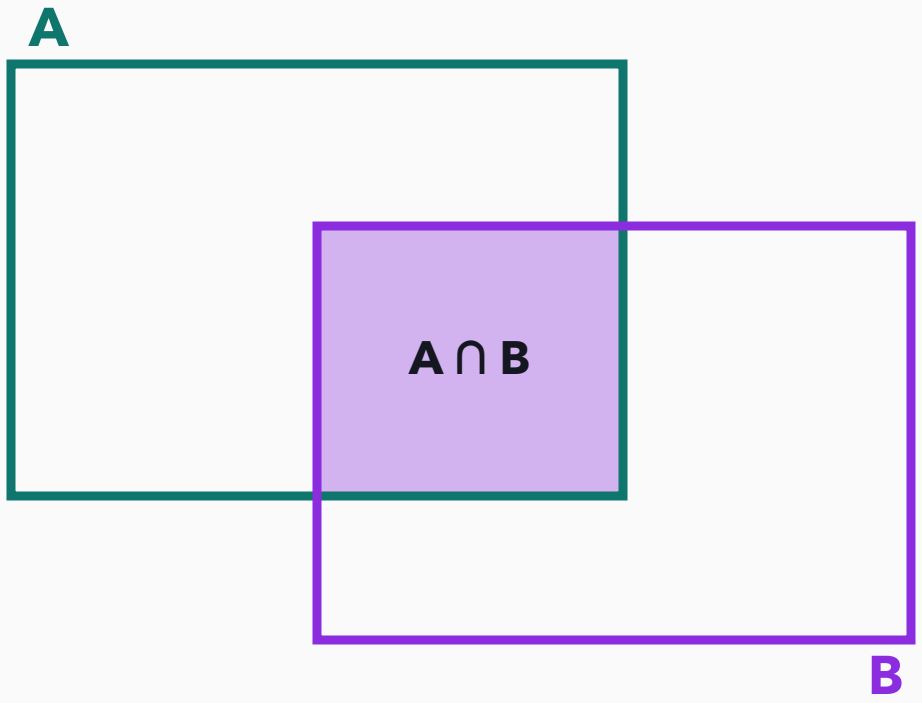
A slip that is fine for the dog misses the small person completely. We need a score that scales with the box.

> The same 72-pixel slip leaves the guess off the person entirely. For the dog it was a small error; here it is a miss. Compare overlap, not distance.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Intersection over union (IoU)

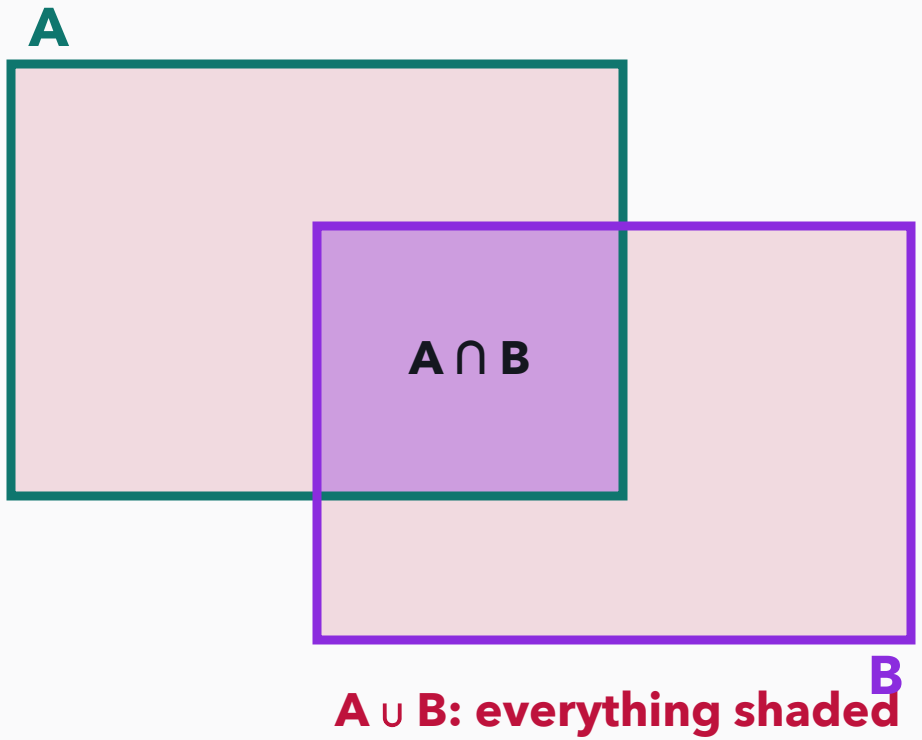


Intersection: the area covered by both boxes.

> The intersection: the region inside both boxes.

▶ Notebook, part 3

Intersection over union (IoU)



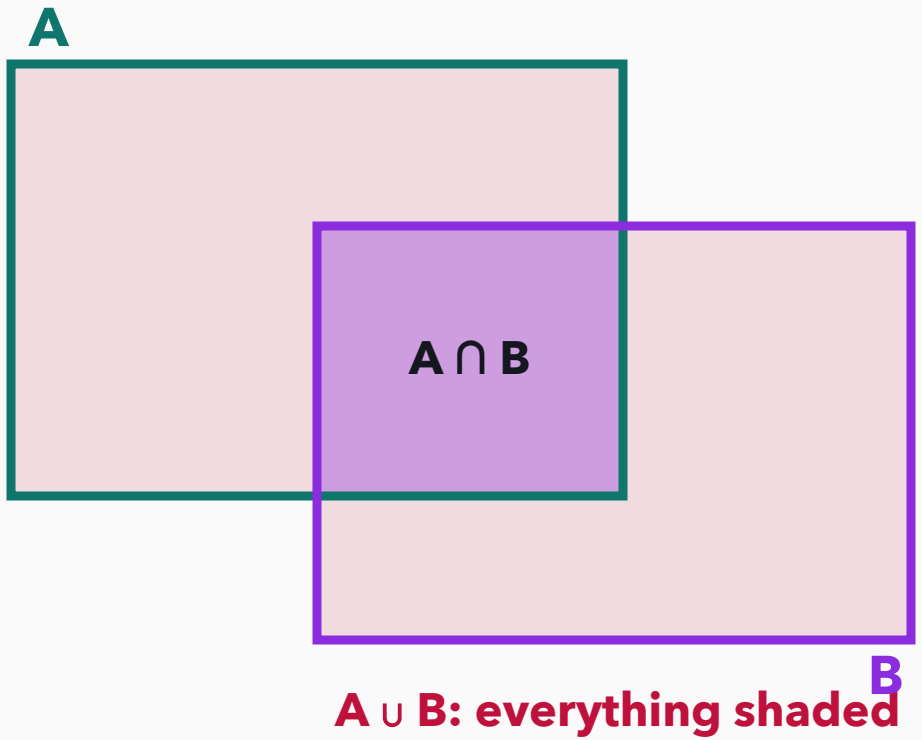
Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

> The union: the region inside either box.

▶ Notebook, part 3

Intersection over union (IoU)



Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

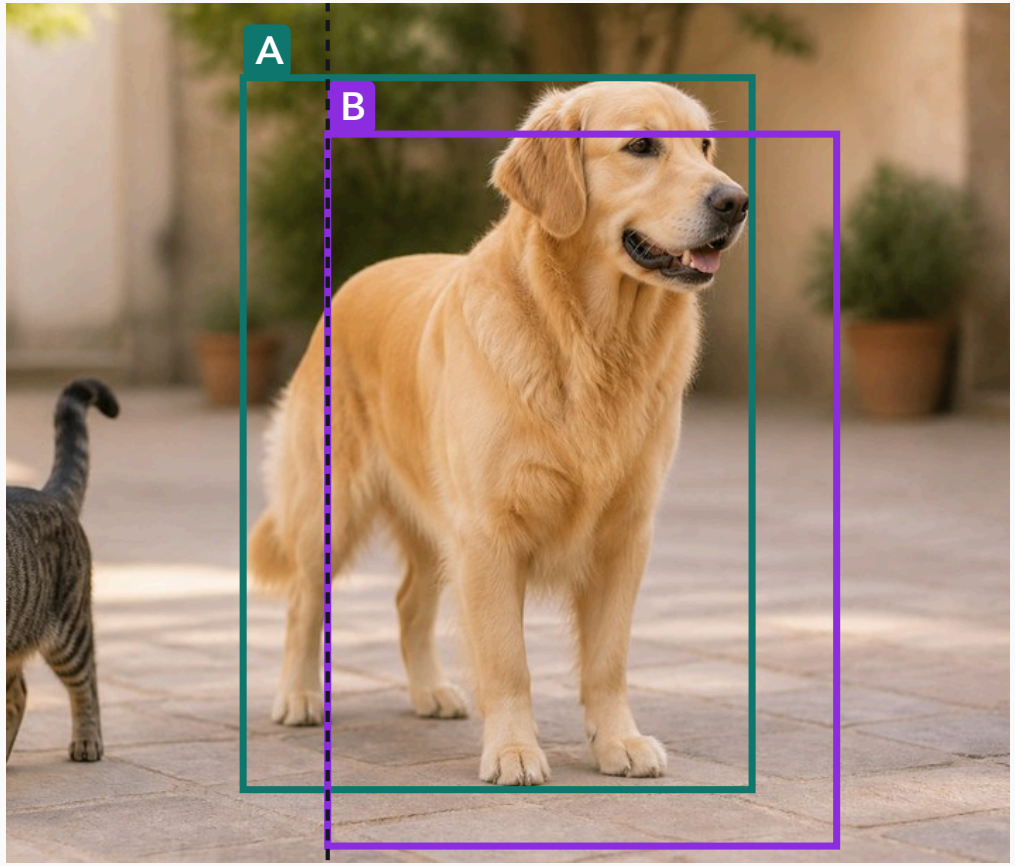
$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

Identical boxes: IoU = 1. Boxes that do not touch: IoU = 0.

IoU divides the first by the second. It is a fraction of area, so it does not care how big the boxes are.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



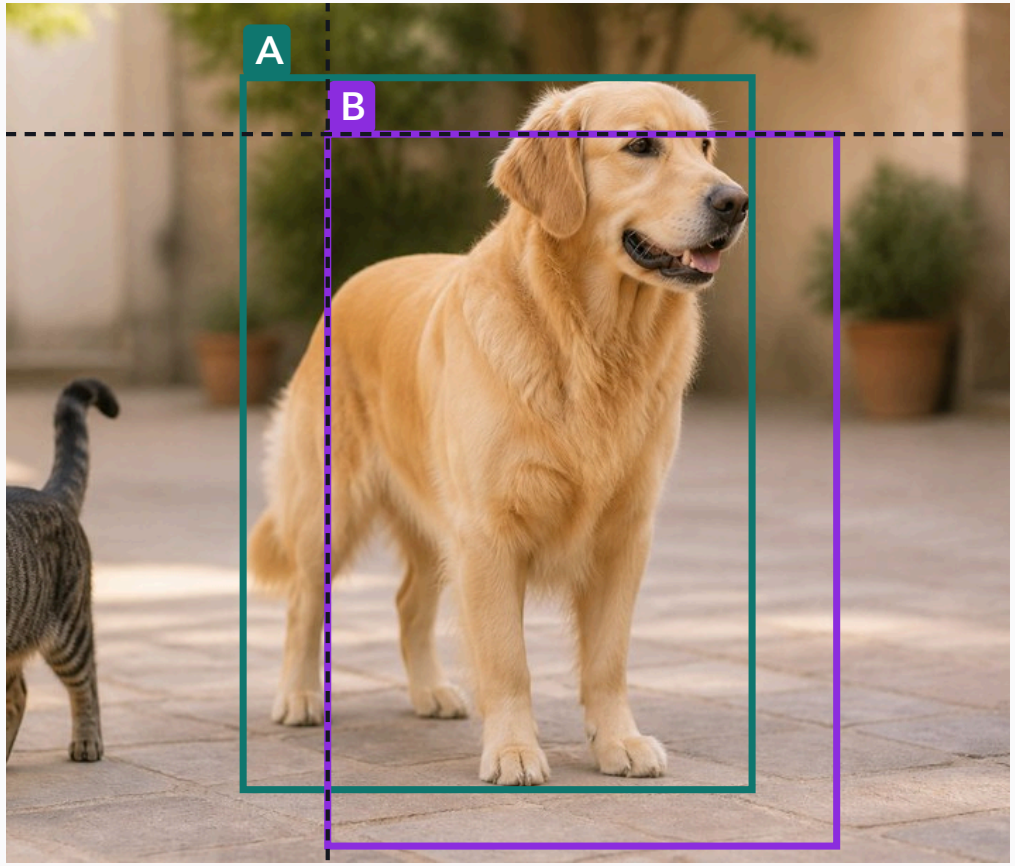
$$\text{left} = \max(407,\ 467) = 467$$

> The overlap starts where both boxes have started: the larger of the two left edges.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

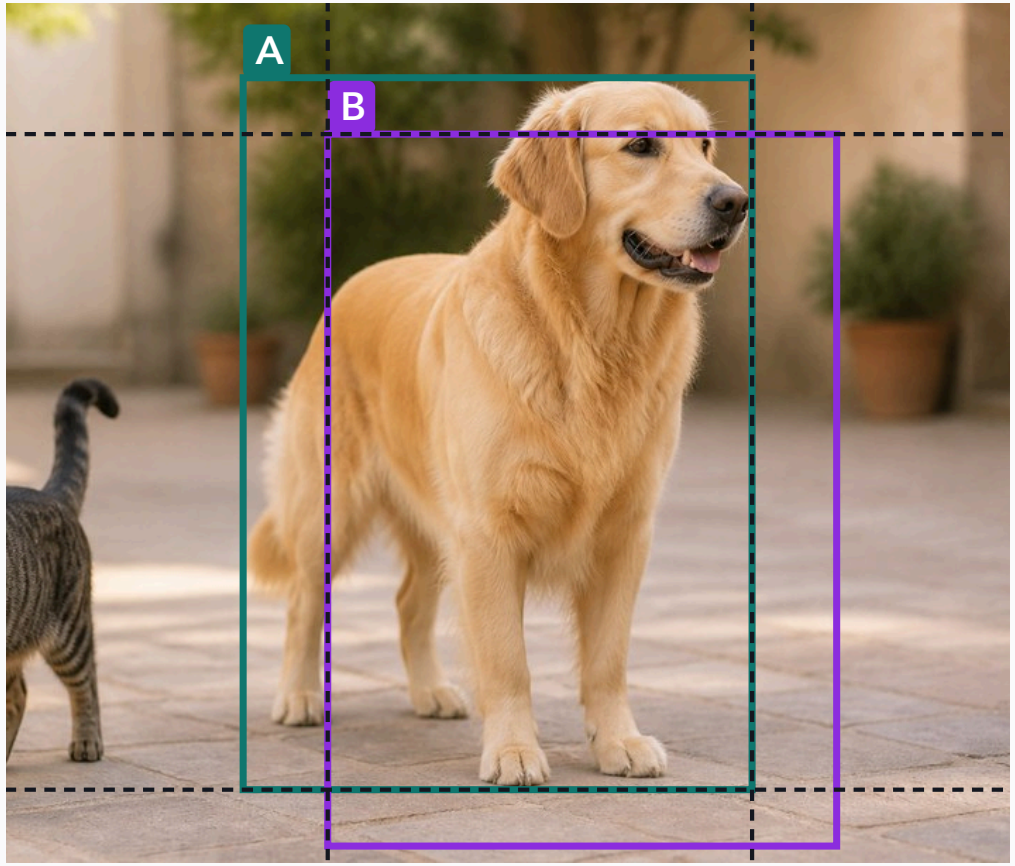
$$\text{top} = \max(83,\ 123) = 123$$

> Likewise its top is the larger of the two tops.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

$$\text{top} = \max(83,\ 123) = 123$$

$$\text{right} = \min(768,\ 828) = 768$$

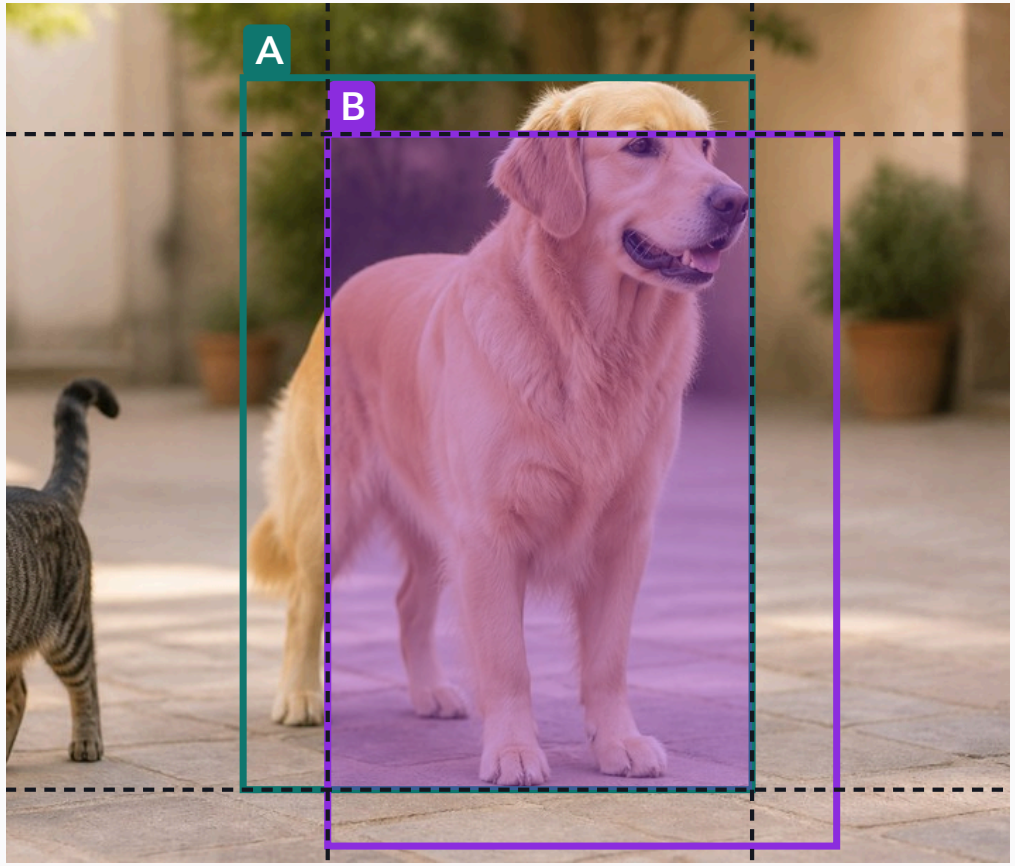
$$\text{bottom} = \min(588,\ 628) = 588$$

> It ends where the first box ends: the smaller right edge and the smaller bottom edge.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

$$\text{top} = \max(83,\ 123) = 123$$

$$\text{right} = \min(768,\ 828) = 768$$

$$\text{bottom} = \min(588,\ 628) = 588$$

$$\text{area} = 301 \times 465 = 139{,}965$$

> Its area: 301 × 465 = 139,965 square pixels. (If a width came out negative, the boxes would not touch: area 0.)

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 2: the union, then the ratio

$$\text{area}(A) = 361 \times 505 = 182{,}305, \qquad \text{area}(B) = 182{,}305$$

Each box's own area. The guess has the same size as our box; it is only moved.

▶ Notebook, part 3

IoU, step 2: the union, then the ratio

$$\text{area}(A) = 361 \times 505 = 182{,}305, \qquad \text{area}(B) = 182{,}305$$

$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

> Adding the two areas counts the overlap twice, so subtract it once.

▶ Notebook, part 3

IoU, step 2: the union, then the ratio

$$\text{area}(A) = 361 \times 505 = 182{,}305, \qquad \text{area}(B) = 182{,}305$$

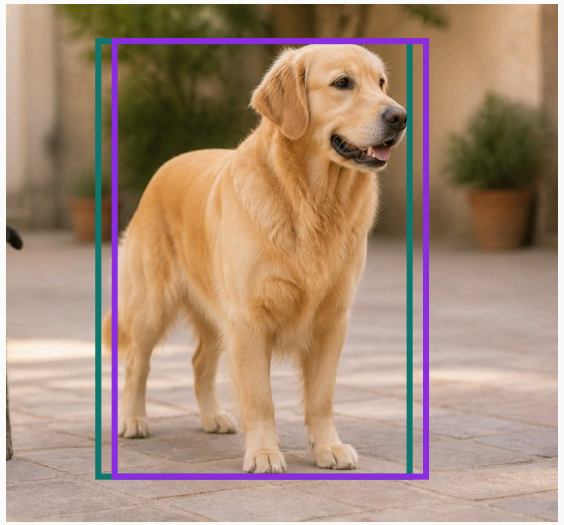
$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

$$\text{IoU} = \frac{139{,}965}{224{,}645} = 0.623$$

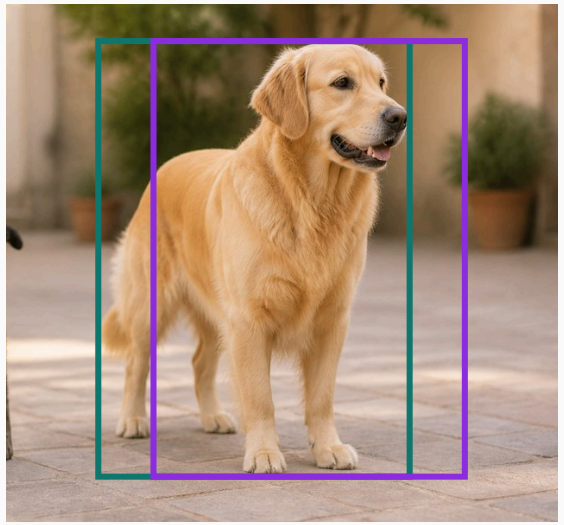
> IoU = 0.623. A move of 60 px right and 40 px down cost more than a third of the score.

▶ Notebook, part 3

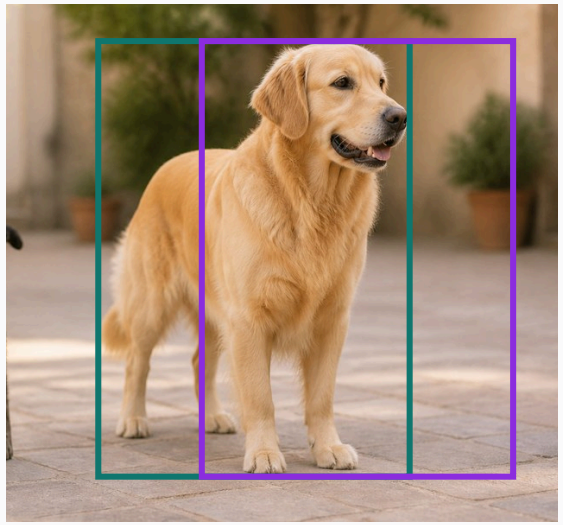
What IoU values look like



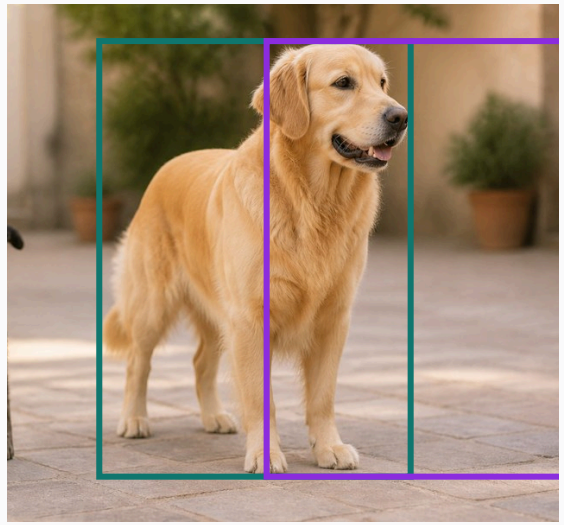
IoU 0.9 · moved 19 px



IoU 0.7 · moved 64 px



IoU 0.5 · moved 120 px



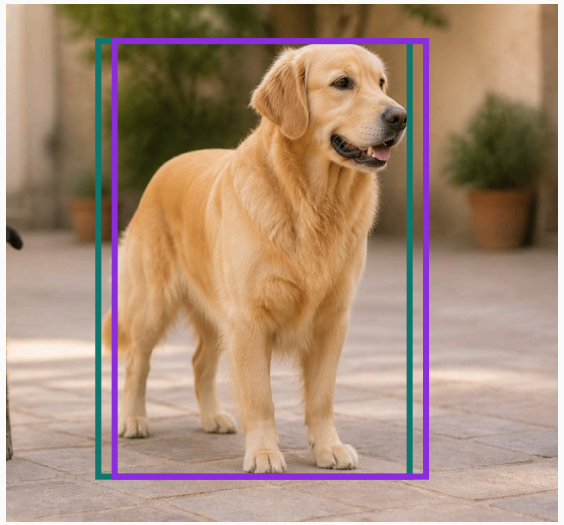
IoU 0.3 · moved 194 px

> The guess slides sideways. At 0.9 the boxes nearly agree; at 0.5 the guess has moved a third of the dog's width; at 0.3, half of it.

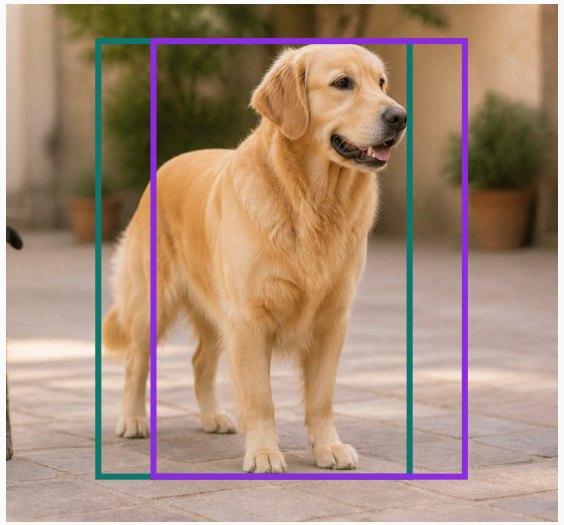
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 3

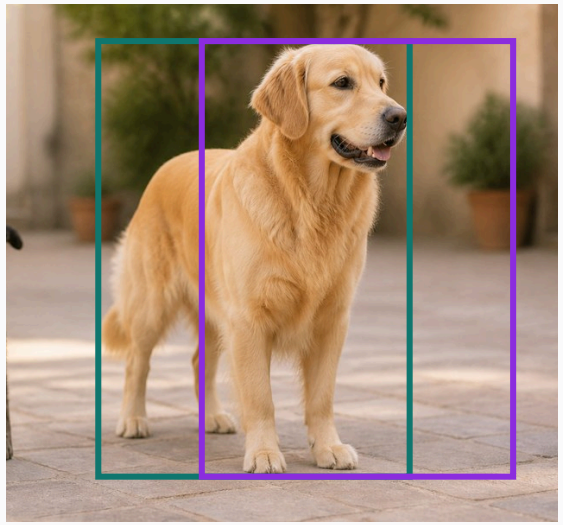
What IoU values look like



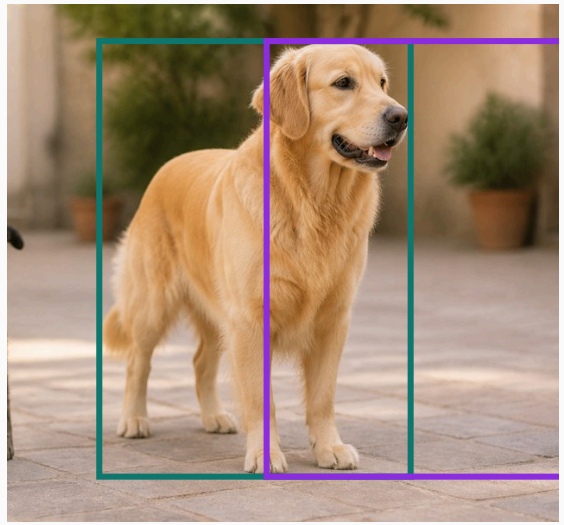
IoU 0.9 · moved 19 px



IoU 0.7 · moved 64 px



IoU 0.5 · moved 120 px



IoU 0.3 · moved 194 px

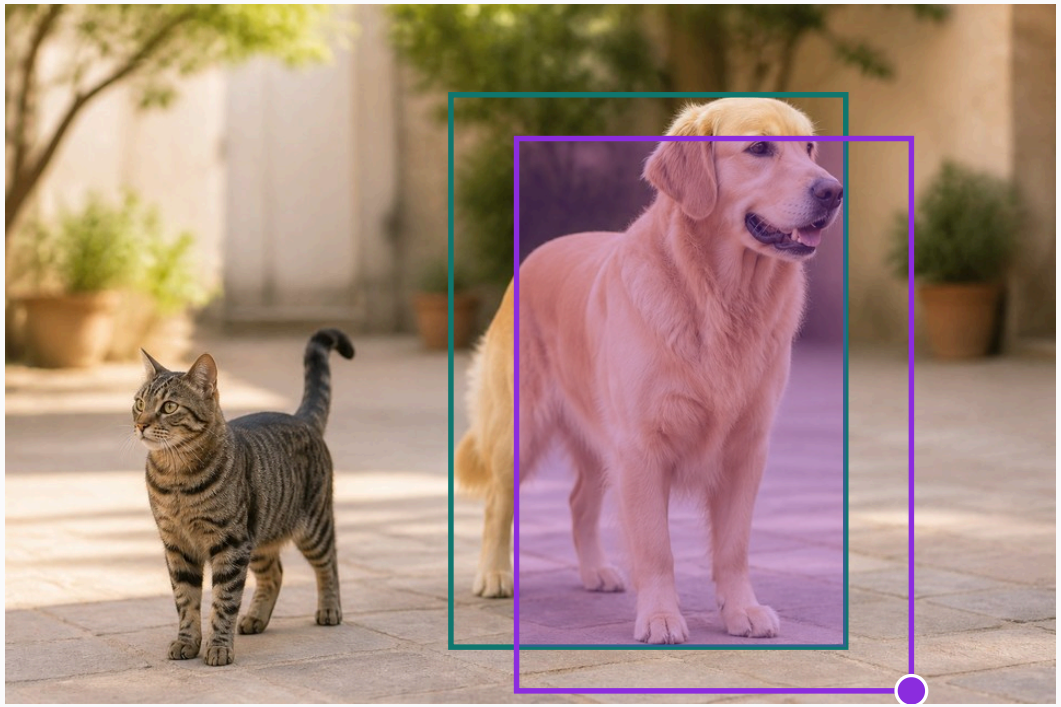
Benchmarks usually count IoU ≥ 0.5 as a hit. The real YOLOv8n box for this dog has IoU 0.983 with ours.

> Most benchmarks call IoU ≥ 0.5 a hit. A trained detector does much better on this dog: 0.983.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 3

Try it: drag a box over the dog



overlap	139,965 px²
union	224,645 px²
IoU	**0.623**

> Drag the purple box. Can you find a box with IoU 0.5 that looks good? One with IoU 0.7 that looks bad?

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):                  # a: (N, 4), b: (M, 4), corners
    lt = torch.max(a[:, None, :2], b[None, :, :2])  # overlap's top-left
    rb = torch.min(a[:, None, 2:], b[None, :, 2:])  # overlap's bottom-right
    inter = (rb - lt).clamp(min=0).prod(-1)         # (N, M); 0 if apart
    area = lambda x: (x[:, 2:] - x[:, :2]).prod(-1)
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)
```

> The same three steps, for N boxes against M boxes at once: max of the starts, min of the ends, clamp at zero, divide.

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):                  # a: (N, 4), b: (M, 4), corners
    lt = torch.max(a[:, None, :2], b[None, :, :2])  # overlap's top-left
    rb = torch.min(a[:, None, 2:], b[None, :, 2:])  # overlap's bottom-right
    inter = (rb - lt).clamp(min=0).prod(-1)         # (N, M); 0 if apart
    area = lambda x: (x[:, 2:] - x[:, :2]).prod(-1)
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)
```

```python
box_iou(torch.tensor([[407., 83., 768., 588.]]), torch.tensor([[467., 123., 828., 628.]]))
# tensor([[0.6230]])  -- torchvision.ops.box_iou gives the same
```

On our two boxes: 0.6230. torchvision.ops.box_iou returns the same N × M matrix.

Part 3 in one slide

1. A box is four numbers: corners (407, 83, 768, 588) or centre and size (587.5, 335.5, 361, 505); divide by the photo's size to normalise.
2. A labelled photo is a photo plus one line per object: a class and four numbers.
3. IoU = overlap ÷ union: 1 for the same box, 0 for boxes apart. The 60-px guess scored 0.623.
4. IoU ≥ 0.5 is the usual "found it".

Next: ask a network to output a box, then several, and see where it breaks.

> Part 3: boxes and IoU. Part 4: a network that outputs boxes.

▶ Notebook, part 4

Part 4 · One box is easy; several are not

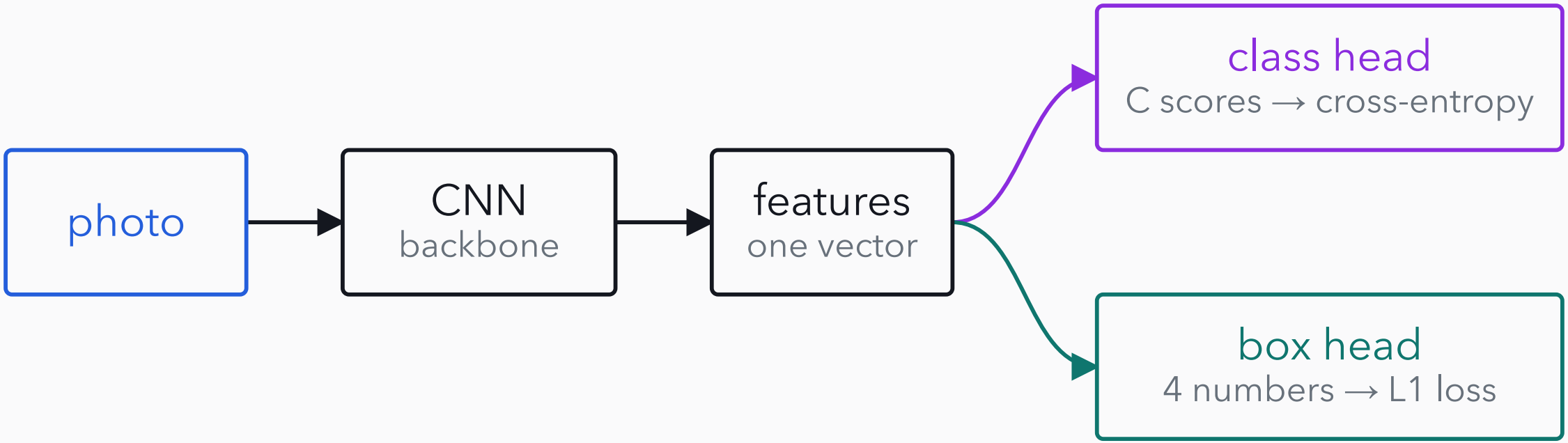
A network can output four numbers for one box. Asking it for several boxes shows the real problem.

1. Add a box head to a classifier; write the target it learns, with and without an object.
2. Ask the same head for two animals, and see it hedge.
3. Give it K slots: count and order break the loss.

> One box is regression; several boxes expose the assignment problem.

▶ Notebook, part 4

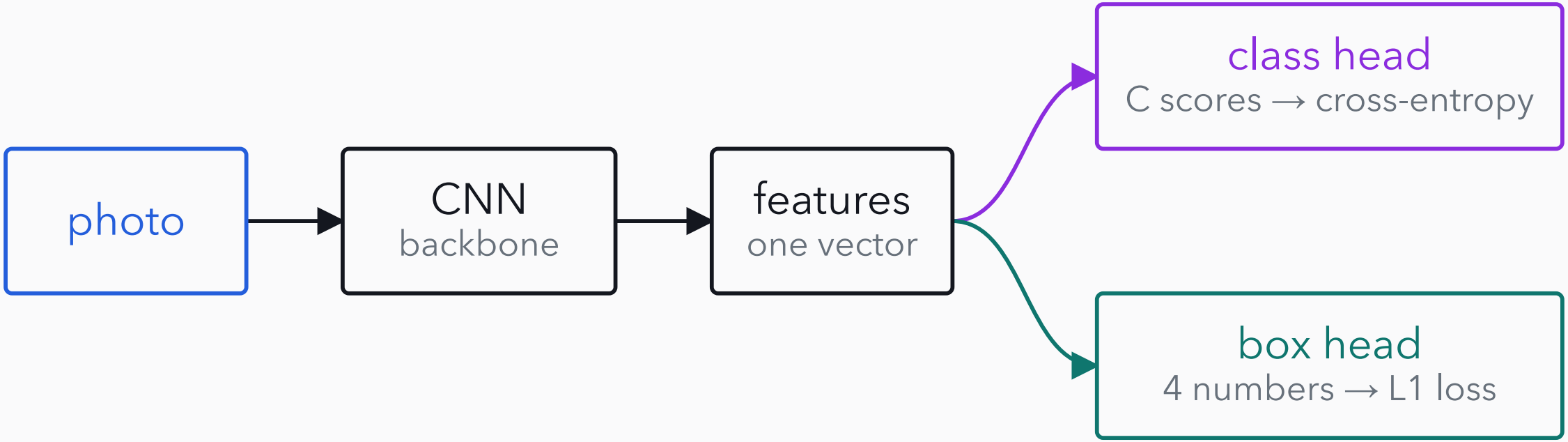
One object: a box head beside the class head



A classifier: a CNN turns the photo into one feature vector; a linear layer turns it into class scores.

▶ Notebook, part 4

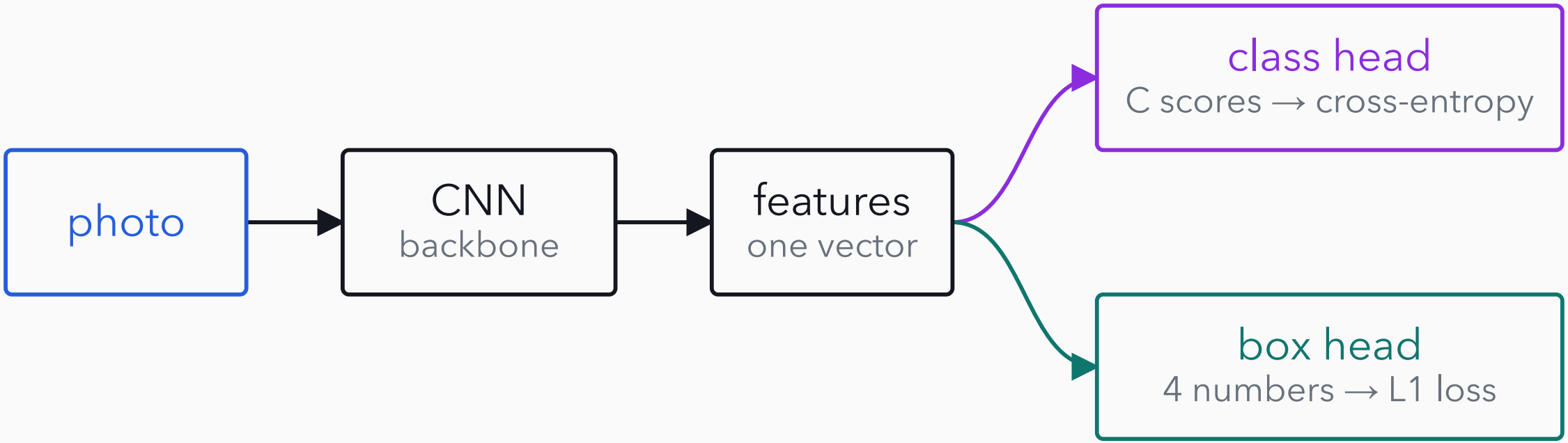
One object: a box head beside the class head



> Add a second small layer on the same vector that outputs four numbers: the box.

▶ Notebook, part 4

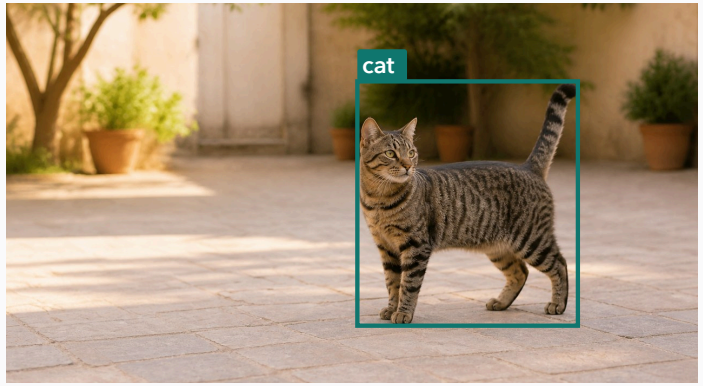
One object: a box head beside the class head



> Train both at once: cross-entropy for the class, and a loss that compares the four box numbers with the annotation's.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



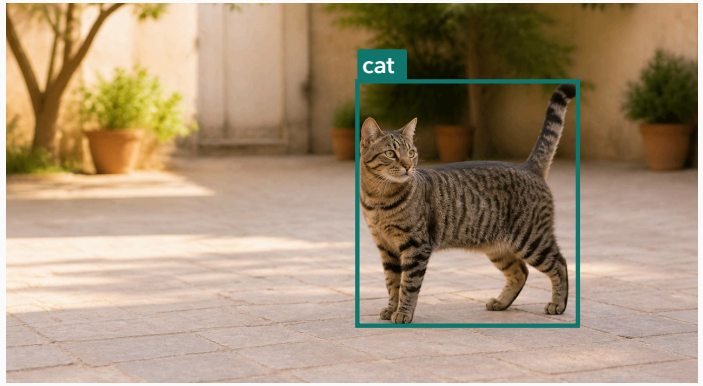
object?	c_x	c_y	w	h	cat	dog
1	0.667	0.527	0.319	0.642	1	0

A localiser for two classes, cat and dog, writes seven numbers: is there an object; its box (normalised centre and size); and which class. For this photo: 1, the cat's box, cat 1, dog 0.

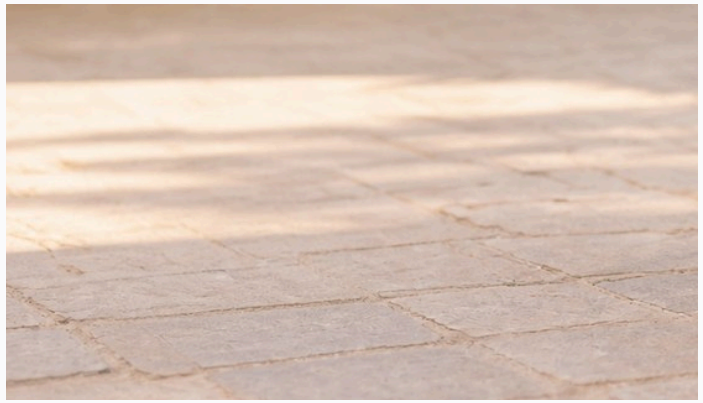
Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



object?	c_x	c_y	w	h	cat	dog
1	0.667	0.527	0.319	0.642	1	0



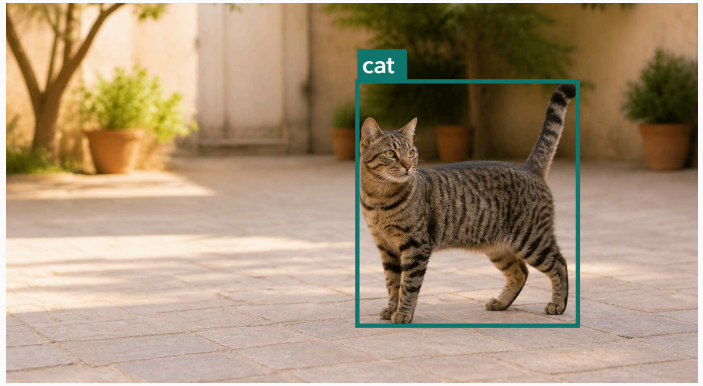
object?	c_x	c_y	w	h	cat	dog
0	?	?	?	?	?	?

> A photo of empty floor. “Object?” is 0. Its box and class have no right answer, so the target there is “?”.

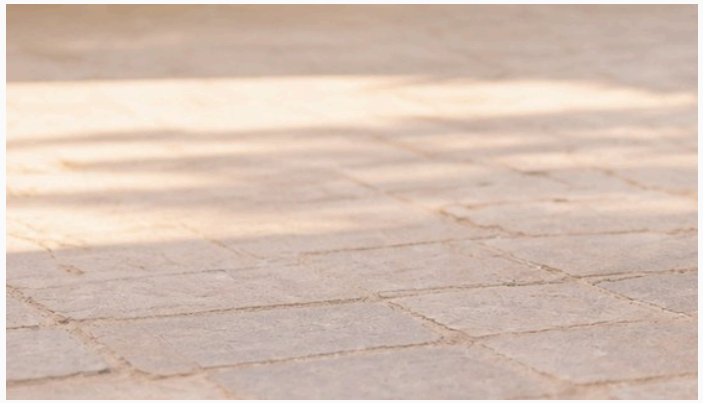
Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



object?	c_x	c_y	w	h	cat	dog
1	0.667	0.527	0.319	0.642	1	0



object?	c_x	c_y	w	h	cat	dog
0	?	?	?	?	?	?

Only “object?” is learned here: each **?** is left out of the loss.

> The loss counts only entries that have a right answer: for the floor, just the first. Keep this “?” in mind: a detector writes it at most places in a photo.

Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Training the box head: compare four numbers

$$\mathcal{L}_{\text{box}} = \sum_{k=1}^{4} \left| \hat{b}_k - b_k \right|$$

The L1 box loss: for each of the four numbers, the absolute difference between prediction and target; add them.

▶ Notebook, part 4

Training the box head: compare four numbers

$$\mathcal{L}_{\text{box}} = \sum_{k=1}^{4} \left| \hat{b}_k - b_k \right|$$

	c_x	c_y	w	h
target (the dog)	0.612	0.524	0.376	0.789
prediction (the guess)	0.674	0.587	0.376	0.789
|difference|	0.062	0.063	0.000	0.000

> Take the 60-px guess from Part 3 as the prediction, in normalised units. The centre is off; the size is right.

▶ Notebook, part 4

Training the box head: compare four numbers

$$\mathcal{L}_{\text{box}} = \sum_{k=1}^{4} \left| \hat{b}_k - b_k \right|$$

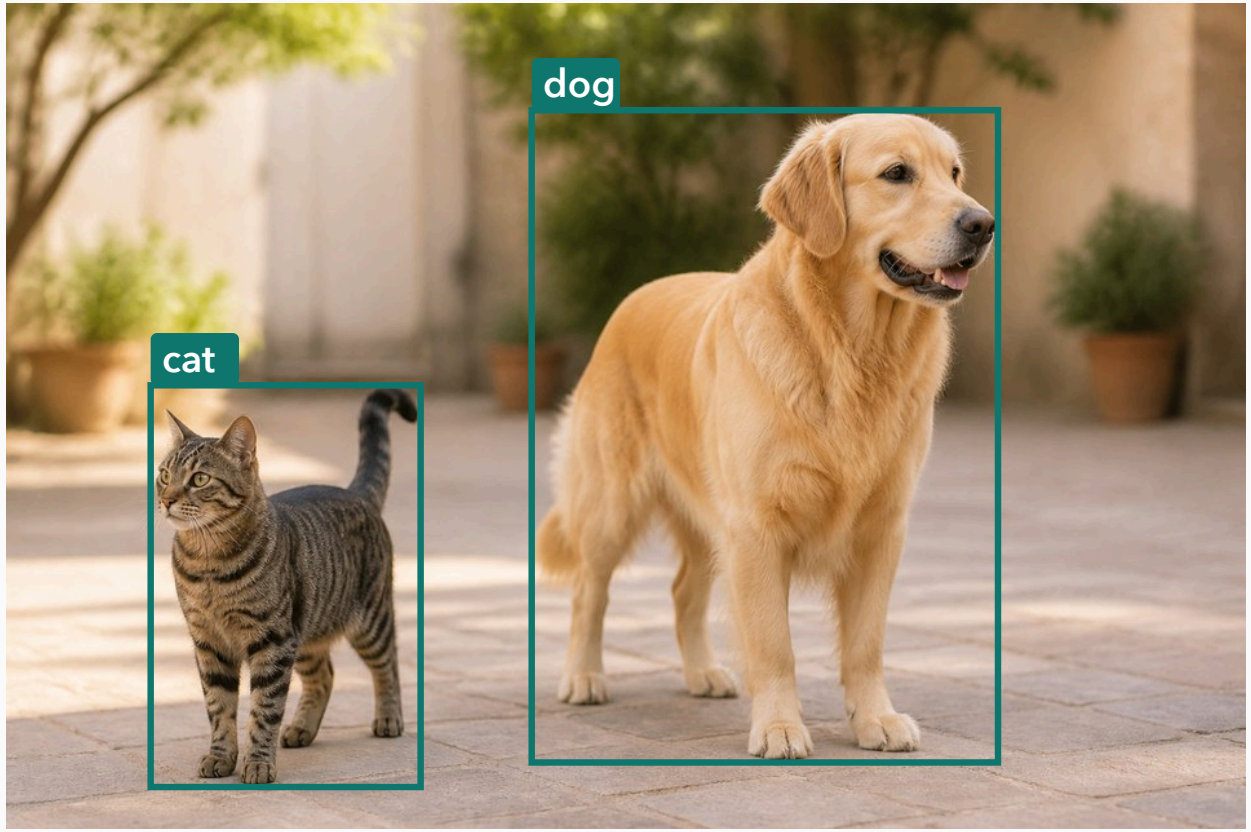
	c_x	c_y	w	h
target (the dog)	0.612	0.524	0.376	0.789
prediction (the guess)	0.674	0.587	0.376	0.789
|difference|	0.062	0.063	0.000	0.000

$$\mathcal{L}_{\text{box}} = 0.062 + 0.063 + 0.000 + 0.000 = 0.125$$

> Loss 0.125. It is zero only if all four numbers match; gradient descent moves the four outputs toward the target.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



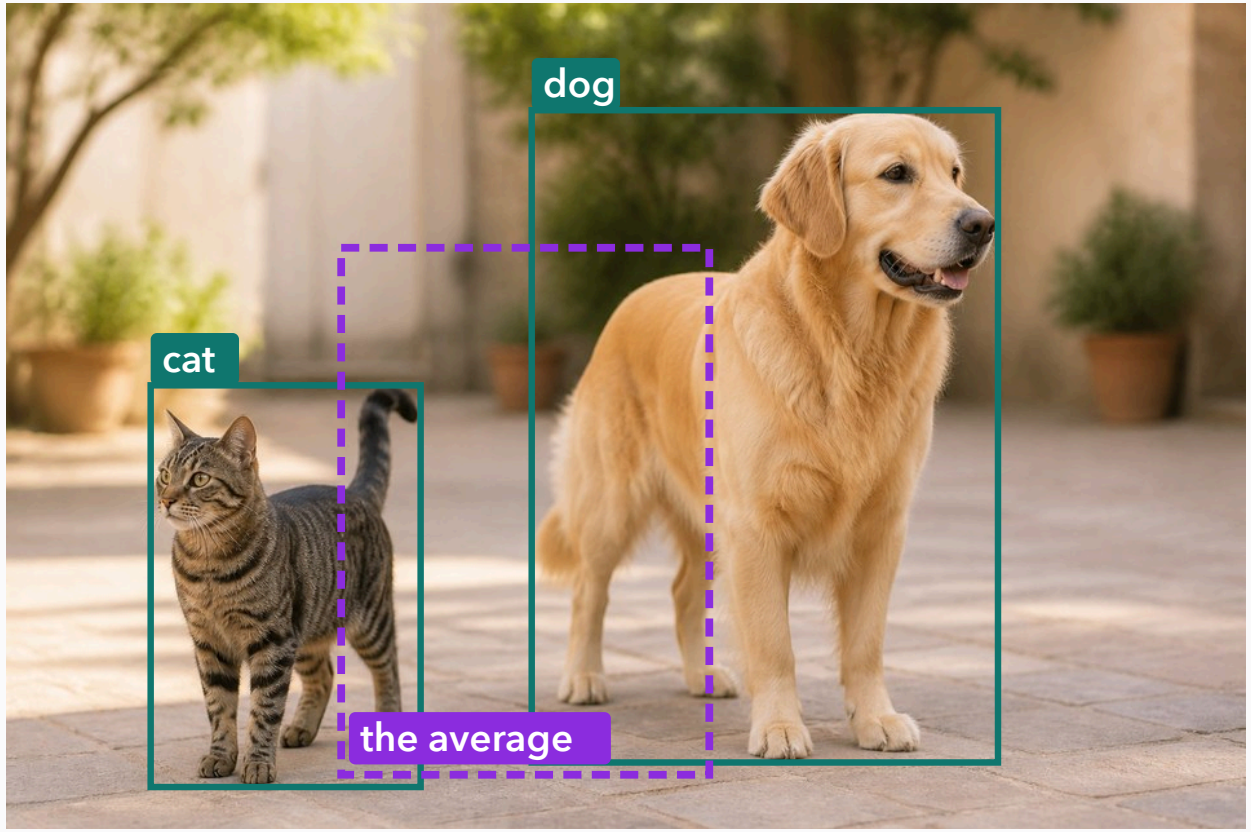
The annotation lists both animals. The head has room for one box.

> One box head, two animals. Which should it draw?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



The annotation lists both animals. The head has room for one box.

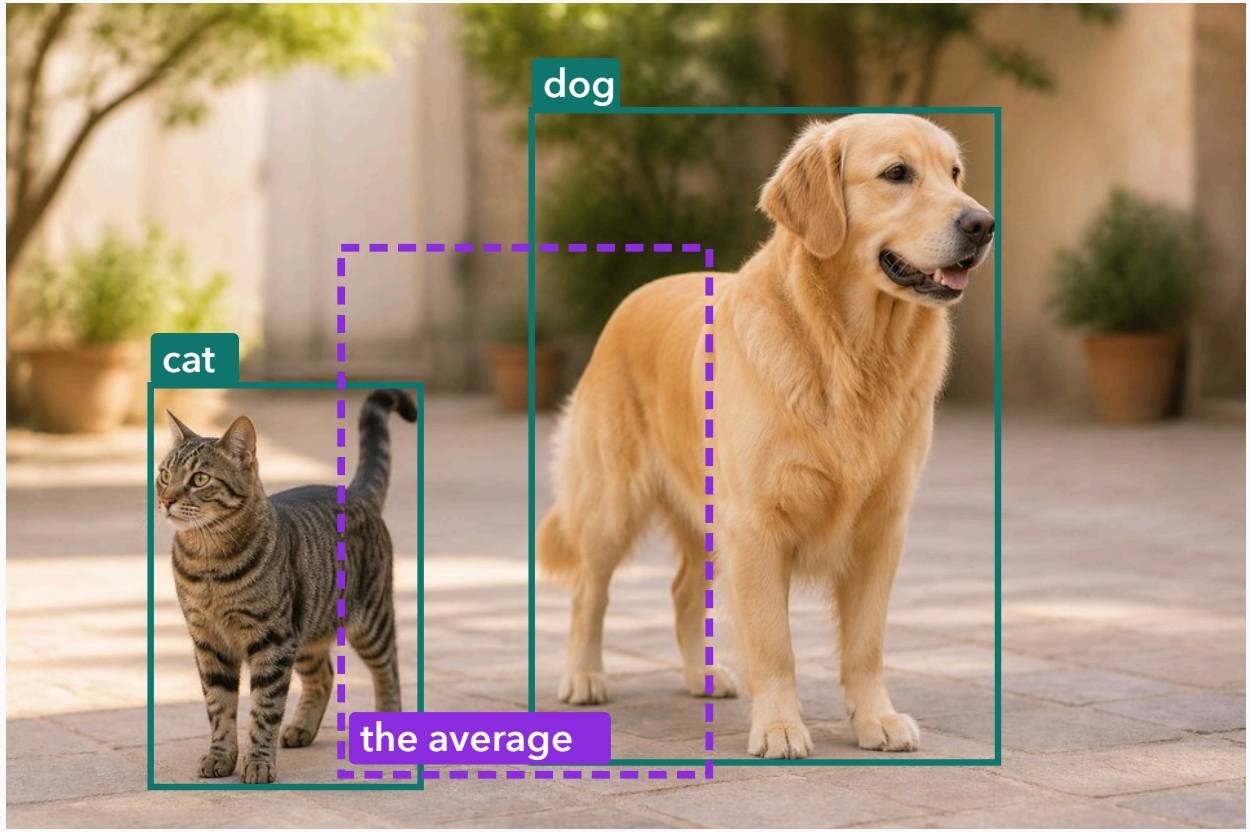
Trained to match "the first animal in the list", and the list order is arbitrary, the safest single answer is in between: the average box.

> If some annotations list the dog first and some the cat, a squared error is smallest for the average of the two boxes: the network learns to hedge.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



The annotation lists both animals. The head has room for one box.

Trained to match "the first animal in the list", and the list order is arbitrary, the safest single answer is in between: the average box.

$$\mathrm{IoU}(\mathrm{avg}, \mathrm{dog}) = 0.225$$
$$\mathrm{IoU}(\mathrm{avg}, \mathrm{cat}) = 0.114$$

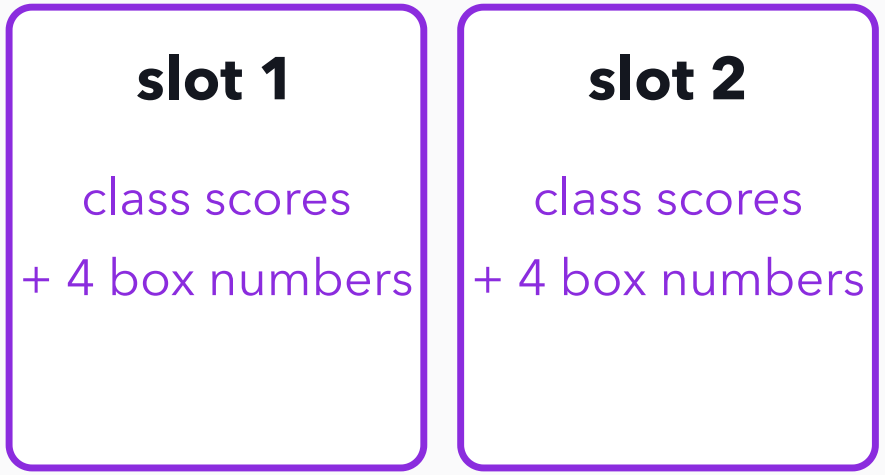
It matches neither animal.

> The average box has IoU 0.23 with the dog and 0.11 with the cat. A box between two objects describes neither.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Give the head K slots



> Give the head K slots instead of one. Each slot writes a class and a box. Two animals: two slots used.

▶ Notebook, part 4

Give the head K slots



The other slots must be able to say “nothing here”: add a no-object class. Output size: K × (C + 1 + 4), fixed.

▶ Notebook, part 4

Give the head K slots

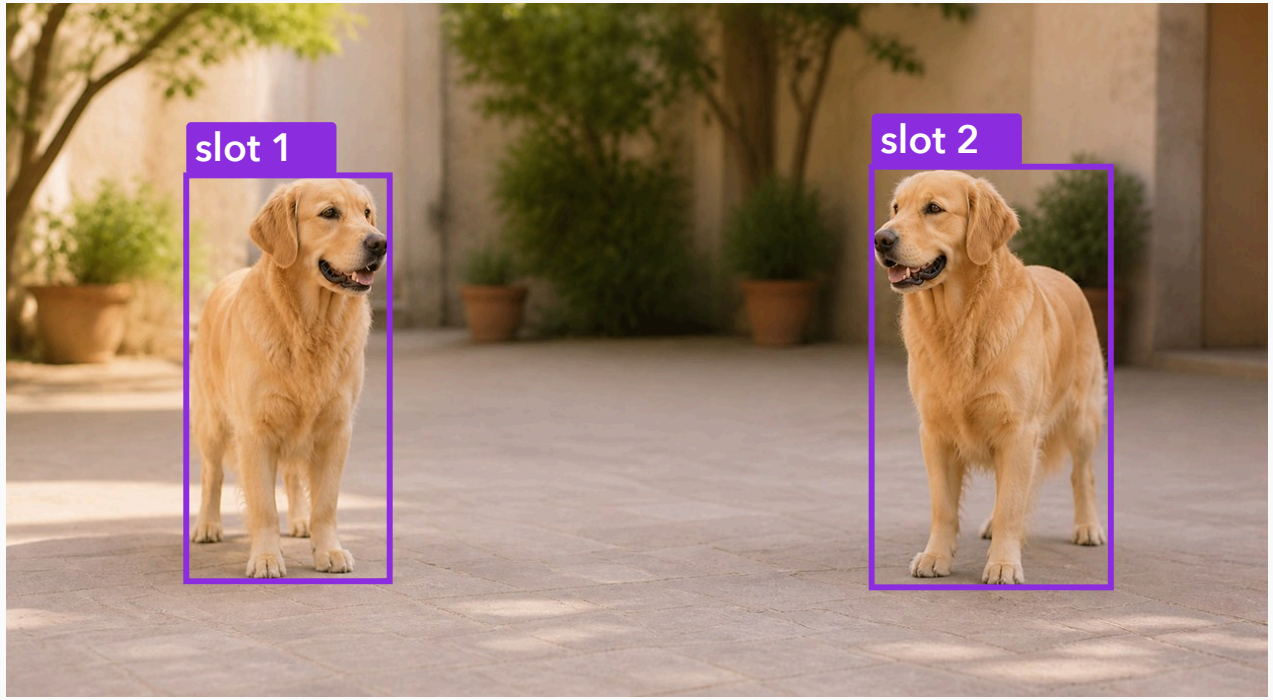


Problem 1: a photo with more objects than slots. The crowd photo has 11 people; 5 slots cannot hold them. So make K larger than any photo needs.

> Problem 1: what if a photo has more objects than K? Then choose K large: DETR (Appendix A) uses K = 100.

▶ Notebook, part 4

Problem 2: which dog belongs in slot 1?



> Two slots, two predictions: slot 1 drew the left dog, slot 2 the right dog. Both boxes are right.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 4

Problem 2: which dog belongs in slot 1?



$$\text{slot 1 vs entry 1: } \mathcal{L} = 1.212$$

> But the annotator happened to list the right dog first. Compare slot 1 with entry 1 and slot 2 with entry 2: the L1 loss, in normalised units, is 1.21.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 4

Problem 2: which dog belongs in slot 1?



$$\text{slot 1 vs entry 1:}\ \ \mathcal{L} = 1.212$$

$$\text{slot 1 vs entry 2:}\ \ \mathcal{L} = 0.012$$

> Compare them the other way round: 0.012.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 4

Problem 2: which dog belongs in slot 1?



$$\text{slot 1 vs entry 1:}\ \ \mathcal{L} = 1.212$$

$$\text{slot 1 vs entry 2:}\ \ \mathcal{L} = 0.012$$

The same correct answer costs 100 times more because of the list's order. The loss must not depend on the order.

> Nothing about the prediction changed, only the list order. A loss that pairs outputs with targets by position punishes correct answers.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 4

Two ways to decide which output answers for which object

A · by place

- Tie each output to a place in the image
- The outputs at the dog's place answer for the dog
- No slot ever chooses

Parts 5–7: YOLO and most fast detectors

> Fix A: give every output a fixed place in the image. Whatever is at that place is its job. Then order cannot matter.

▶ Notebook, part 4

Two ways to decide which output answers for which object

A · by place

- Tie each output to a place in the image
- The outputs at the dog's place answer for the dog
- No slot ever chooses

Parts 5–7: YOLO and most fast detectors

B · by matching

- Keep free slots
- Before the loss, pair slots with objects in the cheapest way
- Each object gets exactly one slot

Appendix A: DETR

> Fix B: keep free slots, and before each loss, match slots to objects so that the total cost is lowest. Then order cannot matter either.

Part 4 in one slide

1. A box head on a classifier learns one box: four numbers, an L1 loss. With no object, its box target is "?": not learned.
2. Two animals, one box: the loss is lowest for the average box, which matches neither (IoU 0.23 and 0.11).
3. K slots: a photo can have more objects than slots; and pairing slot k with list entry k punished a correct answer (loss 1.21 instead of 0.012).
4. Fix A: tie each output to a place in the photo.

Next: build fix A: a grid of cells, each writing a box and class scores.

> Part 4: one box works; many boxes need a rule for which output answers for which object.

▶ Notebook, part 5

Part 5 · A candidate at every place

Tie each output to a place: a grid of cells over the photo, each writing one record.

1. From a classifier on crops, to sliding windows, to one pass of a CNN that gives a grid of cells.
2. What each cell writes: four box numbers and 80 class scores.
3. Which cells answer for each object; their targets and their loss, by hand for one cell.
4. At test time, read every cell's box back out.

> A grid of cells, each writing a record: targets, loss, decoding.

▶ Notebook, part 5

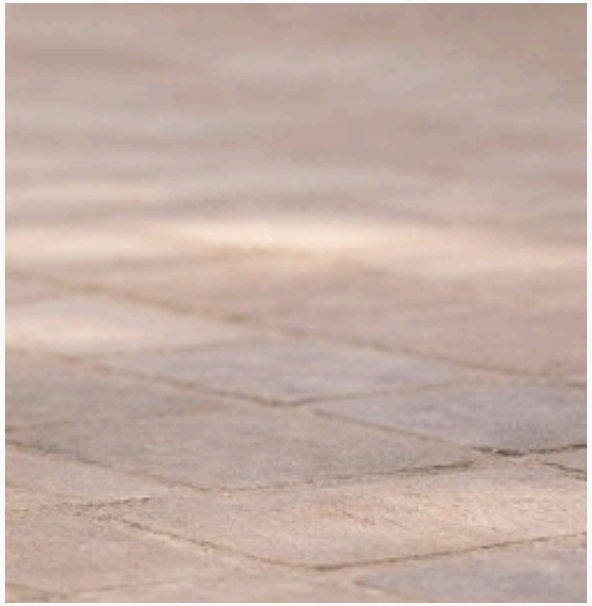
Start from a tool we have: a classifier for crops



dog



cat



nothing

> Three crops cut from the courtyard: a dog, a cat, and floor with nothing on it.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

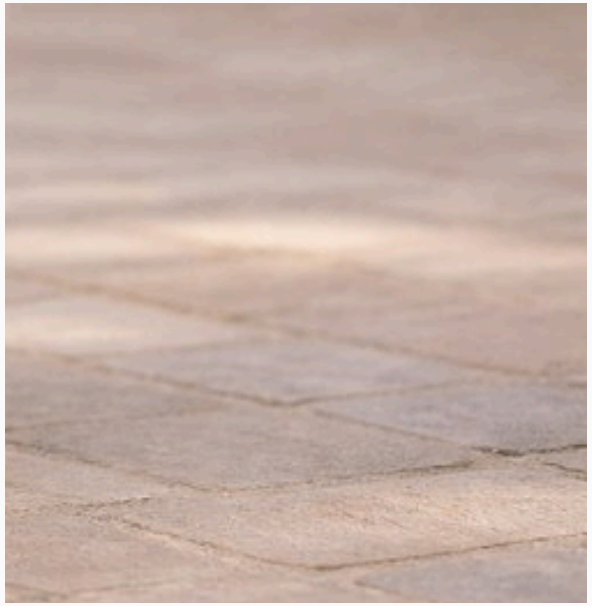
Start from a tool we have: a classifier for crops



dog



cat



nothing

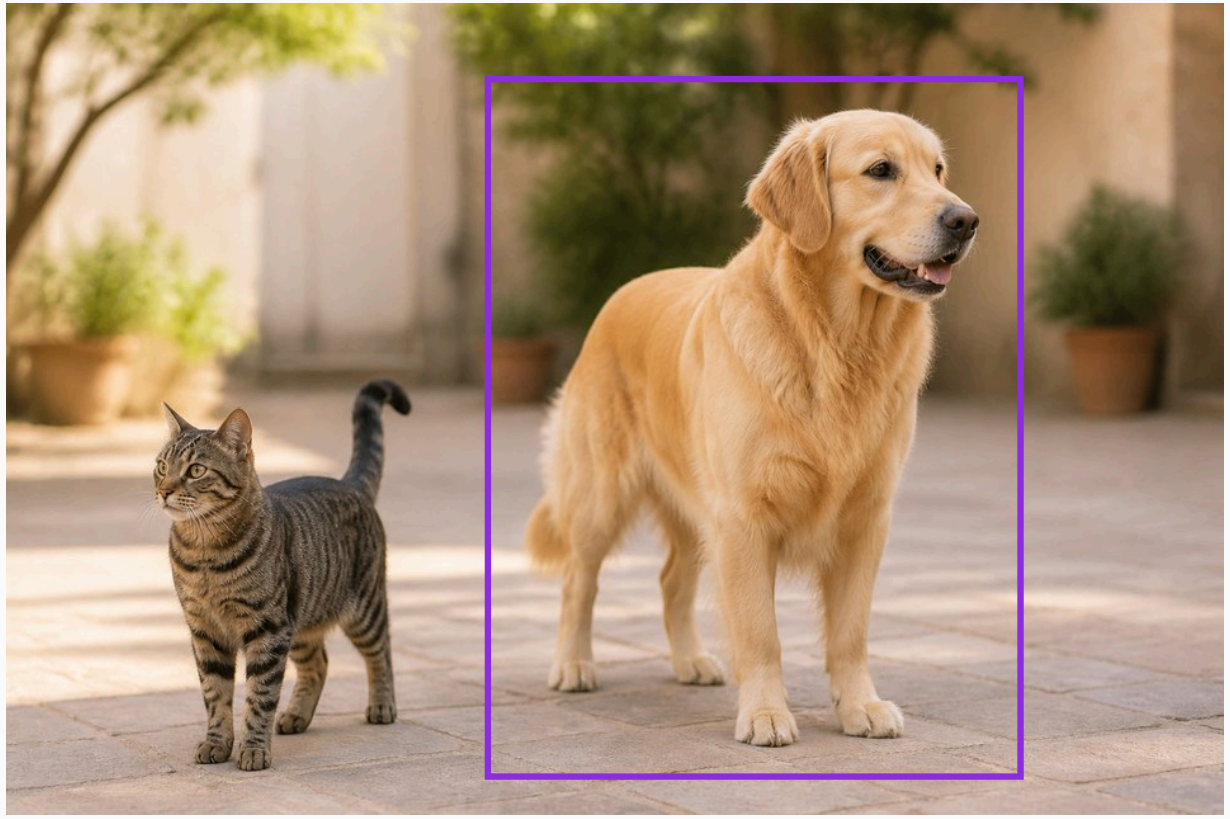
Train a classifier on many crops like these. Shown one crop, it answers: dog, cat, or nothing.

> A classifier trained on thousands of such crops answers for one crop: dog, cat or nothing. It cannot say where anything is in a whole photo. Can we use it to find objects anyway?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



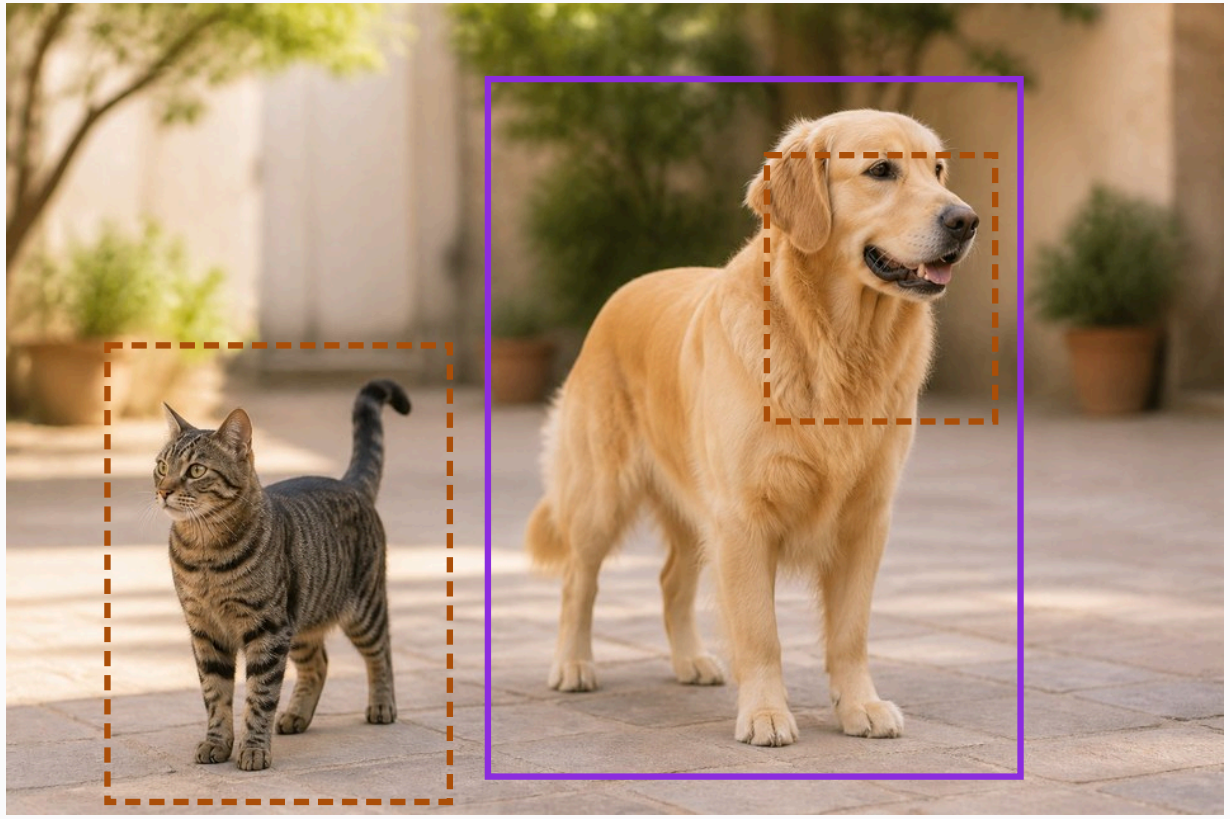
Crop a window and classify it: dog.

> Put a window on the photo, crop it, classify it. This window says “dog”.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



Crop a window and classify it: dog.

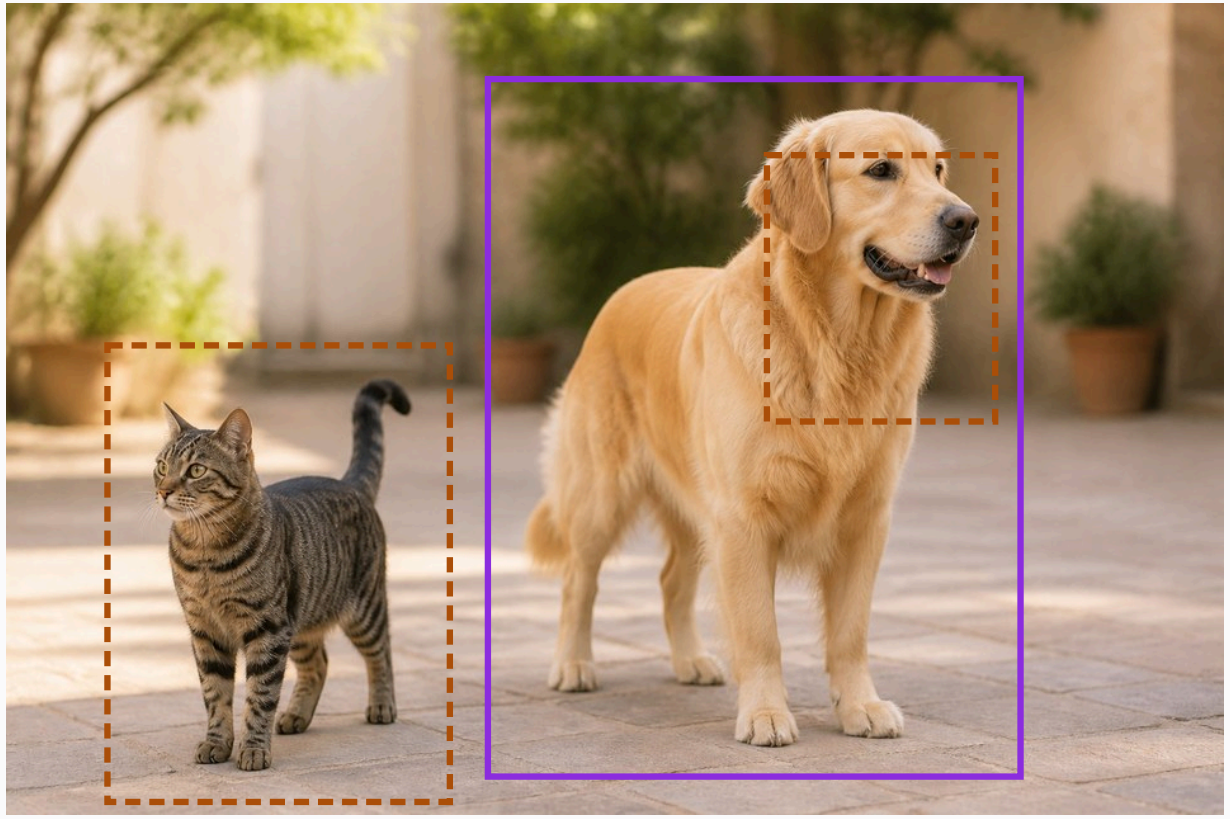
Move it every 32 px across and down: 30 × 20 = 600 positions.

> Move the window every 32 pixels: 30 positions across, 20 down, 600 windows.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



Crop a window and classify it: dog.

Move it every 32 px across and down: 30 × 20 = 600 positions.

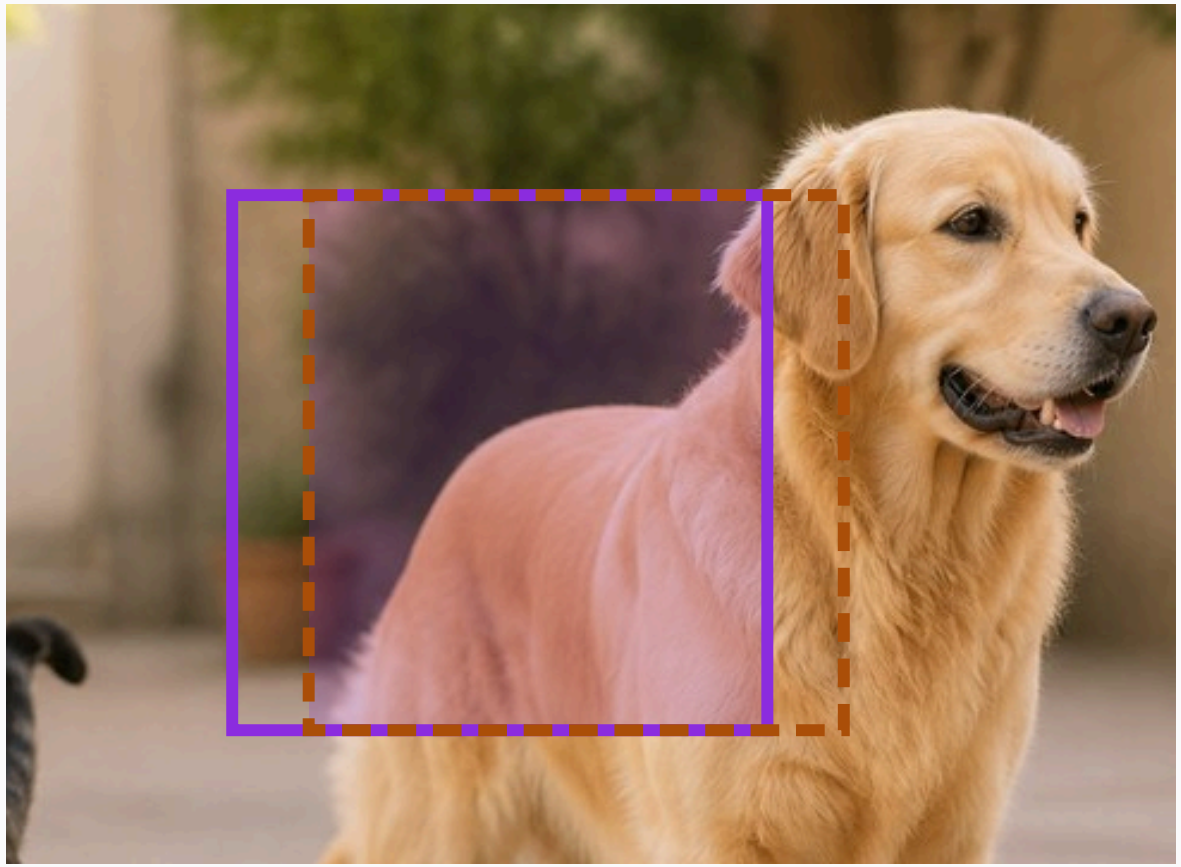
Objects come in many sizes and shapes: try 3 sizes × 3 shapes at each position. **5,400 crops**, each through a whole CNN.

> At each position try 3 sizes and 3 shapes: 5,400 crops per photo, each a full run of the classifier. It works, and it is slow.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Neighbouring windows see almost the same pixels



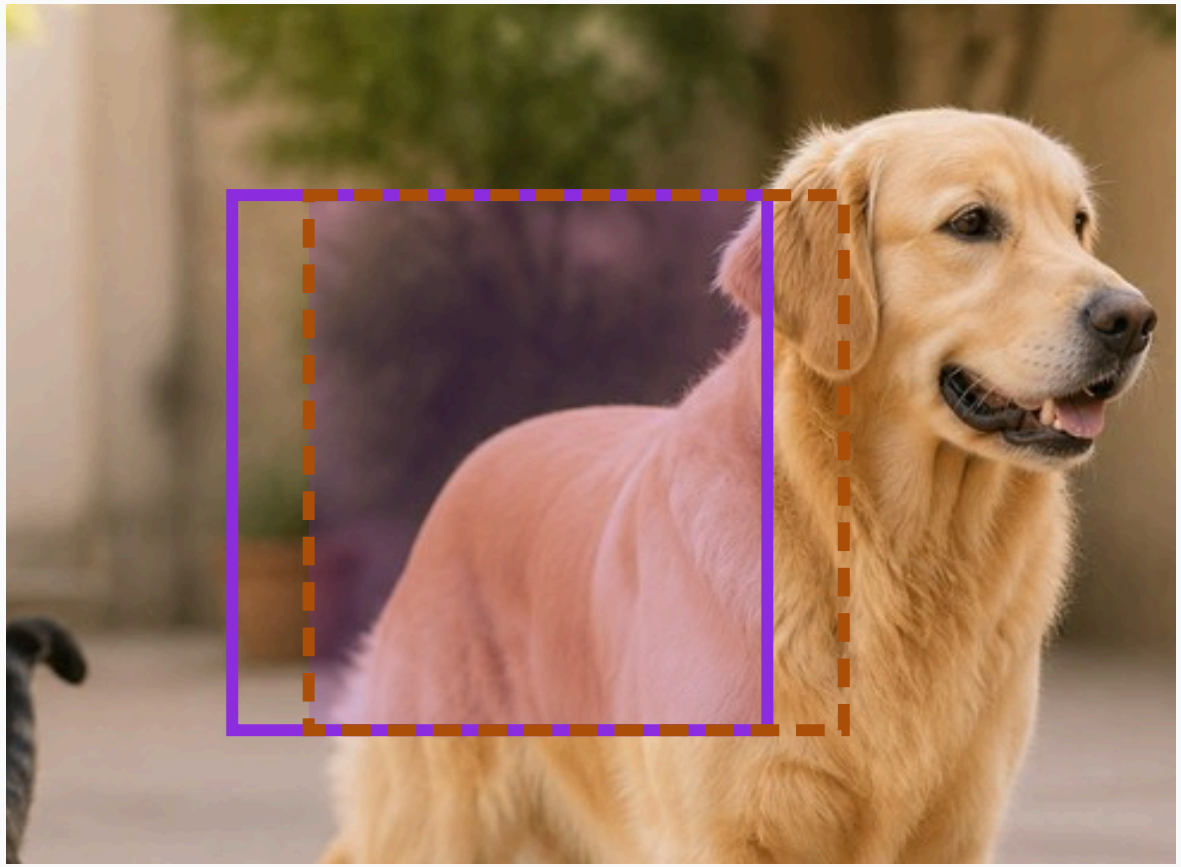
Two windows, 224 px wide, 32 px apart: they share 86% of their pixels.

Two neighbouring windows, 224 pixels wide and 32 pixels apart, share 86% of their pixels. Classifying them separately computes most features twice.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Neighbouring windows see almost the same pixels



Two windows, 224 px wide, 32 px apart: they share 86% of their pixels.

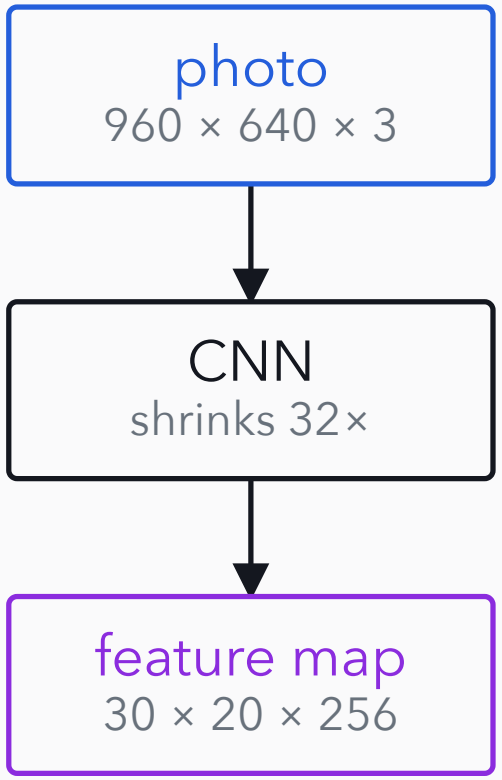
Most of the work for one window is repeated for the next. **A CNN does the sliding itself**: its convolutions already slide over every position, once.

> A convolution already slides the same small filter over every position. Run the CNN once on the whole photo and read one output per position: every window at once, with the shared work done once.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

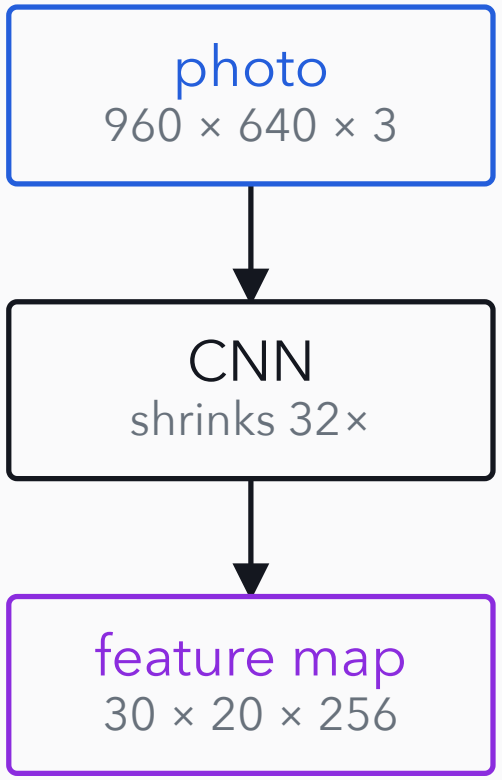
One pass of a CNN: a 30 × 20 grid of feature vectors

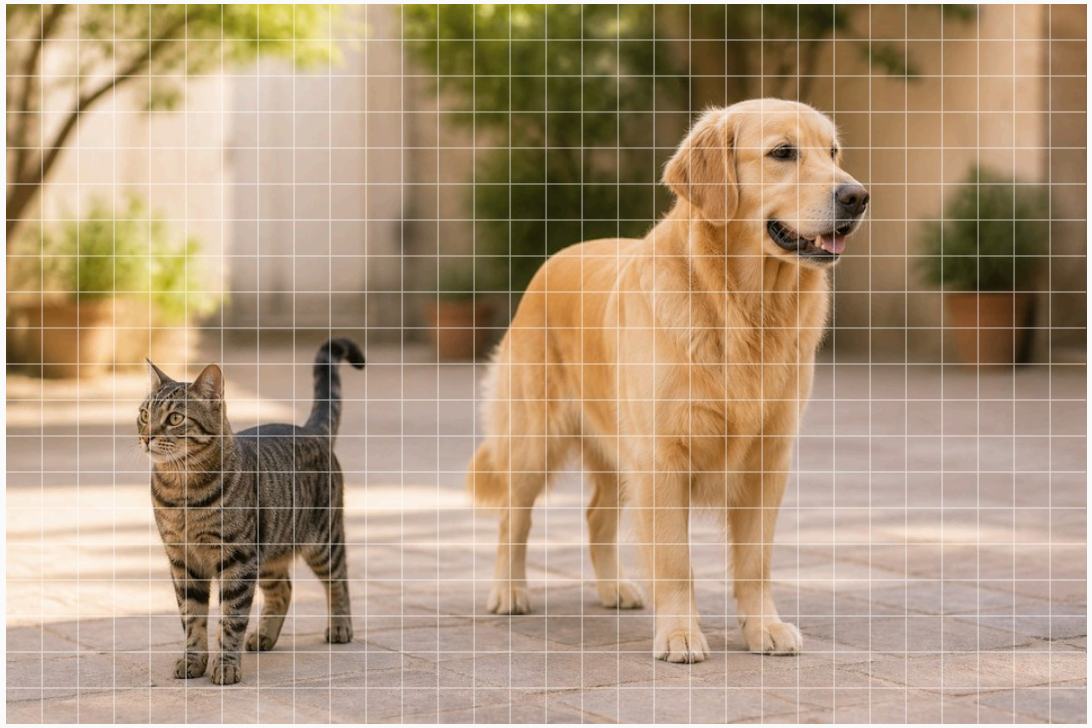


> The photo goes through a CNN that halves the resolution five times: 960 × 640 becomes 30 × 20, with 256 numbers at each position.

▶ Notebook, part 5

One pass of a CNN: a 30 × 20 grid of feature vectors

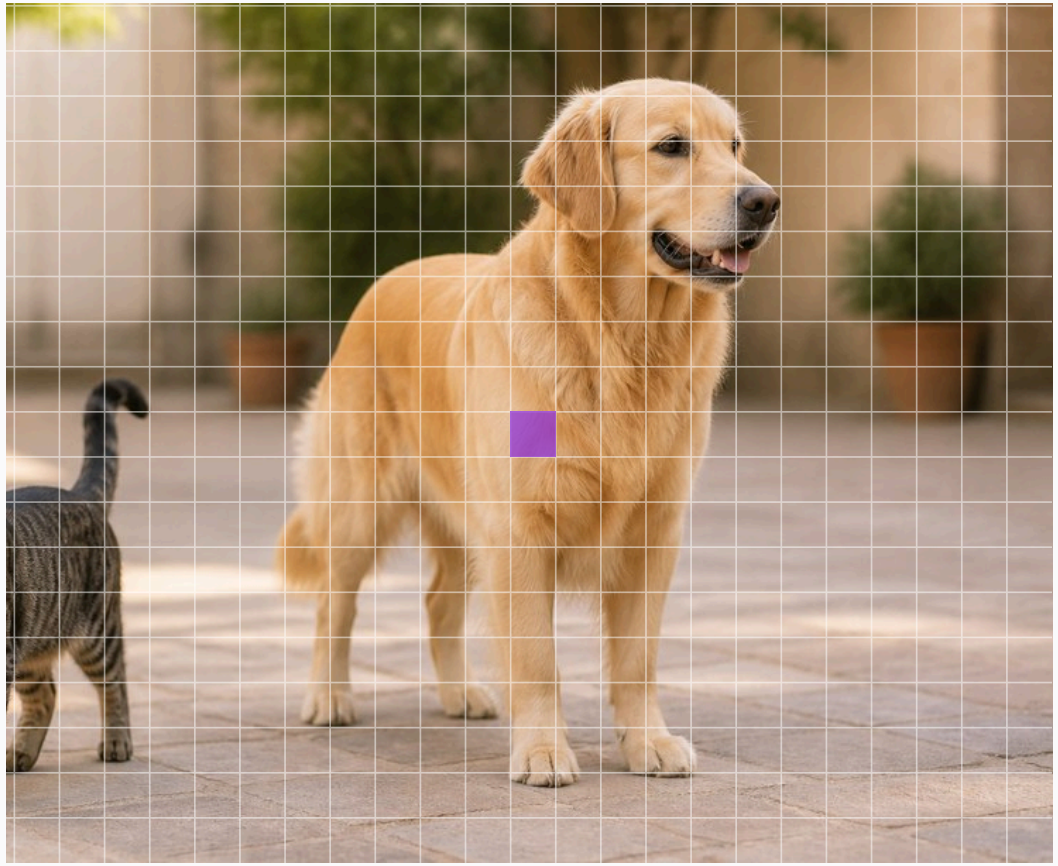




Lay that 30 × 20 grid over the photo. Each square, a cell, stands for one position of the feature map: 600 cells, 600 “windows”.

▶ Notebook, part 5

One cell: an address on the photo, with a wide view



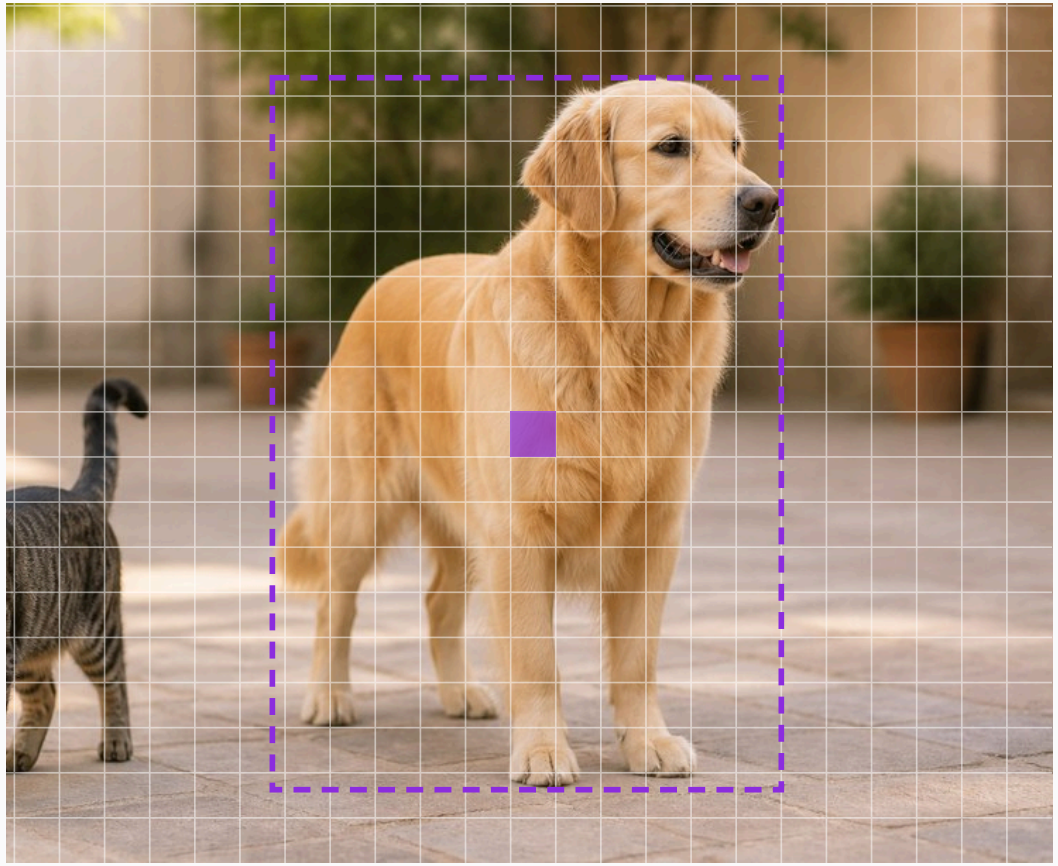
Cell (row 10, column 18) covers a 32 × 32 block: x from 576 to 608, y from 320 to 352.

> Cell (10, 18) is a 32 × 32 block of pixels in the middle of the dog: column 18 starts at x = 576, row 10 at y = 320. Rows and columns count from 0.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

One cell: an address on the photo, with a wide view



Cell (row 10, column 18) covers a 32 × 32 block: x from 576 to 608, y from 320 to 352.

Its 256 numbers were computed from a region **much larger** than its block, wide enough to take in the whole dog.

> But a cell is an address, not a crop: every layer of the CNN mixes in neighbours, so its vector sees far beyond its block. That is why one cell can describe a box much larger than itself.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Every cell writes a record: 4 box numbers, 80 class scores

4 box numbers (px)

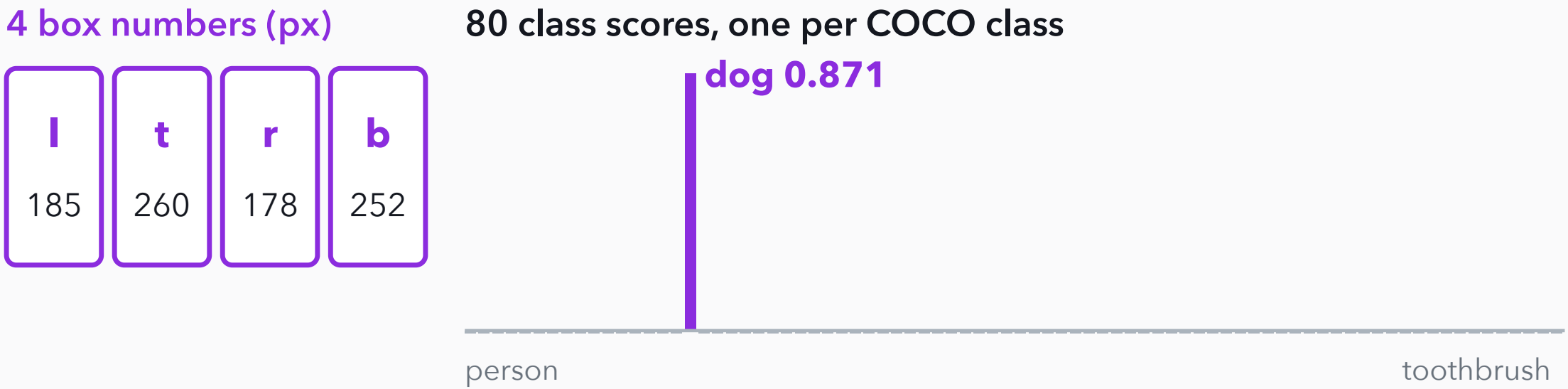
l	t	r	b
185	260	178	252

> Cell (10, 18) writes four numbers that place a box: its distances to the box's left, top, right and bottom edges.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a record: 4 box numbers, 80 class scores

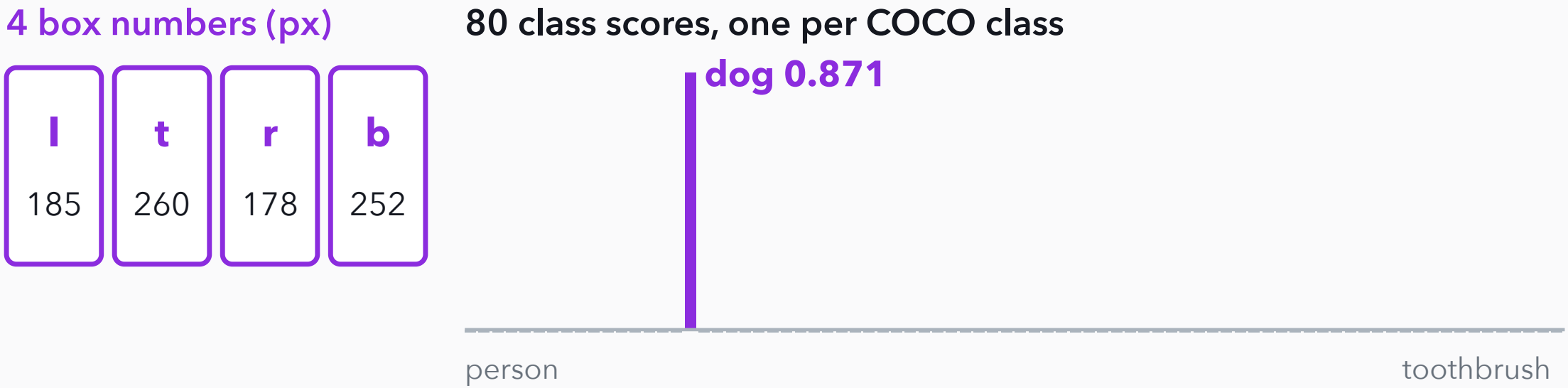


> And 80 scores, one per COCO class, each between 0 and 1. Here dog is 0.871; the other 79 are near zero.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a record: 4 box numbers, 80 class scores



There is no separate “object?” number: a cell says “nothing here” by giving all 80 classes a low score.

> Part 4’s localiser had an “object?” number. YOLOv8 drops it: 80 independent yes/no scores, all low, already mean “no object”.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

What actually comes out: one tensor of fixed shape

```
x = load_photo("cat-dog.jpg")    # (1, 3, 640, 960): RGB values in 0..1
y, _ = net(x)                     # YOLOv8n, before any clean-up
y.shape                           # torch.Size([1, 84, 12600])
```

Run the real network on the courtyard photo and print the shape: 84 rows and 12,600 columns, one column per cell.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

What actually comes out: one tensor of fixed shape

```
x = load_photo("cat-dog.jpg")    # (1, 3, 640, 960): RGB values in 0..1
y, _ = net(x)                     # YOLOv8n, before any clean-up
y.shape                           # torch.Size([1, 84, 12600])
```

```
y[0, :4, k]      # cell k's box: c_x, c_y, w, h in pixels
y[0, 4:, k]      # its 80 class scores, each between 0 and 1
```

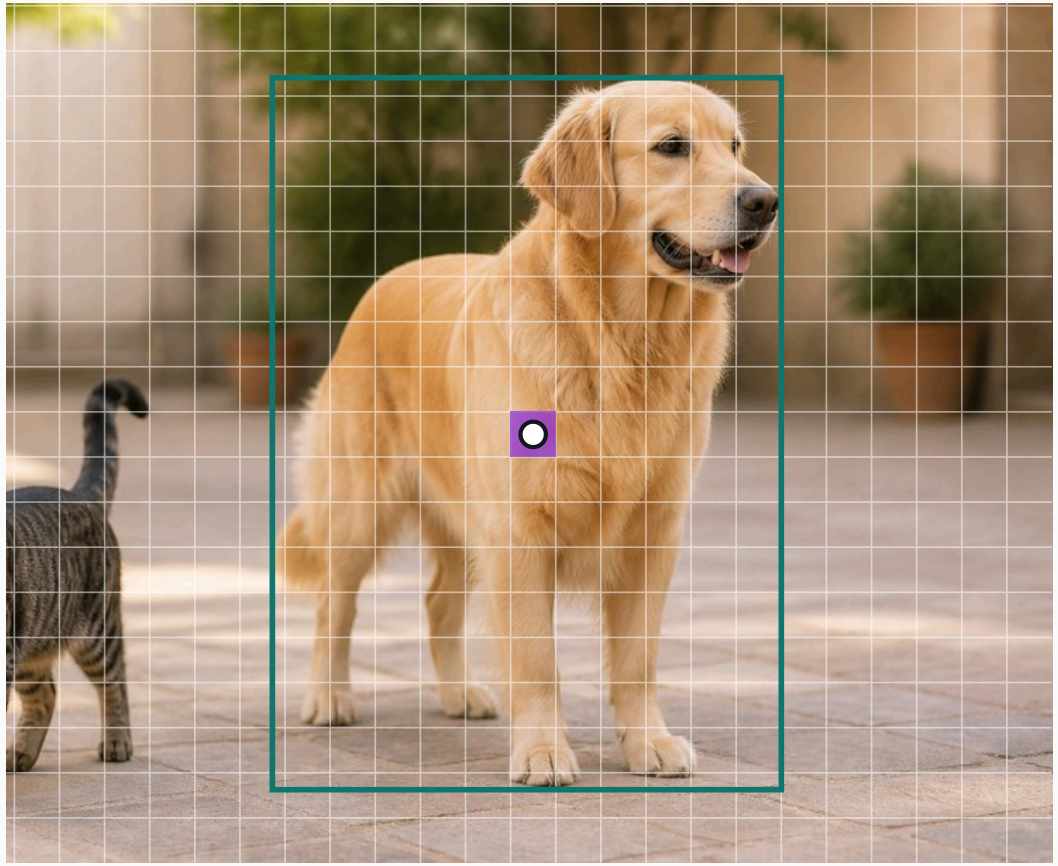
$$84 = \underbrace{4}_{\text{box}} + \underbrace{80}_{\text{classes}}, \qquad 12{,}600 = \underbrace{600}_{\text{this grid}} + \underbrace{12{,}000}_{\text{two finer grids (Part 7)}}$$

Each column is one record: 4 box numbers (the head has already turned l, t, r, b into a centre and a size) and 80 class scores. 600 columns come from our 30 × 20 grid; the rest from two finer grids, in Part 7.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



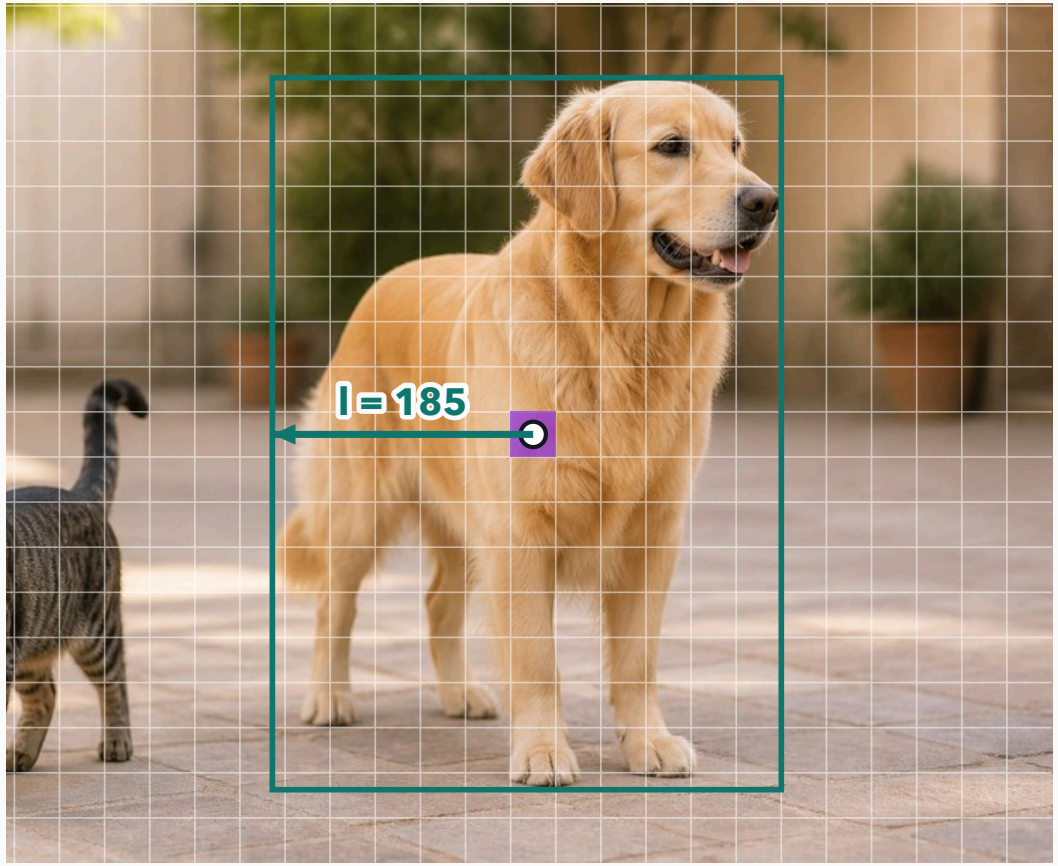
$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

> Stand at the centre of cell (10, 18): column 18 and row 10, plus a half, times 32 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

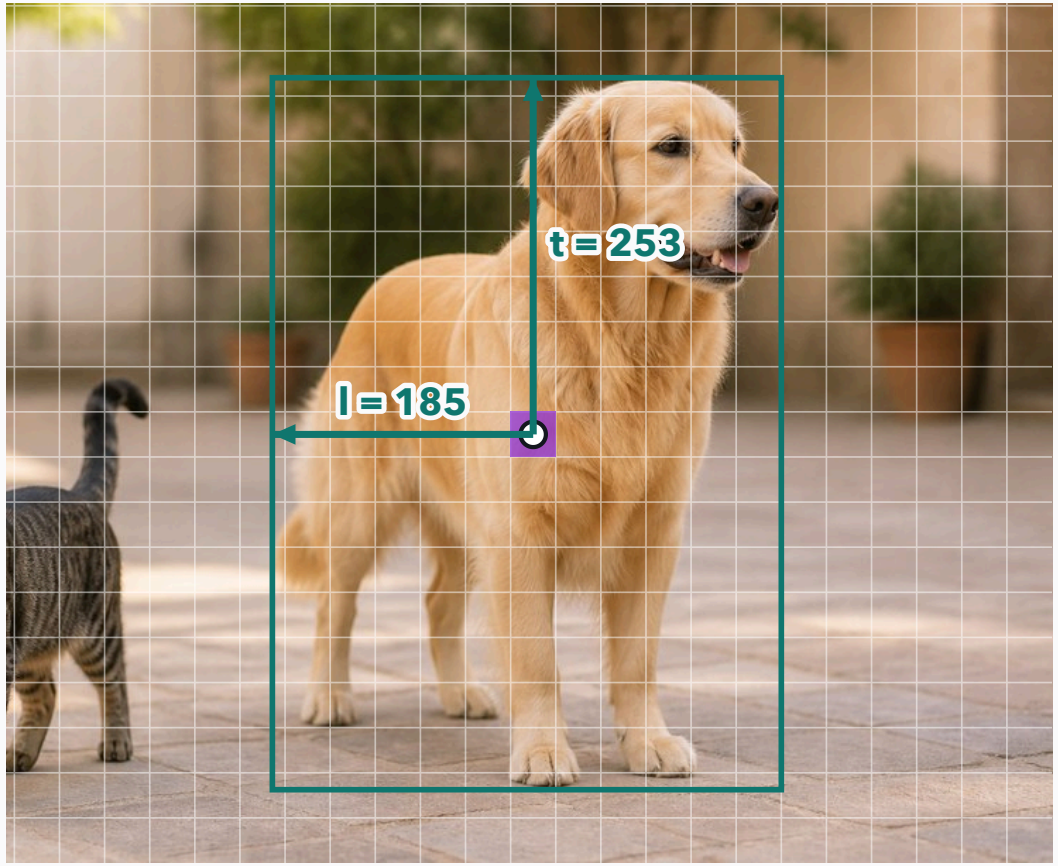
$$l = 592 - 407 = 185$$

Left: how far to the box's left edge. 592 − 407 = 185 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

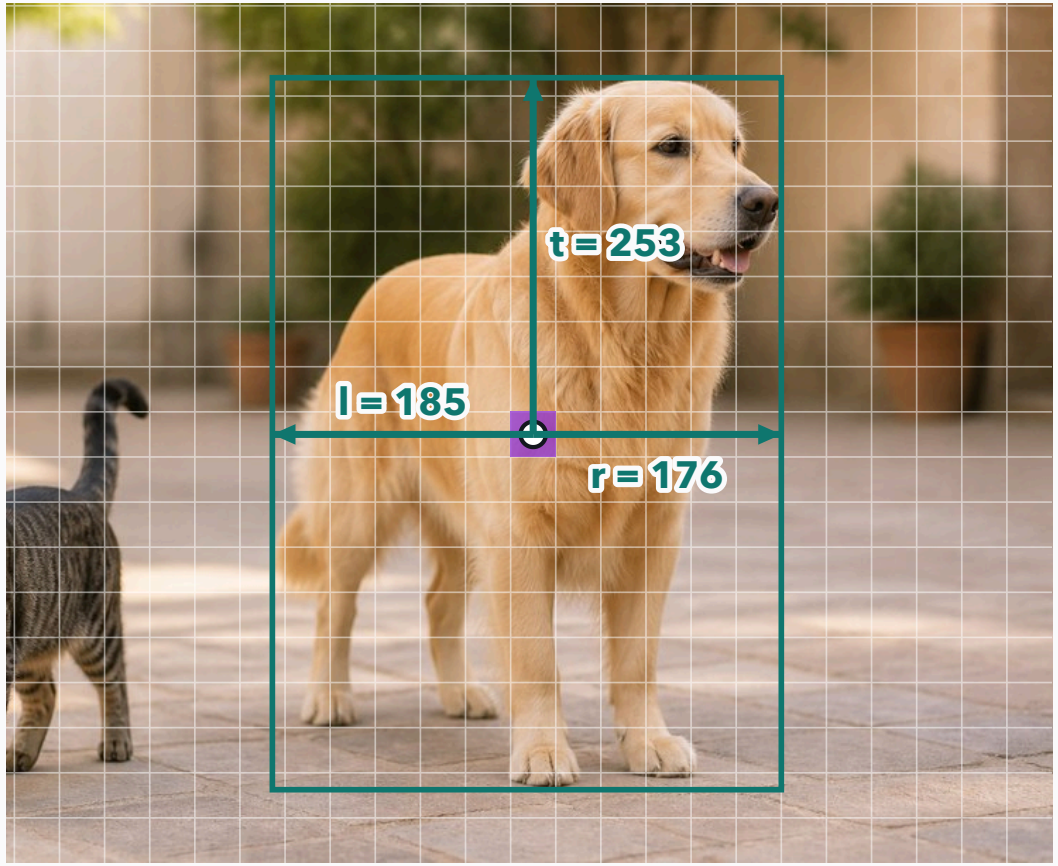
$$t = 336 - 83 = 253$$

> Top: 336 − 83 = 253.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

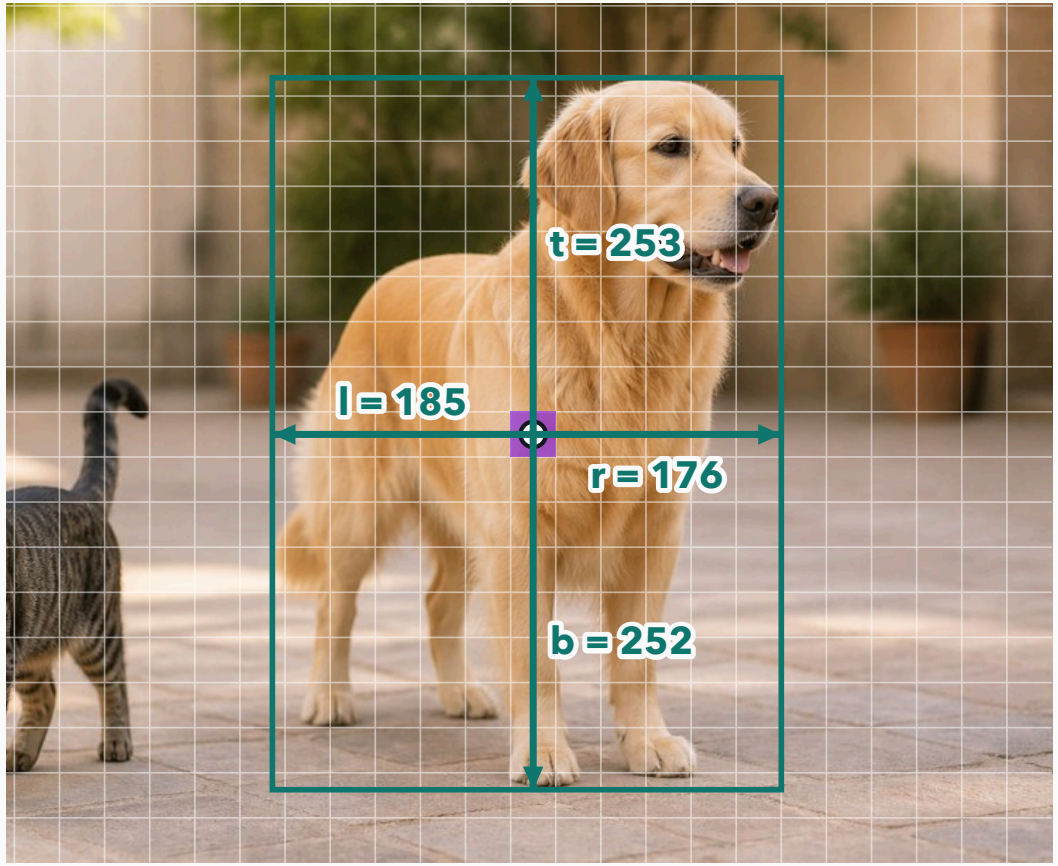
$$r = 768 - 592 = 176$$

> Right: 768 − 592 = 176.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

$$r = 768 - 592 = 176$$

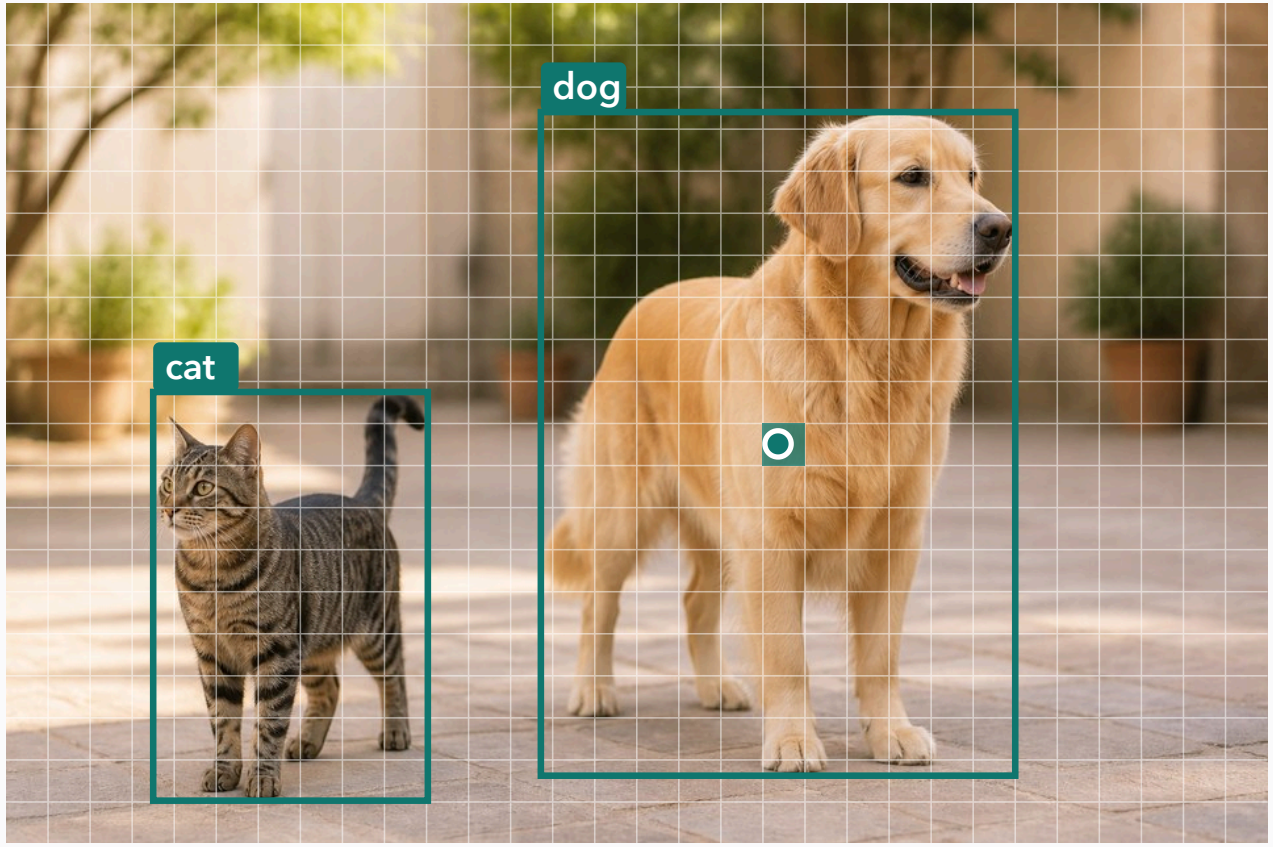
$$b = 588 - 336 = 252$$

> Bottom: 588 − 336 = 252. Like standing in a room and measuring the distance to each wall: four numbers fix the box.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The simplest rule: the cell that holds the centre



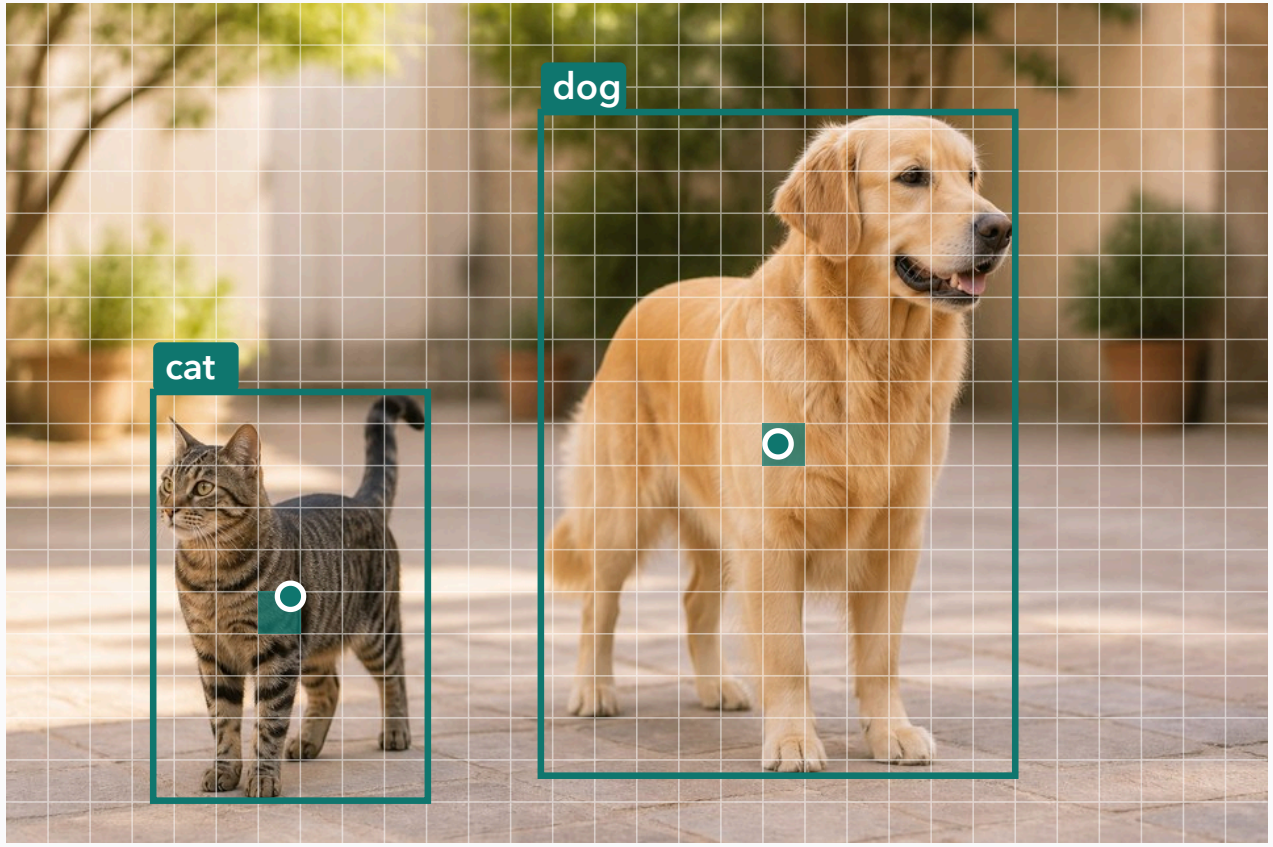
The dog's box centre, (587.5, 335.5), falls in cell (10, 18). That cell is **responsible** for the dog.

> One rule: the cell that contains the box's centre answers for the object. The dog's centre (587.5, 335.5) is in cell (10, 18), our cell.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The simplest rule: the cell that holds the centre



The dog's box centre, (587.5, 335.5), falls in cell (10, 18). That cell is **responsible** for the dog.

The cat's centre, (216.5, 451.5), falls in cell (14, 6).

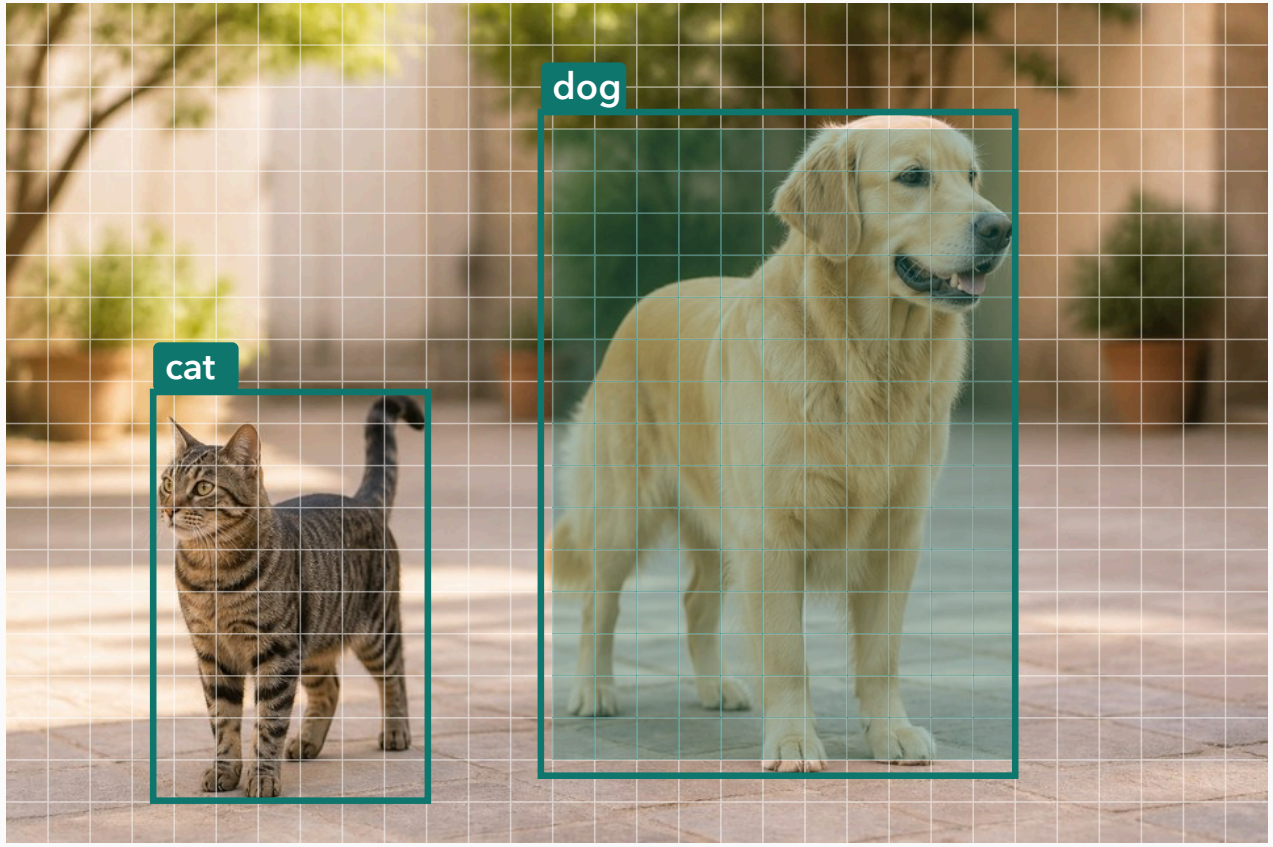
Two cells learn an object; the other 598 learn "nothing here".

> The cat's centre is in cell (14, 6). Every object now has exactly one cell, decided by place alone: no slot order to argue about.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

More cells per object: a block around the centre



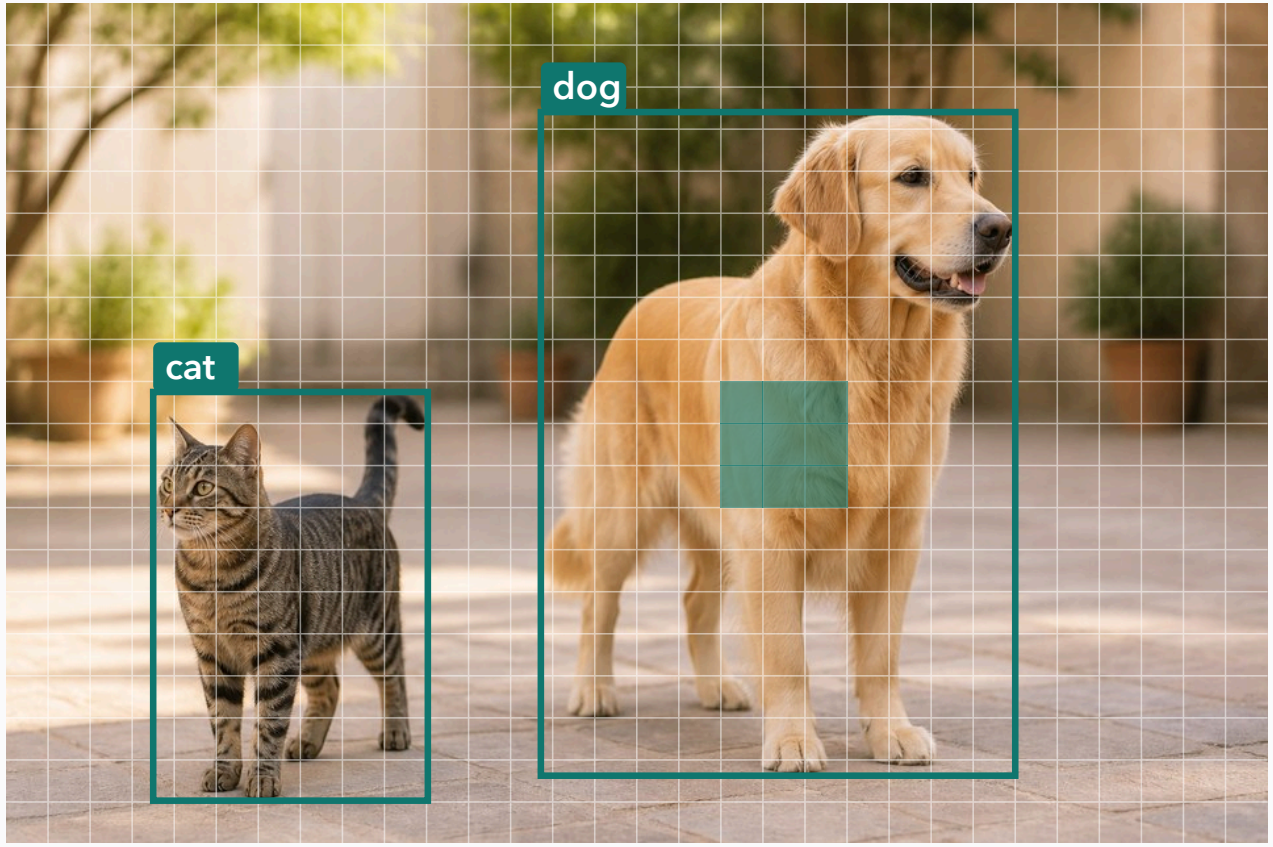
One cell per object gives little to learn from. Use every cell inside the dog's box? That is 165 cells, many of them mostly floor.

> One positive cell per object is a weak training signal. All 165 cells inside the box would include cells near its edges that see mostly floor: a noisy target.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

More cells per object: a block around the centre



One cell per object gives little to learn from. Use every cell inside the dog's box? That is 165 cells, many of them mostly floor.

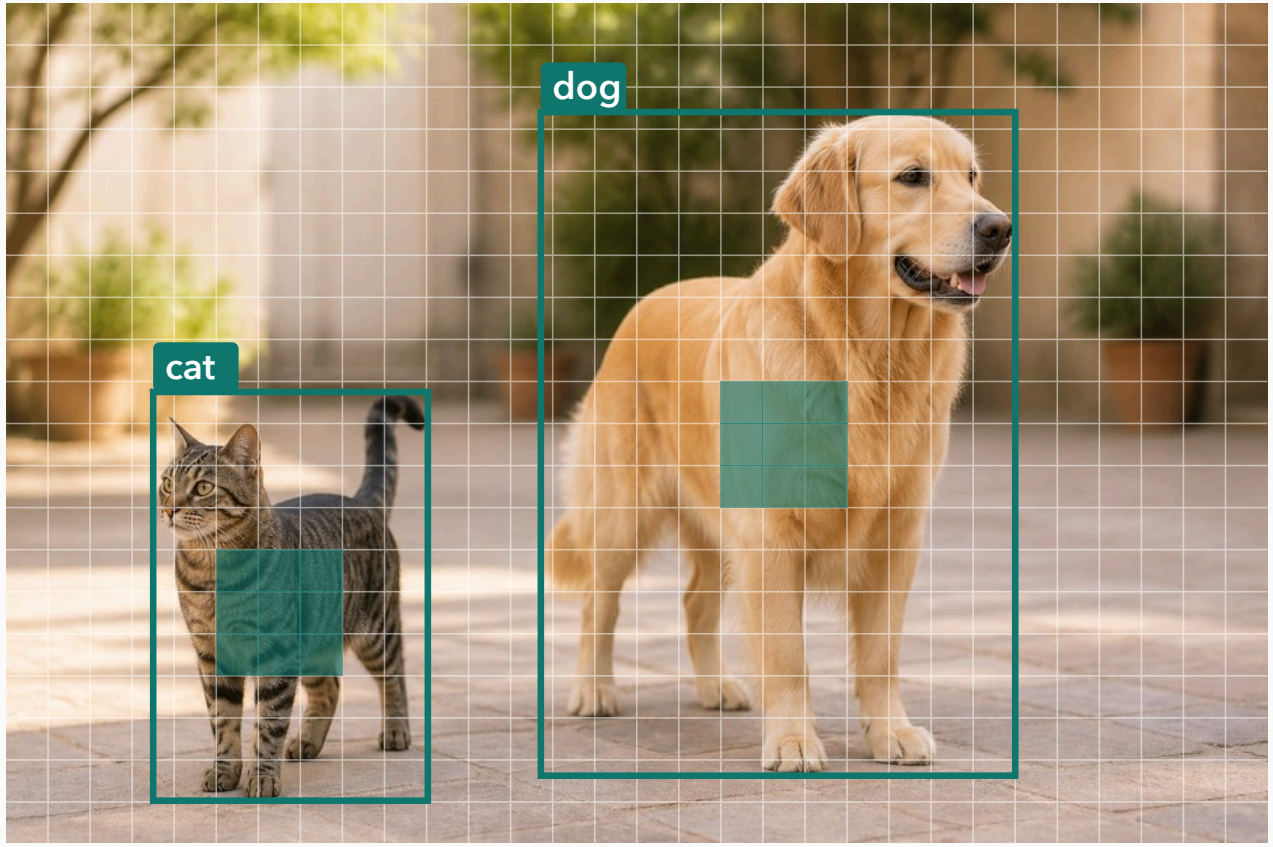
A middle way, FCOS's "centre sampling": the cells within 1.5 cells of the box's centre. For the dog, rows 9–11 and columns 17–19.

> The middle way: the cells near the centre, within 1.5 cells of it, a 3 × 3 block around cell (10, 18).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

More cells per object: a block around the centre



One cell per object gives little to learn from. Use every cell inside the dog's box? That is 165 cells, many of them mostly floor.

A middle way, FCOS's "centre sampling": the cells within 1.5 cells of the box's centre. For the dog, rows 9–11 and columns 17–19.

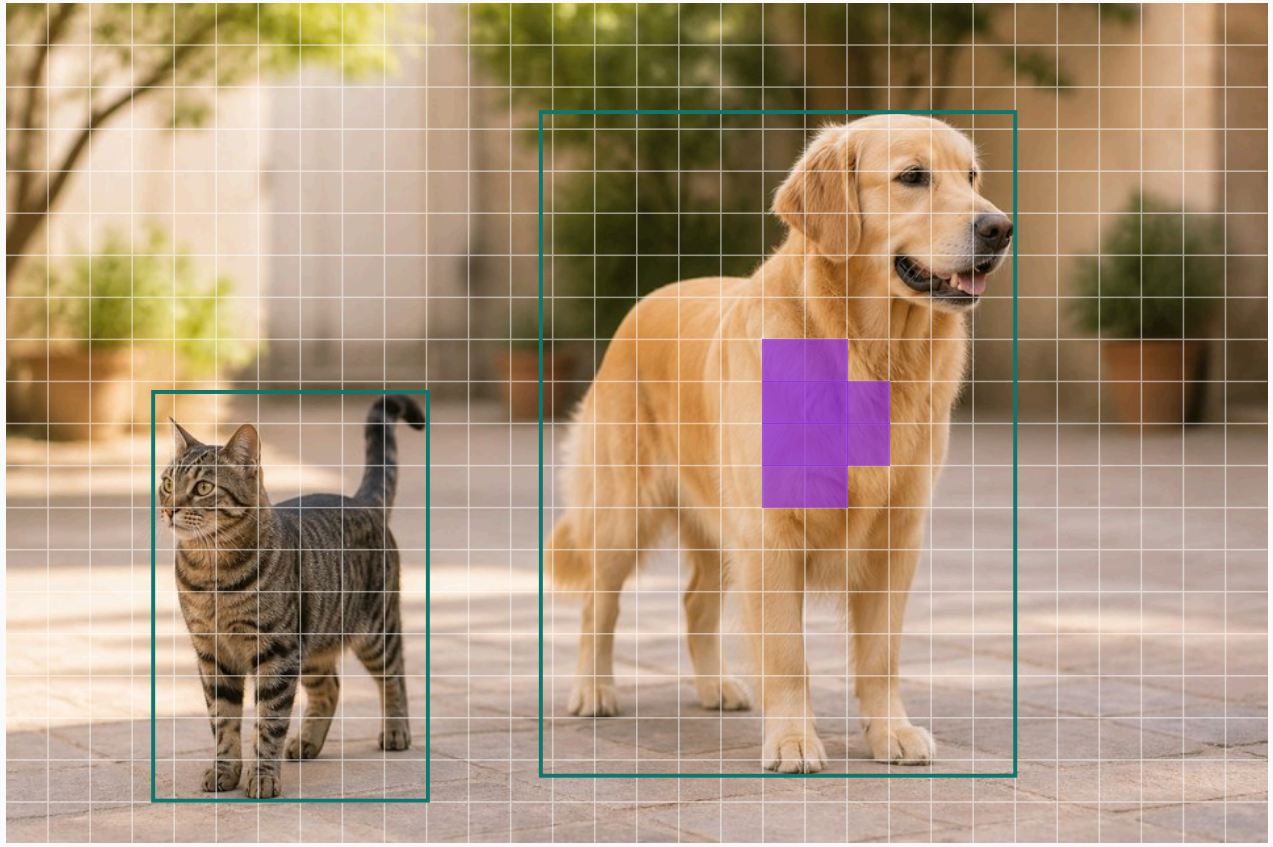
Same for the cat: rows 13–15, columns 5–7. These 18 cells are **positives**; the other 582 are **background**.

> Do the same for the cat. 18 cells get an object target; 582 get "no object of any class".

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The trained YOLOv8n: about ten cells fire per object



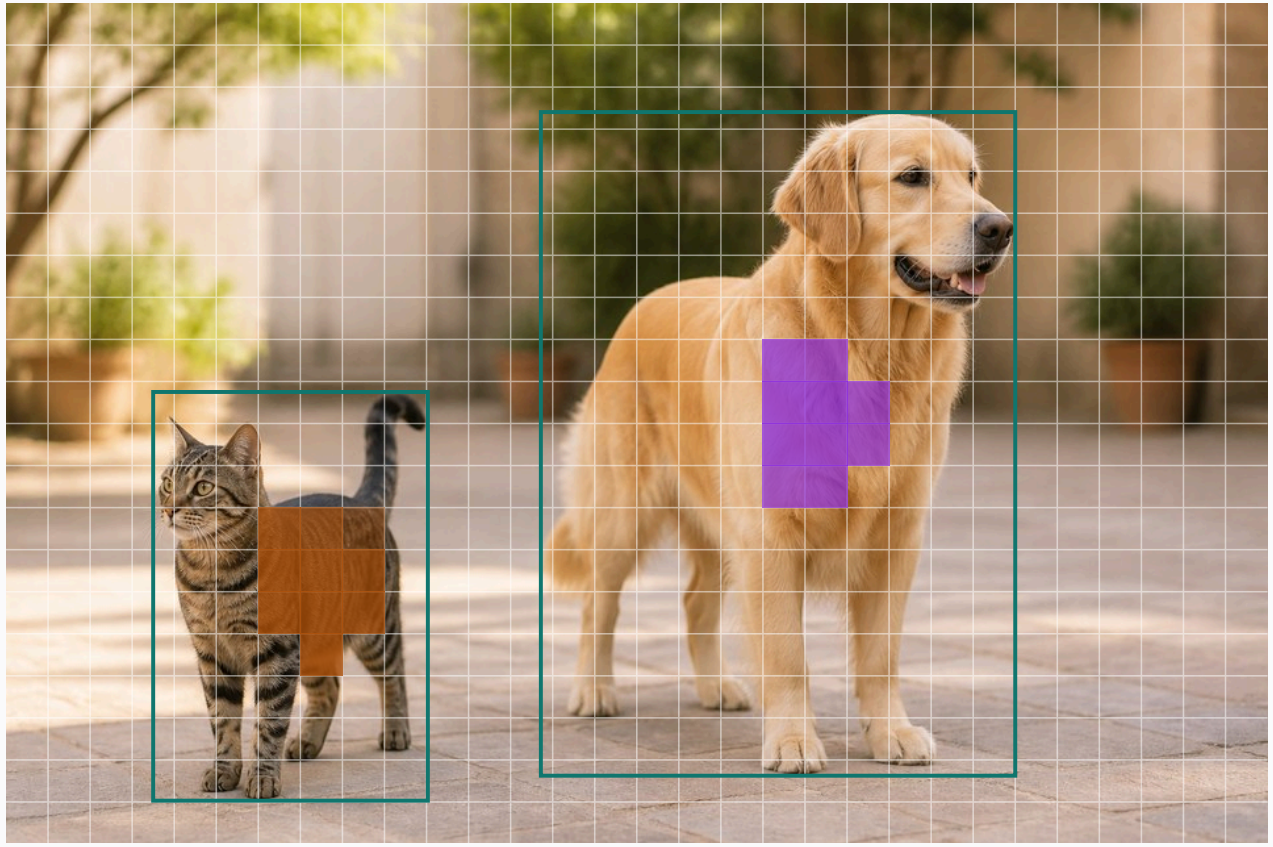
Shade = the cell's dog score. 10 cells score above 0.25, all near the dog's centre; 6 of them are in our 3 × 3 block.

> Each cell's dog score from the real network: dark where it is high. A small cluster near the dog's centre, nothing elsewhere.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

The trained YOLOv8n: about ten cells fire per object



Shade = the cell's dog score. 10 cells score above 0.25, all near the dog's centre; 6 of them are in our 3 × 3 block.

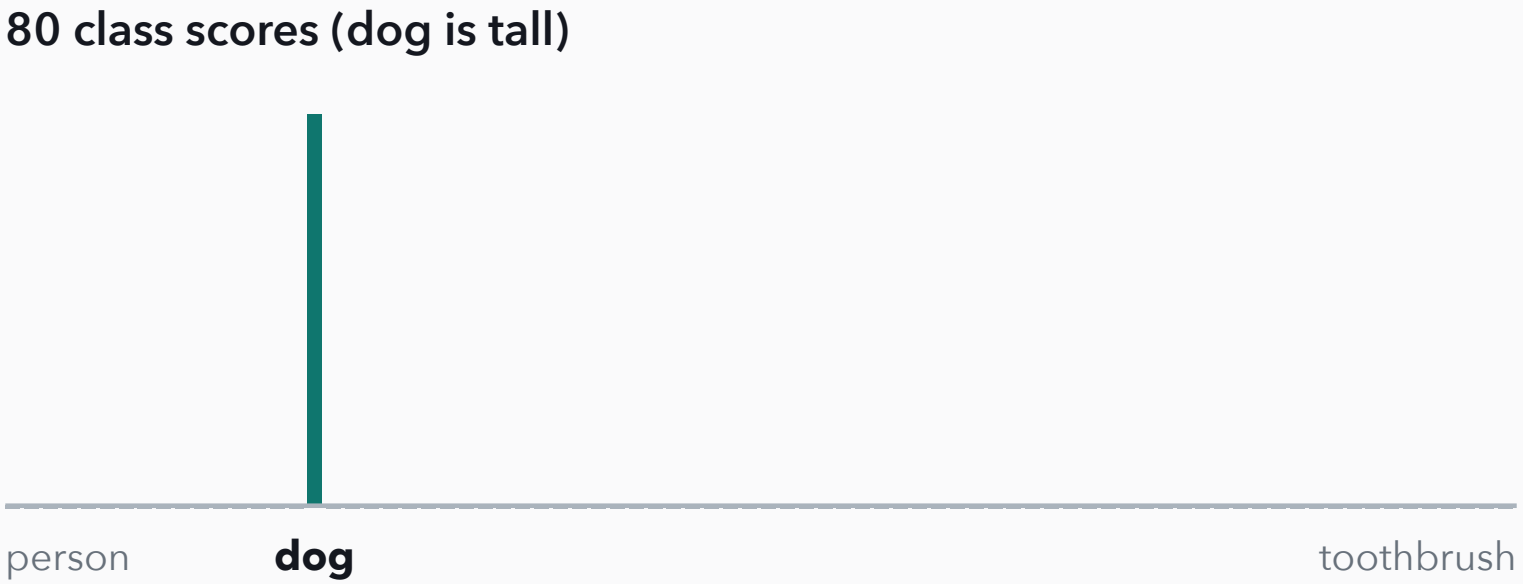
Cat scores: again 10 cells. YOLOv8 trains 10 positive cells per object, and the network learned to fire in about that many.

> The cat's scores make a second cluster. About ten cells per object: exactly what YOLOv8's training asked for, and the source of the duplicates we must remove later.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

The target at a positive cell, (10, 18)



Class targets: **1 for dog**, 0 for each of the other 79 classes. Each is the answer to its own yes/no question.

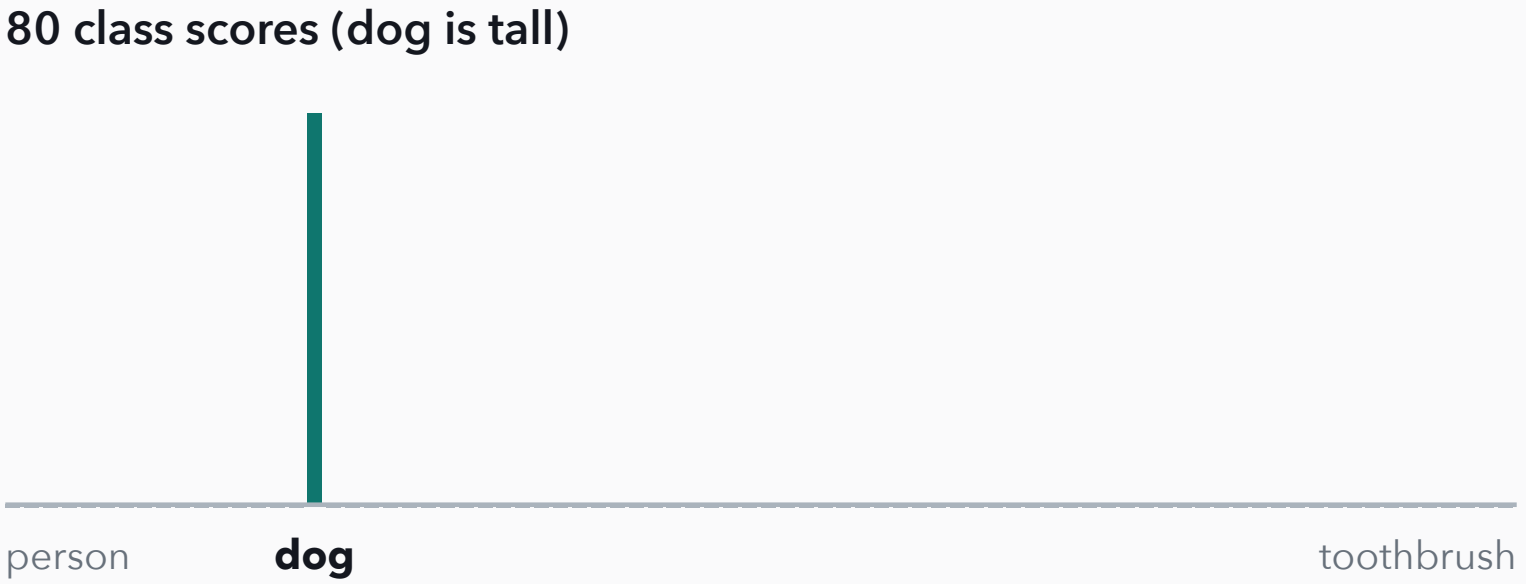
> A positive cell's class targets: 1 for dog, 0 for the other 79 classes. Each is the answer to its own yes/no question.

▶ Notebook, part 5

The target at a positive cell, (10, 18)

box target (px)

l	t	r	b
185	253	176	252

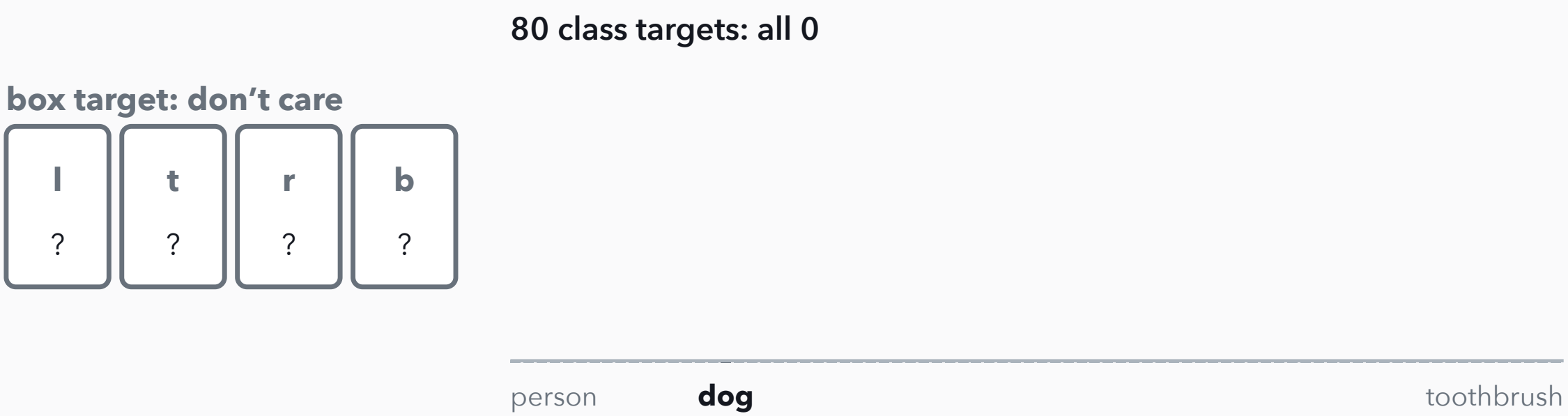


Box target: the four distances, (185, 253, 176, 252) px; in cells (÷ 32), (5.78, 7.91, 5.50, 7.88). A box much larger than a cell is fine.

> Its box target: the four distances we just measured, (185, 253, 176, 252).

▶ Notebook, part 5

The target at a background cell: all zeros, and “?”



Cell (3, 3), on the wall: all 80 class targets are 0, and its box target is “?”, like the empty floor of Part 4.

> A background cell, such as (3, 3) on the wall: all 80 class targets are 0. Its box target is “?”: there is no box to learn, so its box numbers are left out of the loss.

▶ Notebook, part 5

What the network wrote at cell (10, 18)



Dog: **0.871** (target 1). Cat 0.0001, skateboard 0.00005; the other 77 are smaller still (targets 0).

> The real network's scores at this cell: dog 0.871 against a target of 1; every other class near its target of 0.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

What the network wrote at cell (10, 18)

target

l	t	r	b
185	253	176	252

prediction

l	t	r	b
184.8	260.4	177.6	252.1



Distances: close to the target, the top edge furthest off. The loss turns these differences into one number to reduce.

> Its four distances: (184.8, 260.4, 177.6, 252.1) against the target (185, 253, 176, 252). Close; the loss measures how close.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

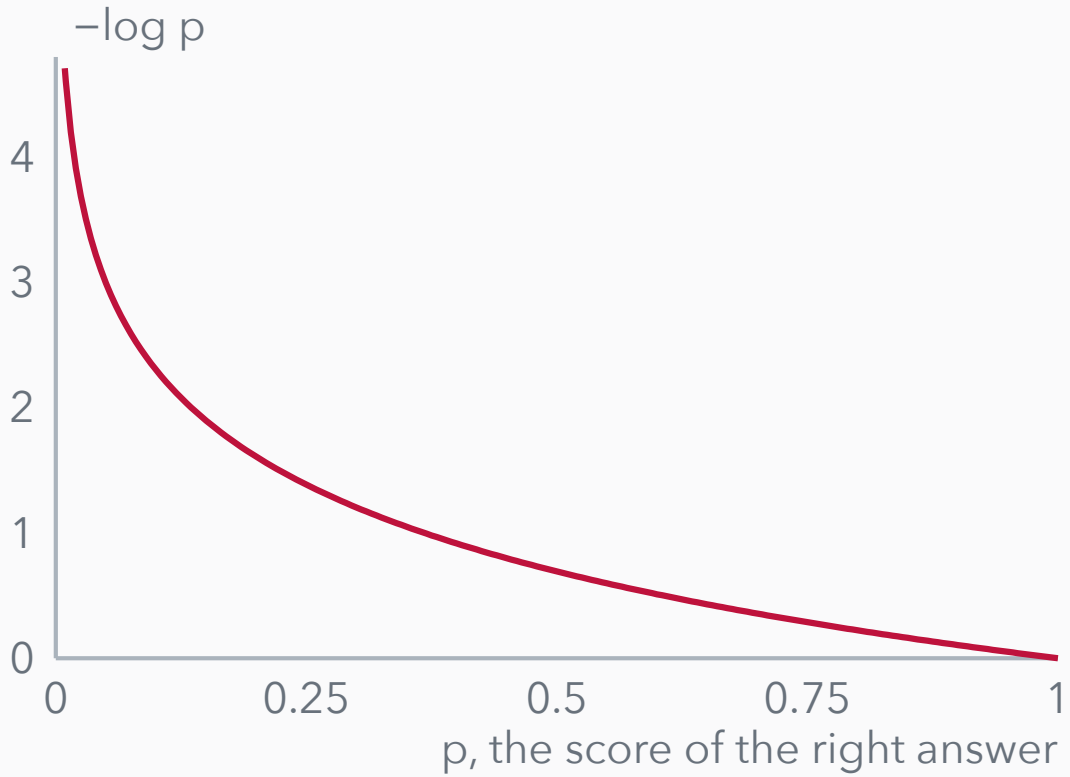
▶ Notebook, part 5

Scoring one yes/no answer: binary cross-entropy

$$\text{yes } (y = 1): \quad -\log p$$

$$\text{no } (y = 0): \quad -\log(1 - p)$$

p: the score the network gave this class; y: the target, 1 or 0. Natural logarithms.



A class score answers one yes/no question. If the answer is yes, the loss is −log p; if no, −log(1 − p), the same curve mirrored.

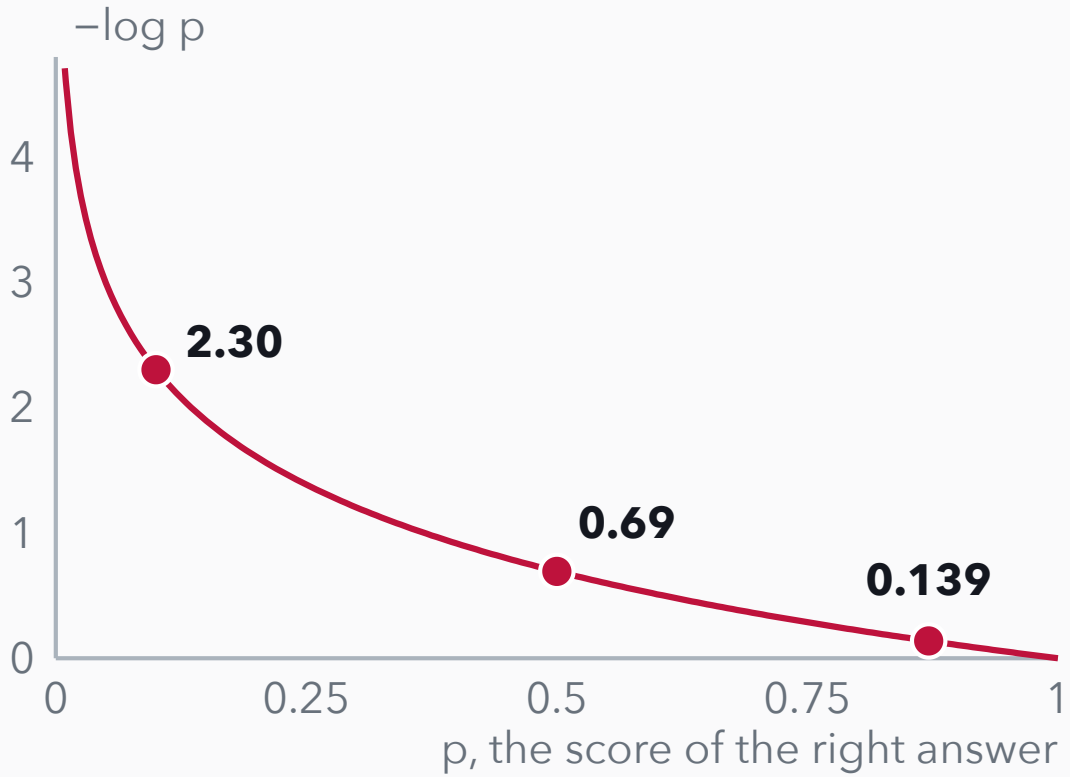
▶ Notebook, part 5

Scoring one yes/no answer: binary cross-entropy

$$\text{yes } (y = 1): \quad -\log p$$

$$\text{no } (y = 0): \quad -\log(1 - p)$$

p: the score the network gave this class; y: the target, 1 or 0. Natural logarithms.



Read the curve for a yes: sure and right (p near 1) costs almost nothing; unsure (0.5) costs 0.69; sure and wrong (0.1) costs 2.30, and it grows without bound. Our dog cell: 0.871 costs 0.139.

▶ Notebook, part 5

Class loss at cell (10, 18): 80 yes/no questions

$$\text{dog: } y = 1,\ p = 0.871: \quad -\log 0.871 = 0.139$$

Dog is a yes: −log 0.871 = 0.139.

▶ Notebook, part 5

Class loss at cell (10, 18): 80 yes/no questions

$$\text{dog: } y = 1,\ p = 0.871: \quad -\log 0.871 = 0.139$$

$$\text{the other 79: } y = 0: \quad \sum_c -\log(1 - p_c) = 0.0004$$

Every other class is a no: −log(1 − p) for each. All 79 scores are near zero, so together they add only 0.0004.

▶ Notebook, part 5

Class loss at cell (10, 18): 80 yes/no questions

$$\text{dog: } y = 1,\ p = 0.871: \quad -\log 0.871 = 0.139$$

$$\text{the other 79: } y = 0: \quad \sum_c -\log(1 - p_c) = 0.0004$$

$$\mathcal{L}_{\text{class}}(10, 18) = 0.139 + 0.0004 = 0.139$$

The cell's class loss: 0.139, almost all from the dog question.

▶ Notebook, part 5

Box loss: how far off are the four distances?

$$\mathcal{L}_{\text{box}} = \sum_{k \in \{l,t,r,b\}} \left| \hat{d}_k - d_k \right|$$

> Compare the four predicted distances with the four target distances, and add the absolute differences: the L1 loss of Part 4.

▶ Notebook, part 5

Box loss: how far off are the four distances?

$$\mathcal{L}_{\text{box}} = \sum_{k \in \{l,t,r,b\}} |\hat{d}_k - d_k|$$

$$= |184.8 - 185| + |260.4 - 253| + |177.6 - 176| + |252.1 - 252|$$

$$= 0.2 + 7.4 + 1.6 + 0.1 = 9.3 \text{ px}$$

> 0.2 + 7.4 + 1.6 + 0.1 = 9.3 pixels. Most of it is the top edge: the predicted box starts 7.4 pixels higher than ours.

▶ Notebook, part 5

A box loss that does not care about size: 1 − IoU

$$1 - \mathrm{IoU}(\text{predicted box}, \text{target box}) = 1 - 0.980 = 0.020$$

9.3 px is nothing for the dog, and a miss for Part 1's 13-px pedestrian. IoU, a fraction of area, scores both on one scale: Part 3's argument again.

YOLOv8 uses an IoU-based box loss (CIoU), plus a loss on how it spreads each distance (DFL).

> Another choice: one minus the IoU of the predicted and target boxes, here 0.020. It does not depend on the box's size, so small objects count as much as large ones.

▶ Notebook, part 5

A background cell: only a class loss, and it is small

$$\text{cell } (3,3): \quad \sum_{c=1}^{80} -\log(1-p_c) = 0.0038 \qquad (\text{largest } p_c = 0.001,\ \text{potted plant})$$

A background cell has only class targets, all 0. Here every score is tiny, so the loss is 0.0038. No box loss: there is no box to learn.

▶ Notebook, part 5

A background cell: only a class loss, and it is small

$$\text{cell } (3,3): \quad \sum_{c=1}^{80} -\log(1-p_c) = 0.0038 \qquad (\text{largest } p_c = 0.001,\ \text{potted plant})$$

But there are 582 background cells on this grid and only 18 positives. Tiny terms, many of them: detectors weight them down. YOLOv1 multiplied them by 0.5; RetinaNet's focal loss shrinks the easy ones (optional slide).

> The imbalance matters: hundreds of easy background terms against a handful of positives. Without care they dominate the sum and the network learns to say "nothing" everywhere.

▶ Notebook, part 5

The loss for the whole photo

$$\mathcal{L} = \frac{1}{N_{\text{pos}}} \Bigg[\underbrace{\sum_{\text{all cells}} \sum_{c=1}^{80} \text{BCE}(p_c, y_c)}_{\text{class: every cell}} + \lambda \underbrace{\sum_{\text{positive cells}} \mathcal{L}_{\text{box}}}_{\text{box: positives only}} \Bigg]$$

> Class loss from every cell; box loss only from positive cells; divide by the number of positives so photos with many objects do not dominate. λ balances the two parts.

▶ Notebook, part 5

The loss for the whole photo

$$\mathcal{L} = \frac{1}{N_{\text{pos}}} \Bigg[\underbrace{\sum_{\text{all cells}} \sum_{c=1}^{80} \text{BCE}(p_c, y_c)}_{\text{class: every cell}} + \lambda \underbrace{\sum_{\text{positive cells}} \mathcal{L}_{\text{box}}}_{\text{box: positives only}} \Bigg]$$

Here: 600 cells contribute class terms; the 18 positives also contribute box terms; divide by N_{pos} = 18.

> For our photo: 600 class terms, 18 box terms.

▶ Notebook, part 5

The loss for the whole photo

$$\mathcal{L} = \frac{1}{N_{\text{pos}}} \Bigg[\underbrace{\sum_{\text{all cells}} \sum_{c=1}^{80} \text{BCE}(p_c, y_c)}_{\text{class: every cell}} + \lambda \underbrace{\sum_{\text{positive cells}} \mathcal{L}_{\text{box}}}_{\text{box: positives only}} \Bigg]$$

Here: 600 cells contribute class terms; the 18 positives also contribute box terms; divide by N_{pos} = 18.

One number for the photo. Backpropagation sends its gradient into the shared head and the CNN; every cell's weights are the same weights, so every cell learns from every object.

> Gradient descent on this one number trains the shared layers.

▶ Notebook, part 5

The same loss in code

```
logits = head_cls(feats)                          # (600, 80): one row per cell
dist   = head_box(feats)                          # (600, 4): l, t, r, b in cells
y = torch.zeros(600, 80)
y[pos, labels] = 1.0                              # positives: 1 for their class
cls_loss = F.binary_cross_entropy_with_logits(logits, y, reduction='sum')
box_loss = (dist[pos] - dist_target[pos]).abs().sum()
loss = (cls_loss + box_loss) / len(pos)
```

> The target is a 600 × 80 table of zeros with a 1 at each positive cell's class. Class loss over everything, box loss over the positives.

▶ Notebook, part 5

The same loss in code

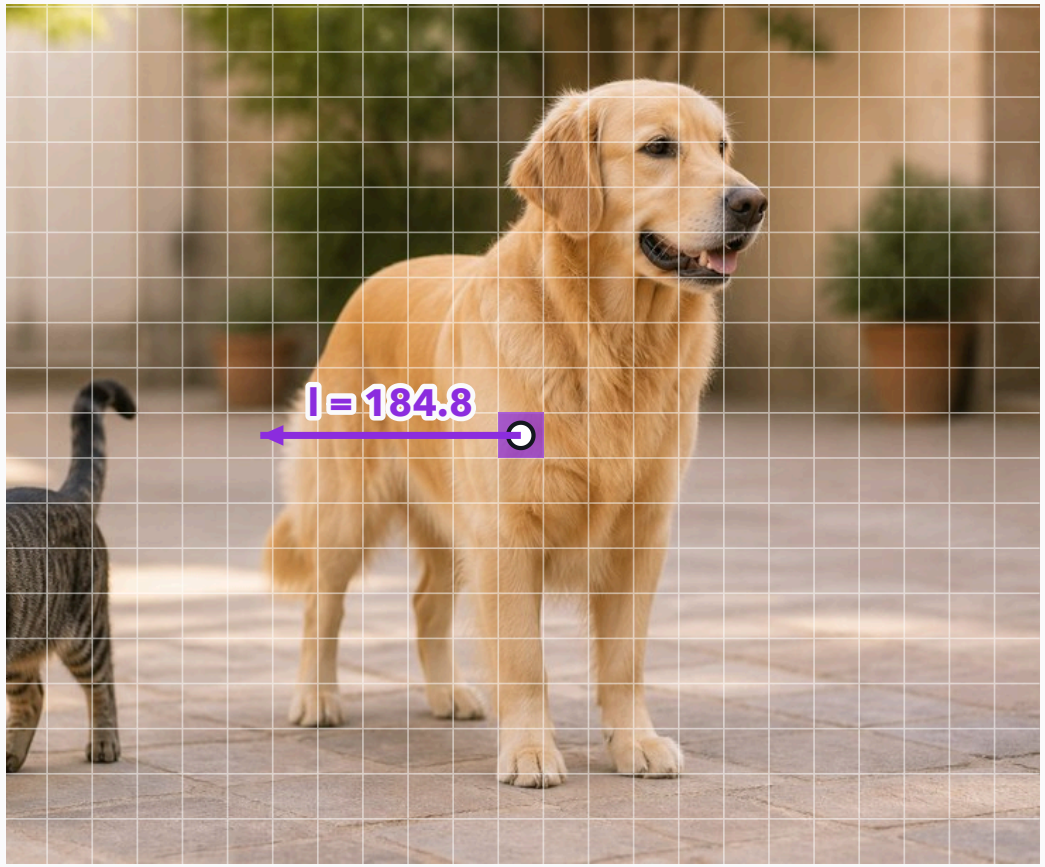
```
logits = head_cls(feats)                          # (600, 80): one row per cell
dist   = head_box(feats)                          # (600, 4): l, t, r, b in cells
y = torch.zeros(600, 80)
y[pos, labels] = 1.0                              # positives: 1 for their class
cls_loss = F.binary_cross_entropy_with_logits(logits, y, reduction='sum')
box_loss = (dist[pos] - dist_target[pos]).abs().sum()
loss = (cls_loss + box_loss) / len(pos)
```

pos: the indices of the 18 positive cells; labels: their class (16 for dog, 15 for cat); dist_target: their (l, t, r, b) targets.

> Building pos, labels and dist_target from the annotations is the “assignment” step: the 3 × 3 rule written as code. The notebook does it for this photo.

▶ Notebook, part 5

At test time: read the box back out of the cell



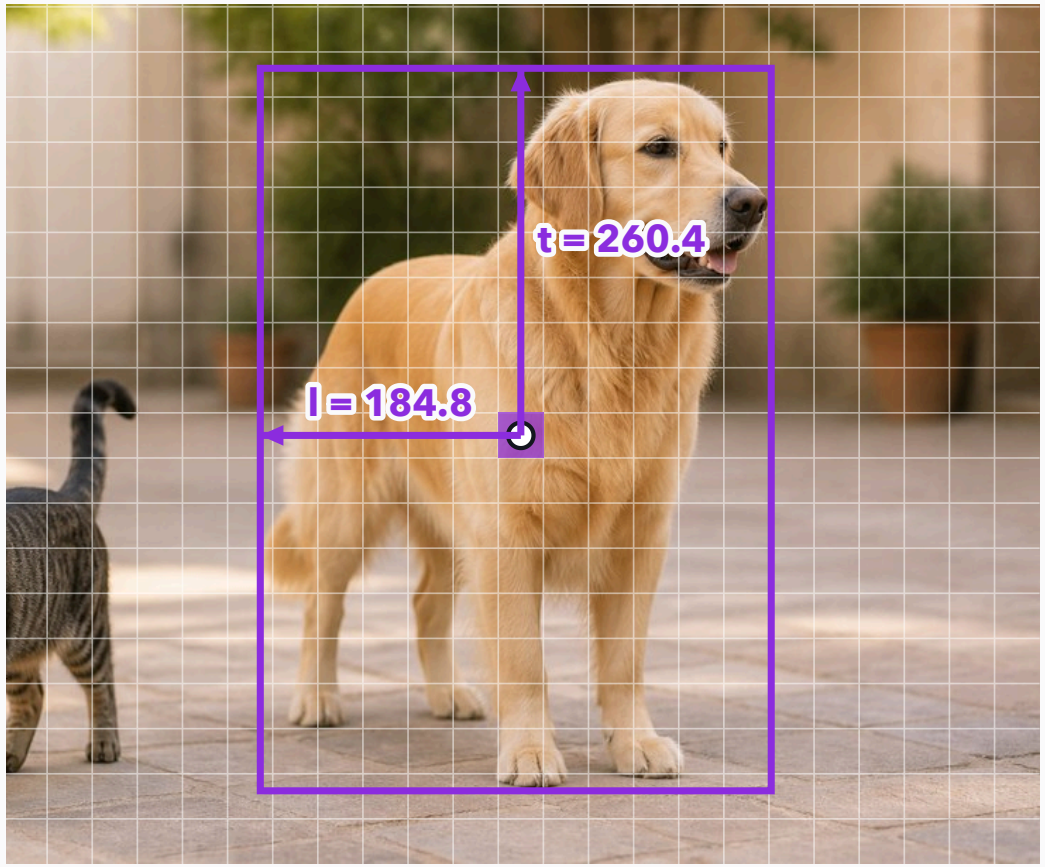
No annotation now: only the record.

> At test time we know only what the cell wrote: four distances and 80 scores.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

At test time: read the box back out of the cell



No annotation now: only the record.

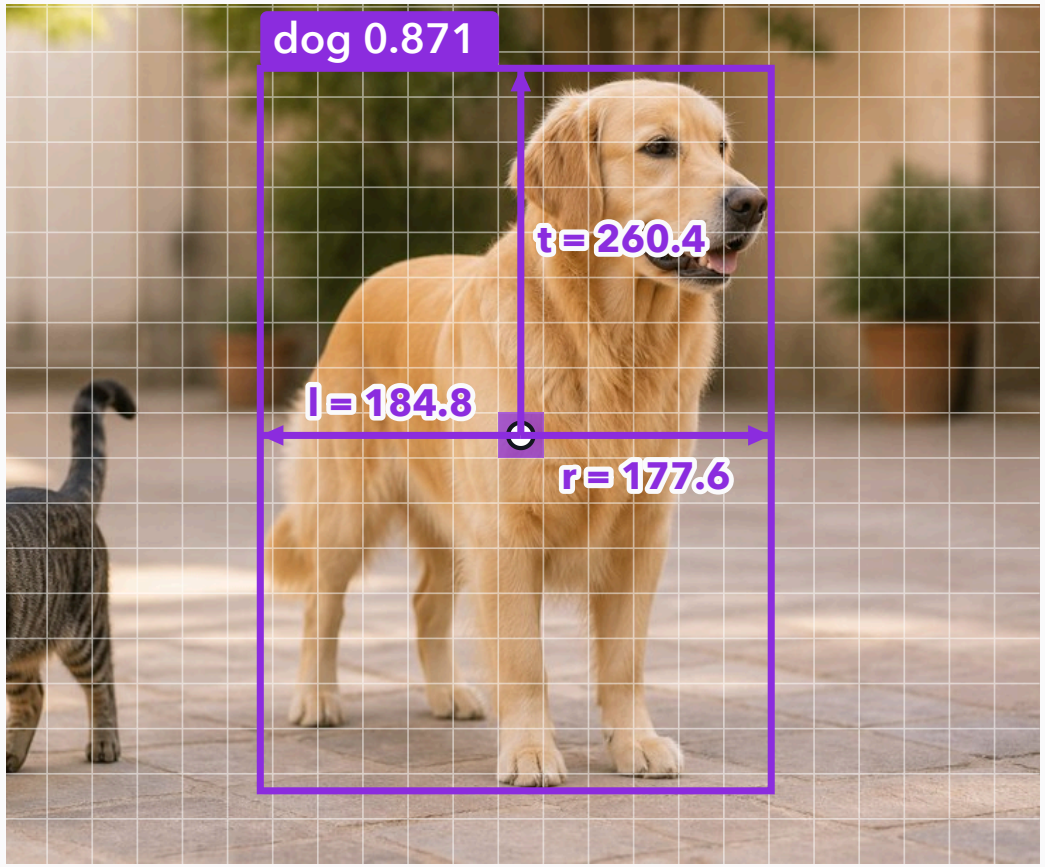
$$x_1 = 592 - 184.8 = 407.2$$
$$y_1 = 336 - 260.4 = 75.6$$
$$x_2 = 592 + 177.6 = 769.6$$
$$y_2 = 336 + 252.1 = 588.1$$

> Undo the distances: left edge = centre minus l, and so on. A box in pixels.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

At test time: read the box back out of the cell



No annotation now: only the record.

$$x_1 = 592 - 184.8 = 407.2$$
$$y_1 = 336 - 260.4 = 75.6$$
$$x_2 = 592 + 177.6 = 769.6$$
$$y_2 = 336 + 252.1 = 588.1$$

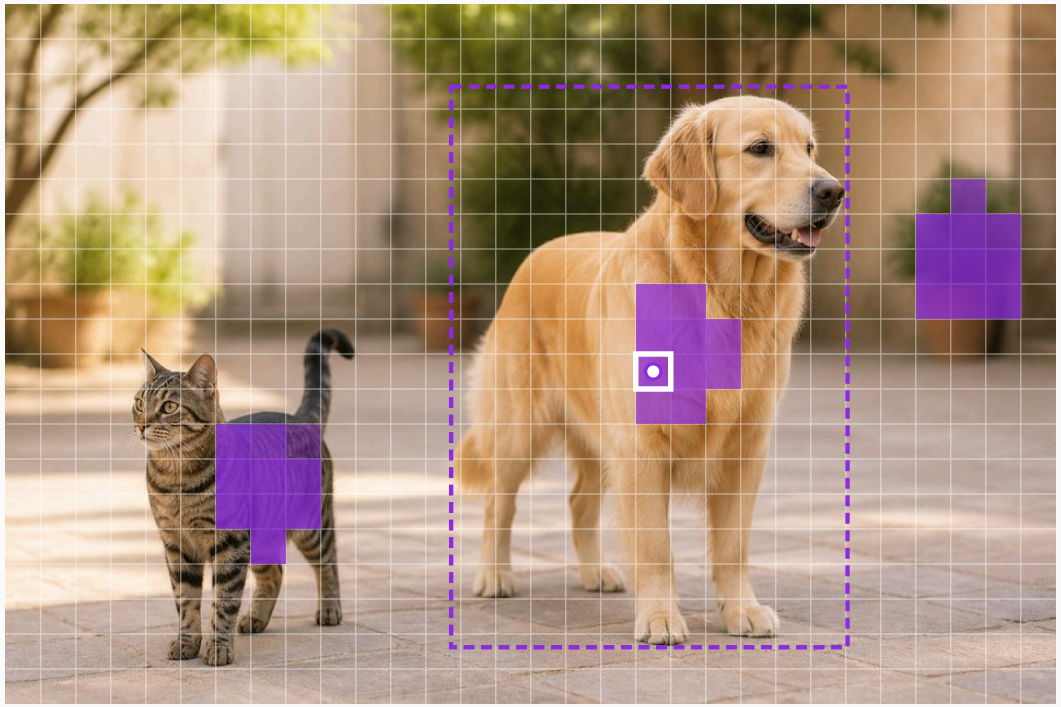
Score: the largest class score, 0.871, so the label is dog.

> Its score is its largest class score, 0.871, and its label that class. One cell, one scored box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a record: point at one



Cell: row 10, column 18

centre (592, 336) px

dog	0.871
cat	0.0001
skateboard	0.0001

its box: (407, 76, 770, 588)

> Move over the grid. Cells near the dog say dog and draw nearly the same box; cells on the wall give every class a tiny score, and their boxes mean nothing.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Decode every cell, keep scores ≥ 0.25: 31 boxes



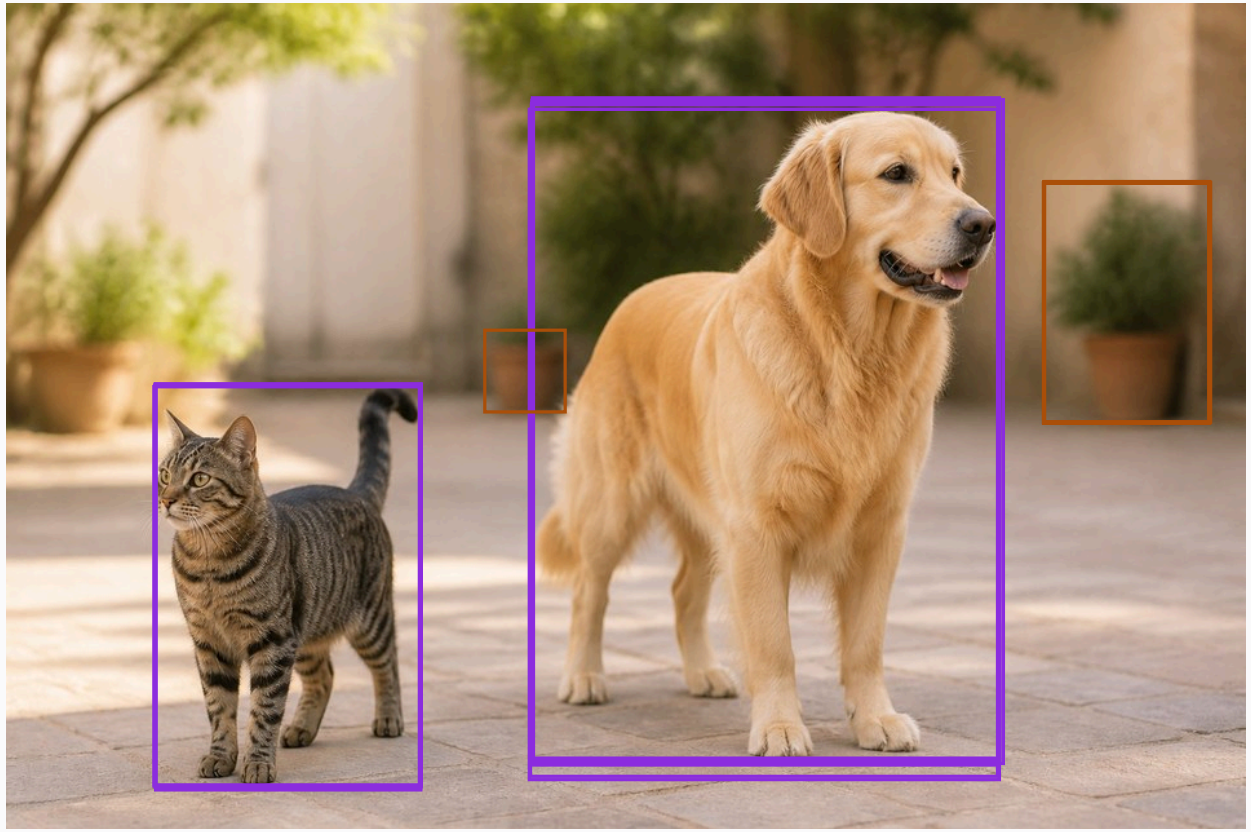
The network always writes 12,600 records for a photo this size: this grid's 600 and two finer grids' (Part 7).

> Every photo of this size gets exactly 12,600 records: the fixed-size output.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Decode every cell, keep scores ≥ 0.25: 31 boxes



The network always writes 12,600 records for a photo this size: this grid's 600 and two finer grids' (Part 7).

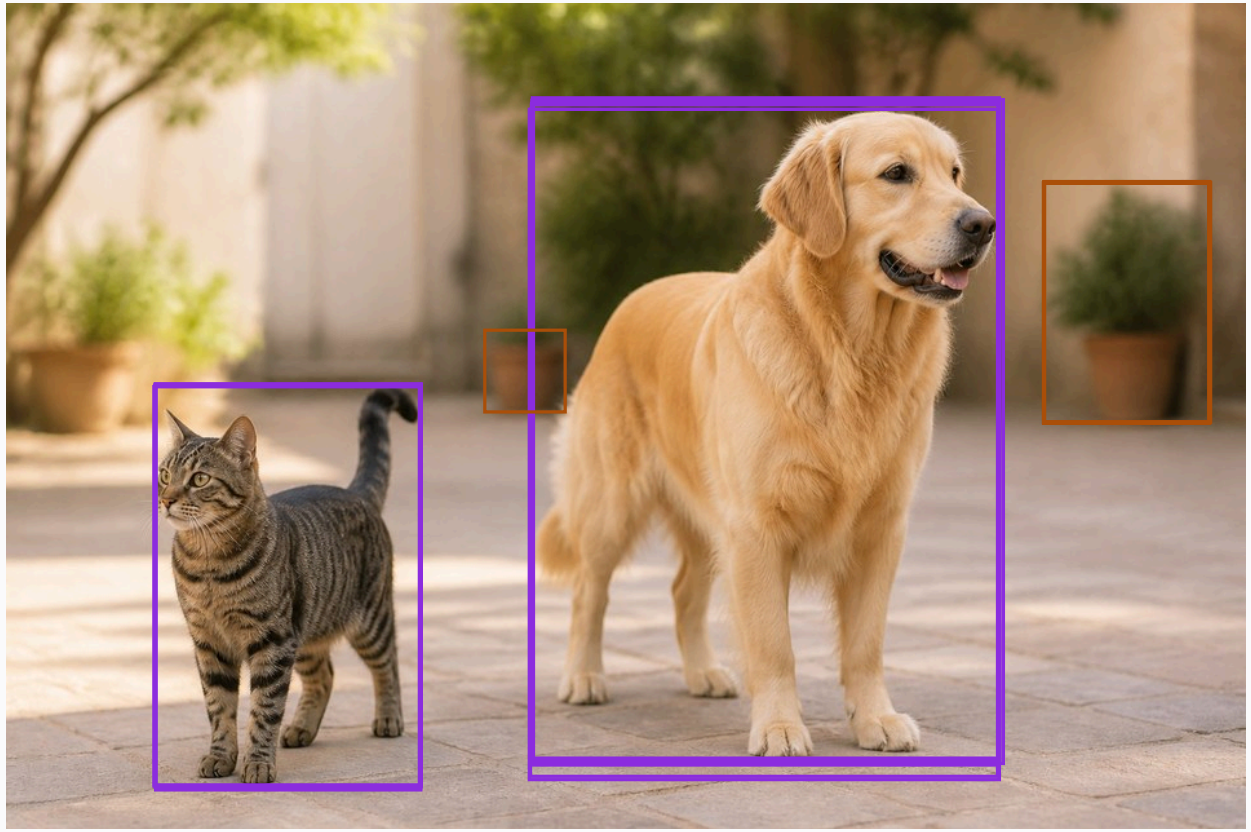
Scores ≥ 0.25: 31 boxes. 10 on the dog, 10 on the cat, 11 on plants.

> Throw away the unconfident ones: 31 boxes have a score of at least 0.25.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Decode every cell, keep scores ≥ 0.25: 31 boxes



The network always writes 12,600 records for a photo this size: this grid's 600 and two finer grids' (Part 7).

Scores ≥ 0.25: 31 boxes. 10 on the dog, 10 on the cat, 11 on plants.

We want one box per object. Each animal has about ten nearly identical boxes, one per firing cell.

> The ten cells that fired for the dog each drew a dog box. To get one box per object, remove the duplicates.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Part 5 in one slide

1. Sliding a classifier over 5,400 windows works but repeats work; one pass of a CNN gives a grid of cells instead.
2. Every cell writes a record: four distances to a box's sides and 80 yes/no class scores.
3. Cells near an object's centre are positives (18 here); the rest are background, with box target "?".
4. Loss: binary cross-entropy for every class at every cell, plus a box loss at the positives.
5. At test time: decode every cell, keep scores ≥ 0.25: 31 boxes for 4 objects.

Next: turn about ten boxes per object into one.

> Part 5: a candidate at every place, trained near each object. Part 6: one box per object.

▶ Notebook, part 6

Part 6 · One box per object: non-maximum suppression

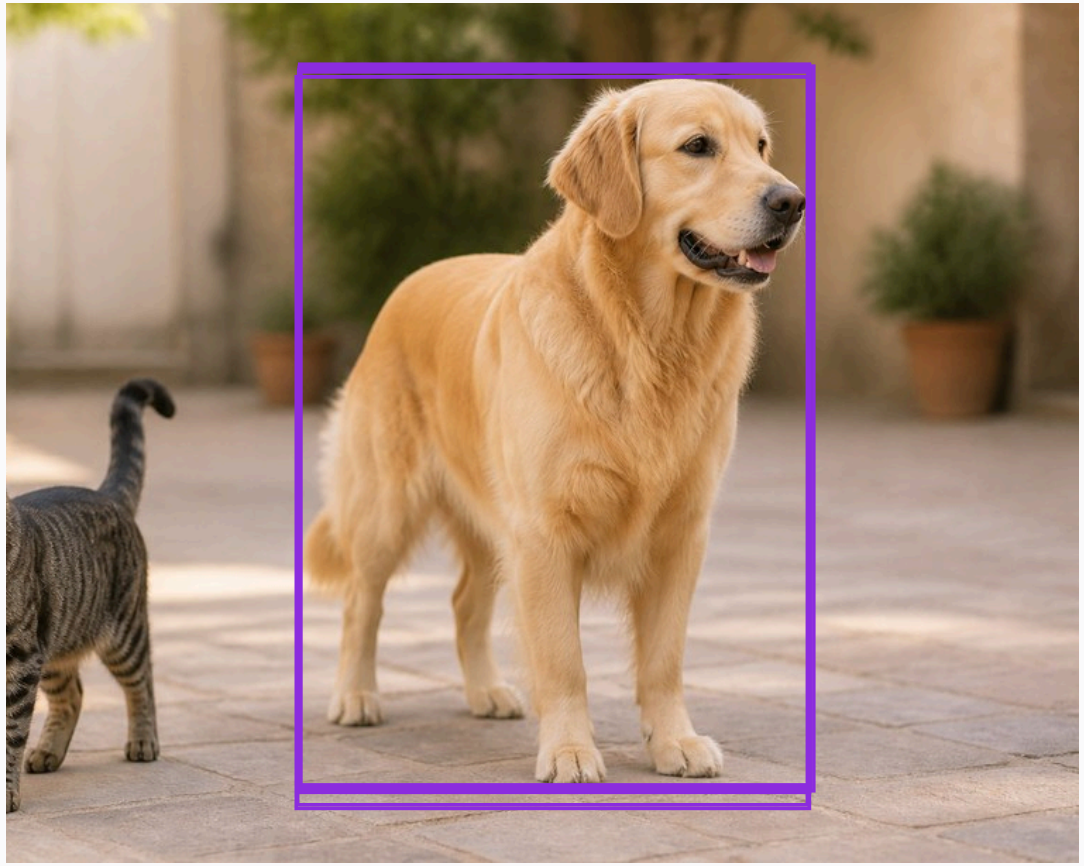
Ten boxes for one dog. Keep the best and delete the ones that overlap it heavily.

1. Run the greedy recipe on the dog's ten real boxes, one IoU at a time; then on two dogs.
2. See what the IoU threshold trades off, and why classes are kept apart.
3. Try it live; write it in eight lines; then the whole detector in code.

> NMS keeps the highest-scoring box and removes boxes that overlap it.

▶ Notebook, part 6

Ten boxes for one dog



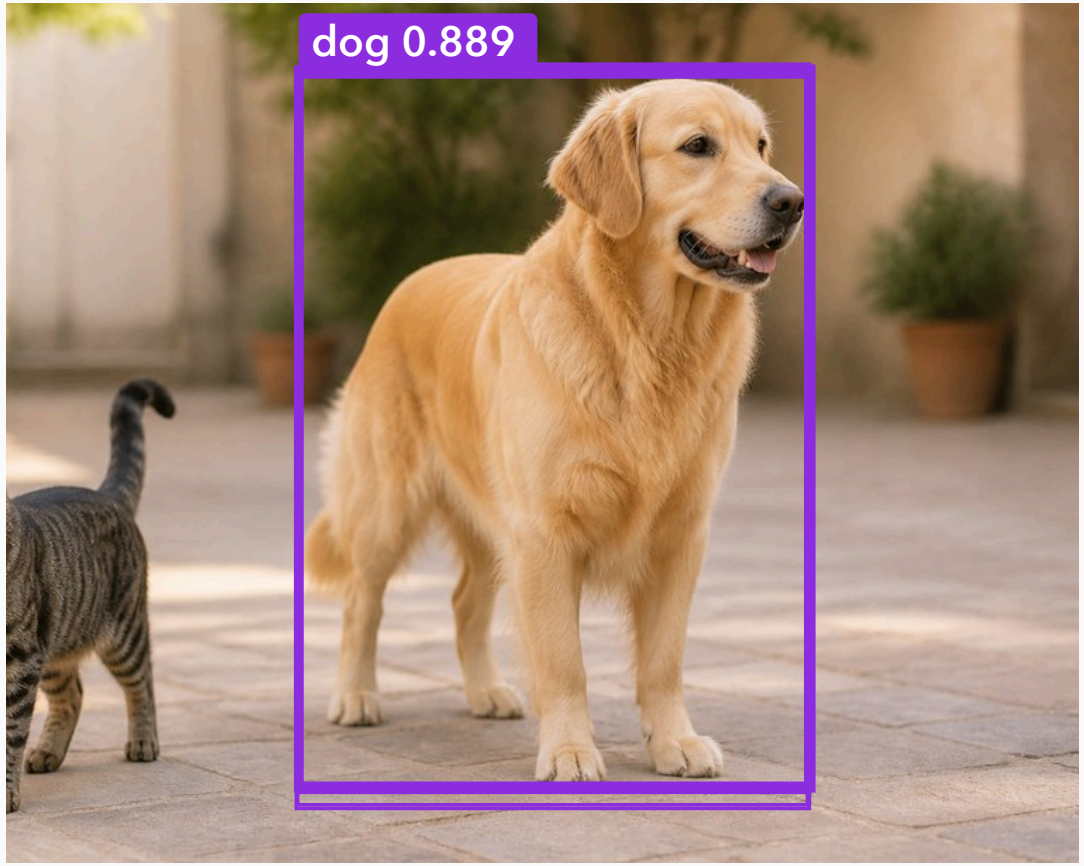
cell (row, col)	score
(9, 19)	**0.889**
(9, 18)	0.888
(10, 18)	0.871
(10, 19)	0.862
(8, 18)	0.822
(10, 20)	0.797
(8, 19)	0.793
(11, 19)	0.792
(9, 20)	0.792
(11, 18)	0.766

The ten cells that fired for the dog, and their scores, from 0.889 down to 0.766. Their boxes lie almost on top of each other.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Ten boxes for one dog



cell (row, col)	score
(9, 19)	**0.889**
(9, 18)	0.888
(10, 18)	0.871
(10, 19)	0.862
(8, 18)	0.822
(10, 20)	0.797
(8, 19)	0.793
(11, 19)	0.792
(9, 20)	0.792
(11, 18)	0.766

> They describe one dog. Keep the most confident box, from cell (9, 19), and delete the rest.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.

> Sort by score.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.

> The best box is kept: no other box can beat it.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).

> Boxes that overlap it heavily describe the same object: delete them. Boxes that overlap little may be other objects: keep them for now.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).
4. Repeat from step 2 with what is left, until nothing is left.

> Then repeat: the best of what is left is kept, and so on. “Non-maximum suppression”: suppress what is not the local maximum.

▶ Notebook, part 6

NMS on the dog's ten boxes

rank	cell	score	IoU with the kept box	fate
1	(9, 19)	0.889		
2	(9, 18)	0.888		
3	(10, 18)	0.871		
4	(10, 19)	0.862		
5	(8, 18)	0.822		
6	(10, 20)	0.797		
7	(8, 19)	0.793		
8	(11, 19)	0.792		
9	(9, 20)	0.792		
10	(11, 18)	0.766		

> Step 1: the ten boxes sorted by score.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

rank	cell	score	IoU with the kept box	fate
1	(9, 19)	0.889	kept box	**keep**
2	(9, 18)	0.888		
3	(10, 18)	0.871		
4	(10, 19)	0.862		
5	(8, 18)	0.822		
6	(10, 20)	0.797		
7	(8, 19)	0.793		
8	(11, 19)	0.792		
9	(9, 20)	0.792		
10	(11, 18)	0.766		

> Step 2: keep the top one, cell (9, 19).

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

rank	cell	score	IoU with the kept box	fate
1	(9, 19)	0.889	kept box	**keep**
2	(9, 18)	0.888	0.993	
3	(10, 18)	0.871	0.988	
4	(10, 19)	0.862	0.985	
5	(8, 18)	0.822	0.961	
6	(10, 20)	0.797	0.981	
7	(8, 19)	0.793	0.956	
8	(11, 19)	0.792	0.980	
9	(9, 20)	0.792	0.996	
10	(11, 18)	0.766	0.977	

> Step 3: each other box's IoU with it. All are above 0.9: these boxes are the same box, drawn by neighbouring cells.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

rank	cell	score	IoU with the kept box	fate
1	(9, 19)	0.889	kept box	**keep**
2	(9, 18)	0.888	0.993	**delete (> 0.7)**
3	(10, 18)	0.871	0.988	**delete (> 0.7)**
4	(10, 19)	0.862	0.985	**delete (> 0.7)**
5	(8, 18)	0.822	0.961	**delete (> 0.7)**
6	(10, 20)	0.797	0.981	**delete (> 0.7)**
7	(8, 19)	0.793	0.956	**delete (> 0.7)**
8	(11, 19)	0.792	0.980	**delete (> 0.7)**
9	(9, 20)	0.792	0.996	**delete (> 0.7)**
10	(11, 18)	0.766	0.977	**delete (> 0.7)**

> All above the 0.7 threshold: delete all nine. Nothing left for the dog; one dog box remains.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



Two dogs: 20 dog boxes with score ≥ 0.25, about ten per dog.

Two dogs in one photo: 20 dog boxes survive the score filter, about ten on each dog.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



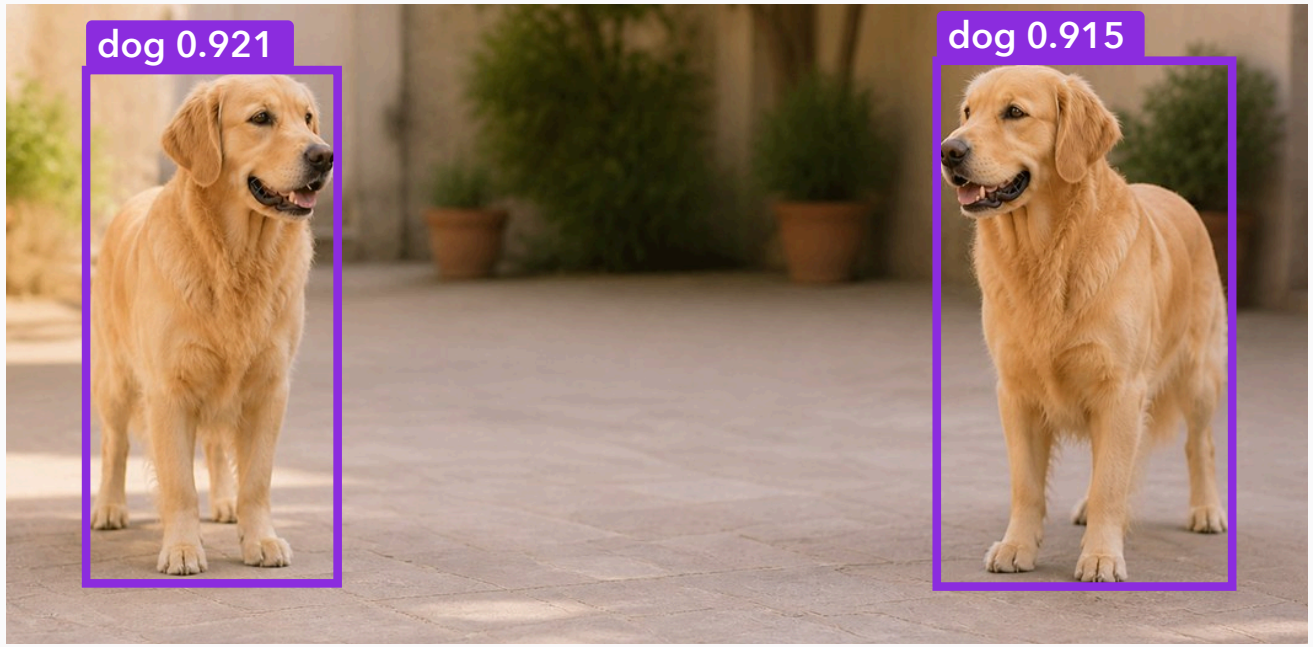
Round 1 keeps the best box, 0.921 on the left dog, and deletes its 9 twins (IoU ≥ 0.97). The right dog's boxes have IoU **0.0** with it: untouched.

> Round 1: keep the best box, 0.921, on the left dog; delete every dog box that overlaps it by more than 0.7. The right dog's boxes do not overlap it at all, so they stay.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



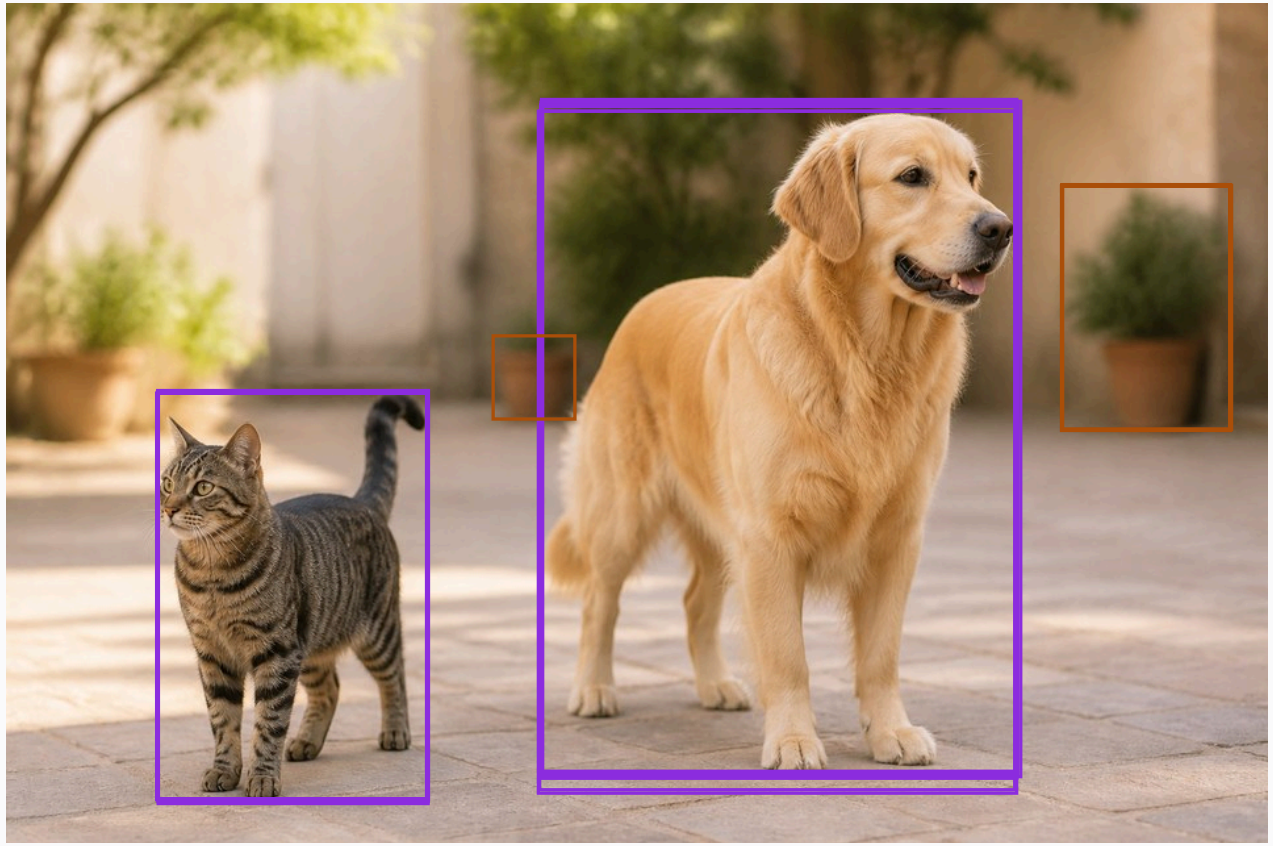
Round 2 keeps the best of what is left, 0.915 on the right dog, and deletes its 9 twins. Two dogs, two boxes.

> Round 2: the best remaining box, 0.915, is on the right dog. Keep it, delete its twins. Keeping only the single best box overall would have lost a dog; the loop keeps one per object.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

All 31 boxes → 4 detections



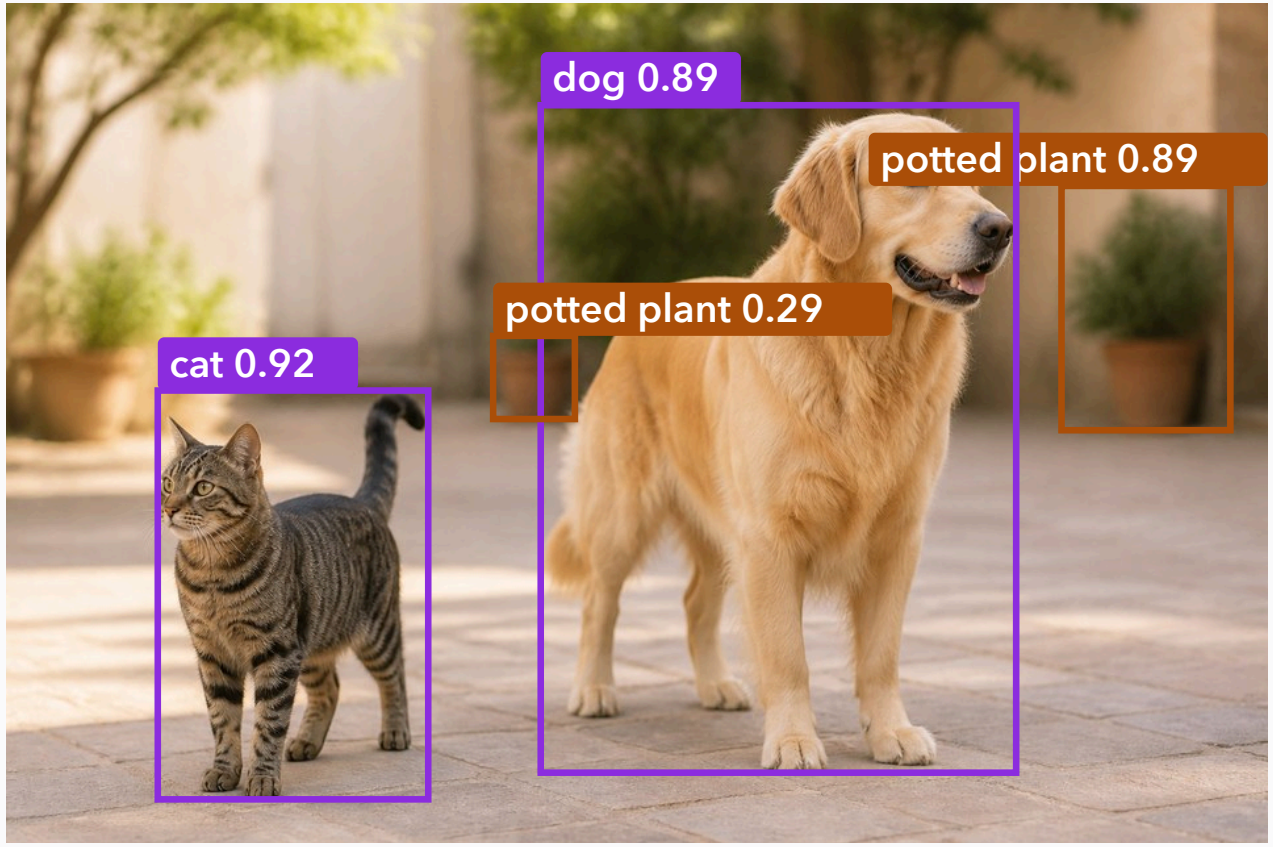
Before: 31 boxes.

> The same loop runs on every class: 10 cat boxes become one, 10 plant boxes become one.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

All 31 boxes → 4 detections



Before: 31 boxes.

After: 4. cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290.

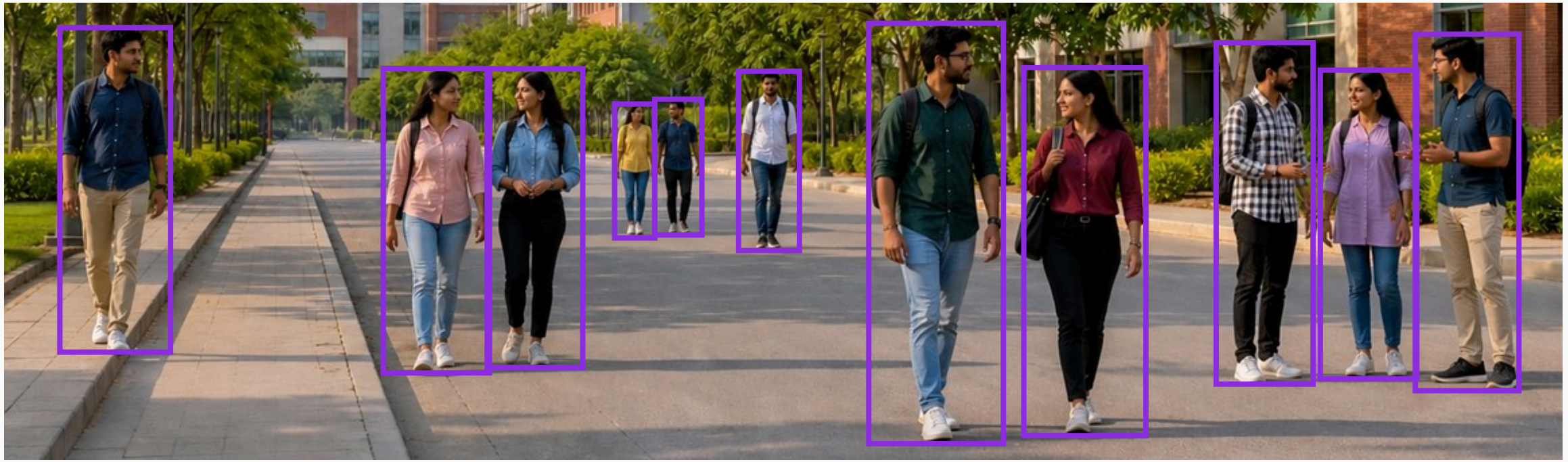
Our eight-line loop keeps exactly what Ultralytics' own post-processing keeps (checked in scripts/science.py).

> Four detections: cat, the big potted plant, dog, and a small plant on a finer grid. A fixed 12,600 records became a set of four.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



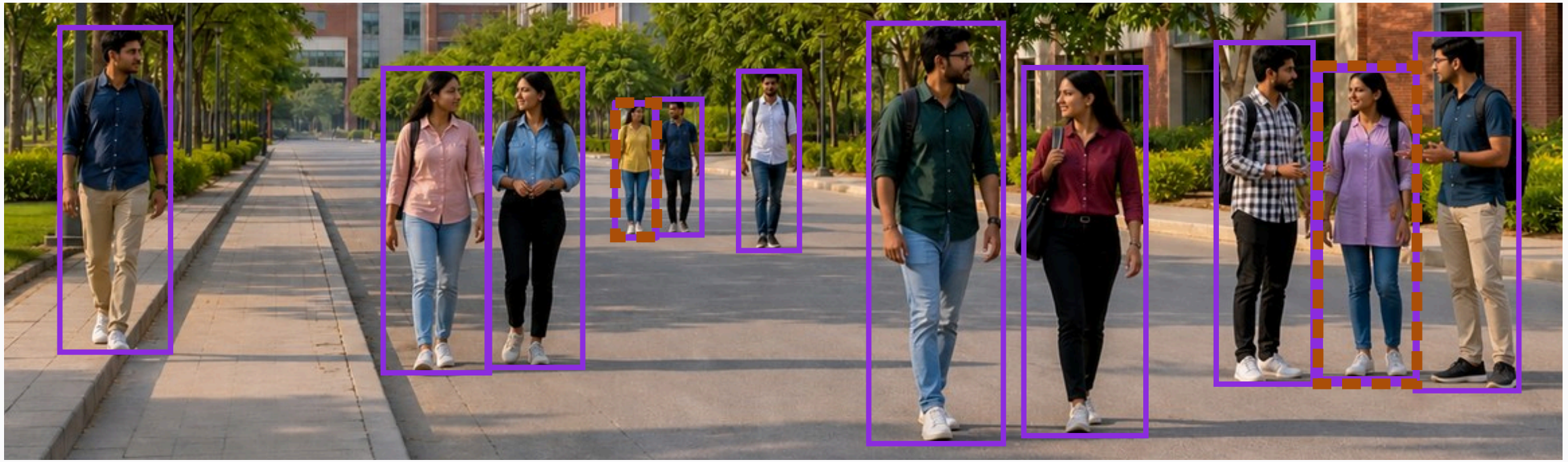
Threshold 0.7: 11 people, one box each. (From 112 candidates.)

> The crowd photo: 112 person candidates, NMS at 0.7: eleven people, eleven boxes.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



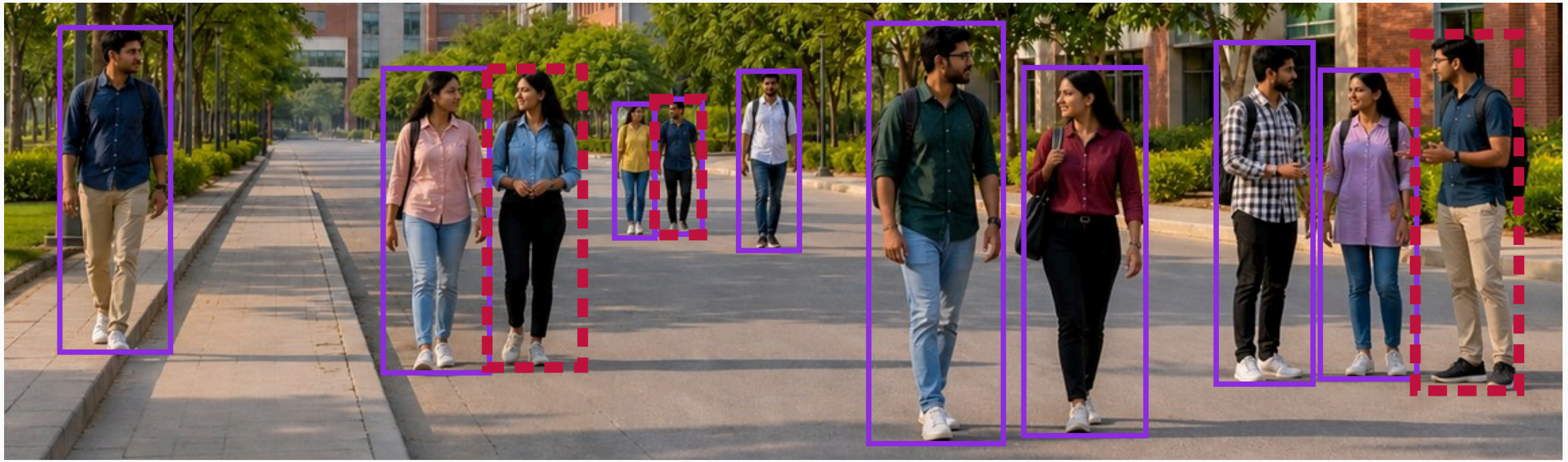
Too high, 0.95: 13 boxes. Two duplicates (amber) overlap their twin by less than 0.95 and survive.

> Raise the threshold to 0.95 and NMS deletes only near-identical boxes: two duplicates slip through.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



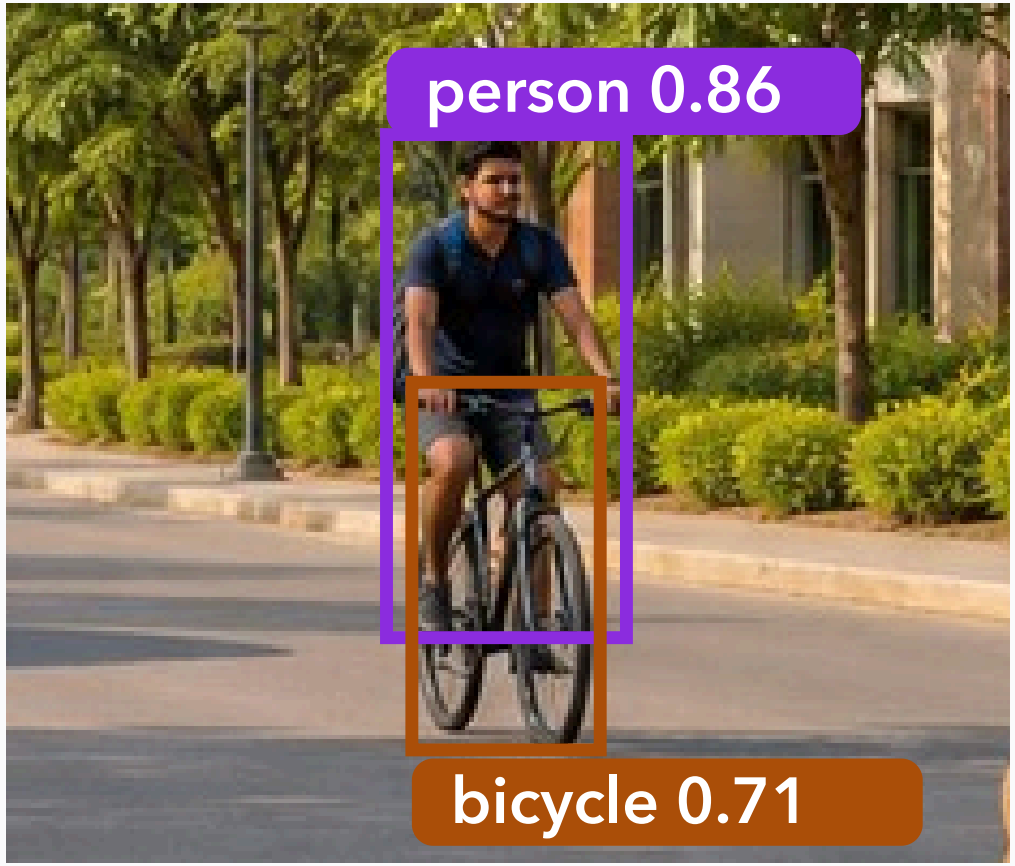
Too low, 0.0 (any overlap at all): 9. Two people whose boxes merely touch a neighbour's are deleted (red).

> Lower it to 0 and any touching box is deleted, including real neighbours. People in a queue or a crowd overlap far more than this; no single threshold suits every scene.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



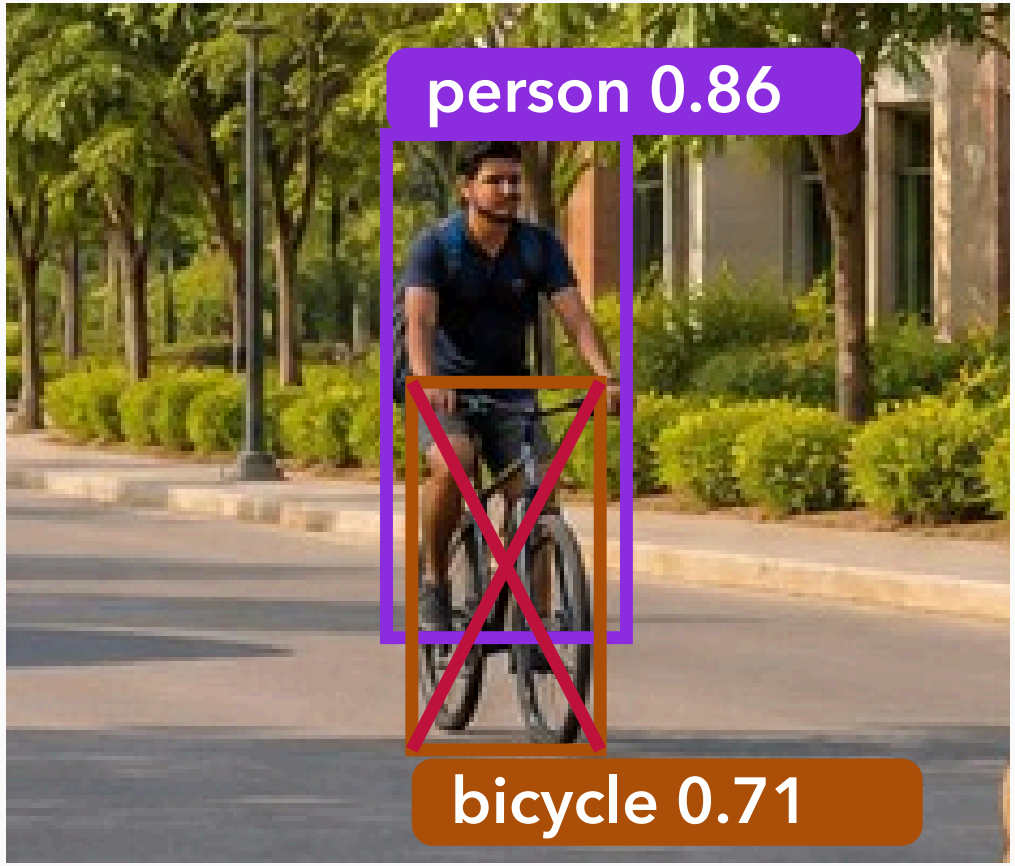
$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

A cyclist: a person box and a bicycle box. They overlap, IoU 0.339, and both are right.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

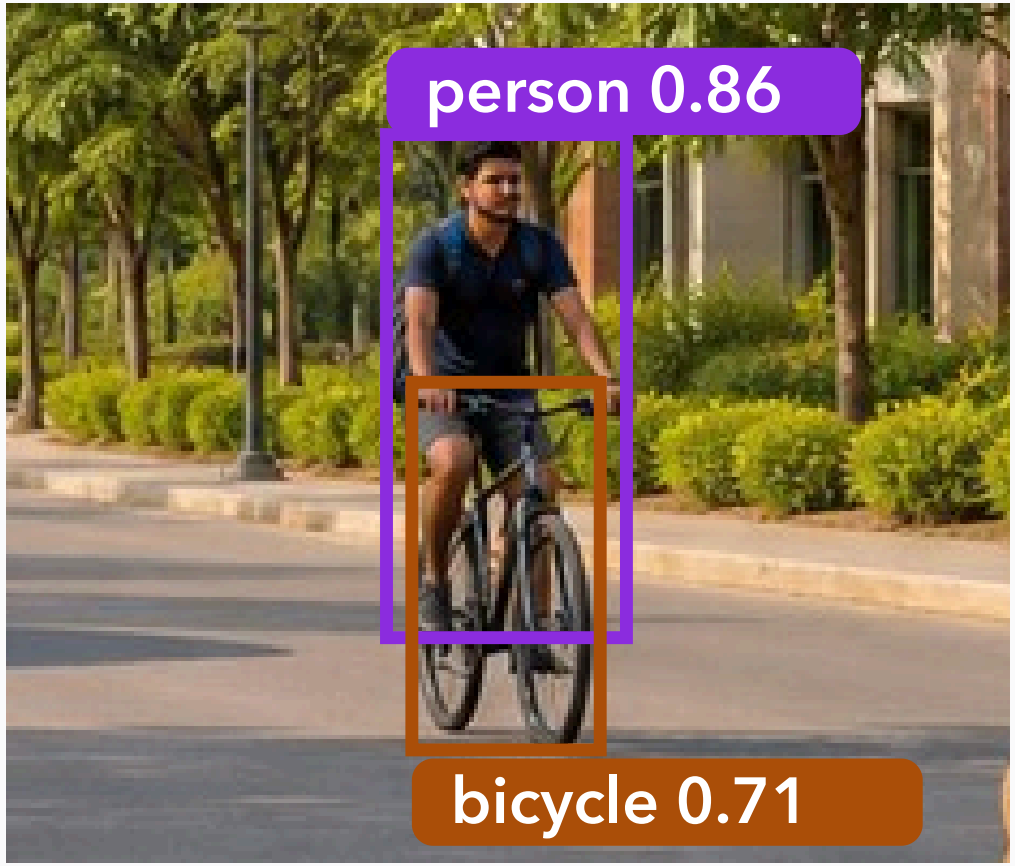
Ignore classes, threshold 0.3: the bicycle overlaps the higher-scoring person by more than 0.3, so it is deleted.

> If NMS ignored classes, with threshold 0.3, the bicycle would be suppressed by the person.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

Ignore classes, threshold 0.3: the bicycle overlaps the higher-scoring person by more than 0.3, so it is deleted.

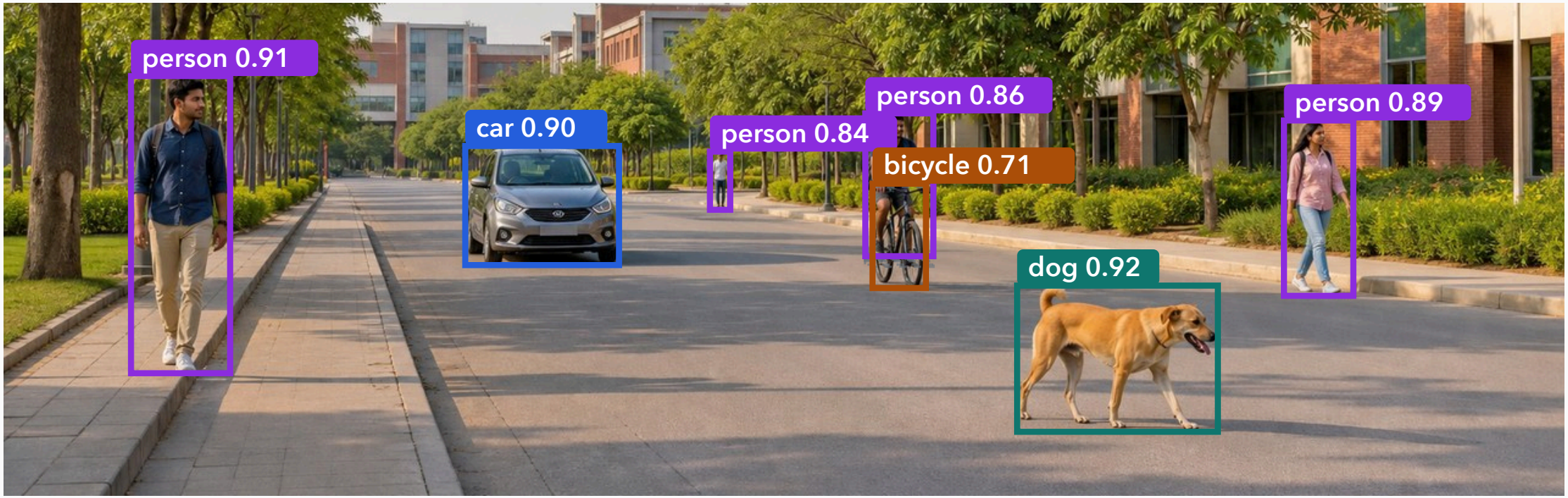
Class-aware NMS only compares boxes of the same class: both survive. This is the usual choice (torchvision's batched_nms).

> Run NMS separately for each class and both are kept. Steps 3 of the recipe said "of the same class" for this reason.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Try it: thresholds on the street photo



> Real candidates from YOLOv8n. Move the score threshold, then the IoU threshold; switch to “all classes together” and watch the bicycle; press Step to run the loop one round at a time.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)          # step 1
    keep = []
    while order.numel() > 0:
        i = order[0]; keep.append(i)                 # step 2
        ious = box_iou(boxes[i, None], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]               # step 3, then repeat
    return torch.stack(keep)
```

> The recipe line by line: sort, keep the first, drop everything that overlaps it more than the threshold, repeat.

▶ Notebook, part 6

NMS in code

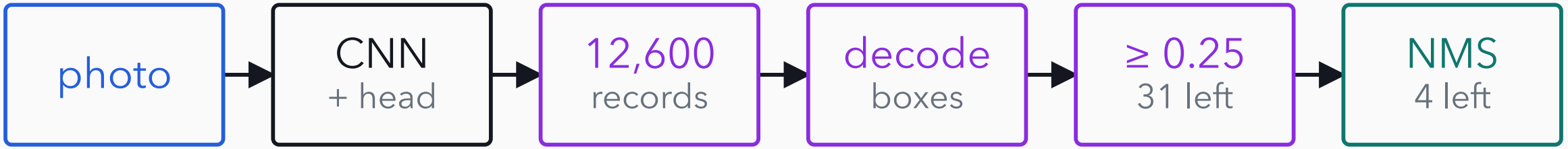
```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)          # step 1
    keep = []
    while order.numel() > 0:
        i = order[0]; keep.append(i)                 # step 2
        ious = box_iou(boxes[i, None], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]               # step 3, then repeat
    return torch.stack(keep)
```

```python
torchvision.ops.nms(boxes, scores, 0.7)              # same indices
torchvision.ops.batched_nms(boxes, scores, labels, 0.7)  # per class
```

> torchvision has it built in; batched_nms runs it separately for each class label.

▶ Notebook, part 6

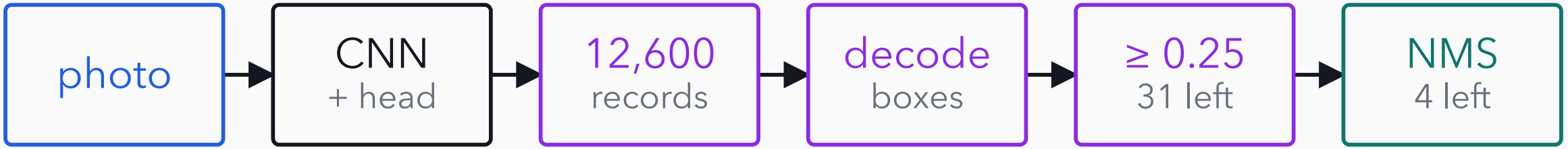
The dense answer, end to end



> Photo, CNN, a fixed 12,600 records, decoded into boxes, filtered by score, cleaned by NMS: four detections.

▶ Notebook, part 6

The dense answer, end to end



Fixed size in, any size out. The network's output never changes shape; the score filter and NMS decide how many objects come out: here 4, for the crowd 11, for an empty wall 0.

> This is answer A to the lecture's question: place-based outputs, many per object in training, one per object after NMS.

▶ Notebook, part 6

The whole detector at test time, in eight lines

```python
def detect(photo, net, names, score_thr=0.25, iou_thr=0.7):
    y, _ = net(photo)                          # (1, 84, 12600): always this shape
    boxes = cxcywh_to_xyxy(y[0, :4].T)         # (12600, 4), in pixels
    scores, labels = y[0, 4:].max(0)           # each cell's best class and score
    keep = scores >= score_thr                 # 31 cells left
    boxes, scores, labels = boxes[keep], scores[keep], labels[keep]
    keep = batched_nms(boxes, scores, labels, iou_thr)   # 4 left
    return [(names[l], s, b) for b, s, l in zip(boxes[keep], scores[keep], labels[keep])]
```

> Parts 5 and 6 as code: run the network once, read every cell's box and best class, keep the confident cells, run NMS separately for each class.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, in eight lines

```python
def detect(photo, net, names, score_thr=0.25, iou_thr=0.7):
    y, _ = net(photo)                          # (1, 84, 12600): always this shape
    boxes = cxcywh_to_xyxy(y[0, :4].T)         # (12600, 4), in pixels
    scores, labels = y[0, 4:].max(0)           # each cell's best class and score
    keep = scores >= score_thr                 # 31 cells left
    boxes, scores, labels = boxes[keep], scores[keep], labels[keep]
    keep = batched_nms(boxes, scores, labels, iou_thr)   # 4 left
    return [(names[l], s, b) for b, s, l in zip(boxes[keep], scores[keep], labels[keep])]
```

```python
detect(courtyard, net, names)
# cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290, each with its box
```

> On the courtyard photo it returns the 4 detections of the lecture's first slide. Nothing in it depends on how many objects there are.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

Part 7 · Small and large objects: three grids

A 32-pixel cell is coarse for a person 13 pixels wide. YOLOv8 predicts from three grids at once.

1. See the range of sizes in one street photo, now with the grid.
2. Count the cells of the three grids: where 12,600 came from.
3. Check which grid found each real detection.

> Three grids, three cell sizes, one record format.

▶ Notebook, part 7

Part 1’s street again, with the 32-pixel grid



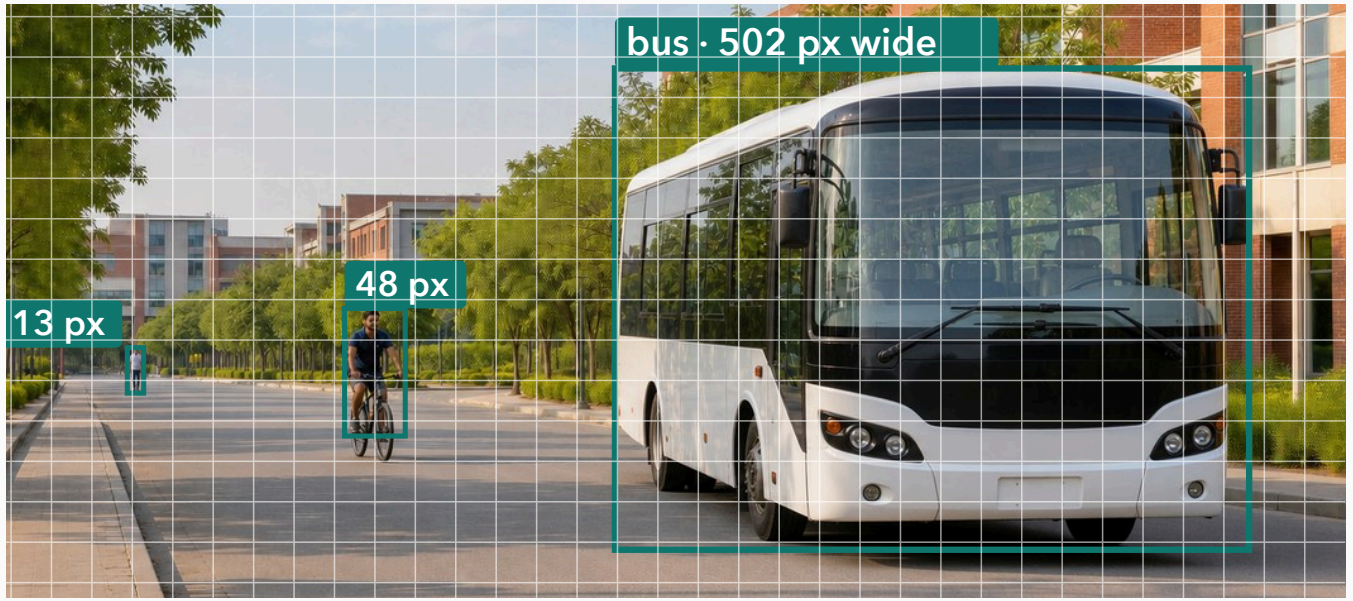
The bus is 502 px wide; the cyclist 48 px; the far pedestrian 13 px.

> Part 1’s street: a bus 502 px wide, a cyclist 48 px, a pedestrian 13 px far down the road.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

Part 1's street again, with the 32-pixel grid



The bus is 502 px wide; the cyclist 48 px; the far pedestrian 13 px.

On a 32-px grid the pedestrian is under half a cell. Two such people side by side would put both centres in one cell, which can describe only one of them.

> Draw the 32-px grid. The bus spans about 16 cells; the pedestrian sits inside one. A coarse grid suits big objects; small ones need finer cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all			

> YOLOv8 reads three feature maps from different depths of the CNN: cells of 8, 16 and 32 pixels.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all	**12,600**	**18,480**	

> Add them up: 12,600 for the courtyard, the number we decoded in Part 5. A bigger photo gets more cells: 18,480 for the street.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all	**12,600**	**18,480**	

Every cell of every grid writes the same 84-number record. The fine grid catches small objects; the coarse grid sees enough to box a bus.

> Each grid writes the same kind of record. Feature pyramids (FPN, 2017) mix deep and early features so even the fine grid knows what it is looking at.

▶ Notebook, part 7

Which grid found each object? The real answer



object	grid
bus	32 px
person	16 px
person	8 px
bicycle	16 px

> After NMS, look up which grid each kept box came from: the bus from the 32-px grid, the cyclist and bicycle from the 16-px grid, the far pedestrian from the 8-px grid.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

Which grid found each object? The real answer



object	grid
bus	32 px
person	16 px
person	8 px
bicycle	16 px

Street photo: dog and near person from 32 px; car, cyclist and bicycle from 16 px; the far person from 8 px.

> The street photo agrees: large from coarse, small from fine. Training assigns each object to cells whose grid suits its size.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Part 7 in one slide

1. One grid cannot suit a 502-px bus and a 13-px pedestrian.
2. YOLOv8 predicts from three grids, with cells of 8, 16 and 32 pixels: 12,600 records for the courtyard.
3. Every cell of every grid writes the same 84-number record; NMS runs on all of them together.
4. Small objects come from the fine grid, large ones from the coarse grid.

Next: measure how good the detections are.

> Part 7: three grids for three sizes. Part 8: measuring a detector.

▶ Notebook, part 8

Part 8 · Measuring a detector

Compare detections with annotations: decide which are right, then summarise.

1. Decide true positive, false positive and miss, on the street photo's real detections.
2. Precision and recall at one threshold, then at all of them: walk down a ranked list.
3. Average precision and mAP; then back to Part 1's bird.

> Right or wrong for each detection, then precision, recall and AP.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…

> A detection counts as correct if its class is right and it overlaps an annotated box with IoU of at least 0.5.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.

> Each annotated object can be claimed once. Detections go in score order; the first good one claims it.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.

> Every detection that claims nothing is a false positive, including a duplicate that NMS missed.

▶ Notebook, part 8

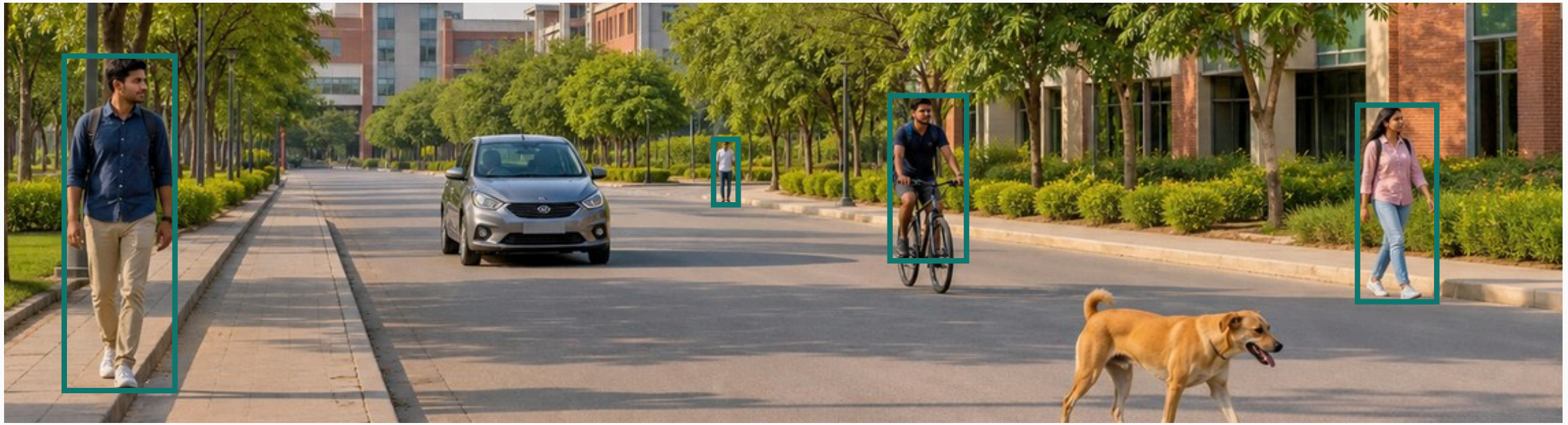
When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.
4. **False negative (FN)**: an annotated object that no detection claimed: a miss.

> Every annotated object left unclaimed is a false negative.

▶ Notebook, part 8

Real check: people in the street photo



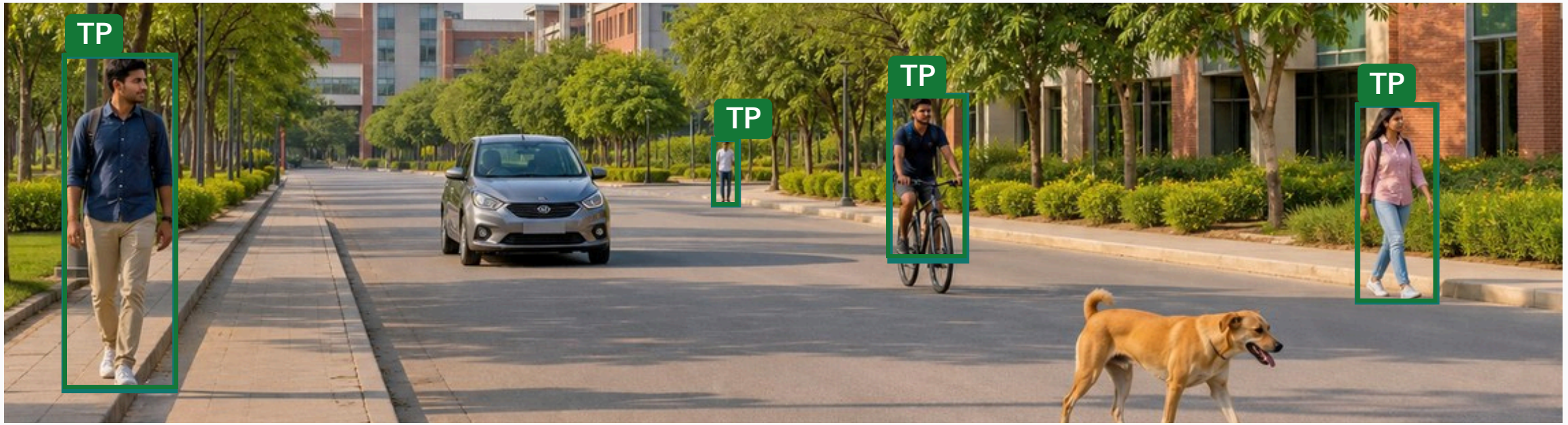
Teal: the 4 annotated people. YOLOv8n's person detections after NMS, down to score 0.01: 12.

> Four annotated people (teal). YOLOv8n's person detections after NMS, down to score 0.01: 12.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Real check: people in the street photo



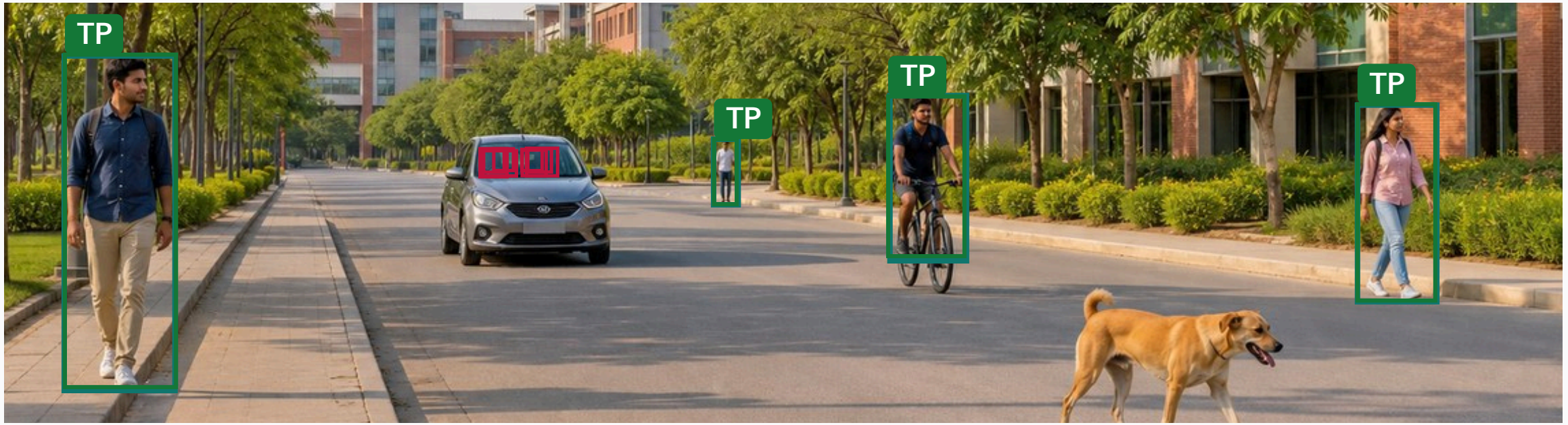
The 4 most confident (0.907, 0.893, 0.864, 0.845) each claim a different person, IoU 0.97, 0.97, 0.92, 0.91: **4 TP**.

> The four most confident each claim a different person with IoU above 0.9: four true positives.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Real check: people in the street photo



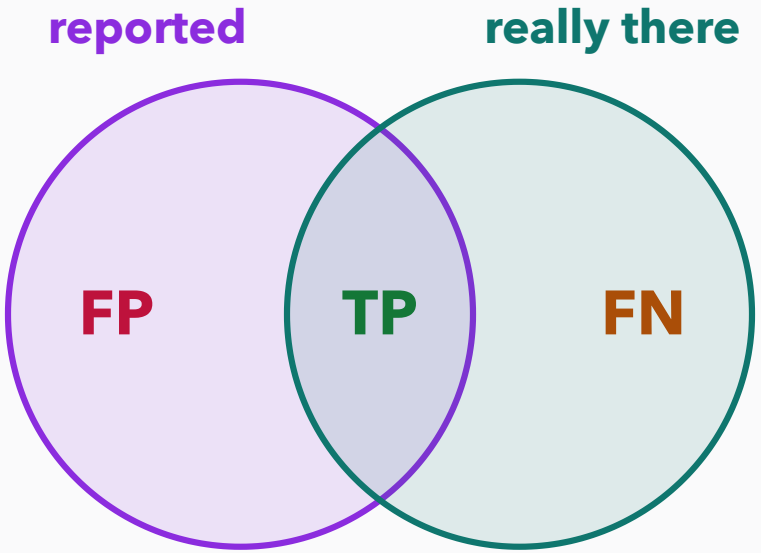
The other 8 (scores 0.042 down to 0.011) sit on the car's windscreen, IoU 0 with every person: **8 FP**. No person is missed: 0 FN.

> The other 8, scores 0.04 and below, sit on the car's windscreen, where nobody is: false positives. No person is missed.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Precision and recall: two questions



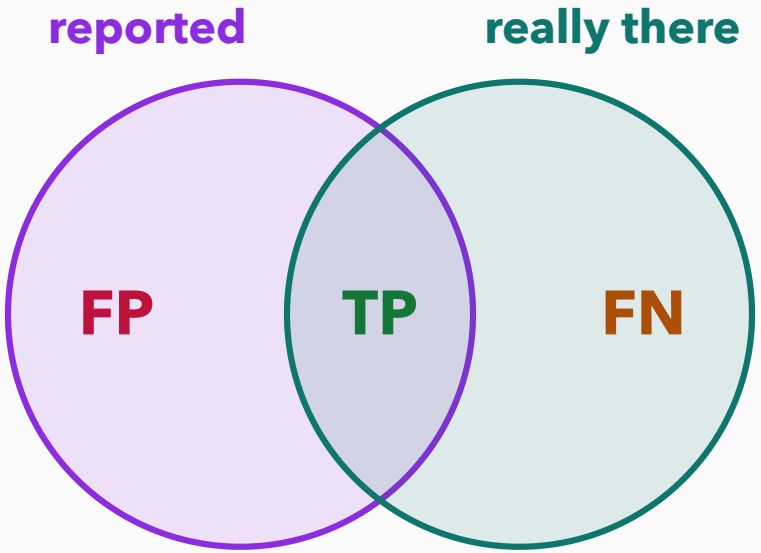
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of what we reported, how much was right?

Precision looks at the left circle, everything we reported: the fraction of it that was right.

▶ Notebook, part 8

Precision and recall: two questions



$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of what we reported, how much was right?

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Of what was really there, how much did we find?

Recall looks at the right circle, everything that was really there: the fraction of it we found.

▶ Notebook, part 8

The street's people at two score cut-offs

keep scores ≥	TP	FP	FN	precision	recall
0.25	4	0	0	4/4 = 1	4/4 = 1

> With the usual 0.25 score cut-off, the street photo is perfect: four people found, nothing extra. Precision 1, recall 1.

▶ Notebook, part 8

The street's people at two score cut-offs

keep scores ≥	TP	FP	FN	precision	recall
0.25	4	0	0	4/4 = 1	4/4 = 1
0.01	4	8	0	4/12 = 0.33	4/4 = 1

> Keep everything down to 0.01 and the 8 windscreen boxes join: recall stays 1, precision drops to 0.33. The cut-off trades one for the other.

▶ Notebook, part 8

Back to the bird: a lower cut-off finds it



keep scores ≥	found	TP	FP	FN	precision	recall
0.25	cat, dog	2	0	1	1.00	2/3

At the usual 0.25: the cat and the dog are found, the bird (0.098) is missed. Precision 1, recall 2/3.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Back to the bird: a lower cut-off finds it



keep scores ≥	found	TP	FP	FN	precision	recall
0.25	cat, dog	2	0	1	1.00	2/3
0.05	bird, cat, dog	3	0	0	1.00	3/3

> Lower the cut-off to 0.05 and the bird's box gets through: recall 3/3. On this photo nothing false comes in with it; on the street, going lower brought eight false people.

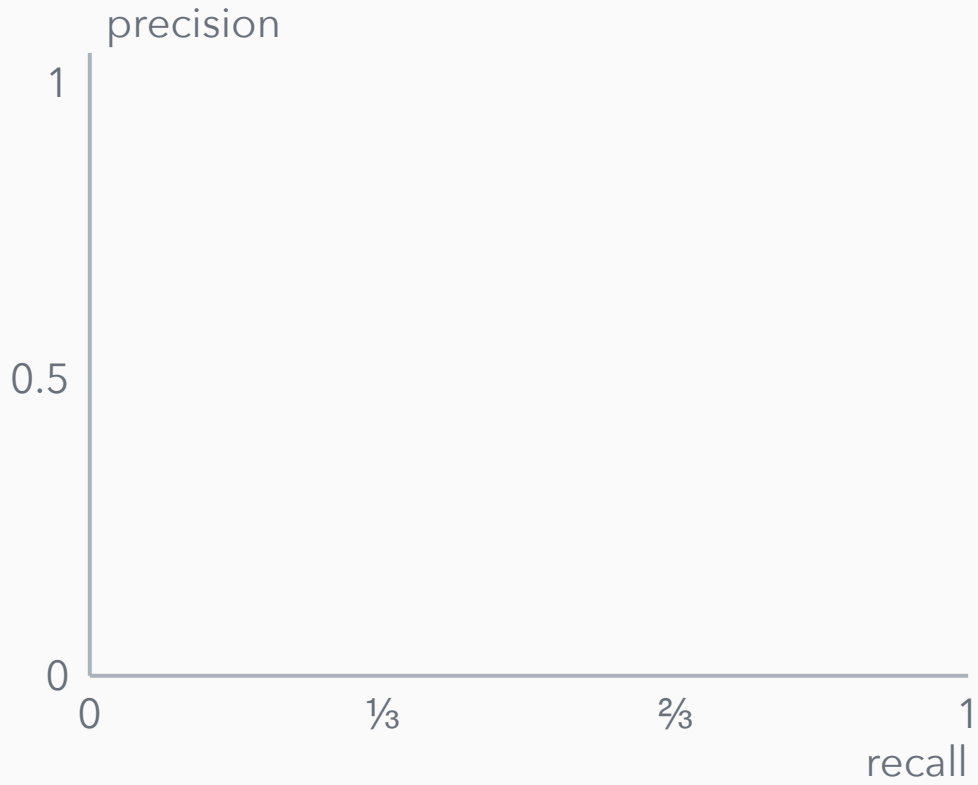
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95				
#2	0.90				
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



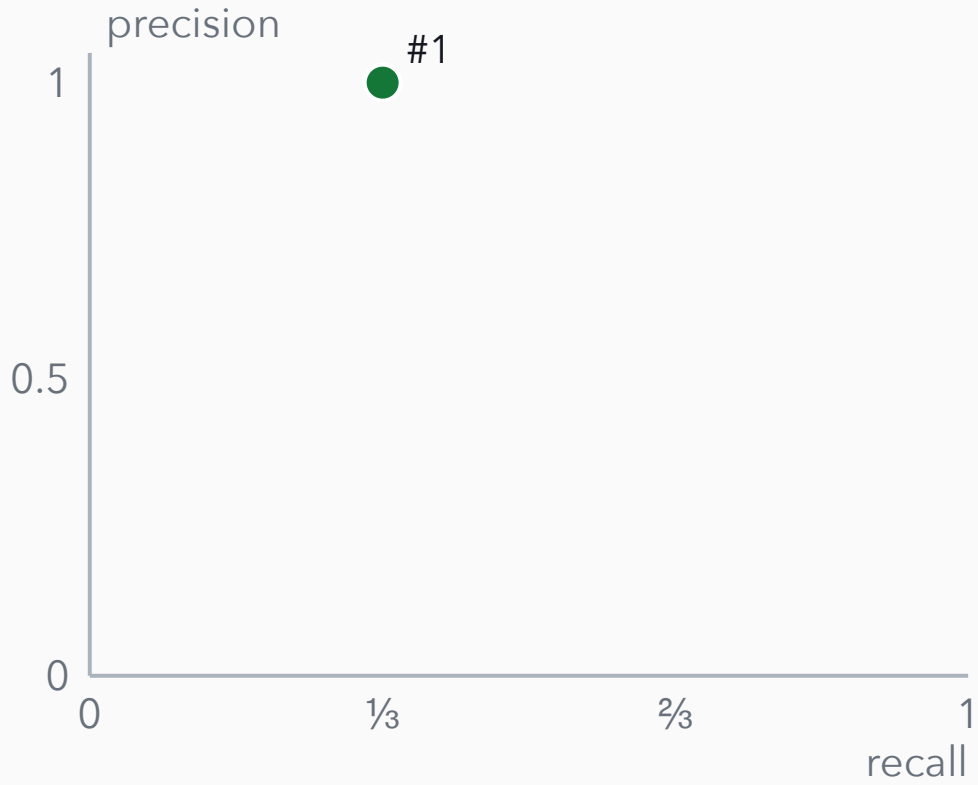
> Six detections sorted by score; three objects in the photos. Lower the threshold one detection at a time and track precision and recall.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90				
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



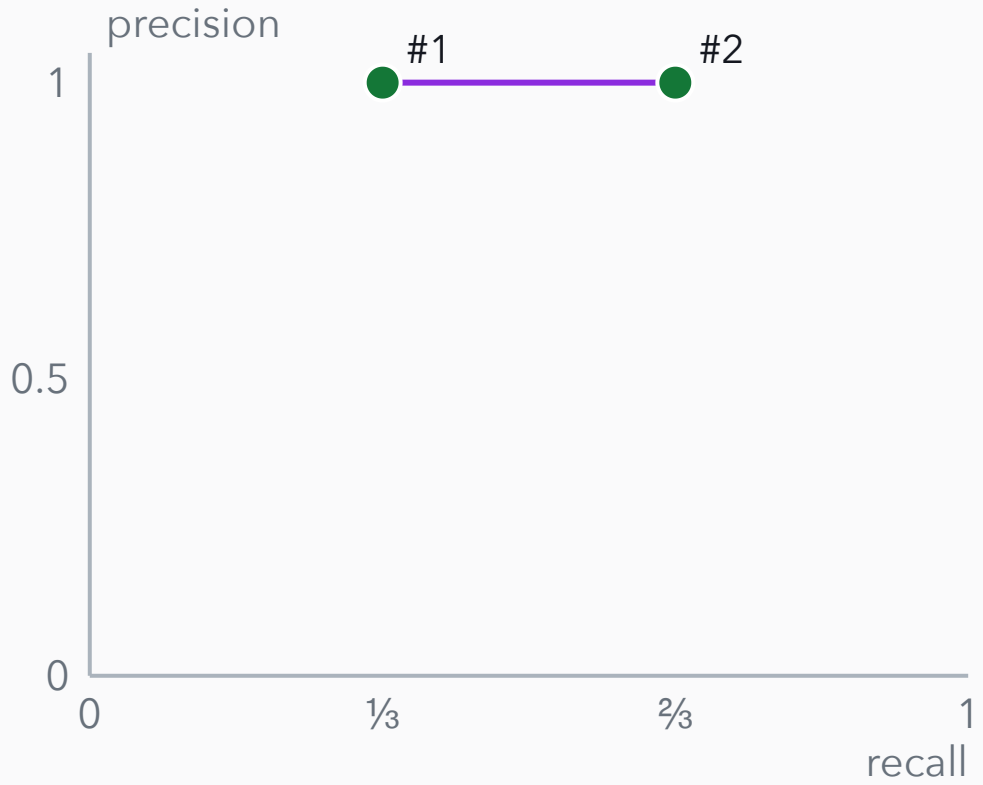
> #1 (0.95) claims object A: TP. Precision 1/1, recall 1/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



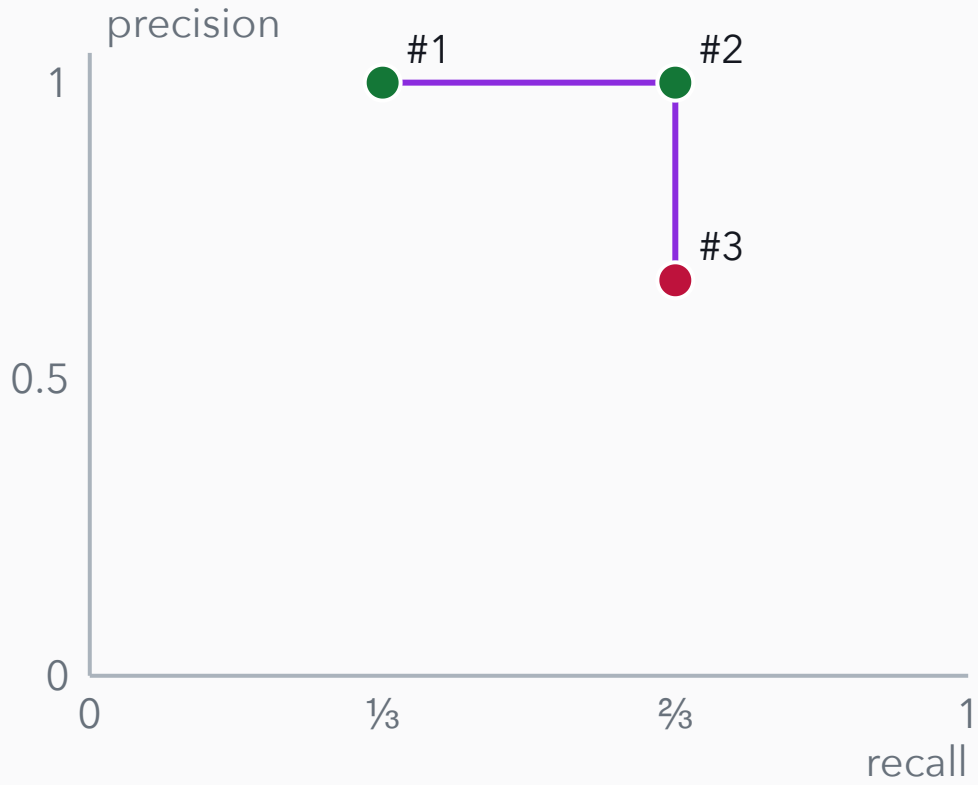
> #2 (0.90) claims B: TP. 2/2, 2/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



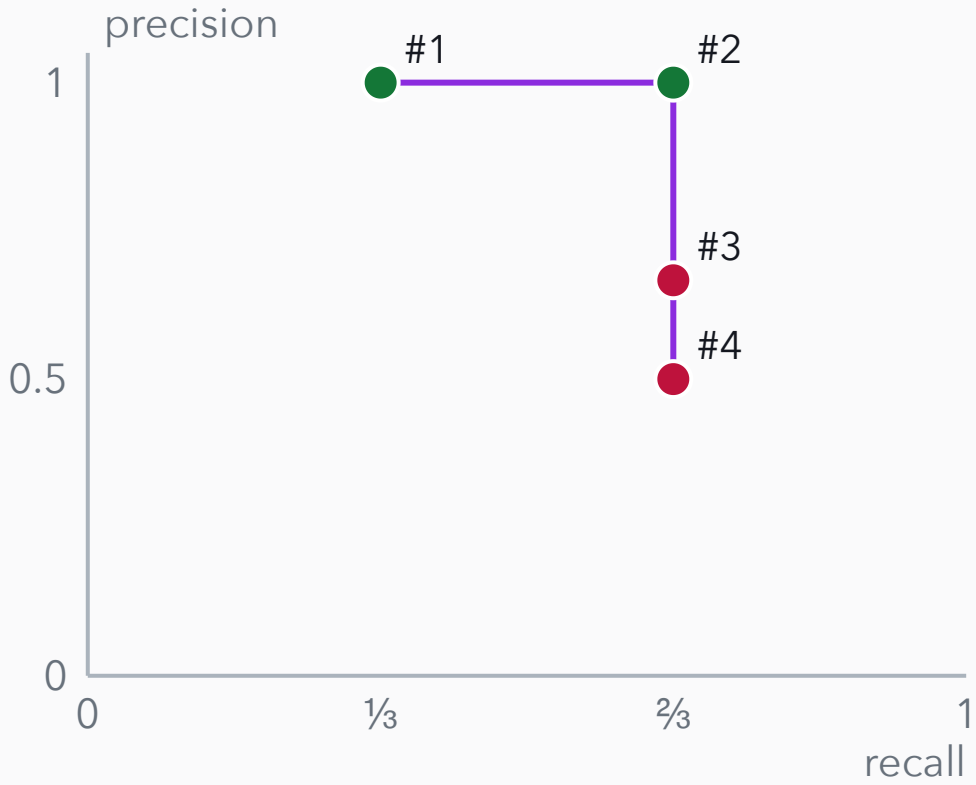
> #3 (0.80) lands on A again, already claimed: a duplicate, FP. Precision falls to 2/3; recall stays.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



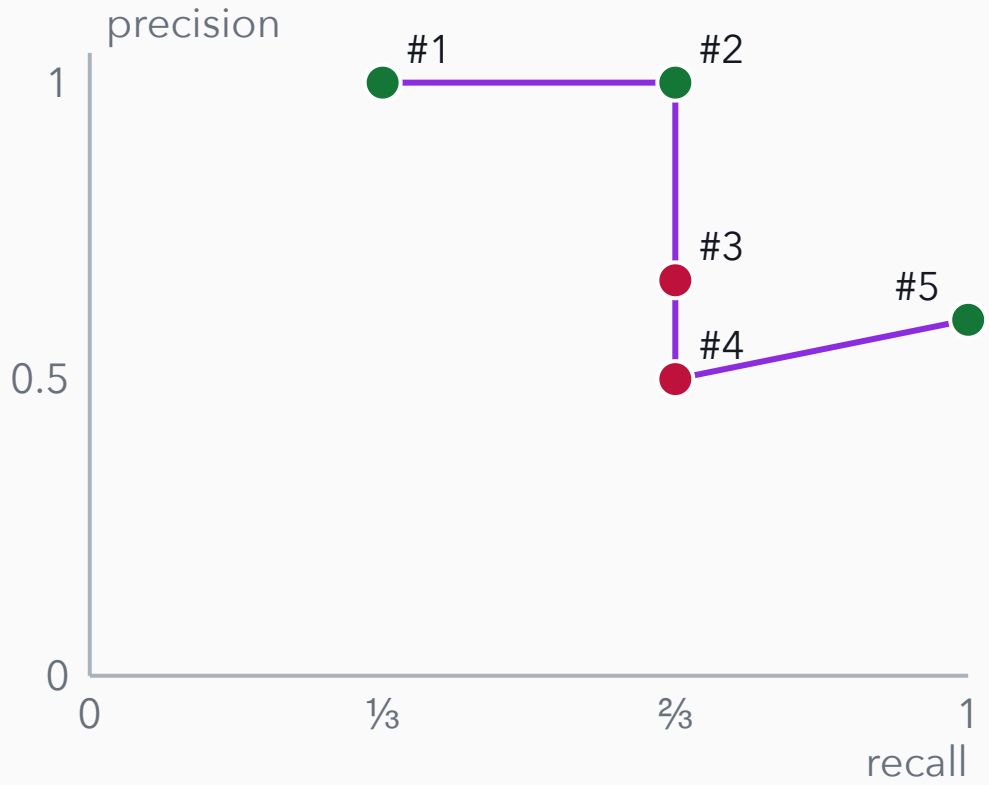
> #4 (0.70) where nothing is: FP. 2/4, 2/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60	**TP**	match	3/5	3/3
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



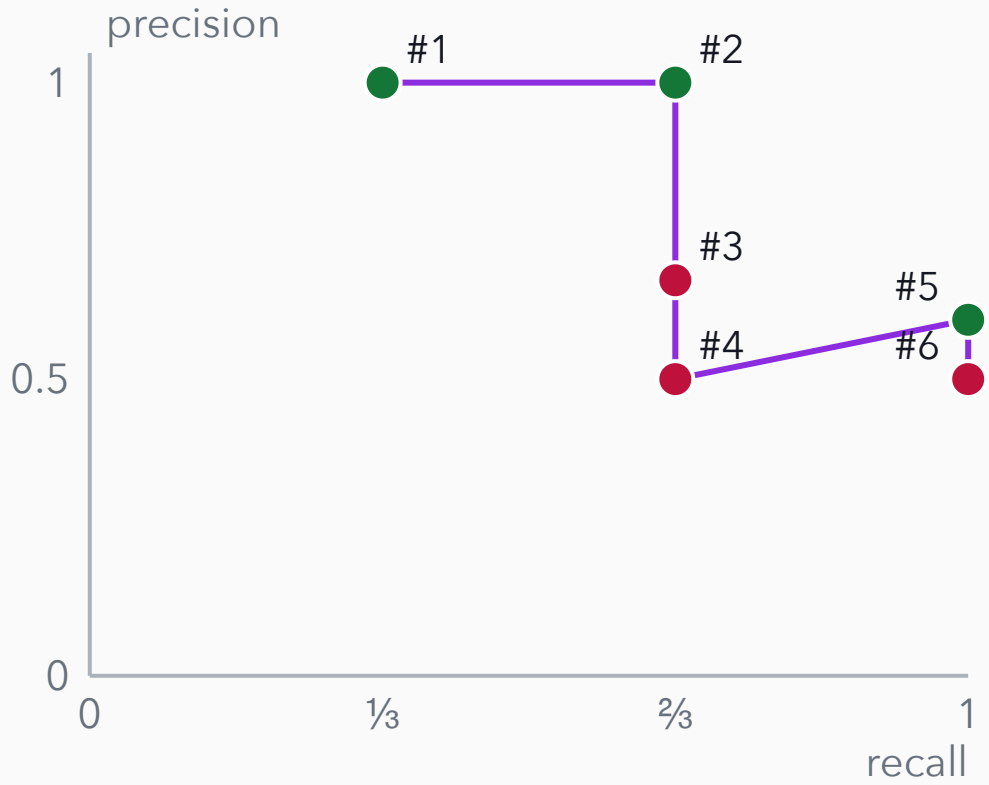
> #5 (0.60) claims C: TP. Recall reaches 1; precision 3/5.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60	**TP**	match	3/5	3/3
#6	0.40	**FP**	no truth here	3/6	3/3

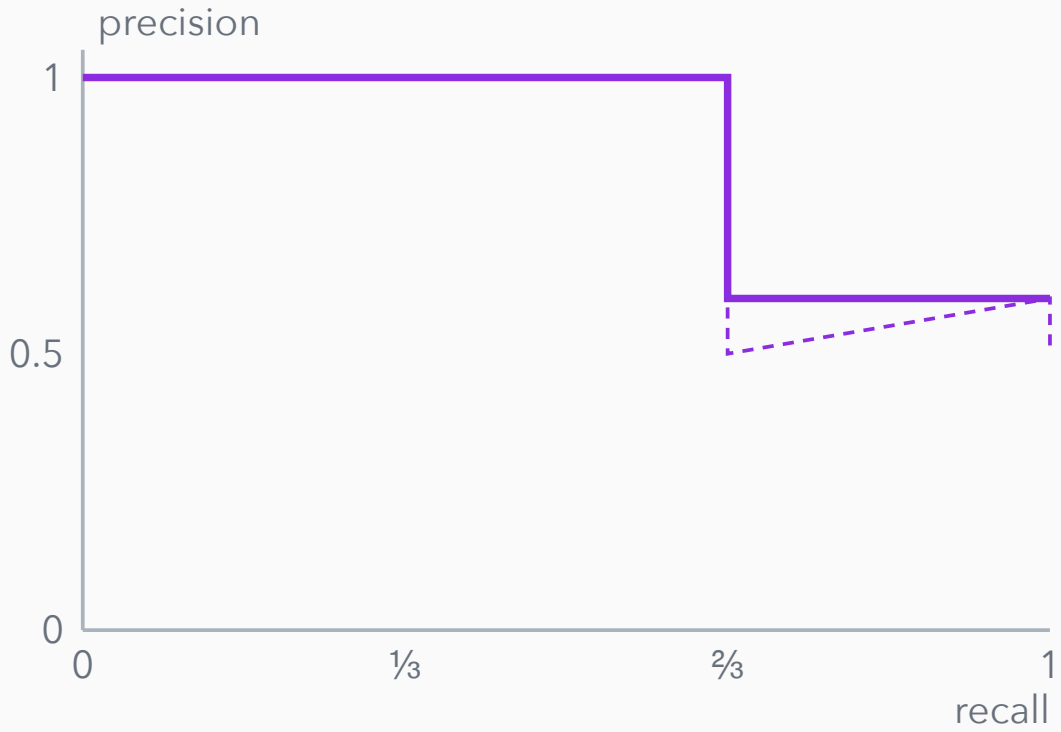
A small invented list (3 objects, 6 detections) so that hits and misses interleave.



> #6 (0.40): FP. 3/6, 3/3. Every point is one possible threshold.

▶ Notebook, part 8

Average precision: the area under the curve

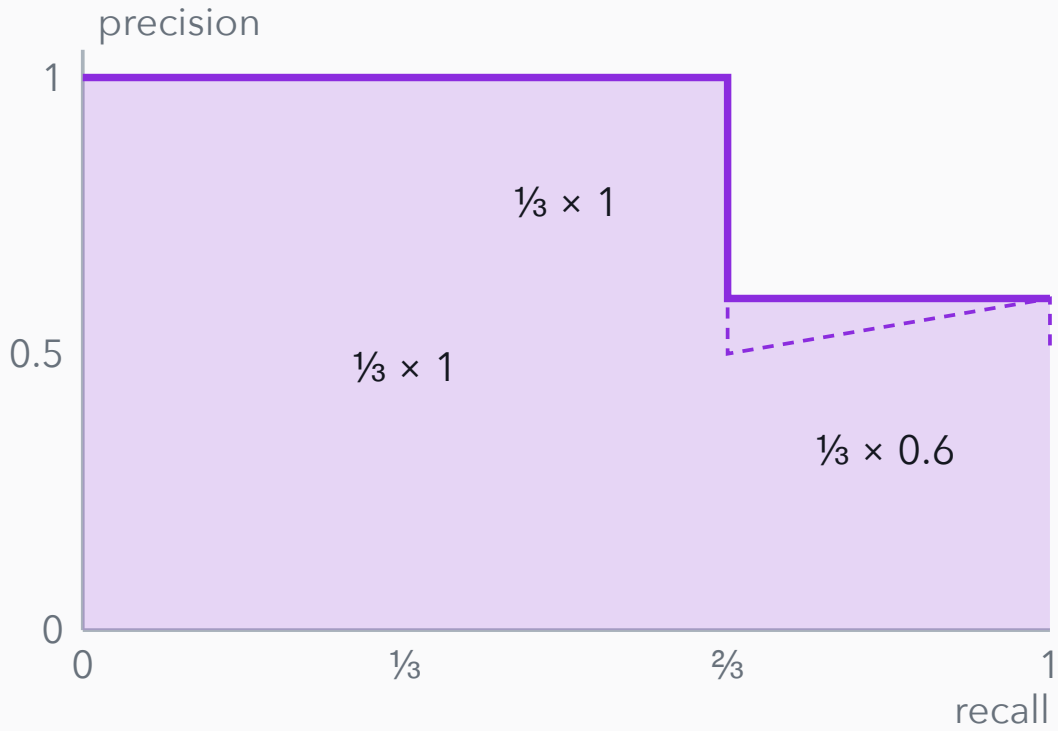


Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

> Smooth the zig-zag: at each recall, take the best precision reachable by lowering the threshold further. That is the envelope.

▶ Notebook, part 8

Average precision: the area under the curve



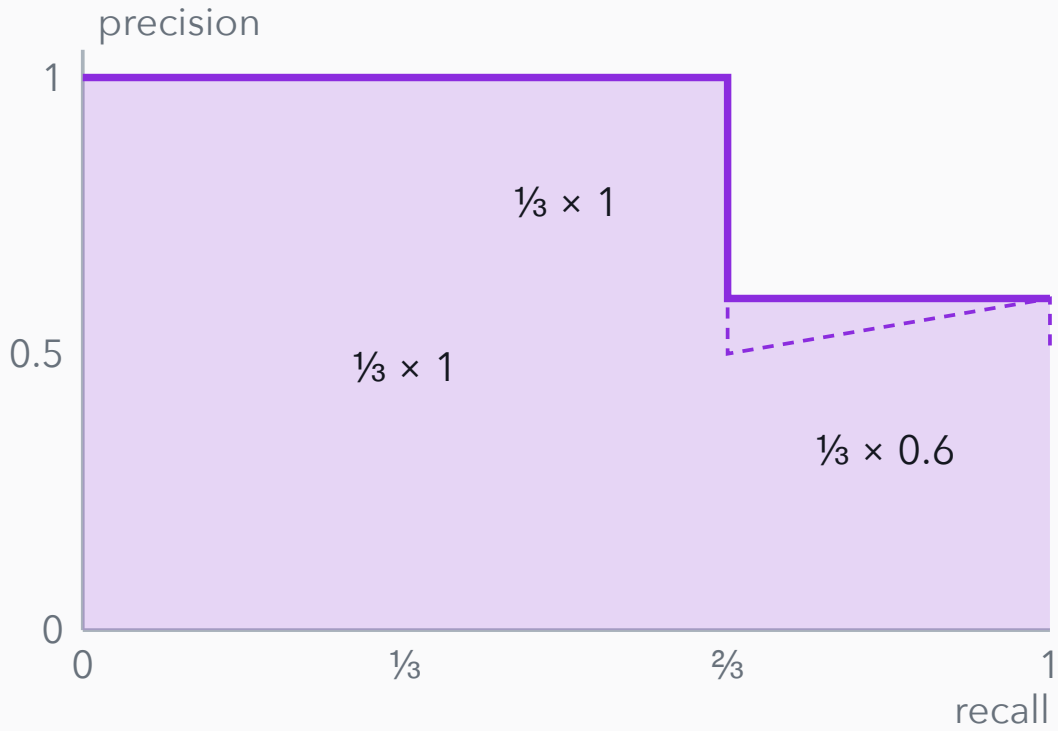
Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

Its area: a third of the recall at precision 1, another third at 1, the last third at 0.6. AP = 0.867.

▶ Notebook, part 8

Average precision: the area under the curve



Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

AP = 1 only if every object is found before the first false positive, as for the street's people.

> AP rewards ranking: correct detections should come before wrong ones. The street people scored AP = 1.

▶ Notebook, part 8

mAP: average over classes, and over IoU thresholds

$$\mathrm{mAP}_{50} = \frac{1}{C}\sum_{c=1}^{C}\mathrm{AP}_c \quad (\text{a hit needs IoU} \geq 0.5)$$

mAP averages AP over classes. COCO goes further and averages over ten IoU thresholds, from loose (0.5) to strict (0.95), which rewards tight boxes.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

mAP: average over classes, and over IoU thresholds

$$\mathrm{mAP}_{50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c \quad (\text{a hit needs IoU} \geq 0.5)$$

$$\mathrm{AP}_{[.5:.95]} = \text{mean of mAP at IoU } 0.50, 0.55, \ldots, 0.95$$

> Raising the IoU threshold turns loose boxes into false positives and their objects into misses: AP can only fall.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

mAP: average over classes, and over IoU thresholds

$$\mathrm{mAP}_{50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c \quad (\text{a hit needs IoU} \geq 0.5)$$

$$\mathrm{AP}_{[.5:.95]} = \text{mean of mAP at IoU } 0.50, 0.55, \ldots, 0.95$$

YOLOv8n	mAP@0.5	AP@[.5:.95]
our 8 photos, 7 classes, 31 objects	1.000	0.936
COCO validation, 5,000 photos, 80 classes (published)	–	0.373

Our eight clean photos are easy; COCO's crowded, cluttered photos are not.

> YOLOv8n on our eight photos: 1.00 at IoU 0.5, 0.94 averaged over the strict thresholds. On COCO's validation set its published AP is 0.373: real scenes are far harder.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

Evaluation in code

```python
from torchmetrics.detection import MeanAveragePrecision
metric = MeanAveragePrecision(iou_type="bbox")
metric.update(preds=[{"boxes": boxes, "scores": scores, "labels": labels}],
              target=[{"boxes": gt_boxes, "labels": gt_labels}])
res = metric.compute()          # res["map"] = AP@[.5:.95], res["map_50"] = AP@0.5
```

> Libraries do the matching, ranking and averaging. The notebook also has a twenty-line version that reproduces the walk-through.

One question, one answer, three uses of IoU

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

> The lecture's question: how can a network with a fixed output return a variable, unordered set?

One question, one answer, three uses of IoU

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

1. A candidate at every place: each cell of three grids writes four box numbers and 80 class scores.
2. In training, the cells near each object learn it; every other cell learns "nothing here".
3. At test time, keep the confident cells, then NMS keeps one box per object.

> The dense answer: outputs tied to places, many per object in training, one per object after NMS. The list's length is decided after the network.

One question, one answer, three uses of IoU

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

1. A candidate at every place: each cell of three grids writes four box numbers and 80 class scores.
2. In training, the cells near each object learn it; every other cell learns "nothing here".
3. At test time, keep the confident cells, then NMS keeps one box per object.

IoU used for	pairs	when
assignment	cells ↔ annotated objects	training
NMS	predictions ↔ predictions	test time
evaluation	predictions ↔ annotated objects	measuring

> Three places where boxes are paired up, easy to confuse: assignment in training, NMS between predictions, evaluation against annotations. Each uses IoU.

Back to the six jobs: what you would do now

job	classes	what is hard	what to do
road	person, bicycle, car, bus…	far pedestrians are tiny	fine grid (Part 7); follow boxes across frames
crowd	person	people stand close and overlap	check the NMS threshold (Part 6)
camera trap	each species	small animals get unsure scores, like the bird	a lower cut-off; more labelled photos of small animals

> The three everyday jobs: a pretrained COCO detector already knows their classes. What remains is tuning: grids, NMS threshold, score cut-off.

Back to the six jobs: what you would do now

job	classes	what is hard	what to do
road	person, bicycle, car, bus…	far pedestrians are tiny	fine grid (Part 7); follow boxes across frames
crowd	person	people stand close and overlap	check the NMS threshold (Part 6)
camera trap	each species	small animals get unsure scores, like the bird	a lower cut-off; more labelled photos of small animals
microscope	each cell type	no COCO class; cells touch	label cells by hand and fine-tune; NMS keeps touching cells
warehouse	parcel	new class, many sizes	label parcels and fine-tune the same detector
factory	crack, scratch, dent	thin and small defects	higher resolution, the fine grid; measure recall

> The three specialist jobs need new classes: label a few hundred photos with boxes, fine-tune the same detector, and measure precision and recall on photos it has not seen.

Next: beyond boxes



A box is a rough answer. Next lecture: which pixels belong to each object (masks), keypoints, depth, and boxes found from words: “the dog on the right”.

Beyond Boxes · Language-grounded vision

Practice: the notebook runs every number in this lecture. Appendices: A, DETR · B, more depth.

> Boxes are the coarsest description of where an object is. Next: masks, keypoints, and finding objects named in words.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py).

▶ Notebook, appendix A

Appendix A · DETR: a fixed set of slots, matched to objects

Back to Part 4's slots: make 100 of them, let each read the whole image, and match slots to objects before the loss. No NMS.

1. Follow a photo through DETR to 100 predictions, and see where two queries looked.
2. Watch duplicates vanish inside the decoder.
3. Build the matching cost one cell at a time, solve the matching, and compute the loss.

> The other answer to the lecture's question: DETR, 100 slots that read the image with attention, matched to objects instead of cleaned up by NMS.

▶ Notebook, appendix A

Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.

> Problem 1 was capacity: use K = 100 slots. Unused slots predict a 92nd class, “no object”.

▶ Notebook, appendix A

Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.

> A slot needs a way to look at the photo: give it a learned query vector that attends to the image features.

▶ Notebook, appendix A

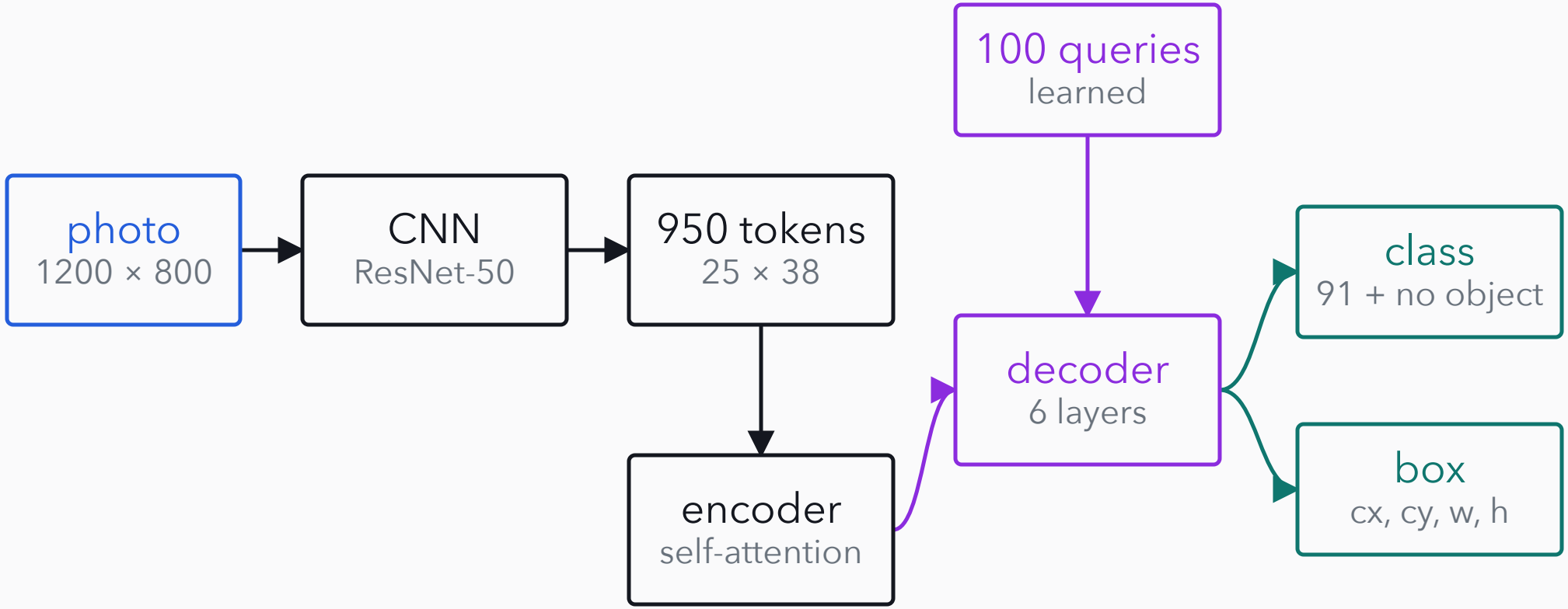
Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.
3. **Match before the loss**: pair slots with objects so that the total cost is lowest.

> Problem 2 was order: before computing the loss, find the cheapest one-to-one pairing of slots and objects. Order in the annotation list stops mattering.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

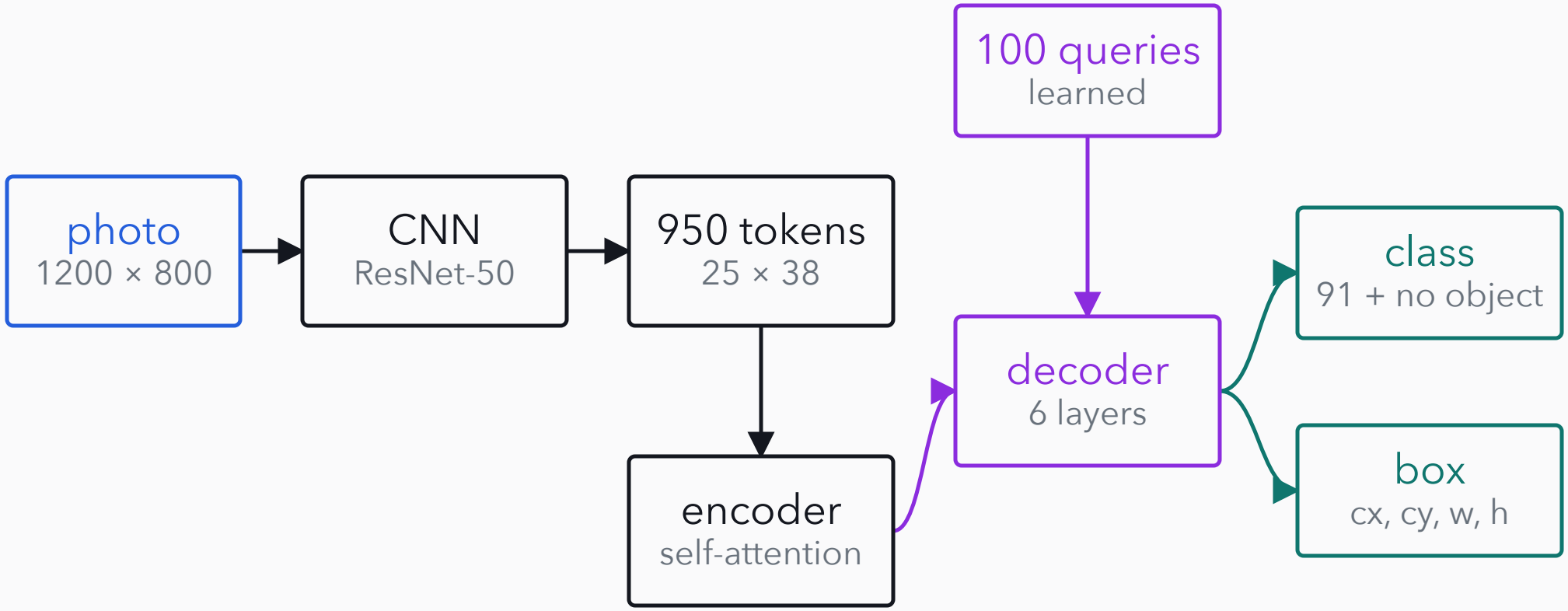


A ResNet-50 turns the photo, resized to 1200 × 800, into a 25 × 38 grid; each position becomes a token of 256 numbers. 950 tokens.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

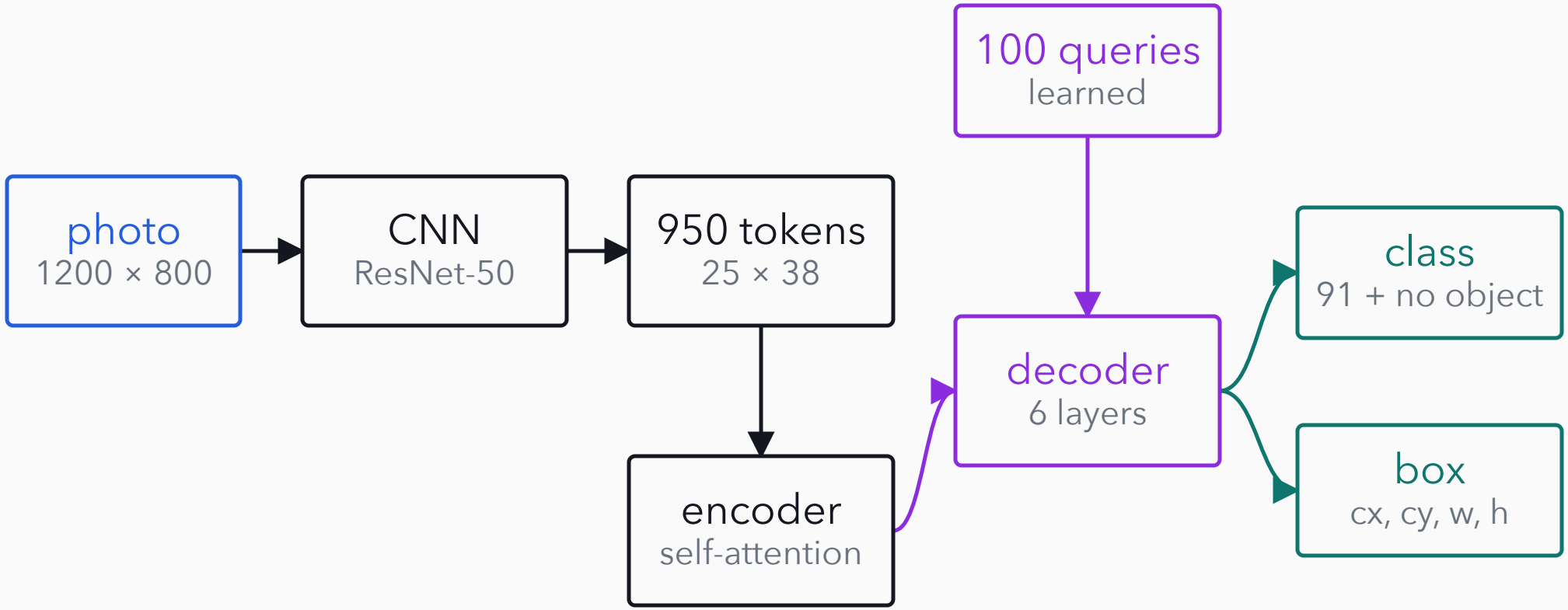


A transformer encoder lets the 950 tokens attend to each other, so each token knows about the rest of the photo.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

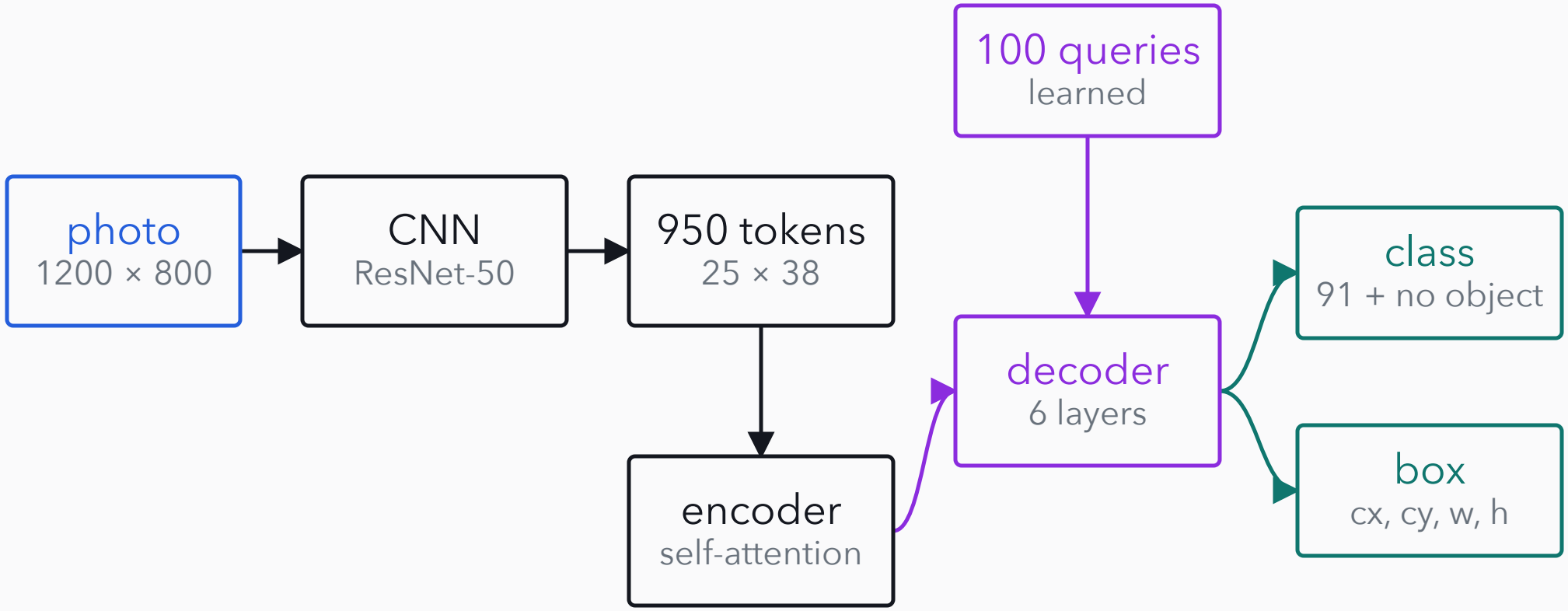


> The decoder starts from 100 learned query vectors. In each of 6 layers, the queries attend to each other, then to the 950 image tokens (cross-attention).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions



> Two small heads read each of the 100 outputs: a class among 91 COCO ids plus “no object”, and a box as normalised (c_x, c_y, w, h).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

A query is learned once and reused for every photo

Stored: 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

> The 100 queries are stored parameters: 100 × 256 numbers learned in training, identical for every photo.

▶ Notebook, appendix A

A query is learned once and reused for every photo

Stored: 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

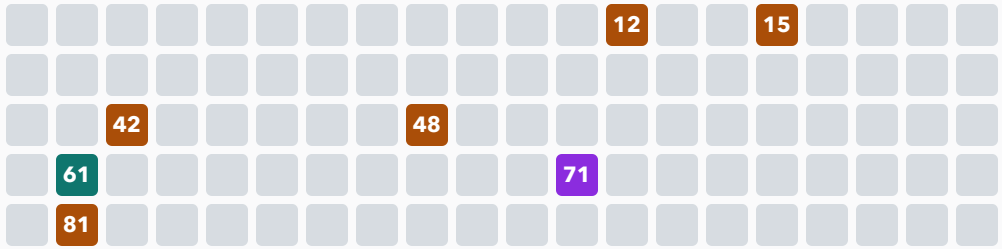
Computed: what each query writes depends on the photo, because it reads the photo through cross-attention.

As in the VLM lectures: stored query slots, computed per-image summaries.

> Their outputs are computed: the same query writes a dog box for one photo and “no object” for another, depending on what it read.

▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



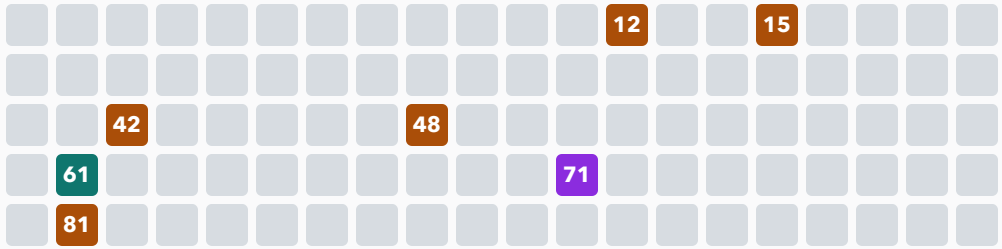
Grey: “no object”. 88 of the 100 slots give it more than 0.9.

> One square per query, numbered 0 to 99. Coloured if the query names an object; grey if it says “no object”.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



Grey: “no object”. 88 of the 100 slots give it more than 0.9.

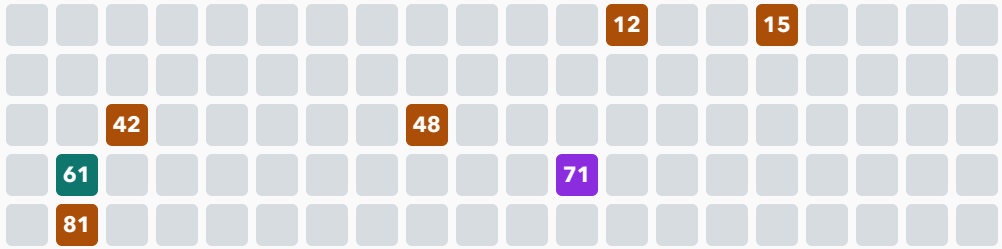
query	class	p	p(no object)
71	dog	0.999	0.001
61	cat	0.999	0.001
12	potted plant	0.998	0.002
81	potted plant	0.989	0.011
48	potted plant	0.973	0.027

> The busy ones: query 71 says dog (0.999), query 61 cat (0.999), others plants. Their numbers are arbitrary: training decided which slot does what.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

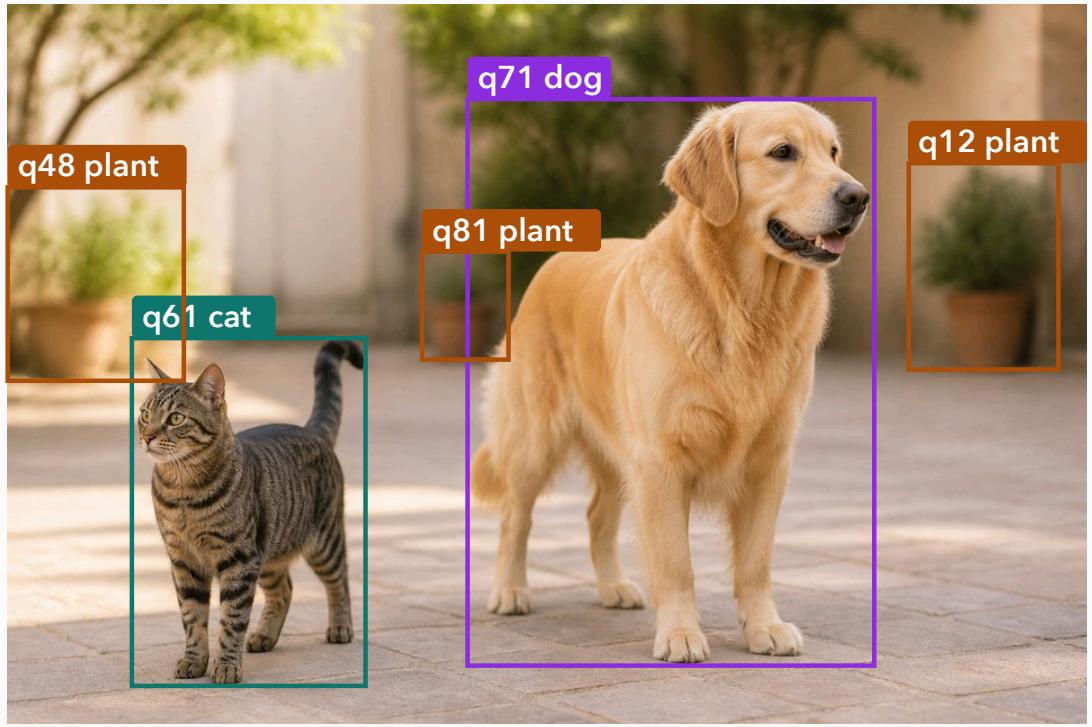
▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



Grey: “no object”. 88 of the 100 slots give it more than 0.9.

query	class	p	p(no object)
71	dog	0.999	0.001
61	cat	0.999	0.001
12	potted plant	0.998	0.002
81	potted plant	0.989	0.011
48	potted plant	0.973	0.027



> Their boxes: one per object. No duplicates to remove, and no NMS.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Where two queries looked



query 71 → dog

> Query 71's cross-attention in the last decoder layer, averaged over its 8 heads: where its weights over the 950 tokens went. Bright = much attention.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Where two queries looked



query 71 → dog



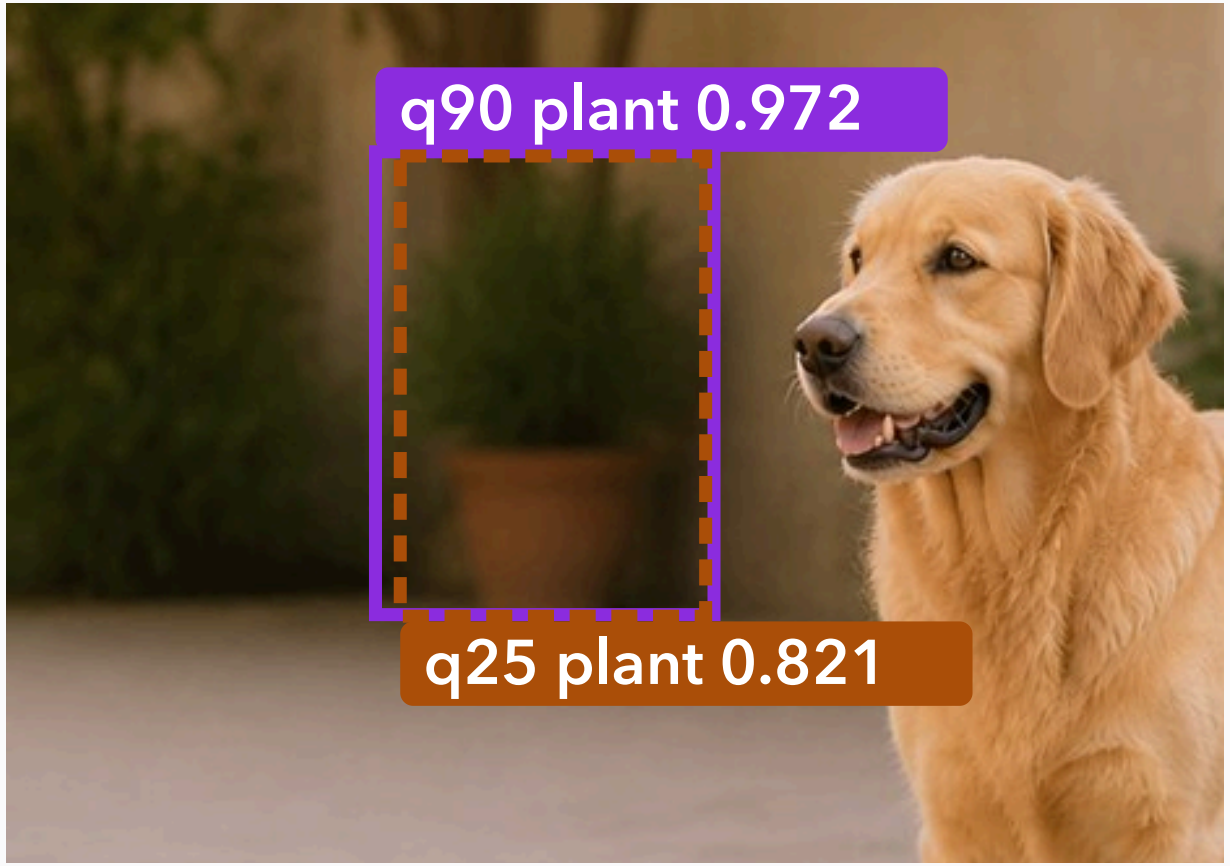
query 61 → cat

> Query 61, the cat: its attention sits on the cat. Each query attends mostly to the parts of the photo that fix its box: the extremities more than the middle.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



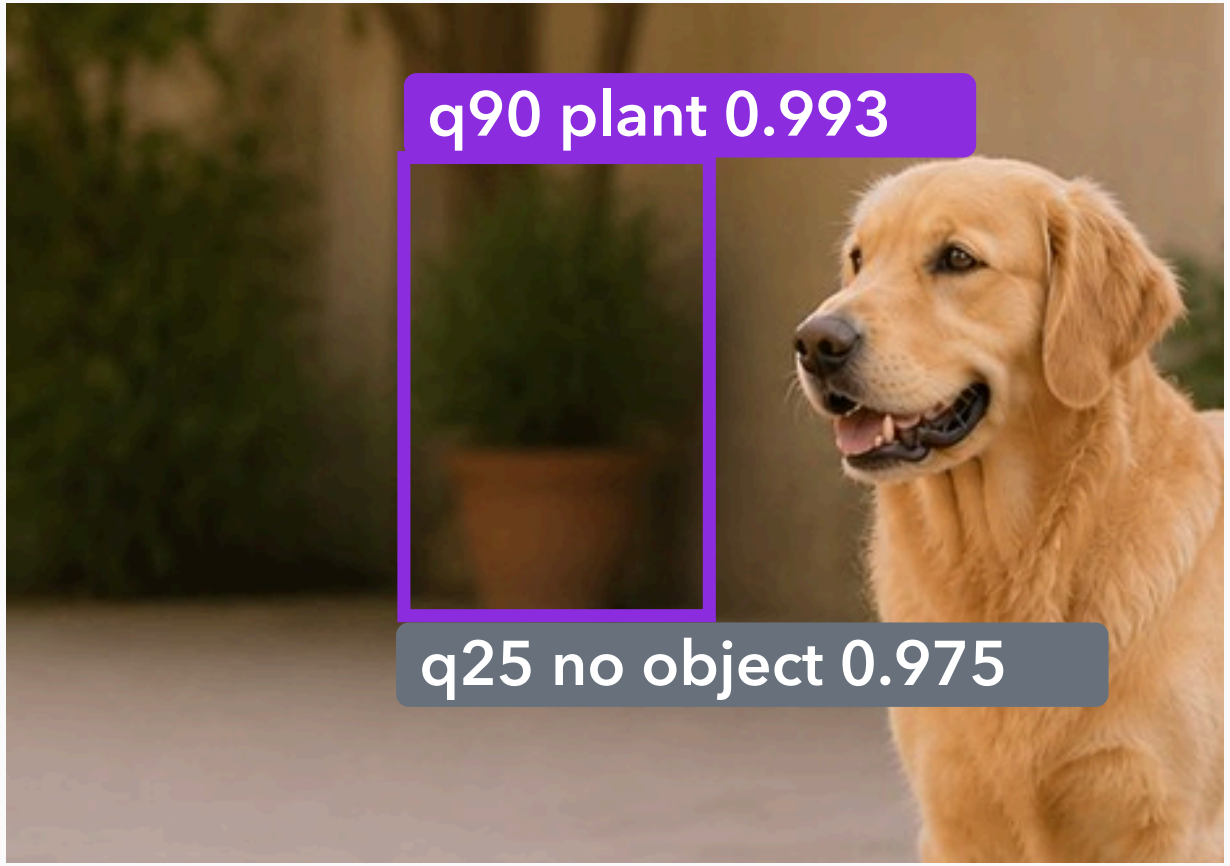
After layer 1: queries 90 and 25 both say “potted plant” for the same pot (IoU 0.891).

> DETR’s heads can read the queries after any decoder layer. After layer 1, two queries describe the same pot in the two-dogs photo.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



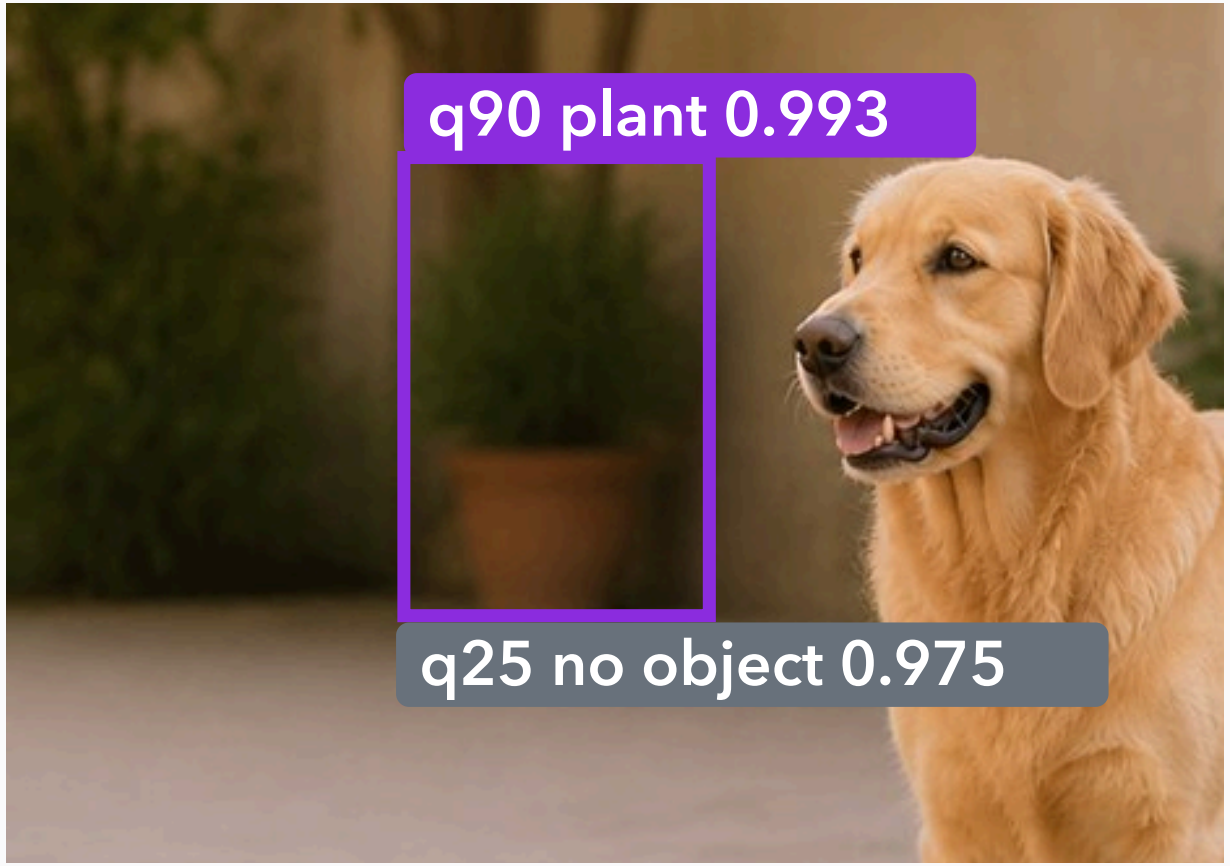
After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

> After layer 2, query 25 has dropped out: “no object” at 0.975. The duplicate was removed by the network, not by a post-processing rule.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

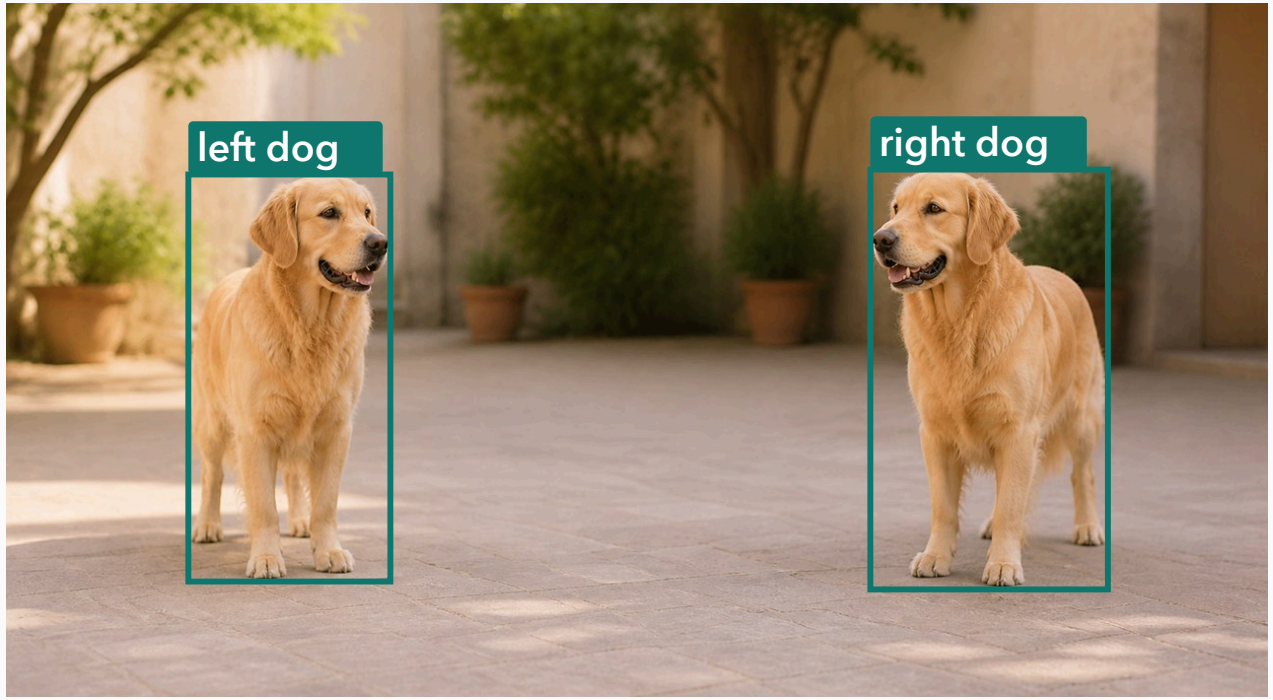
Queries attend to each other in every decoder layer, so one can see that another already has the pot. Training rewards exactly one of them per object (next slides).

> This works because training never rewards two queries for one object. That comes from the matching.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Training: which prediction should learn which dog?



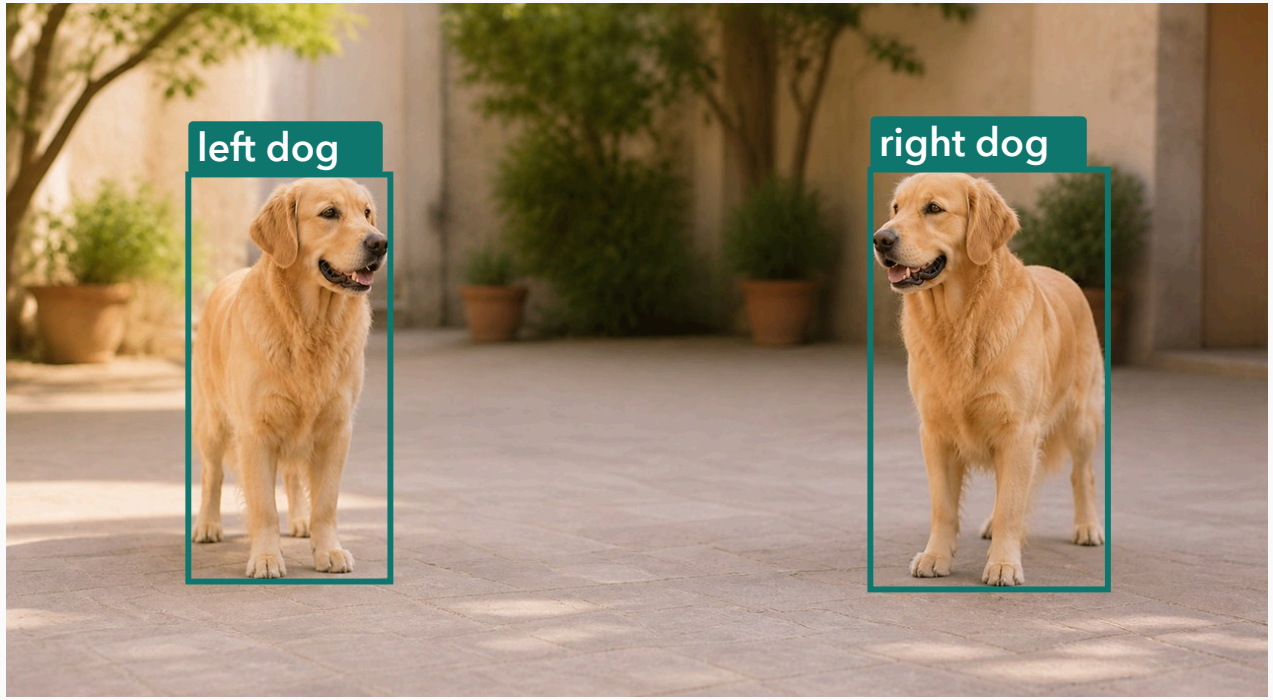
Two targets. 100 predictions. In Part 4, comparing slot k with list entry k punished a correct answer.

> Training photo: two annotated dogs. DETR made 100 predictions. Which prediction is compared with which dog?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Training: which prediction should learn which dog?



Two targets. 100 predictions. In Part 4, comparing slot k with list entry k punished a correct answer.

Instead: give every (prediction, target) pair a **cost**, then choose the one-to-one pairing with the lowest total.

> Score every pairing with a cost: low when the prediction has the right class and a box close to the target's. Then pick, among all one-to-one pairings, the cheapest.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

> IoU cannot tell a near miss from a far miss: both are 0. A loss built on IoU gives no direction to move.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

> Generalised IoU subtracts the empty fraction of the smallest box C that encloses both. Far apart, C is mostly empty and GIoU approaches −1.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

DETR's right-dog query against the left dog: IoU 0, GIoU −0.529.

> Real example: query 98 predicts the right dog; against the left dog its IoU is 0 but its GIoU is −0.529.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

DETR's matching cost has three parts: minus the probability of the target's class, the L1 distance between boxes, and minus the GIoU. Weights 1, 5 and 2.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

Query 61 gives “dog” probability 0.999: the class part is −0.999.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

> Its normalised box differs from the left dog's by 0.004 in total (L1), and GIoU is 0.984.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

$$C = -0.999 + 0.020 - 1.968 = -2.95$$

Total −2.95. Low cost = good pairing; negative is fine, only comparisons matter.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	−2.95	3.09	6.07	−1.71
right dog	?	?	?	?

Row 1: the left dog against four of the queries. q61 is cheapest; q2 is second: its box is on the left dog too.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	−2.95	3.09	6.07	−1.71
right dog	3.08	−2.90	4.47	4.26

> Row 2: the right dog. q98 is cheapest.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	**−2.95**	3.09	6.07	−1.71
right dog	3.08	**−2.90**	4.47	4.26

Cheapest pairing: left dog ↔ q61, right dog ↔ q98, total −5.85. Every other query is matched to nothing and learns “no object”.

q2’s box also sits on the left dog (cost −1.71), but it says “no object”: the class term makes q61 cheaper.

> Choose one query per dog with the lowest total: q61 and q98. Here each dog’s cheapest query is different, so the choice is easy. It is not always.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	**1**	2
right dog	2	**9**

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

> A made-up two-by-two case. Greedy: the left dog grabs q1, its cheapest at 1; the right dog is stuck with q2 at 9. Total 10.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	1	**2**
right dog	**2**	9

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

> The other pairing costs 2 + 2 = 4. Grabbing the cheapest first was wrong for the total.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	1	**2**
right dog	**2**	9

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

Trying every pairing is hopeless with 100 queries. The **Hungarian algorithm** (Kuhn, 1955) finds the cheapest one-to-one pairing in polynomial time: `scipy.optimize.linear_sum_assignment`.

> The best pairing must be found globally. The Hungarian algorithm does it efficiently; scipy has it.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

A matched query learns its object: cross-entropy for the class, plus L1 and 1 − GIoU for the box. q61 is already almost perfect: tiny loss.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \; 0.1 \times (-\log 0.997) = 0.0003$$

> An unmatched query learns “no object”, its cross-entropy weighted by 0.1 because most queries are empty. q2 already says no object: almost no loss.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \;\; 0.1 \times (-\log 0.997) = 0.0003$$

$$\text{q90 (a plant)} \to \text{no object:} \;\; 0.1 \times (-\log 0.003) = 0.581$$

Our target list holds only the two dogs, so the loss would teach q90 to call the pot nothing. What is annotated decides what is learned; COCO does label potted plants.

> q90 named a real plant, but plants are not in this target list: it pays 0.58 for being right. Annotation policy matters.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why DETR needs no NMS

DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

> Matching is one-to-one: in every training step, a duplicate query is told “no object”. The network learns not to duplicate, using self-attention among queries.

▶ Notebook, appendix A

Why DETR needs no NMS

DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

YOLO · one-to-many

- About ten cells per object are trained as positives
- All ten learn to fire
- So NMS must delete nine of them

> The dense detector trained ten cells per object on purpose (more positives, more signal) and pays for it with NMS.

▶ Notebook, appendix A

DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                  # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())         # one query per object
```

> The cost matrix is three lines: class, L1 and GIoU for every (query, object) pair. One call solves the matching. The loss then uses the matched pairs.

▶ Notebook, appendix A

DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                  # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())         # one query per object
```

```
model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
out = model(**processor(images=img, return_tensors="pt"))
dets = processor.post_process_object_detection(out, threshold=0.9,
                                               target_sizes=[img.size[::-1]])
```

> At test time there is nothing to match: run the model, keep the queries whose best class beats a threshold. No NMS.

▶ Notebook, appendix A

Two answers to one question

	dense (YOLOv8)	set (DETR)
outputs	one record per cell: 12,600	100 learned slots
who answers for an object	cells near it, by place	the matched query
training targets	one-to-many: ~10 cells	one-to-one: Hungarian matching
at test time	score filter + NMS	threshold only
on our courtyard	4 detections	5 objects at $p \geq 0.9$
our 8 photos, AP@[.5:.95]	0.936	0.882
COCO validation, AP (published)	0.373	0.420

> Side by side. The structural difference is how outputs are assigned to objects in training: by place, several per object; or by matching, one per object.

▶ Notebook, appendix A

Two answers to one question

	dense (YOLOv8)	set (DETR)
outputs	one record per cell: 12,600	100 learned slots
who answers for an object	cells near it, by place	the matched query
training targets	one-to-many: ~10 cells	one-to-one: Hungarian matching
at test time	score filter + NMS	threshold only
on our courtyard	4 detections	5 objects at $p \geq 0.9$
our 8 photos, AP@[.5:.95]	0.936	0.882
COCO validation, AP (published)	0.373	0.420

Both turn a fixed-size output into a set of any size: by scores and NMS, or by “no object” and matching.

> Both solve the lecture’s question with a fixed tensor plus a way to say “nothing here”.

YOLOv1 (2016): one responsible cell per object

$$7 \times 7 \text{ cells} \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Each cell: two boxes, each (x, y, w, h, confidence), and one set of 20 class probabilities (PASCAL VOC). Only the cell containing an object's centre is responsible for it; x, y are offsets inside that cell, w, h fractions of the image.

Confidence was trained toward the IoU between the box and the truth. A cell could hold at most one object's class.

> Why did YOLOv1 struggle with flocks of birds?

Anchors: predict offsets from template boxes

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = p_w\, e^{t_w}, \quad b_h = p_h\, e^{t_h}$$

Faster R-CNN, SSD and YOLOv2–v5 place several template boxes (anchors) of chosen sizes and shapes at every cell, and predict small corrections to each. Positives are anchors with high IoU to an object.

Anchor-free detectors (FCOS, YOLOv8) predict distances from the cell centre instead, as in Part 5.

> What must be chosen by hand for an anchor-based detector?

Two-stage detectors: propose, then classify

Stage 1, a region proposal network: about 2,000 boxes that might contain something.
Stage 2: crop features for each proposal (RoI pooling), classify it and refine its box.

R-CNN (2014) ran a CNN per proposal; Fast R-CNN (2015) shared one CNN pass; Faster R-CNN (2015) learned the proposals too. Mask R-CNN adds a mask per proposal: Lecture 23.

torchvision's fasterrcnn_resnet50_fpn returns post-processed detections: its NMS is already inside.

> What does the first stage output?

Focal loss: quiet the easy background

$$\mathrm{FL}(p_t) = -(1-p_t)^{\gamma}\log p_t, \qquad p_t = \begin{cases} p & y=1 \\ 1-p & y=0 \end{cases}$$

A background cell with p = 0.001: $p_t = 0.999$, so with $\gamma = 2$ its loss is multiplied by $(1 - p_t)^2 \approx 10^{-6}$. Easy examples nearly vanish; hard ones keep their weight.

RetinaNet (Lin et al., 2017) made one-stage detectors as accurate as two-stage ones this way.

> Which examples does focal loss down-weight?

Box formats in practice

where	format	the dog
torchvision	corners, pixels	(407, 83, 768, 588)
COCO json	(x, y, w, h), top-left, pixels	(407, 83, 361, 505)
YOLO label file	normalised (c_x, c_y, w, h)	(0.612, 0.524, 0.376, 0.789)
DETR output	normalised (c_x, c_y, w, h)	same as YOLO

> Why do format bugs often go unnoticed?

Resizing and padding move the boxes too

Detectors often resize a photo to a fixed size and pad the rest (letterboxing). The boxes must get the same treatment: scale every coordinate by the resize factor, then add the padding offset.

$$960 \times 640 \;\to\; 640 \times 427 \;(r = \tfrac{2}{3}),\ \text{pad } 106 \text{ px above}: \qquad y' = \tfrac{2}{3}\, y + 106$$

Our photos are multiples of 32 on both sides, so this lecture ran the models without padding.

> What happens to the dog's $y_1 = 83$ after this letterbox?

After DETR

- **Deformable DETR** (2021): each query attends to a few sampled points per feature level; trains about 10× faster.
- **DINO, RT-DETR** (2022–23): better queries and matching; real-time DETRs.
- **YOLOv10** (2024): one-to-many and one-to-one heads; NMS-free YOLO.
- **Open vocabulary** (OWL-ViT, Grounding DINO): classes given as text, matched with CLIP-like embeddings (the language-grounded vision lecture).

> What did DETR's successors mainly improve?